

Consumer Application Audit

Playora (rush-x.xyz) · Artifact reviewed: `index.html`, the shipped compiled bundle · 14 September 2026

Scope. The consumer-facing application: all 68 player-facing routes and the shared modules behind them — 85 Metro modules, 47,661 lines. The administrator and organizer interfaces were audited separately and are out of scope here.

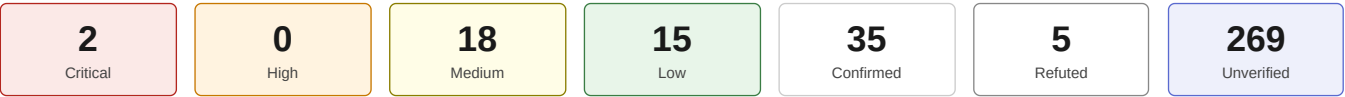
Method. Ten screen-group auditors and six cross-cutting sweeps (authorization, money integrity, data exposure, input handling, architecture, deployment readiness) read the shipped code in parallel, each following every screen into the backend functions it calls. Every finding was then put to two independent adversarial verifiers, one checking the evidence against the code and one checking that a real consequence follows; split votes were adjudicated by a third. 5 of 304 candidate findings were refuted and dropped. Severity is the more conservative of the two verifiers' assessments. A further 269 candidates could not be put through verification and are listed separately in section 5, unchecked.

How to read. Section 1 is the executive summary. Section 3 gives the four dimension reviews asked for — security and compliance, architecture and scalability, code quality and maintainability, testing and deployment readiness. Section 4 is the findings register. Section 5 is an explicit account of what this audit did *not* cover.

1. Executive summary

Not yet — two defects close the front door and the group-payment flow, and both are one-screen fixes rather than design problems.

The consumer app is substantially built and its business rules are more careful than most pre-launch codebases. What it is missing is not architecture but exercise: the two worst defects found are both cases of code that no test and, apparently, no manual run has ever executed. One await is missing in the phone verification path, which closes account creation entirely. One React effect is gated on the state it is itself the only producer of, which makes group reservations impossible to pay for. Neither is subtle once you look; both survived because the test harness injects a ready-made session and never walks a user in through the front door. The rest of the register is the ordinary work of a pre-launch app: error states that leave a screen blank, privacy toggles that the backend does not always honour, and a discovery surface with no automated coverage at all.



Must fix before real users and real money

- F-CQUAL-1 — phone sign-in and sign-up throw "wrong code" for every user, because the stored OTP record is read without await. Fixed in this pass.
- F-CARCH-1 — a group booking can be reserved but its payment view can never mount, so held seats expire unpaid and re-reserving throws. Fixed in this pass.
- F-CSEC-1 — OTP sessions were written without an expiry and with a Math.random token, and the session loader honoured any session lacking an expiry stamp. Fixed in this pass.
- F-CSEC-3 — password reset routes the recovery code through the SMS provider addressed to an email, so email account recovery cannot succeed and the user is told their code expired.
- F-CSEC-5 — guest mode issues one shared constant identity that can write to the social graph, so guest actions are indistinguishable between people.

What is genuinely well built

- The backend rules are centralised in one module rather than scattered through screens, so an authorization fix lands in one place and cannot be half-applied.
- Error codes are mapped to localised strings through a single table, and both locales are key-for-key complete with a test that enforces it.
- The money paths use a real ledger with explicit statuses rather than mutating balances, which is why the money-integrity sweep found no way to create or destroy funds.
- Audience partitioning is applied at the data layer, not the view layer, so a screen cannot accidentally leak across it.
- The admin and organizer surfaces, audited separately and since fixed, share these same backend rules — so consumer fixes inherit that work rather than duplicating it.

Risk if ignored. As shipped before this pass, the product could not acquire a single user: phone is the primary sign-in route and it failed unconditionally. Group bookings, the feature most likely to drive weekend volume, took seats out of inventory and then stranded them. Both are now fixed and covered by tests; the remaining register is quality and privacy work that should be scheduled, not emergency work.

2. Highest-priority findings

- F-CQUAL-1** (Critical, ./auth/sign-up.tsx and ./auth/sign-in.tsx (defect in mod/631.js)): mockVerifyOtp reads the OTP record without awaiting it, so every phone sign-in and sign-up fails with "wrong code"
- F-CARCH-1** (Critical, ./group/[gameId]): Group booking detail view is unreachable — reserved seats can never be paid for

By risk area

Risk area	Count
Data consistency	2
Error handling	2
Navigation dead end	2
Account creation / authentication	1
Dead-end state	1
Session management	1
Account recovery	1
Identity / unauthenticated writes	1

By screen

Interface	Count
./(tabs)/index	7
./booking/search	4
./discover	3
./auth/sign-up.tsx	3
./auth/sign-in.tsx (email tab, defect in mod/631.js)	2
./auth/sign-up.tsx and ./auth/sign-in.tsx (defect in mod/631.js)	1
./group/[gameId]	1
./auth/sign-in.tsx (defect in mod/631.js)	1

Risk area	Count
Date and time correctness	1
Query cost	1
App shell resilience / first-run routing	1
First-run routing / deep links	1
Misleading data	1
Request volume and result races	1
Consent and social graph	1
Data exposure	1
Authentication and identity	1
Regression coverage	1
Data integrity and rating manipulation	1
Localization / error handling	1
Form validation / recovery UX	1
Observability / release validation	1
Error handling / localization	1
Pre-consent PII retention	1
Stale state	1
Dead end	1
Localization	1
Error handling / silent data loss	1
Access control	1
Localization and labelling	1
Session management / identity confusion	1
Auditability	1

Interface	Count
./(auth)/sign-in.tsx, ./(auth)/sign-up.tsx (defect in mod/630.js and mod/631.js)	1
./_layout.tsx (defect in mod/630.js)	1
./_layout.tsx and ./index.tsx	1
./booking/venue/[id]	1
./discover, ./map, ./venue/[id], ./booking/search, ./booking/venue/[id]	1
./map	1
./venue/[id]	1
./(auth)/sign-in.tsx (forgot-password flow)	1
./blocked.tsx	1
./discover, ./map, ./booking/search, ./venue/[id]	1
./how-it-works.tsx and ./blocked.tsx	1
mod/630.js (AuthProvider)	1
mod/631.js (mock backend, phone auth)	1

3. Dimension reviews

Security & Compliance

7 Medium 4 Low

Eleven confirmed findings, none of them an open door to another user's money, but several that weaken the guarantees the product makes in its own words. The pattern is that a privacy or session promise is made in the UI and then not enforced on the read path: sessions written by the OTP route carried no expiry, guest mode is one shared identity rather than one per device, and Discover returns the full attribute set for profiles the viewer is only supposed to see in outline. None of these require a server to fix.

Priorities

#	Action	Why	Effort
1	Route password-reset codes through an email transport and persist them, so account recovery can succeed	Recovery is currently impossible for email accounts	Hours
2	Issue a per-device guest identity instead of one shared constant	Guest writes to the social graph are currently unattributable	Hours

#	Action	Why	Effort
3	Make the Discover DTO honour the viewer relationship rather than filtering in the client	Followers-only attributes are currently sent to non-followers	Days

Architecture & Scalability

1 Critical 5 Medium

Six findings, and the worst of them is a state-machine error rather than a scaling one: the group booking screen gates its loader on the state that loader alone produces. Beyond that, the home screen pulls the entire future game table on every focus and builds its list and its map from two different sets, so the two views can disagree. These are cheap to fix now and expensive once the game table is real.

Priorities

#	Action	Why	Effort
1	Build the home list and map from one filtered set	The two views can currently disagree about what exists	Hours
2	Paginate the home feed instead of pulling the whole future game table on focus	Cost grows with the game table, not the screen	Days

Code Quality & Maintainability

1 Critical 5 Medium 10 Low

Sixteen findings, and the highest-severity defect in the entire audit sits here rather than in security: a missing await. The recurring shape is a promise treated as a value and an error path that was never walked — raw error codes shown to users, router.back() as the only exit from screens that can be deep-linked into, and spinners with no failure state. Individually minor, collectively the reason a user who hits trouble has no way out.

Priorities

#	Action	Why	Effort
1	Give every screen that can be deep-linked a real back target instead of router.back()	Deep links currently dead-end	Hours
2	Map every thrown code to a localised string; several paths still surface raw codes	Arabic users see English identifiers	Hours
3	Add a failure state to the four discovery screens that currently spin forever	A failed load is indistinguishable from a slow one	Days

Testing & Deployment Readiness

1 Medium 1 Low

Only two findings, and that number is itself the finding. The discovery, map and venue-browsing surface has no automated coverage at all, and the harness injects a ready-made session, which is precisely why an unconditional failure in phone authentication reached the shipped bundle. Coverage now exists for the two Critical paths; the discovery surface remains untested.

Priorities

#	Action	Why	Effort
1	Cover the discovery, map and venue-browsing surface	It is the largest untested consumer area	Days
2	Keep driving auth through the real OTP path in tests rather than injecting a session	That injection is why the Critical shipped	Hours

4. Findings register

Sorted by severity. IDs: CSEC = security & compliance, CARCH = architecture & scalability, CQUAL = code quality & maintainability, CTEST = testing & deployment readiness. Evidence cites `mod/<id>.js:<line>` in the consumer module dump, where `<id>` is the Metro module id.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CQUAL-1 Critical	./auth/sign-up.tsx and ./auth/sign-in.tsx (defect in mod/631.js)	Account creation / authentication	mockVerifyOtp reads the OTP record without awaiting it, so every phone sign-in and sign-up fails with "wrong code" `s` is bound to a Promise, not the stored code record. `s.expires` is undefined so the expiry check is skipped, `s.verified` is undefined so the verification branch is always entered, `s.attempts += 1` yields NaN so the attempt cap never trips, and `t.trim() !== s.code` compares a string against undefined, which is always true. The function therefore throws OTP_WRONG unconditionally. It should await the lookup and operate on the persisted record. Evidence: mod/631.js:883	No user can create an account or sign in by phone. Sign-in (mod/613.js:176) shows "wrong code" for a correct code; sign-up (mod/917.js:134) fails at the final fork-confirm step, and the catch at mod/917.js:149-156 bounces the user back to the OTP step and re-sends a code, producing an unbreakable loop after the user has already entered their name, date of birth and audience. Phone is the primary (and for OTP-created accounts the only) sign-in route, so the product is effectively closed to new users. Scenario: New user on rush-x.xyz enters +965 phone, receives the demo code 123456, mockCheckOtp (631:851-861) accepts it and the wizard advances to name/DOB/audience; the final confirm calls mockVerifyOtp (917.js:135) which throws OTP_WRONG, the catch at 917.js:149 matches OTP_WRONG and bounces the user back to the OTP step and re-sends a code — an unbreakable loop. Existing phone users hit the same throw at 613.js:176 and at 917.js:541 (existing-account fork). No account can be created or signed into by phone.	In `ya` (mod/631.js:883) change `s = _a.get(o)` to `s = await _a.get(o)`, and await the three fire-and-forget `_a.delete(o)` calls at mod/631.js:886, 893 and 901 so the code is actually invalidated before a session is returned. Add a regression test that drives mockRequestOtp -> mockCheckOtp -> mockVerifyOtp end to end; tools/smoke.mjs currently injects a ready-made session into localStorage (seedScript) and never executes this path.
F-CARCH-1 Critical	./group/[gameId]	Dead-end state	Group booking detail view is unreachable — reserved seats can never be paid for After `createGroupBooking` succeeds the screen sets only the group id (`E(t.id)`, mod/2444.js:604) and re-renders the friend-picker form. It should load the group and switch to the hold/payment view. As written the reservation view, the countdown, the per-member statuses and the 'Pay your share' button can never be displayed. Evidence: mod/2444.js:13, 46-59, 87, 98, 120-271 (pay button at 260), 604; expiry at mod/631.js:13610-13631; already_booked at mod/631.js:13676; mod/2457.js:205-211	A user reserves seats for themselves plus friends/guests, is shown no confirmation, and is returned to the same builder form. The seats are held for 10 minutes (GROUP_HOLD_MS) and then silently released by the expiry sweeper (mod/631.js:13613-13631). There is no other route to pay a group hold — /my-bookings renders 'Pay to keep your spot' as static text with no action (mod/2457.js:205-211) — so every group booking ever made expires unpaid. Pressing Reserve again throws `already_booked` (mod/631.js:13676), leaving the user with no path forward at all. Scenario: A user opens /group/<gameId>, picks two friends and taps Reserve: createGroupBooking succeeds and seats go to status 'reserved', but the screen re-renders the same builder form (R is still null, so the F & R view and its 'Pay your share' button never mount). Ten minutes later Bd cancels every held booking; tapping Reserve again throws already_booked while the hold is live, so the group can never be paid for from anywhere in the app.	In the `createGroupBooking` success handler and in `pe()` (mod/2444.js:27-40), fetch the booking immediately (`O(await fetchGroupBooking(M.id, id))`) after setting `F`, and key the poll effect on `F` alone, stopping it when `R.group.status !== 'reserving'` inside the interval body rather than in the dependency guard.
F-CSEC-1 Medium	./auth/sign-in.tsx (defect in mod/631.js)	Session management	OTP sign-in writes a session with no expiry and a Math.random-derived token, and loadSession fails open when expires_at is absent The OTP branch hand-rolls session persistence instead of reusing `li`, dropping the 30-day TTL, and derives the bearer token from Math.random instead of randomToken. loadSession's type guard makes the expiry optional rather than mandatory, so a session lacking it is honoured forever. Evidence: mod/631.js:894	Every account that signs in by phone gets a session that never expires — the 30-day TTL the product claims does not apply to the primary sign-in route. The token is generated from a predictable PRNG, so once these tokens are sent to a real server (mod/673.js:56-62 attaches them as `Bearer`), they are guessable from observed values rather than unforgeable. Scenario: With the Finding 1 `await` applied, an existing phone user signs in at mod/613.js:176; ya returns at 631:895 having written a session with no expires_at, and 613.js:177 adopts it without calling persistSession. loadSession (631:2263) then skips the expiry branch on every future boot, so that device stays authenticated indefinitely — the documented 30-day TTL never applies to the primary sign-in route.	Replace mod/631.js:894-895 with a call to `li({ user: ..., token: await randomToken() })` so the expiry stamp and CSPRNG token are applied, and tighten mod/631.js:2263 to reject any session whose `expires_at` is missing or not a number.
F-CSEC-2 Medium	./auth/sign-in.tsx (email tab, defect in mod/631.js)	Account recovery	Password reset sends the code through the SMS provider addressed to an email address, and keeps codes in a non-persisted Map with no rate limit Three defects in one function: (1) the recovery code is handed to an SMS provider with an email address as the destination, so it is never delivered; the send throws, the catch at mod/631.js:930-931 deletes the code, and the user is later told E_CODE_EXPIRED_REQUEST_A_NEW_ONE (mod/631.js:942). (2) `ka` is a plain in-memory Map, so any page reload — including the app-switch to a mail client on mobile — destroys the pending code. (3) There is no throttle at all, unlike mockRequestOtp. Evidence: mod/631.js:929	Email/password account recovery cannot succeed in production: users who forget their password are permanently locked out and the UI blames them for an expired code. The unthrottled endpoint also lets an attacker trigger unlimited provider sends and repeatedly overwrite a victim's pending code (mod/631.js:926 resets it on every call), denying recovery to anyone under attack. Scenario: With demo:false in index.html, a user with an email/password account taps Forgot password on the sign-in screen. mockRequestPasswordReset generates a random code, stores it in `ka`, then calls ca(email, code); smsProvider rejects (an email address is not a dialable destination, or smsConfigured() is false and it throws SMS_NOT_CONFIGURED), the catch at 631:930-931 deletes the code, and the UI still reports sent. Every subsequent submit hits 631:942 and shows 'code expired, request a new one' forever — recovery is impossible. Even with demo:true, a page reload between request and submit empties the in-memory `ka` and produces the same expired-code error.	Route reset codes through an email transport, not `ca`/smsProvider; persist them in the same durable store as `_a` (mod/631.js:798-808) so a reload does not void them; and apply the existing `ma()` throttle keyed by email in mockRequestPasswordReset.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CSEC-3 Medium	./(auth)/sign-in.tsx, ./(auth)/sign-up.tsx (defect in mod/630.js and mod/631.js)	Identity / unauthenticated writes	Guest mode issues one shared, constant identity, and the backend guest guard covers only 4 of its write entry points Two rules are missing in 631 itself, so both survive a port to a server. First, the guest id carries no per-device entropy, so it is not an identity at all — two guests are the same principal. Second, `md()` is applied ad hoc to four join-related functions instead of centrally, leaving post creation, team creation, direct messages, blocks, stories and media uploads open to a principal that has no phone verification, no accepted terms (mod/631.js:903 `if (tr) throw new Error("TERMS_REQUIRED")` is only on the real signup path) and no age check (mod/631.js:902, MIN_AGE 13). Evidence: mod/631.js:14374	Content written by one guest is attributed to, visible to and mutable by every other guest, since they share `guest:male`/`guest:female`. Unverified, un-aged, terms-unaccepted visitors can author feed posts, create clans and send messages — the exact surface the E_SIGN_UP_TO_JOIN_BROWSING_IS error exists to close. Scenario: A visitor taps 'browse as guest (male)' on the sign-in screen, opens the Feed tab, taps the compose button (mod/1622.js:63), writes a caption and posts. mod/2375.js:60 calls createPost('guest:male', ...) -> mod/631.js:14374, which has no md() guard, and the post is stored with author_id 'guest:male'. The same principal can create a club via mod/2370.js:90 -> mockCreateTeam. None of the checks the real signup path enforces (TERMS_REQUIRED at 631:900, MIN_AGE at 631:899, phone verification) ever ran, and every guest on the device shares that one id.	Give each guest a random per-device id (`guest:<audience>:<uuid>`) in mod/630.js:74, and hoist the guest check out of the four ad-hoc call sites into a single write-path assertion in 631 alongside the existing audience (`ra`) and sanction (`wd`) guards, so every mutating mock* function rejects `guest:` actors by default.
F-CARCH-2 Medium	./(tabs)/index	Data consistency	The home screen's list and map are built from two different game sets, so results appear in one and not the other `suggested` is a personalised, de-duplicated top-24 slice that deliberately drops games the viewer has already joined and games they organise; `ce` is every public scheduled future game in the partition. Both are then filtered by the same sport/day/search predicates and presented as the same list under a single view toggle. One source should feed both views, with the recommender only controlling ordering. Evidence: mod/1677.js:112	Flipping list/map changes which games exist. A game the user has already confirmed shows as a pin on the map and is absent from the list; so does any game past the 24-item cap. Worse, the search result line at mod/1677.js:736 counts `ve.length` and the empty state at mod/1677.js:824-830 renders "Nothing matches X" off the same `ve`, so a user can search a venue, be told there are no games, switch to map, and see them. Scenario: With 30 upcoming public padel games, a user types a venue name that only matches games ranked outside the recommender's 24-item cut. The list renders `homeSearchNoneTitle` ("Nothing matches X") and the result line says 0; toggling to map shows pins for those exact games at that venue. The day strip is also built from both sources (mod/1677.js:145-150), so a day chip can be marked as having games and then show an empty list.	Drive both views from `ce` (`fetchUpcomingGames`) and use `G.suggested` only to sort and to attach `reasons`/`predicted`; or pass `ve`'s underlying rows into `VenueMap` at mod/1677.js:815-822 so the toggle cannot change membership.
F-CQUAL-2 Medium	./(tabs)/index	Date and time correctness	Home groups games by a UTC date key and formats them in the device zone, ignoring the configured region time zone Kuwait is UTC+3, so `toISOString().slice(0,10)` yields the previous calendar date for any start time between 00:00 and 02:59 local. Those games are bucketed under yesterday's divider, and the divider label is rebuilt from the key as local noon (mod/1677.js:874 `new Date(`\${e}T12:00:00`)`), so the heading and the card time disagree. Separately, the list ignores `getRegionSettings().timeZone` entirely while the "next up" card honours it. Evidence: mod/1677.js:1047	A 00:30 match on Tuesday is listed under the Monday heading, counted in Monday's `dayGamesCount`, labelled "MORNING" with a sunrise icon, and cannot be found by selecting Tuesday in the day strip. For any user whose device zone differs from the region they configured in /settings/region, the same game shows one time in the "next up" card and a different time in the list below it. Scenario: A Kuwait user (UTC+3) opens home with a game starting Tue 00:30 local (2026-09-15T00:30+03:00 = ...T21:30Z). L() returns "2026-09-14", so the card is filed under the Monday divider, counted in Monday's total, labelled MORNING, and disappears entirely when the user taps the Tuesday chip — while the card itself prints 12:30 AM. Separately, a user whose device is on UTC sees the next-up card (Asia/Kuwait) and the list card (device zone) print two different times for the same game.	Replace `L` at mod/1677.js:1047 with a zone-aware key derived from `formatInZone(date, locale, { year:"numeric", month:"2-digit", day:"2-digit" })` in module 912, build `ze` and the band classifier (mod/1677.js:879-882) from the same zone, and swap the raw `toLocaleTimeString`/`toLocaleDateString` calls at mod/1677.js:961, 1176 and 1223 for `formatClock`/`formatDayLabel` from mod/1311.js.
F-CARCH-3 Medium	./(tabs)/index	Query cost	Home pulls the entire future game table on every focus and fires six unbounded queries per refresh There is no pagination, no date window and no field projection on the home screen's heaviest query, and nothing coalesces the six calls into one. The screen only ever renders 24 suggested rows plus map pins, so the payload is unbounded relative to what is shown. Evidence: mod/631.js:2679-2694, mod/631.js:2665-2677, mod/631.js:15395, mod/631.js:16878/16899, mod/1677.js:38-71	Home is the app's most-visited screen and it re-fetches every upcoming public match in the city — twice, once directly and once inside the recommender — every time the tab regains focus or the live-refresh timer ticks. Behind a real API this is the query that will fall over first as the game table grows, and on a slow connection it is the reason the home screen stalls. Scenario: With a few hundred future public games, every focus of the Home tab (and every live-refresh tick) walks the entire future game table twice — once via fetchUpcomingGames and once inside the recommender — awaiting Oi per game and scanning Qt.bookings three times per row, to render at most 24 suggestion rows.	Give `mockGetUpcomingGames` (mod/631.js:2679) a bounded window and limit (e.g. the next 14 days, capped at the number of rows the caller renders), hoist the repeated `Qt.bookings` scans in `li` into one pass, and have mod/1677.js:42-62 fetch the map dataset lazily only when `pe === "map"`.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CQUAL-3 Medium	./(tabs)/index	Data consistency	The month picker marks days that the list view cannot show, dead-ending on an empty day `Le` advertises availability from the union of `ce` and `G.suggested` while the list can only ever render `G.suggested`. The set should be derived from the same source the list renders, or the picker should filter to dates that survive `ve`'s predicate. Evidence: mod/1677.js:145-150, mod/1677.js:998-1007, mod/1677.js:112-123 and mod/1677.js:824-871	A user opens the month dropdown, sees a date flagged as having games, taps it, and lands on the empty-day state at mod/1677.js:831-865 ("nothing on tonight"). Because `G!` excludes games the user has already joined and games they organise (mod/631.js:15396) and caps the rest at 24 (mod/631.js:16899), this happens routinely for active users — the picker points them at their own matches and then denies they exist. Scenario: A user who has already joined Thursday's match opens the month dropdown: Thursday is flagged as having games (it is in ce), tapping it sets the day filter, and the list — built only from suggested, which excludes joined and self-organised games — renders the "nothing on" empty-day card for a day the picker just said had games.	Build `Le` at mod/1677.js:145 from the same rows the list renders (or, preferably, fix the underlying single-source issue so `ce` drives both views), so a flagged date always yields at least one row.
F-CARCH-4 Medium	./_layout.tsx (defect in mod/630.js)	App shell resilience / first-run routing	AuthProvider's session bootstrap has no catch, so a storage read failure leaves the app permanently in loading and the shell never routes The bootstrap treats storage as infallible. Any environment where localStorage access throws — Safari with "Block all cookies", an embedded/third-party context, a profile with site data blocked — rejects the promise, `y(!1)` never runs, `loading` stays true forever, and the root effect returns early on every render. Evidence: mod/630.js:21	Affected visitors get a permanently half-booted app: the tab shell renders with no session, they are never redirected to sign-in, no screen can gate on `loading`, and the only console output is an unhandled rejection. There is no retry, no error state and no way out except clearing the site. Scenario: Safari with 'Block all cookies' (or the page embedded in a storage-partitioned third-party context): window.localStorage access throws SecurityError, module 619's getItem promise rejects, loadSession rejects at mod/631.js:2258, the un-caught chain at mod/630.js:21 never calls y(!1), loading stays true forever, mod/1809.js:51 returns early on every render, and index.tsx has already redirected to /(tabs). The visitor sits on the tab shell with session null, is never sent to /(auth)/sign-in, and the only output is an unhandled rejection — no retry, no error state.	Add `.catch(() => { w(null); y(!1); })` to the chain at mod/630.js:21-25 so a storage failure degrades to signed-out, and move the `secureGet` call at mod/631.js:2258 inside the existing try/catch so `loadSession` resolves to null instead of rejecting.
F-CARCH-5 Medium	./_layout.tsx and ./index.tsx	First-run routing / deep links	The shell renders the signed-in tabs before the session is known and throws away the deep-link destination when redirecting to sign-in There is no loading gate between mount and the first routing decision, and the sign-in redirect drops the requested URL instead of forwarding it as `next`, which the sign-up screen already supports. Evidence: mod/1809.js:54	Every cold load flashes the authenticated home tab to a signed-out visitor and mounts each tab screen with a null user before bouncing to sign-in. Worse, a shared match link (https://rush-x.xyz/game/<id>) opened by a signed-out invitee is lost: after signing in they land on the tabs with no path back to the match they were invited to — the single most common acquisition flow for a pickup-sports app dead-ends. Scenario: A signed-out invitee opens a shared match link https://rush-x.xyz/game/<id>. mod/1809.js:54 fires S.replace(`/auth/sign-in`) with no next param; after a successful sign-in mod/613.js:177 (OTP) or 613.js:361 (password) replaces to `/(tabs)`. The invitee lands on the home tab with no route back to the match they were invited to, and the original URL is gone from history because both redirects use replace.	Render a splash while `loading` is true in mod/1809.js:49, and at line 54 pass the current path as a `next` param; add `useLocalSearchParams` handling to mod/613.js so both the OTP (line 177) and password (line 361) success paths honour it, matching the validated pattern already in mod/917.js:16.
F-CQUAL-4 Medium	./booking/search	Misleading data	"km away" distances and the default result ordering on booking search are fabricated from a hash of the user id There is no geolocation anywhere in the flow. When the user has not set a `home_area` — which nothing in the booking flow asks them to do — `Sa` silently substitutes a deterministic pseudo-random point in a $\pm 0.11^\circ$ box around Kuwait City and the result is presented as a real distance and used as the primary sort key. The fallback should return null and the UI should omit the distance and fall back to the price ordering. Evidence: mod/631.js:1025	Users see confident claims like "3.2 km away" that are a hash of their user id, and the venue list they are shown first is ordered by that fiction rather than by price or proximity. Two users pick different venues for the same query for no real reason, and the nearest venue can be sorted last. Scenario: A user signs up (profile has no home_area), is approved as an organizer, and opens /booking/search. Every row shows "- N km away" where N is derived from a hash of their UUID, and the whole list is ordered by that number, so a venue two streets away can sort last while one across the city sorts first. Two users running the identical query get different orderings and different distances for the same venue, and nothing in the flow ever asks for their location or area.	Make `Sa` (mod/631.js:1025) return null when there is no `home_area` and reserve `demoUserLocation` for the demo flag (`FaQ`), let `mockSearchBookableVenues` fall through to the price sort at mod/631.js:6969 when `i` is null, and prompt for an area in mod/1840.js when the distance line would otherwise be hidden.
F-CARCH-6 Medium	./booking/search	Request volume and result races	Both search screens re-query the backend on every keystroke; the /discover debounce is dead code and /booking/search has no ordering guard `useFocusEffect` re-runs whenever its callback identity changes while the screen is focused, so binding it to a callback that closes over the query text turns it into a per-keystroke trigger. The focus effect should depend on a stable refetch reference, and each in-flight request needs the cancelled-flag guard already used in mod/1842.js. Evidence: mod/1840.js:16-32 and mod/1840.js:104-107; mod/2374.js:38-54; correct shape at mod/1842.js:28-49	Typing an eight-character venue name issues eight full `searchBookableVenues` calls (each of which re-scans every venue and every court, mod/631.js:6937-6969) and, on /discover, sixteen `searchPlayers` calls. With a real backend behind mod/673.js this is a burst of uncancelled requests whose responses can land out of order, so the list can settle on the results for a prefix of what the user typed while the input shows the full query. Scenario: On /booking/search the user types an 8-character venue name: each character sets J(true) and issues a full searchBookableVenues (mod/631.js:6937 re-scans every venue and its courts), so the result list is replaced by a spinner on every keystroke and, with no cancellation token, a slower earlier response can overwrite the results for the full query.	Hold the query in a ref or pass a stable `refetch` to `useFocusEffect` at mod/1840.js:28 and mod/2374.js:50, debounce `F` in mod/1840.js as mod/2374.js:38-49 intends to, and add the cancelled-flag guard from mod/1842.js:30-47 to both loaders.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CQUAL-5 Medium	/booking/venue/[id]	Error handling	An unknown or failed venue id on the booking screen renders a spinner forever with no error and no way back The loading flag `K` and the not-found state `!A` are conflated into one branch that renders only a spinner — no header, no back chevron, no message, no retry. A rejected fetch is worse still: `J(!1)` never runs, so the promise rejects unhandled and the spinner is permanent. Compare mod/2502.js:30-34, which renders `EmptyState { title: V("venueNotFound") }` for the same case on the other venue screen. Evidence: mod/1842.js:64	A stale bookmark, a shared link to a venue that was removed, or one failed request leaves the user staring at a spinner on a screen that has no chrome at all. Because `/booking/venue/[id]` sits outside `(tabs)` there is no dock either, so on web the only escape is the browser back button. The user never learns that anything went wrong. Scenario: Open a stale bookmark or shared link to /booking/venue/<id of a venue no longer in the store>: fetch resolves null, J(!1) flips loading off, `!A` keeps the same branch, and the screen renders only a spinner forever — no title, no back chevron, no message. Same outcome if the loader rejects (E_STORAGE_FULL from the Dr() seed write, mod/631.js:1763-1775), except J(!1) never runs and the rejection is unhandled. Downgraded from High because the browser back button / native swipe still exits.	Wrap the loader in try/catch, keep a separate `error` state, and split the branch at mod/1842.js:64: render the header (with the back control from mod/1842.js:78-84) plus `EmptyState`/retry when `!A`, and the spinner only while `K` is true.
F-CSEC-4 Medium	/discover	Consent and social graph	The female "Send message" button on Discover silently sends a follow request and can never open a chat For a female viewer looking at any female player she does not already follow, `can_message` is false and `dm_state` is "none", so tapping the button first calls `followPlayer` without asking, then calls `startConversation`, which throws `messages_followers_only` because the follow it just created is still `pending`. The bare `catch { await le(); }` swallows the error and only refetches the list. The code should not create a follow as a side effect of a message tap, and it should surface the "this player only accepts messages from followers" outcome instead of discarding it. Evidence: mod/2374.js:61-76	Every female user sees a prominent "Send message" CTA on essentially every other female player that does nothing visible no matter how many times she taps it, while each tap quietly files a follow request in her name that she never consented to send and that the recipient receives as a notification. The single most important social action in the women's experience is a permanent dead end. Scenario: A female user on /discover taps "Send message" on any female player she does not already follow (the default case): a pending follow request plus a follow_request notification with her name is created at mod/631.js:14053-14072 without her asking, startConversation then throws messages_followers_only, the catch refetches, and the row comes back identical — same "Send message" button, Follow button still reading "Follow" — so nothing visible happens no matter how many times she taps.	In `re` (mod/2374.js:61-76) remove the implicit `followPlayer` call, and in the `catch` set an error state rendered next to the row (reusing `storeErrorText`) rather than swallowing it. Render the button as "Follow to message" when `!can_message && dm_state === "none"`, driven off the same `allow_messages` rule that `gl` enforces in mod/631.js:14650-14655.
F-CSEC-5 Medium	/discover	Data exposure	Discover returns the full attribute set for followers-only profiles that the profile endpoint deliberately withholds Two endpoints over the same records disagree about what a non-follower may see. `mockGetPlayerProfile` treats `profile_visibility === "followers"` as a gate; `mockSearchPlayers` ignores it entirely and only filters `private`. The search result should apply the same `d` computation and null out the gated fields, or exclude followers-only profiles the viewer does not follow. Evidence: mod/631.js:14202	Because `followers` is the default privacy setting for every female-audience account, the entire women's directory leaks sports, skill level, home area, availability (`looking_for_game`) and live presence to any signed-in viewer in that partition — including a guest session — even though opening the same player's profile correctly shows a "limited" card. The gate is real in the product and absent in the listing, so the setting silently does nothing where it matters most. Scenario: A signed-in female viewer (or a guest:female session, since aa() at mod/631.js:750-752 maps guest ids to an audience) opens /discover: every female player on default privacy is listed with her skill level, her sports and a "looking for a game" flag, while tapping the same player's profile returns limited:true with skill_level `all`, favorite_sports [] and looking_for_game false — the followers-only setting is enforced on one endpoint and ignored on the other.	In `mockSearchPlayers` (mod/631.js:14197-14258) compute the same gate as mod/631.js:14281 and either `continue` on a gated profile or push a reduced row (`skill_level: "all", `sports: [], `area: null`, `looking_for_game: false`, `limited: true`), so the listing and `mockGetPlayerProfile` can never disagree.
F-CSEC-6 Medium	/discover	Authentication and identity	Guest sessions are a single shared identity that can write to the social graph `mockFollow`, `mockUnfollow` (mod/631.js:14076) and `mockStartConversation` (mod/631.js:14656) accept any actor id, including one with no profile row behind it. Writes should require an actor that resolves to a real profile, and guest sessions should be read-only. Nothing in 631 asserts either. Evidence: mod/631.js:14046-14048 (no actor check) with mod/630.js:72-76 and mod/631.js:12395 (the guard that exists for joins but not for follows)	An anonymous visitor with no email, phone or password can follow and unfollow real players and generate notifications to them. Because all guests share one id, the follow rows, DM conversations, presence and audit actor refs of every anonymous visitor collide into one account. The recipient sees a follower called "A player" whose profile page throws `not_found` (mod/631.js:14274), so the follower list contains an unopenable phantom entry, and the rows outlive the guest session, which `clearGuestSession` wipes on sign-out. Scenario: A visitor taps 'browse as guest', gets id guest:female, opens /discover from the feed link and taps Follow: mockFollow accepts an actor with no profile row, writes a follow with follower_id 'guest:female' and sends the target a follow_request notification named "A player" (14071). Opening that follower's profile throws not_found (14274), leaving an unopenable row that outlives the guest session cleared by signOut.	Add an actor assertion in mod/631.js next to `on`/`sn` (mod/631.js:4504-4519) — e.g. `assertRealUser(e)` that throws unless `Qt.profiles.some(p => p.id === e)` — and call it at the top of `mockFollow`, `mockUnfollow`, `mockStartConversation` and `mockUpsertReview`. On the client, gate the Follow and message buttons in mod/2374.js:326-342 behind `!user.id.startsWith("guest:")` and route guests to sign-up.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CTEST-1 Medium	./discover, ./map, ./venue/[id], ./booking/search, ./booking/venue/[id]	Regression coverage	The discovery, map and venue-browsing surface has no automated coverage at all Five routes in this group — including the whole booking funnel and both error-prone id-parameter screens — have zero end-to-end assertions. The harness to do it already exists (tools/smoke.mjs exports `openApp` with a seeded session and a console/page error collector), so the missing piece is only the suites. Evidence: tools/flows-search.mjs:7 and 36-64 (and every other openApp call in tools/*.mjs); harness at tools/smoke.mjs:58-71; uncovered behaviour at mod/1842.js:64 and mod/2450.js:31	Nothing in CI would catch the permanent spinner on an unknown `./booking/venue/<id>`, the inert back button on a deep-linked `./map`, the mislabelled price filter, or the list/map divergence — all of which are present in the shipped bundle today. Any future edit to module 631's venue or search functions can break this surface silently, since the suites that do run never touch it. Scenario: An edit to mockSearchBookableVenues (mod/631.js:6937) or mockGetVenueBookingDetail (mod/631.js:6971) that returns null or throws leaves ./booking/venue/<id> stuck on the spinner at mod/1842.js:64; all seven suites still pass because none of them ever opens that route.	Add a `tools/flows-discovery.mjs` that drives `openApp` against `./discover`, `./map`, `./venue/<seeded id>`, `./venue/does-not-exist`, `./booking/search` and `./booking/venue/does-not-exist` for both `user` and `organizer` roles, asserting that each renders a titled state (not a bare spinner) and that `errors.length === 0`, and register it alongside the existing seven suites.
F-CQUAL-6 Medium	./map	Navigation dead end	Every discovery screen's back control is an unguarded router.back(), which does nothing on a directly opened URL These screens assume they were pushed from inside the app. On a deep link, a shared URL, or a page refresh the navigation stack contains only that route, so `back()` silently does nothing. Each back control should fall back to `router.replace("/(tabs)")` exactly as `GateScreen` does. Evidence: mod/2450.js:31	`./map` is declared as a `fullScreenModal` on the root stack (mod/1809.js:81-84), so it renders no dock and no tabs — a user who opens or refreshes `./map` is trapped on a full-screen map whose only button is inert. `./venue/[id]` (also a root-stack screen, mod/1809.js:65-68) traps the same way, which is precisely the URL the map's "View venue" button and any shared venue link produce. Scenario: A user opens the map from home and refreshes the page (or follows a shared rush-x.xyz/map link). The stack now holds only `map`, so pressing the close button at mod/2450.js:31 dispatches GO_BACK against an empty stack and nothing happens. The fullScreenModal renders no tabs or dock, so there is no in-app route out of the screen. Identical trap on a shared ./venue/<id> link, which is exactly what the map's venue action produces.	Replace each `router.back()` in mod/2450.js:31, mod/2502.js:42, mod/1840.js:41, mod/1842.js:79 and mod/2374.js:85 with the `canGoBack` fallback helper already written in mod/9001.js:77, exported from module 9001 so all five screens share it.
F-CSEC-7 Medium	./venue/[id]	Data integrity and rating manipulation	Venue reviews accept a client-supplied user id with no proof of visit and no venue check There is no check that the venue exists, that the caller is the `user_id` they claim, that the user has ever had a confirmed booking or match at that venue, and no rate limit. `mockReserveCourt` shows the house style for such guards (mod/631.js:7339 `await on(e)`, mod/631.js:7350-7357 rate limit); the review path has none of it. Evidence: mod/631.js:2948-2964 (mockUpsertReview: only rating clamp + sanitizeText, no venue/visit/rate checks); mod/631.js:2494-2498 (reviews averaged into the public venue rating); mod/2502.js:126-133 (client sends venue_id/user_id)	Any caller can post or overwrite a review as any user id, for any venue id, and move that venue's displayed star rating and review count. A guest session (`user_id: "guest:male"`) can do it too, and mod/631.js:2781-2790 renders those rows with no author, which mod/2502.js:165 displays as the generic "Player". Venue ratings are the main trust signal in the booking funnel and they are unauthenticated, forgeable, and unbounded. Scenario: Call upsertReview({venue_id: "does-not-exist", user_id: <any id>, rating: 5}) — or simply review a venue you have never booked or attended — and the row is written unconditionally; for a real venue id it is folded into the displayed star rating and rating_count at mod/631.js:2494-2498, and for a bogus venue id it creates an orphan review row that no reader ever reclaims.	In `mockUpsertReview` (mod/631.js:2948): derive the author from the session rather than the payload, throw `E_VENUE_NOT_FOUND` when `fi().some(v => v.id === e.venue_id)` is false, and require a prior confirmed booking or attendance at that venue before accepting the row. Add a new error code to mod/674.js and both locales in module 909.
F-CQUAL-7 Low	./(auth)/sign-in.tsx (email tab, defect in mod/631.js)	Localization / error handling	mockSignIn throws a hardcoded English sentence instead of an error code, so Arabic users see untranslated text The most common authentication failure is the one message in the auth path that is not an error code, so it can never be mapped in 674 or translated in 909. This also violates the project's own stated convention ("New error codes: throw E_SOME_CODE, map it in 674, add seSomeCode to both locales in 909"). Evidence: mod/631.js:2341	Arabic-locale users — the primary audience for a Kuwait app whose sign-up screen is written in Arabic (mod/917.js:666) — get an English sentence on the single error they are most likely to hit. It is also invisible to the localization audit, since 909's en/ar key sets are otherwise perfectly in sync. Scenario: An Arabic-locale user on rush-x.xyz mistypes their password on the email tab of /(auth)/sign-in; mockSignIn throws the raw English sentence, storeErrorText returns it unchanged, and the RTL Arabic form renders "Invalid email or password." in English under the password field — the one auth error with no `se*` key in 909.	Throw `E_INVALID_EMAIL_OR_PASSWORD` at mod/631.js:2341, add it to the map in mod/674.js, and add `seInvalidEmailOrPassword` to both `en` and `ar` in 909.
F-CQUAL-8 Low	./(auth)/sign-in.tsx (forgot-password flow)	Form validation / recovery UX	Password-reset form accepts an 8-character password the backend will reject, and discards the structured strength reasons The form's own rule (8 characters) is weaker than the rule that actually applies (12 + four character classes), and the per-reason feedback the validator already produces is thrown away. Evidence: mod/613.js:316	A user in the middle of account recovery types an 8-character password, the button is enabled, submission fails with a single generic message, and nothing tells them the real requirement. Repeated guessing is the only path, and each retry burns one of the five attempts on the reset code (mod/631.js:943) — after which the code is deleted and recovery must start over. Scenario: A user resetting their password enters the correct 6-digit code and "Summer2026" (10 chars, no symbol): the button is enabled, mockResetPassword bumps n.attempts, passes the code check, then throws E_PASSWORD_DOES_NOT_MEET_THE_STRENGTH with no hint about the 12-char/symbol rule; after five such tries `n.attempts > 5` deletes the code entirely (mod/631.js:943) and the user must restart recovery.	Import `checkPasswordStrength` in mod/613.js, gate line 316 on `isStrongPassword(oe)`, and render the returned reason list as live requirement hints beneath the new-password input.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CTEST-2 Low	./(auth)/sign-up.tsx	Observability / release validation	Every sign-up funnel event is silently discarded because the wizard emits a vocabulary the funnel logger rejects The screen's event names were never reconciled with the whitelist the logger enforces, and the facade's blanket <code>`catch(() => {})`</code> plus the logger's silent <code>`return`</code> mean nothing surfaces the mismatch. Evidence: mod/631.js:15585	The signup funnel reports zero for every stage, permanently. <code>`signup_start`</code> and <code>`signup_done`</code> — the two counters the dashboard exists to show — are never incremented by anything, so the team has no signal that account creation is broken (see the mockVerifyOtp defect) and no way to measure drop-off between the phone, OTP, name, fork and clan steps. Scenario: A user completes the sign-up wizard: <code>pe('wizard:start') -> logFunnel -> mockLogFunnel</code> receives <code>`wizard:start`</code> , <code>FUNNEL_EVENTS.has()</code> is false at mod/631.js:15585 and it returns without writing; the same happens for all eight wizard events and for <code>`firstweek:tap1`</code> from mod/1677.js:338. <code>playora.mock.funnel.v1</code> therefore records zero <code>signup_start</code> and zero <code>signup_done</code> for every user, so the funnel readout stays permanently empty and gives no signal that account creation is broken.	Map the wizard's steps onto the existing vocabulary (emit <code>`signup_start`</code> at mod/917.js:34 and <code>`signup_done`</code> at mod/917.js:164), or extend <code>FUNNEL_EVENTS</code> at mod/631.js:15557 with the eight <code>`wizard:*`</code> names; and make <code>mockLogFunnel</code> warn on an unknown event instead of returning silently.
F-CQUAL-9 Low	./(auth)/sign-up.tsx	Error handling / localization	The sign-up OTP step prints raw error codes like "OTP_EXPIRED" to the user instead of translated text The OTP step passes the state variable straight to <code>FormError</code> instead of through <code>`authErrorText`</code> , unlike the four sibling steps and unlike the sign-in screen. Evidence: mod/917.js:522	A user who mistypes the code, lets it lapse, or exceeds five attempts sees the literal string <code>"OTP_WRONG"</code> , <code>"OTP_EXPIRED"</code> or <code>"OTP_ATTEMPTS"</code> on the most failure-prone screen in the product, in both English and Arabic. It reads as a crash, gives no instruction, and is the first thing many users see. Scenario: On the sign-up OTP step a user mistypes the 6-digit code and taps Continue: <code>mockCheckOtp</code> throws <code>OTP_WRONG</code> (mod/631.js:855), the catch at 917.js:546 clears the field and sets <code>Fe</code> to the raw <code>'OTP_WRONG'</code> , and <code>FormError</code> at 917.js:521-522 prints that literal string under the input — in Arabic locale too. Today this also fires for every existing phone user, since <code>mockVerifyOtp</code> at 917.js:541 always throws <code>OTP_WRONG</code> (Finding 1).	Change mod/917.js:522 to <code>`text: (0, w.authErrorText)(Fe, U)`</code> to match mod/917.js:371, and apply the same to the clan step at mod/917.js:783.
F-CSEC-8 Low	./(auth)/sign-up.tsx	Pre-consent PII retention	The sign-up wizard writes phone, full name and date of birth to device storage before terms are accepted, and never clears the draft when the flow is abandoned Identifying data — a Kuwaiti mobile number, a full legal name and a date of birth, which for a <code>MIN_AGE</code> of 13 (mod/631.js:857) frequently belongs to a minor — is written before the user has agreed to anything, and the only deletion path is completing registration. The module's 30-minute TTL (module 1617) is the sole bound, and it is refreshed on every save, so an idle open tab keeps extending it. Evidence: mod/917.js:62-66	A visitor who starts registration and changes their mind leaves their name, phone and date of birth on a shared or public browser with no action that removes them. Because the flow is the one place the app collects a date of birth, this is the highest-sensitivity data the consumer app touches, retained specifically for the users who declined to consent. Scenario: On a shared family tablet or a kiosk, person A starts sign-up, enters +965 phone, full name and date of birth, then taps the chevron back to sign-in and walks away; the draft stays in <code>localStorage</code> . Within 30 minutes person B opens <code>./(auth)/sign-up</code> and mod/917.js:39-41 prefills A's phone, legal name and birth date into the form — the birth date being data A supplied before ever reaching the <code>TERMS_REQUIRED</code> checkpoint at mod/631.js:903.	Call <code>`clearWizardDraft()`</code> when the user leaves the wizard — on the back-to-sign-in handler (mod/917.js:344), on <code>`at`</code> (mod/917.js:260) when returning to the phone step, and in <code>`signOut`</code> (mod/630.js:61) — and stop writing the birth date to the draft at mod/917.js:65 until after the terms checkpoint.
F-CQUAL-10 Low	./(tabs)/index	Stale state	The 14-day strip is frozen at mount, so after midnight the first chip is yesterday and always empty The strip is derived from <code>`new Date()`</code> once, at mount, on a tab screen that stays mounted for the life of the session (30 days per the session policy). It should be recomputed when the day rolls over — the render-tick interval at mod/1677.js:91 is already running and could carry the current date as its dependency. Evidence: mod/1677.js:108-111 (P=14 at mod/1677.js:1046), chips at mod/1677.js:616-645	Any user who leaves the app open across midnight — the normal case for an evening-sports app — sees a day strip whose first chip is yesterday. Selecting it always yields zero games, because everything upstream filters to <code>`starts_at >= now`</code> . Today's games are still reachable from the second chip, but the strip is silently off by one and the month picker's scroll-to-date lands on the wrong cell. Scenario: A user leaves the Home tab open from 23:00 to 00:30; the first day chip still shows yesterday's date and selecting it filters to a day where every game has <code>starts_at < now</code> , so the list is always empty while the group headers computed fresh at mod/1677.js:124-134 still label today correctly.	Key <code>`ze`</code> on the current day: keep a <code>`todayKey`</code> state updated by the existing interval at mod/1677.js:89-93 and add it to the <code>`useMemo`</code> dependency list at mod/1677.js:111.
F-CQUAL-11 Low	./(tabs)/index	Dead end	The first-week checklist's "join a game" row is a no-op when there is no suggestion The row's <code>`go`</code> is guarded by <code>`G.next_up &&`</code> with no fallback, so when there is no next-up game the tap logs a funnel event and does nothing. The other two goals navigate unconditionally (mod/1677.js:302 <code>`/discover`</code> , mod/1677.js:304 <code>`/(tabs)/games`</code>); this one should fall back to the same place. Evidence: mod/1677.js:293-298 and mod/1677.js:331-339, 366-371; null <code>next_up</code> at mod/631.js:16923-16928	The condition under which the row is both visible and not done — a brand-new account with no bookings — is exactly the condition under which <code>`next_up`</code> is most likely to be null. The onboarding checklist's first and most important step therefore presents an arrow, records a click in the funnel, and leaves the user on the same screen, which will also make the funnel data read as engagement rather than failure. Scenario: A brand-new account (created <7 days ago, no bookings) with no recommendable game sees the "Join your first game" row with a chevron; tapping it logs <code>firstweek:tap1</code> to the funnel and does nothing — the screen does not change and there is no other affordance on that row.	At mod/1677.js:297 change <code>`go`</code> to <code>`()` => (G.next_up ? De(G.next_up.game_id) : F.push("/(tabs)/index"))`</code> — or scroll to the game list / open <code>`/discover`</code> — so the row always leads somewhere.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CQUAL-12 Low	./(tabs)/index	Localization	Counts passed as strings skip Arabic numeral localization `String(n)` turns the value into a string before module 675 can decide whether to convert to Arabic-Indic digits, so these four strings render Western digits inside otherwise fully localized Arabic sentences. Passing the raw number, or `formatNumber(...)` as used elsewhere, fixes it. Evidence: module 675 in index.html (t() numeric-only param conversion) with mod/1677.js:736, mod/1677.js:927, mod/1842.js:102, mod/1842.js:348; formatNumber at mod/1311.js:22	In Arabic the home screen shows "5 مباريات" and "نتيجة ل 3" «...»" while every adjacent count on the same screen uses 0 and ر, and the booking screen's cancellation notice shows a Western hour figure. It is a visible inconsistency on the app's two busiest screens for the primary local language. Scenario: With locale ar, the Home search count renders "3 نتيجة ل" «...»" and a day header renders "5 مباريات" with Western digits, while the day chip immediately above shows 0 via formatNumber (mod/1677.js:639); the venue screen's cancellation notice shows the hour figure in Western digits the same way.	Drop the `String()` wrapper at mod/1677.js:736, mod/1677.js:927, mod/1842.js:102 and mod/1842.js:348, or pass `formatNumber(...)` from mod/1311.js so the region's numeral preference is honoured too.
F-CQUAL-13 Low	./blocked.tsx	Error handling / silent data loss	The blocked-accounts screen has no error handling: a failed unblock silently leaves the row in place Neither handler has a try/catch, no error state exists on the screen, and there is no retry control — unlike the shared `GateScreen`/`RetryButton` pair the codebase provides for exactly this (mod/9001.js:65-71, mod/9001.js:117). Evidence: mod/1837.js:19-21	When device storage is full or blocked, tapping "unblock" appears to do nothing: the row stays, no message appears, and the only trace is an unhandled promise rejection in the console. The user reasonably concludes the person is unblocked when they are not — a safety control that fails silently in the wrong direction. A read failure leaves the screen spinning forever with no way to recover but a reload. Scenario: A user whose localStorage quota is exhausted (long-lived demo store plus media blobs) opens /blocked and taps unblock: Qt.blocks is filtered in memory, Za rejects with E_STORAGE_FULL, `await L()` never runs, so the row stays on screen with no message and only an unhandled rejection in the console; the user taps again with no effect, and after a reload the person is blocked again even though the block was not enforced for the rest of that session.	Wrap both mod/1837.js:11-13 and mod/1837.js:19-21 in try/catch, set an error state, and render it with the existing `RetryButton` from mod/9001.js; treat a failed `fetchBlockedList` as an error state rather than leaving `R` at null.
F-CSEC-9 Low	./booking/search	Access control	Court booking is organizer-only in the backend but ungated in the UI until the final reserve call The app has a role-gate helper (mod/9001.js:39-58 `useRoleGate` / `GateScreen`) and the dock applies the organizer condition before navigating, but the two booking screens themselves apply nothing. A plain user or a guest reaching `/booking/search` by URL, deep link or browser history can browse venues, pick a court, a day, a duration and a slot, open the confirm sheet and only then be told they need an organizer application. Evidence: mod/1620.js:204	Non-organizers are walked through a five-step funnel that cannot succeed, and the failure arrives as a raw alert (mod/1842.js:58) after they have committed to a specific slot. Guests get the same treatment with no route to fix it, since they cannot apply to be an organizer. Scenario: An admin-role account taps "book a court" in the dock: mod/1620.js:36 sets O true for role "admin", so it routes to /booking/search rather than /organizer/apply. The admin picks a venue, court, day, duration and slot, confirms, and mod/631.js:7339 calls on(), which requires an *approved organizer application* the admin does not have — the flow dies in a raw alert (mod/1842.js:58). Same walk-through-then-deny for any plain user or guest who opens /booking/search from a bookmark or browser history.	Add `useRoleGate(["organizer", "admin"])` at the top of mod/1840.js and mod/1842.js and render `GateScreen` (mod/9001.js:72) with a "become an organizer" action when `allowed` is false, matching the dock's behaviour at mod/1620.js:204.
F-CQUAL-14 Low	./booking/search	Localization and labelling	Two filter controls on booking search carry the wrong label The price row's reset chip reuses the area key, and the sport row's heading reuses the sport row's own reset-chip key. The headings should be a sport label and the price reset should use an "any price" key, which does not yet exist in module 909. Evidence: mod/1840.js:64-67, mod/1840.js:85 (with state at mod/1840.js:11), mod/1840.js:95-102	The screen shows two controls labelled "Any area" — one of which silently resets the maximum price instead — and heads the sport filter with "Any sport" rather than "Sport". A user clearing their area filter clears their price ceiling instead and gets a different, more expensive result set with no indication why. Both English and Arabic are affected identically. Scenario: On /booking/search a user who has set a max price and an area taps the chip reading "Any area" in the price row; the area filter is untouched and the price ceiling is cleared instead, returning more expensive venues with no visible reason.	Add `sportLabel` and `anyPrice` to both `en` and `ar` in module 909, then use them at mod/1840.js:66 and mod/1840.js:85.
F-CQUAL-15 Low	./discover, ./map, ./booking/search, ./venue/[id]	Error handling	Four discovery screens have no error path on their initial load and show a permanent spinner when a fetch rejects None of these loaders has a try/catch, so a rejected promise leaves the loading flag set (or, on the map, clears it with an empty dataset) and produces an unhandled rejection. Each needs the `catch (e) { setError(storeErrorText(e?.message ?? ""))    t("error")) }` finally { setLoading(false) }` shape already used at mod/1677.js:41-47, plus a retry control. Evidence: mod/1840.js:16	Any backend hiccup — including the `!f.ok` throw at mod/673.js:69 — leaves /discover, /booking/search and /venue/[id] spinning indefinitely with no message and no retry, and leaves /map showing an empty Kuwait with no pins and no indication that anything failed. On a slow or flaky connection the app looks hung rather than broken, and the only recovery is a full page reload. Scenario: A returning user whose localStorage is near quota opens /booking/search. mockSearchBookableVenues awaits Dr(), whose first-run seed write hits Za (mod/631.js:1763-1775), which converts QuotaExceededError into a thrown E_STORAGE_FULL. mod/1840.js:16-26 has no catch, so J(!1) never runs: the screen renders ActivityIndicator (mod/1840.js:104) forever with no message, no retry, and an unhandled rejection in the console. The only recovery is a full page reload, which reproduces it.	Wrap each loader (mod/2374.js:32, mod/1840.js:16, mod/2502.js:17, mod/2450.js:13) in try/catch/finally, store the localized message via `storeErrorText`, and render `EmptyState` plus a retry button — the pattern already present at mod/1677.js:165-175 and mod/1842.js:252-264.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-CQUAL-16 Low	./how-it-works.tsx and ./blocked.tsx	Navigation dead end	How-it-works and blocked call router.back() unconditionally, so their only exit is dead when the URL is opened directly Both screens assume a navigation stack exists beneath them. On a fresh tab, a bookmark, a shared link or a reload there is no history entry to pop, and `back()` has nothing to do. Evidence: mod/2445.js:17	A signed-in user who opens rush-x.xyz/how-it-works or rush-x.xyz/blocked directly is stranded on a chromeless screen: the chevron does nothing and there is no tab bar, no home link and no other control. The only escape is editing the URL or using the browser's own back button, which on a fresh tab is also empty. Scenario: A signed-in user (session persisted for 30 days) pastes or bookmarks rush-x.xyz/how-it-works and opens it in a fresh tab: mod/1809.js:54 leaves the session alone, the screen renders with only a chevron, `l.back()` is a no-op because the navigation stack has one entry, and the browser's own back is empty too — the only escape is editing the URL.	Replace `l.back()` at mod/2445.js:17 and `l.back()` at mod/1837.js:30 with the guarded fallback already written in mod/9001.js:77, and apply the same treatment to every card-presented route declared in mod/1809.js:56-96.
F-CSEC-10 Low	mod/630.js (AuthProvider)	Session management / identity confusion	Signing in or up never clears the persisted guest session, so an expired real session silently demotes the user back to guest `playora.guest_session` outlives the guest phase. When the real 30-day session lapses, loadSession returns null and the stale guest record is restored instead, with no signal to the user that their identity changed. Evidence: mod/630.js:62	A user who browsed as a guest before registering is, 30 days later, silently signed in as the shared `guest:male` principal rather than sent to sign-in. Their own data disappears from the UI, actions they take are attributed to the guest identity, and nothing prompts them to re-authenticate because mod/1809.js:54 treats the guest session as authenticated. Scenario: A visitor browses as a guest, taps the guest banner and completes sign-up; `playora.guest_session` is never removed. Thirty days later the real session's expires_at lapses, loadSession returns null at mod/631.js:2263-2265, and mod/630.js:22 restores the stale guest record. The user reopens the app as guest:male instead of being sent to sign-in: their profile, teams and bookings are gone, and any post or join they attempt is attributed to the guest principal.	Call `clearGuestSession()` in `signIn`, `signUp` and `adoptSession` (mod/630.js:49-71) whenever the adopted session's user id does not start with `guest`.
F-CSEC-11 Low	mod/631.js (mock backend, phone auth)	Auditability	Every phone-auth audit record is written with a null actor, so OTP sign-ins cannot be attributed to an account The two events that cover the entire phone-authentication path — the primary sign-in route — record `actorRef: null`, so the audit log cannot link an OTP session to the account it created or resumed. Evidence: mod/631.js:889	An investigation into a suspicious phone sign-in has nothing to pivot on: the audit trail shows that some OTP was requested and some OTP was verified, with no indication of which number or which account. Given that OTP sessions also never expire and use a predictable token, this is the path where attribution matters most. Scenario: An existing phone account signs in through mod/631.js:ya (mockVerifyOtp): the `if (!)` branch at 891 returns a session without ever logging auth.sign_in, so the only records are auth.otp_requested(null) and auth.otp_verified(null) — an OTP session is created with zero attributable entries in playora.audit.v1, while an email sign-in of the same account records actorRef(n.id).	Pass `(0, w.actorRef)(t)` at mod/631.js:833 (the canonical phone is already computed) and move the `auth.otp_verified` call at mod/631.js:889 below the account lookup on line 891 so it can log `(0, w.actorRef)(l.id)` for an existing account, or the new account's id on the creation path.

5. Candidates not yet verified

These have not been through the adversarial pass. The auditors produced them, but their verification agents did not run, so nothing has checked the citation against the code or the severity against the consequence. Severity below is as the auditor claimed it. On the portion of the register that was verified, roughly two candidates in three were refuted, so expect a similar rate here. Treat this as a work queue, not a defect list.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CTEST-1 Critical	./(auth)/sign-in.tsx, ./(auth)/sign-up.tsx (mod/631.js)	Release configuration / auth availability	Setting demo:false — the one documented production step — makes every phone OTP request throw SMS_NOT_CONFIGURED, and no configuration path exists index.html line ~10 instructs the operator to ‘Set to false for a real deployment’, and says ‘nothing else in the file needs to change’. But the SMS provider is read from ‘process.env.SMS_PROVIDER_URL’, which is frozen at Metro build time and is empty in a static single-file browser build — there is no field for it in ‘window.__PLAYORA_CONFIG__’ and no server to inject it. With demo on, ‘a === sa’ so ‘ca()’ is never reached and no SMS is ever sent; with demo off, ‘ca()’ always throws. There is no configuration in which a real code is delivered. Evidence: mod/631.js:769-774 — ‘sa = "123456", la = () => (Fa() ? sa : randomSixDigit()), ca = async (e,t) => { if (!smsConfigured() && !Fa()) throw new Error("SMS_NOT_CONFIGURED"); await smsProvider().send(...) }’ and mod/631.js:836-843 inside mockRequestOtp: ‘try { await ca(t,a) } catch (e) { throw (await _a.delete(t), new Error("SMS_NOT_CONFIGURED" === e.message ? "SMS_NOT_CONFIGURED" : "SMS_SEND_FAILED")) }’. ‘smsConfigured’ (module 642, inlined in index.html) is ‘() => !(typeof process !== ‘undefined’ ? process.env?.[‘SMS_PROVIDER_URL’] : undefined)’. Verified at runtime in the shipped bundle: ‘process.env’ is ‘{"NODE_ENV":"production"}’, ‘smsConfigured()’ === false, ‘smsProvider().name’ === ‘log’. Driving the app with ‘__PLAYORA_CONFIG__demo = false’ and calling ‘mockRequestOtp(“+96550001234”)’ returns ‘ERR:SMS_NOT_CONFIGURED’.	The moment the operator performs the single documented go-live step, phone sign-up and phone sign-in — the primary onboarding route for a Kuwait consumer app — stop working for 100% of users, with the raw string SMS_NOT_CONFIGURED on screen. No test covers this: no suite opens /(auth)/sign-in or /(auth)/sign-up, and flows-xcut.mjs only asserts the demo banner's presence, never that auth still works with demo off.	Move the SMS settings into ‘window.__PLAYORA_CONFIG__’ (‘{url, token, senderId}’) and have module 642’s ‘smsConfigured’/‘smsProvider’ read them from there instead of ‘process.env’; add the origin to the CSP connect-src. Add a regression test that boots with demo:false and asserts that ‘/(auth)/sign-in’ can reach the server code.
P-CSEC-1 Critical	./(tabs)/feed.tsx	Content moderation	Post reporting is a no-op: reports are written to a table nothing in the app ever reads Reporting a post should place it in a queue a human or a rule can act on, and at minimum should hide the post from the reporter. Instead the report is appended to storage, a success toast is shown, the post stays in the feed unchanged, and nothing anywhere can ever surface the report. Evidence: mod/631.js:14575 — ‘r.mockReportPost = async (e, t, a) => { (await ei(), Qt.postReports.push({ id: ea(), post_id: t, reporter_id: e, reason: a.trim().slice(0, 500), created_at: ... })), await Za(xe, Qt.postReports), await logAudit("feed.reported", ...)}’. ‘Qt.postReports’ appears exactly four times in the shipped bundle — ‘postReports: []’ (631.js:706), the hydration assignment ‘Qt.postReports = pa’ (631.js:2073), the push at 14577 and the persist at 14584. There is no reader: no facade export in mod/671.js, no admin screen, and no filter in ‘mockGetFeed’ (631.js:14476-14486) that consults it. The screen fires it on a single unconfirmed tap — mod/1622.js:291 ‘onPress: async () => { (await reportPost(t, x.id, "inappropriate"), c(o("reportedToast")))}’ — with a hard-coded reason, no confirmation, no undo, and no dedup or existence check in the backend.	Every user who reports harassment, nudity or abuse is told ‘reported’ and nothing happens — the content stays in their feed and in everyone else’s, forever. The app has no functioning moderation path for user-generated posts at all, which is both a safety failure for a consumer social product and a compliance problem for app-store content policies. Repeated taps silently duplicate rows, and a report can be filed against a post id that does not exist.	Make ‘mockReportPost’ in 631 resolve the post first (reuse ‘cl’ at 631.js:14537), reject a second report from the same reporter for the same post, and record the report where it can be acted on; add a reader for ‘fetchPostReports’ in 671 plus a moderation surface. At minimum ‘mockGetFeed’’s filter ‘i’ (631.js:14480) drop posts the viewer has reported, and change the flag button in 1622 to a confirm sheet that also offers to mute/block of the author.
P-CSEC-2 Critical	./awards/[gameld].tsx	Voting integrity	Live award vote tallies are readable by anyone while the ballot is open, and the organizer can close voting at will The awards UI treats the running count as secret until publication, but the backend hands per-nominee counts to any caller for a match in the ‘voting’ state, and lets the organizer (‘El’ = organizer or admin) end voting at any instant. It should refuse per-nominee counts until the ballot is published and refuse to publish before ‘closes_at’ (or require an explicit forfeit of the remaining window). Evidence: mod/631.js:15100 ‘r.mockGetMvpPanel = async (e, t) => { await ei(); if (!hi(t)) throw new Error("E_MATCH_NOT_FOUND"); ... }’ — no participation, role or audience check follows. mod/631.js:15124 returns ‘participants: i.map(e) => ({ user_id: e, name: Td(e), votes: o.get(e) ?? 0 }))’ for state ‘voting’ (mod/631.js:15110-15116). Contrast mod/631.js:15084, where the ballot deliberately exposes only an aggregate ‘total_votes: Qt.awardVotes.filter((a) => a.match_id === t && a.award_key === e).length’, and mod/1835.js:166-170 which hides even that from a voter (‘N.can_vote ? H(“pickNominee”) : ...’). Publishing is unrestricted in time: mod/631.js:15171 ‘r.mockPublishAwards = async (e, t) => { ... if (!El(e, a)) throw new Error("forbidden"); const i = await D(a); if (!i) throw new Error("not_completed"); await R(a, i); }’ — no check against ‘closes_at’ — and mod/1835.js:243 renders the ‘publishResults’ button whenever ‘N.is_manager && “voting” === N.state’.	An organizer — who is also a nominee, since ‘Sl’ at mod/631.js:14937 adds ‘t.organizer_id’ to the participant set — can poll the live MVP tally and hit Publish the moment their preferred player is ahead, permanently fixing the result before the other participants vote. Any other user can scrape the running tally of a match they are not in. Awards feed the public leaderboard and the player passport, so this corrupts the app's whole reputation surface.	In ‘mockGetMvpPanel’ (mod/631.js:15100) zero out or omit ‘participants[].votes’ and ‘total_votes’ unless ‘d === “published”’ callers not in ‘Sl(t)’. In ‘mockPublishAwards’ (mod/631.js:15171) ‘Ol(i)’ is still true, or record an explicit early-close with an audit error, and notify participants that voting was cut short.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-3 Critical	/awards/leaderboard.tsx	Data correctness / fairness	The awards leaderboard silently drops every non-male winner for every viewer <code>`mockGetAwardLeaderboard`</code> partitions results by the viewer's audience, but the only screen that calls it never supplies one, so the partition silently resolves to <code>`male`</code> for every viewer. Female-audience players (seeded at mod/631.js:5110, 9274, 9328, 9346) are filtered out of the leaderboard unconditionally. Evidence: mod/1836.js:16-18 <code>`const N = (0, l.useCallback)(async () => { M(await (0, S.fetchAwardLeaderboard)({ sport: H ?? void 0, season: R ?? void 0 })); }, [H, R]);`</code> — no <code>`viewerId`</code>. mod/671.js:700 <code>`e.fetchAwardLeaderboard = (o) => t.store.mockGetAwardLeaderboard(o);`</code> passes the object straight through. mod/631.js:15286-15289 <code>`const a = aa(e.viewerId); return [...t.entries()].map(...).filter((e => ia(aa(e.user_id), a)))`</code>. mod/631.js:750-757 <code>`const aa = (e) => e ? ... : "male"`</code> — an absent viewer id defaults to <code>"male"</code>. mod/631.js:758 <code>`ia = (e, t) => e === t    "open" === e    "open" === t`</code>.	A woman who wins match awards never appears on the leaderboard, and a female viewer sees a board she can never place on, with no indication anything was filtered. The season chips make it worse: <code>`mockGetAwardSeasons`</code> (mod/631.js:15293) is computed over the unfiltered <code>`Qt.awardWins`</code>, so a season chip can exist whose leaderboard renders the <code>`noAwardsYet`</code> empty state. For a gender-partitioned Kuwaiti sports product this is a visible fairness and compliance failure, not just a bug.	Pass the signed-in user id from mod/1836.js:17 (<code>`fetchAwardLeaderboard`</code>), widen <code>`fetchAwardLeaderboard`</code> mod/671.js:700 to forward it, and make <code>`mockGetAwardLeaderboard`</code> a missing <code>`viewerId`</code> rather than defaulting to <code>"male"</code> in <code>`aa`</code>. Append the partition to <code>`mockGetAwardSeasons`</code>.
P-CSEC-4 Critical	/booking/[id].tsx	Payment integrity / refunds	When the organizer cancels a court booking late, the players who already paid lose their money and are told they forfeited it <code>`(tr && t)   `</code> short-circuits the entire refund branch whenever the cancellation is late and the payment was paid, so a paid share is left at status <code>`paid`</code> with no refund posting. The cutoff is a penalty on whoever cancels; here it is applied to the payers, who did not cancel anything. Late organizer cancellation should still refund the participants and charge the penalty to the organizer or the venue. Evidence: mod/631.js:7465 <code>`r = (0, H.freeCancellationAllowed)(i.starts_at, n?.cancellation_cutoff_hours ?? 6);`</code> then mod/631.js:7472 <code>`((tr && t)    ((e.status = t ? "refunded" : "expired"), t && ((e.refunded_at = ...), await ac(e.payer_id, e.amount_kwd, e.id))), t && (await bi({ ... type: r ? "refund_issued" : "payment_forfeited", ... })))`</code>. Reached from mod/1838.js:469 <code>`onPress: () => Y(() => (0, _cancelCourtBooking)(c.id, E.id))`</code>.	An organizer cancels a court 2 hours before kickoff. Every player who already paid their split keeps a <code>`paid`</code> payment, receives zero back, and is sent a <code>`payment_forfeited`</code> notification telling them they lost the money through their own inaction. Nothing is written to <code>`seatCancellations`</code>, so <code>/refunds</code> shows no record either.	In <code>`mockCancelCourtBooking`</code> / <code>`Mr`</code> (mod/631.js:7469-7486), <code>`refund`</code> payment unconditionally when the organizer or an admin is canceller — drop the <code>`tr && t`</code> guard, always run <code>`ac(e.payer_id, e.amount_kwd, e.id)`</code> for paid rows, and reserve <code>`payment_forfeited`</code> case where the payer themselves cancelled late.
P-CSEC-5 Critical	/booking/[id].tsx	Payment integrity / revenue loss	The charge-idempotency key is per (game, payer, booking), not per payment, so a re-issued bill settles for free Every payment request for the same (game, payer, court booking) collapses onto one charge key, and the captured charge's <code>`amount_kwd`</code> is never compared against the new payment's <code>`amount_kwd`</code>. Re-running the payment plan — a normal action whenever a player joins or leaves and the equal split changes — leaves the already-paid row in place and adds a new pending row for the same payer. Paying the new row hits the captured key and short-circuits: status becomes <code>"paid"</code>, <code>`gateway_ref`</code> is copied from the old charge, and zero is captured. Evidence: mod/631.js:7655 — <code>`Ur = (e) => `seat:\${e.game_id} ?? "court":\${e.payer_id}:\${e.booking_id} ?? e.id`;`</code> (the payment's own <code>`id`</code> is only used when there is no <code>`booking_id`</code>). mod/631.js:7831-7833 — <code>`const n = Ur(i), r = Qt.paymentCharges.find((e) => e.key === n); if (r && "captured" === r.status) i.gateway_ref = r.gateway_ref;`</code> — when a captured charge exists under that key, <code>`Yr`</code> is never called and the payment is marked paid at 7860 with no money moved. mod/631.js:7583 — <code>`mockCreatePaymentPlan`</code> clears only pending rows: <code>`(Qt.payments = Qt.payments.filter((e) => !(e.booking_id === n.id && "pending" === e.status)))`</code>, then pushes a fresh row per payer at 7586-7601 with the same <code>`booking_id: n.id`</code> and <code>`game_id: n.game_id`</code>. mod/1838.js:269 — the split buttons are always live: <code>`onPress: () => Y(() => (0, _createPaymentPlan)(c.id, E.id, e))`</code>.	Two defects in one flow. The player is double-billed (an old paid row plus a new pending row for the same booking), and if they pay the second bill the platform and venue book revenue that was never collected — <code>`mockGetVenueRevenue`</code> and <code>`mockGetAdminFinancials`</code> count it as real. Any player who has paid once for a booking can settle every later bill on that booking for free simply by getting the organizer to re-press a split button.	Key the charge on the payment row itself — make <code>`Ur`</code> return <code>`charge`</code> — and in <code>`\$r`</code> (mod/631.js:7831) additionally require <code>`r.amount_kwd`</code> before reusing a captured charge. In <code>`mockCreatePaymentPlan`</code>, subtract amounts already paid for the charge from each payer's new share instead of issuing a fresh full-price row.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-6 Critical	/chat/[gameId].tsx	Broken access control	Match chat performs no authorization: any signed-in user can read and post in any game's chat, including both private team channels Reading and writing match chat should require that the caller is a confirmed participant of that game, and the team_a/team_b channels should additionally require membership of that team; the author identity should be derived from the session, not accepted from the request. As written the backend accepts any game id, any channel, and any claimed user id and display name. Evidence: mod/631.js:2855 — `r.mockGetChatMessages = async (e, t, a) => (await ei(), Qt.chatMessages.filter((a) => a.game_id === e && a.channel === t)...)` — filters on game id and channel only; no participant check. mod/631.js:2862 — `const Gi = async (e) => { await ei(); const t = { id: ea(), game_id: e.game_id, channel: e.channel, author_id: `self-\${e.user_id}`, author_name: (0, v.sanitizeName)(e.user_name), ... body: (0, v.sanitizeText)(e.body, 1e3) ... }; return (Qt.chatMessages.push(t), ...) ` — no check that the game exists, that the caller joined it, or that `e.user_id / e.user_name` belong to the caller. mod/1843.js:31 sends those fields straight from the client: `sendChatMessage({ game_id: e, channel: E, user_id: s.id, user_name: !?.full_name    "You", body: n })`. Module 1845 (ChannelTabs, inlined in index.html) renders `const f= [general,team_a,team_b]` unconditionally, so the team channels are one tap away for every viewer. The organizer-scoped assert `zi` exists at mod/631.js:2966 but is never applied to chat.	Anyone who knows or guesses a game id (they are in the URL of every shared match link) can read the full chat history of a match they are not in — including the two private team channels — and post messages into it under an arbitrary display name. Because the identity fields are client-supplied, a poster can impersonate the organizer or another player and, for example, tell a team the match is cancelled. This is not a browser-only weakness: the same code moved behind an RPC server would still trust game_id, channel, user_id and user_name from the request body.	In 631, add a participant assert to both `mockGetChatMessages` and `mockSendMessage`: resolve the game, require an active booking/gamePlayers row for the caller (the pattern already used in 631.js:2970), and require the caller's team to match when channel is team_a/team_b. Derive `author_id` and `author_name` inside the caller id rather than from the payload, and drop `user_id`/`user_name` from the `sendChatMessage` argument in 671/1843. Gate the team channel access to module 1845 on the viewer's team.
P-CSEC-7 Critical	/group/[gameId]	Payment integrity	Seats a group leader pays for are never refunded — one refund per payer per match `Br(game, "guest:<group>:<n>", true)` finds no payment (the payment's payer_id is the leader) and returns `{paid_kwd: 0, refund_kwd: 0}`, and `Br(game, leaderId, true)` refunds only the first matching payment even though the leader may hold several paid seat rows for the same match. Refunds must be keyed on the payment rows for the booking, not on a single find() by payer. Evidence: mod/631.js:13704-13707 gives a guest seat a synthetic booking owner: `user_id: e.user_id ?? `guest:\${m.id}:\${a}``, while the payment row for that same seat is written under the leader: mod/631.js:13807 `payer_id: i.user_id ?? e` (guests have `user_id: null`). Refund lookup is by booking owner and returns at most one row: mod/631.js:7650-7654 `const Gr = (e, t) => Qt.payments.find(a => "seat" === a.kind && a.game_id === e && a.payer_id === t && ("pending" === a.status    "paid" === a.status))`, used by `Br` (mod/631.js:7703-7734). Cancellation walks bookings and refunds by the booking's user id: mod/631.js:3766-3775 `... o.push(t.user_id) ... for (const e of o) await Br(n, e, iO)`.	In split_equal / individual / custom mode any guest seat the leader paid for is money the platform keeps when the organizer cancels the match or the leader leaves; in every mode a leader who paid for more than one seat gets at most one seat refunded. The loss is silent — no error, no ledger entry — and the leader's wallet simply comes up short after a cancelled match.	Refund per payment, not per user: in `Br` (mod/631.js:7703) accept the booking id and resolve payments via `booking_id`, or iterate over all payments whose `booking_id` is in the cancelled set in `cancelGame` (mod/631.js:3766) and `qi` (mod/631.js:3132), crediting `ac(payment.payer_id, ...)`.
P-CSEC-8 Critical	/group/[gameId]	Authorization / consent	Group booking creates seats for other people with no friendship or consent check Anyone can put any other user into a paid match. The guest/sanction gates that `ji` applies to a self-join (mod/631.js:2996-2997 `md(t), await wd(t)`) are applied only to the leader here (mod/631.js:13657-13658), so a banned or suspended player can be seated by a third party. Evidence: mod/631.js:13668-13671 `for (const t of r) { if (!Qt.profiles.some((e) => e.id === t)) throw new Error("not_friends"); oa(e, t); }` — the error is named `not_friends` but the test is only `a` profile with this id exists. There is no friendship lookup (`Hd`/`Qd` are never consulted), no invitation and no acceptance step: the members are written straight to `status: "pending_payment"` with real bookings at mod/631.js:13704-13728. The screen lets you add arbitrary strangers found by search, not just friends: mod/2444.js:417-420 `const t = await searchUsers(M.id, e); J(t.map(...))` feeding `he(e.id, e.name)` at mod/2444.js:429.	A stranger's account acquires a reserved seat, an amount owed (in split_equal/individual mode), and a block on joining any other group for that match (`already_booked`, mod/631.js:13677) — all without their knowledge. It also gives a harasser a way to attach a named victim to any match, and lets suspended users back into matches through a friend.	In `mockCreateGroupBooking` (mod/631.js:13668) require an active booking and a friendship (`Qd(e).has(t)`) or model invited members as `status: "pending"` with no booking until the invitee accepts, and run `md`/`wd` for everyone else, not only the leader.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-9 Critical	/my-bookings.tsx	Payment integrity / consumer refunds	Cancelling from My Bookings takes no refund action and leaves no cancellation record Two buttons express the same user intent — give up my spot — but only one settles the money. `mockCancelBooking` should run the same refund-eligibility and seat-settlement path as `mockLeaveMatch`, or the My Bookings button should call `leaveMatch`. Evidence: mod/2457.js:268 `(await (0, T.cancelBooking))(\$,id, e.id), G(null), O())` routes to mod/631.js:2753 `r.mockCancelBooking = async (e, t) => {...}` whose whole body (2753–2780) only flips `t.status = "cancelled"` and persists bookings — it never calls `Br` (the seat settle/refund helper), never calls `ac` (the wallet refund poster), and never pushes to `Qt.seatCancellations`. The other cancel affordance, `leaveMatch` on the game screen, goes to mod/631.js:3129 `s = Ir    (0, F.isRefundEligible)(r.starts_at, o), d = r ? await Br(r, t, s) : ...` and then mod/631.js:3144 `Qt.seatCancellations.push({... paid_kwd: d.paid_kwd, refunded_kwd: d.refund_kwd, reason: l })`.	A player who paid 5.000 KWD and cancels from My Bookings well before the refund cutoff gets nothing back. The seat payment stays `paid`, so mod/631.js:2736 still returns it and the row renders "Cancelled" beside "✓ Paid 5.000 KWD". Because no `seatCancellations` row is written, /refunds (which reads only that table, mod/631.js:3180) shows no trace of it at all — the user has no receipt, no explanation, and no path to dispute. The same player leaving via the game screen would have been refunded in full.	Make `mockCancelBooking` (mod/631.js:2753) delegate to the `mockLeaveMatch` settlement path — compute `isRefundEligible` `Br(game, user, eligible)`, and push the `seatCancellations` row – mod/2457.js:268 to call `leaveMatch(\$,game.id, e.id)`.
P-CARCH-1 Critical	/notifications.tsx	State consistency	Opening the notifications screen marks everything read in the backend but never updates the list Merely mounting the screen destroys the unread state for every notification the user owns, including ones below the fold that were never rendered. The screen then keeps rendering the pre-write snapshot, so the blue dots and the "N new" chip contradict the persisted store until the next remount. Evidence: mod/2458.js:24-30 `const V = (0, n.useRef)(1); (0, n.useEffect) (() => { 1    V.current    It    (\$.some((t) => !t.read) && (V.current = !0), (0, T.markAllNotificationsRead)(t.id).catch(() => {})); }, [l, \$, t]);` — the result is never folded back into `\$`. mod/631.js:2940 `r.mockMarkAllNotificationsRead = async (e) => { ... Qt.notifications.map((a) => a.user_id !== e    a.read ? a : ((t = !0), Object.assign({}, { read: !0 })) ... )`. The stale UI is then read at mod/2458.js:81 `M = \$.filter((t) => !t.read).length` and mod/2458.js:188 `!t.read && (0, q.jsx)(u.default, { style: [S.dot, ...] })`.	A user who opens notifications, glances at the top row and navigates away has silently acknowledged everything, including a `sanction_issued`, a `payment_request` or a `booking_decision` they never saw — and there is no way back: no unread toggle, no filter, and `fetchUnreadCount` (mod/671.js:407) is never called by any screen, so no badge survives anywhere. Within the session the chip still says items are new, so the user cannot tell what state they are in.	Remove the blanket mark-all effect in mod/2458.js:24-30, or mak (a "mark all read" control) and, when it succeeds, set `z((l) => l.m ...x, read: !0 ))))` so the rendered list matches what was written.
P-CSEC-10 Critical	/passport/[id].tsx	Data exposure / audience partition	mockGetPassport performs no audience, block, or profile-visibility check, exposing any user's future match schedule to any caller Passport reads should be subject to the same three gates as player-profile reads: the audience partition (`oa`), the block list, and profile_visibility. None is applied. `dr` is explicitly viewer-aware (it takes a viewer arg, grants admins an override at 6615, and audit-logs cross-user access at 6646), so cross-user viewing is an intended feature that simply shipped without authorization. Evidence: mod/631.js:6610-6616: `dr = async (e, t) => { (await ei(), await tn(), await ir(e)); const a = ar(e), i = t === e, n = t ? Qt.profiles.find((e) => e.id === t)?role : void 0, r = i    "admin" === n, o = rr(e),` — there is no `oa(t, e)` audience assert, no `el(e, t)` block check, and no `al(e).profile_visibility` check anywhere in the function body (6610-6679). Compare mod/631.js:14271-14282, where mockGetPlayerProfile does all three: `i    oa(e, t); const n = li && el(t, e), ... s = al(t).profile_visibility`. The only gate is the subject's own passport toggles, mod/631.js:6667 `history: c ? sr(e) : [],` with `c = r    o.match_history` defaulting to true (mod/631.js:6546 `nr = (e) => ({ user_id: e, match_history: !0, achievements: !0, hosted: !0, sports: !0 })`). `sr` emits future games: mod/631.js:6593-6605 pushes `{ date: n.starts_at, venue_name: a(n.venue_id), organizer_name: ..., status: "upcoming" }` for every scheduled game the subject is confirmed for. The route is registered and takes an arbitrary id (mod/1809.js:137-140, mod/2473.js:6 `const { id: t } = useLocalSearchParams()`).	Navigating to /passport/<any-user-id> returns that user's match history including upcoming games with venue name, exact start time and sport, plus attendance class, reputation class, streaks and follower counts. A male user can read a female user's schedule across the audience partition the app otherwise enforces; a user who has been blocked can still read the blocker's schedule. In a small market like Kuwait this is a stalking vector, and user ids are readily available from /follows/[id], /player/[id] and discover.	In `dr` (mod/631.js:6610), after resolving the viewer, call `oa(t, e)` present and not self, return the limited shape when `el(e, t)` or wt `al(e).profile_visibility` is `private`/unmet-`followers` — reusing the predicate `mockGetPlayerProfile` builds at mod/631.js:14280-14282. Two reads cannot drift again.
P-CSEC-11 Critical	/pay/[id].tsx	Data exposure / IDOR	Any payment record can be read by any caller — the viewer id passed to mockGetPayment is silently dropped The read path accepts an authorization argument and then ignores it, so ownership is never checked. It should reject when the caller is neither the payer, the payee venue, the booking organizer, nor an admin — exactly the check mockGetBooking already performs. Evidence: mod/631.js:7645 — `r.mockGetPayment = async (e, t) => { await ei(); const a = Qt.payments.find((t) => t.id === e); return a ? Lr(a) : null; };` The second parameter `t` (the viewer) is declared and never referenced. The facade forwards it (mod/671.js:538 `e.fetchPayment = (o, c) => t.store.mockGetPayment(o, c);`) and the screen supplies it (mod/2474.js:20 `A(await (0, S.fetchPayment)(e, c?.id))`). Contrast the sibling reader two functions above, mod/631.js:7560 `if (t && a.organizer_id !== t && !ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");`, and the writer mod/631.js:7821 `if (i.payer_id !== e) throw new Error("E_THIS_PAYMENT_REQUEST_IS_NOT_YOURS");`.	Anyone holding or guessing a payment id (they are handed out in /refunds deep links `pay/\${id}`, in payment_request notifications, and to organizers in bulk via mockGetBookingPayments) can load /pay/<id> and read another player's full payment record: payer name, amount owed, venue, game, method and paid_at. Moving this file to a server changes nothing — the check simply is not there.	In `mockGetPayment` (mod/631.js:7645) apply the viewer argum the `find`, throw `E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW` === a.payer_id`, `vr(t, a.payee_venue_id)`, the caller is the booki organizer (`gr(a.booking_id)?.organizer_id === t`), or `ro(t)`.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-12 Critical	./pay/[id].tsx	Payment integrity / unauthorised charge	Paying with "Wallet" silently charges the user's KNET card for any shortfall When the user picks Wallet and the wallet+credits balance does not cover the amount, the code should stop and tell them to top up or pick a card — the <code>`E_INSUFFICIENT_WALLET_BALANCE_ADD_FUNDS_OR`</code> branch exists for exactly that. Instead <code>`Yr`</code> supplies "knet" on the user's behalf, so <code>`tc`</code> runs a real gateway capture for the shortfall with no card selection, no confirmation and no disclosure. A user with 0.500 KWD in the wallet who taps Pay on a 3.000 KWD seat has 2.500 KWD taken from KNET. Evidence: <code>mod/631.js:7751-7755</code> — <code>`const Yr = async (e, t, a, i, n) => { if ("wallet" === n) { const n = await tc(e, (0, A.toFils)(t), a, i, "knet"); return `wallet_\${await (0, c.randomToken)(8)}\${n.external_fils > 0 ? "+knet" : ""};` } }</code>. The fifth argument to <code>`tc`</code> is the fallback gateway and it is hardcoded to "knet". <code>mod/631.js:16248-16252</code> — <code>`if (r.external_fils > 0) { if (ln "wallet" === n "cash" === n) throw new Error("E_INSUFFICIENT_WALLET_BALANCE_ADD_FUNDS_OR"); const e = `gw_\${n}_\${await (0, c.randomToken)(8)}`; if (!(await Vr(e, r.external_fils / 1e3, n))) throw ... }`</code> — so with <code>n="knet"</code> the guard never fires and a card capture is executed. The payer screen offers "wallet" as a plain method chip (<code>mod/2474.js</code>, <code>`K = (0,S.paymentMethods)`</code>) renders every entry of <code>`PAYMENT_METHODS`</code>, which includes <code>`{ method: "wallet" }`</code> and shows no balance, then alerts <code>`paymentSuccess`</code> on return.	Players are charged on a payment instrument they did not choose or consent to in that transaction. In Kuwait this is an unauthorised card debit: the only trace is a <code>`+knet`</code> suffix buried in <code>`gateway_ref`</code> , the screen says "Payment successful" and the receipt shows the wallet method, so the player cannot tell what was charged and disputes/chargebacks are unanswerable.	In <code>`Yr`</code> (<code>mod/631.js:7751</code>), do not substitute a gateway. Pass the selection through to <code>`tc`</code> so the <code>`E_INSUFFICIENT_WALLET_BALANCE_ADD_FUNDS_OR`</code> guard fires and have <code>./pay/[id].tsx</code> fetch the wallet balance, disable the <code>`Wallet`</code> button, and require an explicit second method selection before <code>wallet+card</code> capture.
P-CARCH-2 Critical	./pay/[id].tsx	Payment integrity	The live payment path always captures; the only gateway path with a real signature check is unreachable from the product The screen tells the user the payment was server-verified while the only code that runs is a stub returning <code>`amount > 0`</code>. The <code>intent/action_url/webhook</code> flow — the one that would redirect to a real PSP and verify a signed callback — has no facade export and no caller, so it is dead code that will silently rot. Evidence: <code>mod/631.js:7907</code> — <code>`Vr = async (e, t, a) => u.sandboxProvider.capture(e, Math.round(1e3 * t));`</code> where module 641's <code>`sandboxProvider.capture`</code> is <code>`async (e,t) => t > 0`</code>, i.e. true for any non-zero amount. <code>mod/671.js:539</code> exports only <code>`e.payRequest = (o, c, s) => t.store.mockPayRequest(o, c, s);`</code> — <code>`mockCreatePaymentIntent`</code> (<code>mod/631.js:7908</code>) and <code>`mockGatewayWebhook`</code> (<code>mod/631.js:7934</code>) are defined in 631 and exported from it, but grepping every module in the consumer dump for <code>`CreatePaymentIntent GatewayWebhook`</code> matches only <code>mod/631.js</code> itself. <code>mod/2474.js:170-186</code> — the Pay button calls <code>`await S.payRequest(c.id, R.id, W)`</code> then immediately <code>`n.default.alert(H("paymentSuccess"), H("paymentVerifiedHint"))`</code>, and <code>mod/2474.js:167</code> renders a lock icon next to <code>`H("serverVerifiedHint")`</code>.	Every KWD charged through the app is fictional. A seat is marked paid and confirmed, the organiser sees a confirmed booking, revenue and wallet ledgers move, and no money has changed hands. Because nothing in the product calls the <code>intent/webhook</code> path, wiring a real gateway is not a config change — it is new plumbing on code no test has ever executed.	Either export <code>`createPaymentIntent`/`gatewayWebhook`</code> from 671 <code>./pay/[id].tsx</code> through the <code>intent + redirect + webhook</code> flow, or delete the <code>gatewayWebhook</code> path from <code>mod/631.js</code> so it is not mistaken for a working one. Until a real provider exists, replace <code>`paymentVerifiedHint`/`serverVerifiedHint`</code> with copy that does not claim server verification.
P-CSEC-13 Critical	./privacy-controls.tsx	Privacy controls / consent	Two of the three controls on /privacy-controls are write-only: 631 never reads privacy_visibility or media_consent The screen presents three privacy choices — audience visibility (Everyone / Same audience / Connections), avatar mode, and media consent — with a live "How others see me" preview that renders the name as hidden for <code>`connections`</code> (<code>mod/2477.js:98</code>). Only <code>avatar_mode</code> has any effect. <code>`profiles.privacy_visibility`</code> is a second, parallel visibility model that no read path consults; the model that IS enforced is <code>`privacySettings.profile_visibility`</code> (<code>public/followers/private</code>), set on a different screen (<code>./privacy</code>) and read at <code>mod/631.js:14052, 14121, 14152, 14202, 14280, 16825</code>. <code>`media_consent`</code> gates nothing in the <code>media/clip/photo</code> code at all. Evidence: <code>mod/2477.js:128`onPress: () => z({ privacy_visibility: t.key })`</code> and <code>mod/2477.js:212`onValueChange: (t) => z({ media_consent: t })`</code>. In the backend these two fields are only ever written or echoed: <code>mod/631.js:1002`t.privacy_visibility && (a.privacy_visibility = t.privacy_visibility);`</code>, <code>mod/631.js:1004`"boolean" === typeof t.media_consent && (a.media_consent = t.media_consent);`</code>, and the read-back in <code>mockGetPrivacy</code> at <code>mod/631.js:991-993</code>. A grep of all 17k lines of 631 finds <code>`privacy_visibility`</code> only at <code>972/991/1002/1007/1012/1987-1988/2162/2296/5092/5111/5130/5153/6875/9329/9513</code> (seed values, the setter, the audit line, and the getter) and <code>`media_consent`</code> only at <code>974/992/1004/1009/1014</code> plus seeds — no authorization, filtering or media path reads either one. By contrast the third field, <code>avatar_mode</code>, IS honoured (<code>mod/631.js:8673, 14132, 14246, 14288</code>).	A user — in practice a woman using the same-audience partition — sets "Connections only" and "do not use my photos in match media", sees a preview confirming her name is hidden, and is fully exposed anyway: her profile is still served to everyone under whatever <code>privacySettings</code> default applies, and her media consent choice is discarded. This is a privacy guarantee the product makes in the UI and does not keep in code, and it survives any move of 631 to a server unchanged.	Collapse the two visibility models into one. Either make <code>`mockGetPlayerProfile`/`dr`/`mockSearchPlayers`</code> consult <code>`profiles.privacy_visibility`</code> (with <code>`same_audience`</code> enforced via <code>`isAudience`</code>) or delete the field and have <code>/privacy-controls</code> write <code>`privacySettings.profile_visibility`</code> through <code>`mockSetPrivacySetting`</code> into the media upload/clip-tagging paths so a false <code>`media_consent`</code> excludes the user, and add a regression test asserting each toggle has at least one read result.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-14 Critical	./privacy-controls.tsx	Privacy controls / data exposure	The "Profile visibility" control on /privacy-controls is stored and audited but never read by any gate or DTO in the backend The screen presents `privacy_visibility` as the profile-visibility control and the copy makes a concrete promise — visConnectionsSub = "Only people you've played with or follow", previewNameHidden = "Name visible to connections only", privacyNeverShown = "These settings are never visible to other players." — but 631 never consults the value, so selecting "connections" or "same audience" changes nothing about who can read the profile, the name, or the DTOs. The app has two parallel, unlinked privacy models and only the other one (profile_visibility on ./privacy.tsx) is enforced. Evidence: mod/2477.js:128 writes it — `onPress: () => z({ privacy_visibility: t.key })` with options everyone / same_audience / connections (mod/2477.js:42-45). mod/631.js:997-1017 persists and audits it: <pre>... r.mockUpdatePrivacy = async (e, t) => { ... t.privacy_visibility && (a.privacy_visibility = t.privacy_visibility), ... }</pre> Every other occurrence of the field in mod/631.js (lines 972, 991, 1002, 1007, 1012, 1987-1988, 2162, 2296, 5092, 5111, 5130, 5153, 6875, 9329, 9513) is a seed value, a default, or the echo back to the settings screen. There is not one comparison of `privacy_visibility` anywhere in the 17k-line backend. The gates that actually run read a different, separately-stored object: `al(e)` → `(profile_visibility, show_online, allow_messages)` (mod/631.js:14031, 14280-14283), which is edited on a different screen (./privacy.tsx, mod/2478.js:74).	A user who deliberately restricts her profile to "connections" stays fully discoverable and readable by every same-partition stranger — name, avatar, sports, skill, follower counts, area. Because the UI renders a live "How others see me" preview showing the name hidden, the user is actively misled into believing a protection exists. In a gender-partitioned Kuwait sports app this is the highest-stakes privacy promise the product makes, and it is inert.	Either make 631 read it — add `privacy_visibility` to the checks in `mockGetPlayerProfile` (mod/631.js:14280), `mockSearchPlayer` (mod/631.js:14197-14202), `mockGetSuggestedFollows` (mod/631.js:14197-14202), and `Td`-based name DTOs — or collapse the two models: delete `privacy_visibility` / `avatar_mode` fields from the profile record and use `al` to read and write the same `al` record that ./privacy-controls.tsx uses through `mockGetPrivacySettings` / `mockSetPrivacySetting`.
P-CSEC-15 Critical	./squad/[gameId]	Consumer protection / refunds	Squad opt-out silently forfeits a paid seat, while ignoring the prompt is refunded in full The opt-out path skips the refund quote the normal leave flow shows (mod/2379.js:1044-1052 `LeaveMatchSheet` with `paidKwd` and `mockGetSeatRefundQuote`), and the squad window sits entirely inside the no-refund window, so `isRefundEligible` is false for essentially every opt-out. Evidence: Opt-out cancels the booking through the ordinary leave path: mod/631.js:4038-4041 `(i.squad_opted_out_at = new Date().toISOString(), ... await q(t, e));` and `qi` applies the 2-hour cut-off: mod/631.js:3131-3132 `s = !r    isRefundEligible(r.starts_at, o), d = r ? await Br(r, t, s) : ...` (SEAT_REFUND_CUTOFF_HOURS = 2, module 670). The squad deadline defaults to one hour before kick-off: mod/631.js:3961 `else ((o = r - 36e5), ...)` , and the host chips offer only 1/2/3 hours (mod/2486.js:689 `const B = [1, 2, 3]`, mod/2486.js:350). The silent player who never answers is refunded in full by the sweeper: mod/631.js:4126-4130 `t.status = "cancelled"; t.squad_dropped_at = ...; await Br(e, t.user_id, t0)` . The confirm dialog says nothing about money: mod/2486.js:230-233 `children: k("squadOptOutBody")` → "You'll lose your spot and the waitlist moves up. Rejoining needs the host."	A player who honestly answers 'I can't make it' loses the full seat fee with no warning and no receipt; a player who ignores the prompt is auto-dropped and refunded in full. The app financially punishes the cooperative behaviour it is asking for, and the opt-out screen never discloses the charge — a refund-disclosure problem, not just a UX one.	Fetch `mockGetSeatRefundQuote` in mod/2486.js before rendering the opt-out card and show paid/refundable amounts, and make `mockOptOut` (mod/631.js:4027) grant the same forced refund the sweeper grants (`Br(game, user, true)`) so answering the prompt is never worse than ignoring it.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-16 Critical	/stories/compose.tsx	Data exposure / visibility bypass	A story can attach any clip id, republishing clips from private uploads to every viewer mockCreateStory accepts an arbitrary clip_id: it never verifies the clip exists, that the author owns it, or that the parent upload's visibility permits publication; and mockGetAuthorStories dereferences it for every viewer without re-checking. It should resolve the clip, require clip.owner_id === author (or an explicit grant), and refuse or strip clips whose parent upload is 'private'. mockGetHighlightCollections should likewise exclude clips belonging to non-public uploads. Evidence: mod/631.js:14829 in mockCreateStory stores the caller-supplied clip id with no check at all: <pre>clip_id: t.clipId ?? null, mod/631.js:14926 in mockGetAuthorStories then hands the whole clip row to every viewer: clip: e.clip_id ? (Qt.mediaClips.find((t) => t.id === e.clip_id) ?? null) : null, The two guards that exist elsewhere are bypassed. mod/631.js:13418 (mockGetMediaUpload): if (!i && "private" === a.visibility) throw new Error("forbidden"); mod/631.js:13452 (mockGetClipSignedUrl): if (!a.owner_id === e !d(e)) && "private" === i?.visibility) throw new Error("forbidden"); It is also reachable with no crafted API call, because mod/631.js:14589 (mockGetHighlightCollections) filters on ownership only and never joins mediaUploads: const t = Qt.mediaClips.filter((t) => t.owner_id === e), and mod/2488.js:60 posts whatever chip was tapped: clipId: F?.cover_clip_id ?? null,</pre>	Two concrete leaks. (1) Accidental, via the UI alone: a user marks a match upload 'private', then posts a highlight story from the chip row in /stories/compose - the private clip's full record (upload_id, owner_id, kind, start/end seconds, detected events, player_id, view and share counts) is served to every follower. (2) Deliberate: a viewer reads another user's clip id straight out of a story payload, then re-posts that same clip id in their own story, republishing a clip they were explicitly forbidden from reading through /media/[id] and from signing a URL for. Because the clip row carries player_id and player_tags-derived reels, this discloses which named players appear in a match the owner chose to keep private.	In mockCreateStory (mod/631.js:14819) look the clip up before b row: reject with 'not_found' when it is missing, and with 'forbidden clip.owner_id !== e or when the parent mediaUploads row's visibi 'private'. Re-apply the same visibility check in mockGetAuthorSto (mod/631.js:14926) so historical stories cannot leak a clip whose later made private, and add a visibility join to mockGetHighlightC (mod/631.js:14587) so the compose screen never offers a private
P-CSEC-17 Critical	/teams/[id].tsx	Access control / membership bypass	Any viewer can read any clan's invite code, and that code grants instant membership of invite-only clans `mockGetTeamInvite` performs no authorization at all — it does not even receive a viewer id — yet the invite code is the sole credential that `mockJoinTeamByCode` accepts for instant membership of an invite_only clan. The screen only hides the code in the UI (mod/2489.js:387 gates rendering on `Ye`/canManageTeam); the value is still fetched and held in client state for every visitor. It should require that the caller be an active manager of the team, exactly as `so(teamId, userId)` (mod/631.js:8430) enforces for every other privileged team action. Evidence: mod/631.js:9199 — the invite lookup takes only a team id, so it has no actor to authorize: <pre>... 9199 r.mockGetTeamInvite = async (e) => { 9200 await ei(); 9201 const t = to(e); 9202 return t ? { code: t.invite_code, link: `https://playora.app/t/\${t.invite_code}` } : null; 9203 }; ... mod/671.js:586 confirms the facade passes no viewer: `e.fetchTeamInvite = (o) => t.store.mockGetTeamInvite(o);` mod/2489.js:68 fetches it for every visitor, member or not (it sits in an unconditional Promise.all at lines 62-72): `(0, L.fetchTeamInvite)(e).catch(() => null);` mod/631.js:8636 lets non-members load an invite_only team, because only `private` is gated: `return a ? (ra(t, a.audience), "private" !== a.privacy    no(e, t)    (t && ro(t)) ? fo(a, t) : null) : null;` mod/631.js:8735-8750 then turns that code into immediate active membership with no approval: ... 8734 "invite_only" === i.privacy 8735 ? eo(i.id, async () => { ... 8749 (a.status = "active"), 8750 (a.joined_at = new Date().toISOString()), ... </pre>	Any signed-in user who opens /teams/<id> for an invite-only clan obtains that clan's join code and can immediately make themselves an active member, seeing the private roster, the clan chat and announcements, and — if later promoted or if they are the first to act — the battle board. The invite_only privacy setting offered at mod/2498.js:247-285 provides no protection whatsoever. This is not a client-side-only issue: the function signature carries no actor, so the same hole ships verbatim if 631 is lifted onto a server.	Change `mockGetTeamInvite` in mod/631.js:9199 to take `(userId and `await so(teamId, userId)` before returning the code; update at mod/671.js:586 and the call site at mod/2489.js:68 to pass `r.id the call behind the `Ye` (canManageTeam) check so non-manage request it. Rotate `invite_code` whenever a member is removed (

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-18 Critical	/teams/[id].tsx	Scoring integrity / ranking manipulation	Clan match results are self-declared with no opponent confirmation and feed the public ladder that battle stakes are paid from Two write paths feed one number. The gauntlet path is carefully two-sided; the `teamEvents` path is entirely unilateral, has no `result`-already-set guard, no `starts_at` < now` guard, and lets the actor supply `attended_count`. The UI restrictions (mod/2489.js:992 only renders the win/loss/draw buttons when `new Date(e.ends_at).getTime() < Date.now()` and `!e.result`) exist only on the client. Evidence: mod/631.js:8964-8975 — a captain writes an arbitrary result for their own team; nothing validates the event is finished, that no result already exists, or that the opponent agrees: <pre>... 8964 r.mockRecordTeamResult = async (e, t, a, i, n, r, o) => { 8965 await ei(), await so(t, e); 8966 const s = Qt.teamEvents.find((e) => e.id === a && e.team_id === t); 8967 if (!s) throw new Error("E_EVENT_NOT_FOUND"); 8968 ((s.result = i), 8969 (s.our_score = Math.max(0, Math.round(n))), 8970 (s.their_score = Math.max(0, Math.round(r))), 8971 null != o && (s.attended_count = Math.max(0, Math.round(o))), 8973 await vo(t), ... mod/631.js:8915-8943 lets the same captain manufacture the events (12 per call, unlimited calls), stamping `invited_count` from their own roster size. mod/631.js:9192 shows the ladder score is derived straight from those rows: `return Object.assign({ team: e, member_count: i, score: Ao(e, a), stats: n }, ...)` with `Ao` (9171-9175) = `rankingScore(computeTeamStats(teamEvents, members))`. module 667 defines `rankingScore = 0.4*winRate + 0.2*min(1, playedInWindow/20) + 0.25*attendanceRate + 0.15*0.7`. Contrast mod/631.js:10224-10312 + 10408-10446, where `battle` results require both captains, a 3-attempt dispute cycle and escalation.</pre>	A single captain of a one-person clan can schedule 20 training events, record 20 wins with full attendance, and reach a ranking score of ~96 — above every honestly-played clan. That rank is what /teams/rankings and the team page show, what `Oo()`/`Ko()` use to compute battle `stake_pts` and the +50% underdog bonus (mod/631.js:9944-9949), and what the ceremony reports as the ladder movement. The entire competitive layer is gameable without ever playing a match, and the dispute machinery built for battles is bypassed.	In `mockRecordTeamResult` (mod/631.js:8964) reject events when `ends_at` is in the future, reject a second write when `s.result != r` (require an admin), and cap `attended_count` at the stored `invited_count`. Longer term, exclude self-reported `teamEvents` from `rankingScore`, or require that the opponent's `opponent_team_id` captain to confirm before `vo(t)` recomputes the ladder.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-19 Critical	/teams/create.tsx	Authentication / identity integrity	Guest sessions share one identity and can create, own and operate clans with full write access to the ladder `md`() is the codebase's own guest guard (used at mod/631.js:2912, 2996, 6197, 13657) but none of `mockCreateTeam`, `mockJoinTeam`, `mockPostTeamChat`, `mockSendClanChallenge`, `mockSubmitBattleResult`, `mockRecordTeamResult` or `mockCreateClub` calls it. A guest therefore becomes a real `owner` row in `Qt.teamMembers` with `display_name` resolved to the literal string "Player" (mod/631.js:6548-6551, no profile exists). Evidence: mod/630.js:72-76 — every guest on every device gets the *same* two user ids: <pre>... 72 enterGuest: (t) => { 73 (0, o.setThemeAudience)(t); 74 const s = { user: { id: `guest:\${t}`, email: "guest@playora.app" }, token: "guest" }; 75 (w(s), (0, h.persistGuestSession)(s)); 76 }, ...</pre> mod/613.js:415-417 makes that reachable from the sign-in screen before any account exists: <pre>`onPress: () => { (P(e), F.replace("/(tabs)")); };` mod/631.js:8427-8429 — the only gate `mockCreateTeam` (8537) and `mockJoinTeam` (8683) apply is a truthiness check, which "guest:male" passes: ... 8427 async function oo(e) { 8428 if (!e) throw new Error("E_YOU_MUST_BE_SIGNED_IN_TO"); 8429 } ...</pre> mod/631.js:12394-12396 shows the guard that exists but is never called from any team or club function: <pre>... 12394 md = (e) => { 12395 if (e.startsWith("guest:")) throw new Error("E_SIGN_UP_TO_JOIN_BROWSING_IS"); 12396 }; ...</pre> mod/2498.js:327 gates creation on nothing more than a truthy user: `onPress: async () => { if (e) { ... await (0, v.createTeam)(e.id, {...})`	Anyone who taps "browse as guest" can create clans, join public clans, post clan announcements, throw gauntlets at real clans and submit battle scores — all as `guest:male`, an identity shared by every guest in the world. Two different anonymous visitors both control the same clans and can undo each other's actions; real users see rosters and rankings polluted by owners named "Player" whom they can never contact, report or hold accountable. Real clans can be dragged into battles by an unattributable actor.	Call `md(userId)` at the top of `mockCreateTeam` (mod/631.js:85 `ho`/`mockJoinTeam` (8680), `mockJoinTeamByCode` (8727), `mockPostTeamChat` (9062), `mockSendClanChallenge` (10665 `mockRespondClanChallenge` (10721), `mockSubmitBattleResu` `mockRecordTeamResult` (8964) and `mockCreateClub` (13093) the write CTAs when `useRoleGate().isGuest` (mod/9001.js:55) i:
P-CSEC-20 Critical	/venue/apply.tsx	Authorization / account integrity	Guests can register a venue, and the resulting venue is owned by the shared guest:male / guest:female pseudo-identity mockApplyVenue has no guest guard, the apply screen gates only on the admin role, and the root layout deliberately lets guest sessions roam the whole app. The profile tab links straight there (mod/1805.js:683 `J.push(Oe ? "venue/portal" : Ee ? `\\clubs/\${Ee.club.id}` : "venue/apply")`). Evidence: mod/2503.js:16 — `if ("admin" === pr?.role) return ... GateScreen` is the screen's only gate; a guest has `profile === null` (mod/630.js:32-33 `if ((lv?.user && lv.user.id.startsWith("guest:") && markReturning(), lv?.user)) return (P(null), ...)`). mod/1809.js:52-54 — `const n = "(auth)" === c[0], t = o?.user.id.startsWith("guest:") ?? !1; o    n ? o && !t && n && S.replace("/(tabs)") : S.replace("/(auth)/sign-in");` — a guest session is never redirected off /venue/apply. mod/630.js:74 — `const s = { user: { id: `guest:\${t}`, email: "guest@playora.app" }, token: "guest" };` — guest ids are the two fixed strings `guest:male` / `guest:female`. mod/631.js:6978-7013 — mockApplyVenue checks only `if (ro(e)) throw new Error("E_ADMINS_CANNOT_REGISTER_VENUES")` and then writes `owner_id: e`. The codebase's own guest guard exists and is used elsewhere: mod/631.js:12394-12396 — `md = (e) => { if (e.startsWith("guest:")) throw new Error("E_SIGN_UP_TO_JOIN_BROWSING_IS"); }`.	A browsing guest can create a venue business entity. Because every guest on the device shares the id `guest:male` or `guest:female`, the venue profile it creates is then owned by that shared identity: the next person who taps Browse as guest lands in /venue/portal for that venue with full access to revenue figures, booking accept/reject, court pricing, staff management and the check-in scanner. The owner can never be identified, contacted or held to the commission agreement, and cannot reclaim the venue by signing up.	Call the existing guest guard (`md(e)`, mod/631.js:12394) at the t mockApplyVenue, and add the same `isGuest` check to the /veni screen (mod/2503.js:16) using `useRoleGate`'s `isGuest` flag fro mod/9001.js:55 so the guest sees a sign-up prompt rather than a fails on submit.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-21 Critical	/venue/scan.tsx	Data integrity / reputation manipulation	Court-booking organizers can write attendance through scanCheckin, bypassing every guard on the organizer attendance API mockScanCheckin writes the same `booking.attendance` field that mockSetAttendance writes, but accepts the court booking's own `organizer_id` as an authorized scanner and applies none of mockSetAttendance's checks: no approved-organizer role check (`on`), no organizer-of-that-game check (`zi`), no match-finished check, no cancelled-match check and no 48-hour freeze. It also accepts an arbitrary `player_id` and creates a check-in row for it (mod/631.js:8335-8348) with no check that the player is a participant. Evidence: mod/631.js:8330-8359 — `r.mockScanCheckin = async (e, t, a, i) => { await ei(); const n = Qt.courtBookings.find((e) => e.qr_token === t); if (!n) throw new Error("E_INVALID_QR_CODE"); if (n.organizer_id !== e && !vr(e, n.venue_id)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_SCAN");` ... `const e = Qt.bookings.find((e) => e.game_id === n.game_id && e.user_id === a); e && ((e.attendance = "no_show" === i ? "no_show" : "pending" === i ? null : "attended"), ... await Za(W, Qt.bookings));` Compare the sanctioned path, mod/631.js:3859-3869 — `r.mockSetAttendance = async (e, t, a, i) => { (await ei(), await on(t)); ... zi(n, t); if ("cancelled" === n.status) throw new Error("E_THIS_MATCH_IS_NO_LONGER_OPEN"); if (new Date(n.ends_at).getTime() > Date.now()) throw new Error("E_MATCH_NOT_FINISHED"); if (n.score_submitted_at && Date.now() - new Date(n.score_submitted_at).getTime() > 1728e5) throw new Error("E_ATTENDANCE_LOCKED");` Consumer of the field: mod/631.js:5506-5511 and 5552-5560 — `"attended" === r.attendance && (i += 1), "no_show" === r.attendance && (n += 1)` feeding `attendance_rate` and `isVerified`.	The organizer of a court booking is an ordinary consumer, not the venue. They can mark themselves and everyone in their match `present` without anyone attending, and mark a rival `no_show`, at any time — before the match, on a cancelled match, or weeks after the 48-hour lock that the organizer UI enforces. Attendance drives attendance_rate, the verified badge and the player passport, so the whole reliability signal the app sells to players is forgeable by the person with the most incentive to forge it.	In mockScanCheckin (mod/631.js:8330) drop `n.organizer_id !== authorization test and require `vr(e, n.venue_id)` alone, then route attendance write through the same preconditions mockSetAttend applies: reject when the match is cancelled, when `ends_at` is in and when the 48-hour post-score lock has passed. Reject a `play` has no confirmed booking on `n.game_id` instead of inventing a c for it.
P-CSEC-22 Critical	mod/631.js (shared backend: tc / ac)	Payment integrity / money creation	Card-funded payments are never posted to the ledger, but refunds credit the full amount to the wallet — creating money The ledger is treated as the record of money movement but only records the wallet/credits portion of a payment. Refund, however, always credits `owner: <user>:available` with the full `amount_kwd`. A seat paid entirely on KNET produces no debit anywhere; cancelling it produces a full credit to spendable wallet balance. There is also no reversal of the gateway capture — nothing in 631 calls the provider to refund the card — so the card charge stands while the wallet is credited. Evidence: mod/631.js:16253-16263 — `tc` builds postings only from the internal legs: `(r.credits_fils > 0 && o.push({ account: A.acct.credits(e), delta_fils: -r.credits_fils }), r.wallet_fils > 0 && o.push({ account: A.acct.available(e), delta_fils: -r.wallet_fils }));` const s = r.credits_fils + r.wallet_fils; return (s > 0 && (o.push({ account: A.acct.external("clearing"), delta_fils: s }), await Yl({ kind: a, postings: o, ... })), r);` — `r.external_fils` (the card leg) is posted nowhere, and when the whole amount is external, `s === 0` and no ledger row is written at all. mod/631.js:16269-16278 — `ac` refunds the whole face value regardless: `const i = (0, A.toFils)(t); i <= 0 (await q(e, () => Yl({ kind: "refund", postings: (0, A.transferPostings)(A.acct.platform("refunds"), A.acct.available(e), i), ... })))` where `t` is always `payment.amount_kwd` (callers at 7474, 7715, 7796, 7994).	Every card-paid booking that is later cancelled inside the refund window mints its full value as new, spendable, transferable wallet balance while the original card debit is untouched. A player can pay 20 KWD on a card, cancel, receive 20 KWD of wallet money, and dispute the card charge as well. The `platform:refunds` account drifts unboundedly negative and nothing reads or reconciles it, so the loss is invisible in every report.	Post the external leg in `tc` (`external:gateway` → `external:clear` `r.external_fils`) so every payment is fully double-entered, and re on the payment row. In `ac`, refund each funding source to where from — card portion back through the gateway, wallet portion to `credits` portion to `credits` — rather than crediting `A.acct.available` whole face value.
P-CARCH-3 Critical	mod/631.js (shared store — affects every route)	Availability / cold start	Hydration is all-or-nothing across ~97 localStorage keys and one unreadable key bricks every screen in the app Every one of the ~450 `mock*` entry points begins with `await ei()`, and `ei()` resolves only if all ~97 keys parse. A single corrupt or shape-changed key makes `ti()` reject, `Qt.hydrated` stays false, `Xa` resets to null in the `.finally`, and the next call re-runs the same failing read. `xa` already accepts a per-key default but never falls back to it on a parse failure, and nothing quarantines or resets the bad key. Evidence: mod/631.js:1690-1703 — `xa = async (e, t) => { try { ... const n = await l.default.getItem(e); return n ? JSON.parse(n) : t; } catch (t) { throw ((0, w.noteReadFailure)(e, t, t); } }` — the default value `t` is only used when the key is absent; a parse error rethrows. mod/631.js:1898-1985 awaits ~92 `xa(...)` calls inside one `Promise.all`, then mod/631.js:2005-2009 adds five more serial awaits (`Qt.pushTokens = await xa(V, [])`), `matchEvents`, `seatCancellations`, `paymentIntents`, `paymentCharges`. mod/631.js:1780-1787 — `ei = () => Qt.hydrated ? Promise.resolve() : (Xa    (Xa = ti()).finally(() => { Xa = null; })), Xa)`.	One damaged key — a half-written value, a value written by an older build, an extension or devtools edit — takes the whole app down permanently for that user. `/feed`, `/games`, `/wallet`, `/profile` all reject on every load, most of them into a spinner that never resolves (see the loading-state findings). There is no in-app recovery: the user must clear site data for rush-x.xyz, losing every booking, message and wallet entry.	In `xa` (mod/631.js:1690), catch the parse error, call `noteReadFailure` return the supplied default instead of rethrowing; optionally move value to a ` <key>.corrupt` backup so it is not silently discarded. Set read-failure count from `persistenceHealth` (mod/643.js:83) in a task degraded hydration is visible.</key>

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-4 Critical	mod/631.js (shared store — affects every route)	Write amplification / storage quota	The store's row-storage, advisory-lock and blob-offload paths are guarded on AsyncStorage methods that do not exist, so every table write re-serialises the entire table Three separate scalability escape hatches were written and are all dead code on the shipped build. Consequently: (a) every mutation writes the whole table — <code>`mockCreatePost`</code> at mod/631.js:14393-14394 does <code>`Qt.posts.unshift(i), await Za(Ce, Qt.posts)`</code>; a single like at mod/631.js:14506 rewrites all of <code>`playora.mock.post.likes.v1`</code>; a DM at mod/631.js:14733 rewrites all of <code>`playora.mock.dm.messages.v1`</code>. (b) Per-game serialisation via <code>`Xt`/`Ha`</code> degrades to no locking at all. (c) <code>`Ya`</code> (mod/631.js:1727-1738) returns the raw <code>`data:`</code> URI, so base64 images are stored inline in the JSON arrays. Evidence: mod/631.js:1763-1778 — <code>`Za = async (e, t) => { ... const a = l.default.setRows; if (a && Ga[e] && Array.isArray(t)) return void (await a(e, t)); await l.default.setItem(e, JSON.stringify(t)); ... }`</code>. mod/631.js:1713-1722 — <code>`Ha = async (e, t, a) => { const i = l.default.withAdvisoryLock; return i ? i(...) : a(); }`</code>. mod/631.js:1733-1734 — <code>`const n = l.default.putBlob; if (!n) return e;`</code>. <code>`l`</code> is dep index 2 = module 618, which is <code>`@react-native-async-storage/async-storage`</code> (index.html module 618: <code>`e.default = u.default`</code> re-exporting the <code>async-storage</code> default). Grepping the whole shipped index.html finds <code>`setRows`</code> exactly once, <code>`getRows`</code> twice, <code>`withAdvisoryLock`</code> once and <code>`putBlob`/`deleteBlob`</code> once each — all of them are the reads above inside 631; none is ever defined.	Posting the 500th message in a team chat serialises 500 messages; the cost of every write grows linearly with history, on the UI thread. Worse, <code>`Ya`</code> permits images up to 700 KB decoded (mod/631.js:1732), which is ~933 KB of base64 stored as UTF-16 in localStorage ≈ 1.9 MB of quota each. Three image posts exhaust a typical 5 MB origin quota, after which <code>`Za`</code> throws <code>`E_STORAGE_FULL`</code> (mod/631.js:1772-1774) for every table, not just posts — the user can no longer join a game, send a message or top up their wallet.	Either implement <code>`getRows`/`setRows`/`putBlob`</code> in the storage a behind module 618, or delete the dead branches and replace <code>`Za`</code> append/patch layer (IndexedDB via idb-keyval is already reachat bundle's sandbox). At minimum, route <code>`Ya`</code> output through Indexe storage rather than leaving images inline in <code>`playora.mock.posts.`</code> <code>`playora.mock.stories.v1`</code> , <code>`playora.mock.teamchat.v1`</code> and <code>`playora.mock.teams.v1`</code> (call sites mod/631.js:9067, 9100, 1438
P-CARCH-5 Critical	mod/673.js (backend transport, used by every screen through 672/671)	Data integrity / silent degradation	A single failed fetch permanently downgrades the whole session to the in-browser store, with only a console.warn The proxy is designed to fail open into the local mock store. One transient network error — a tunnel drop, a Railway cold start, a dropped mobile connection — flips the session to local-only and it never recovers without a full page reload. Nothing in the UI says the app is no longer talking to the server. Evidence: mod/673.js:26-34 — <code>`k = () => null !== o ? Promise.resolve() : (!    l = (async () => { c = n(); o = c && (await u(c, 900)) ? "remote" : "mock"; })(), l)`</code> and mod/673.js:65-67 — <code>`}`</code> catch { return ((o = "mock"), console.warn("[backend] unreachable, falling back to local store"), l()); }`. <code>`o`</code> is module-level state; once it is set to "mock" the guard at line 27 (<code>`null !== o`</code>) short-circuits forever, so no later call ever re-probes. Verified: with <code>`backendUrl`</code> set to an unreachable https origin, <code>`backendMode()`</code> === 'mock', <code>`remoteEnabled()`</code> === false, and <code>`store.mockGetVenues()`</code> returned 32 local rows with no user-visible error.	A user mid-session keeps joining matches, paying, cancelling and messaging; every one of those writes lands in localStorage only and is invisible to the real backend and to everyone else. When they reload, the app re-probes, goes remote, and their bookings are gone — or worse, they show a paid seat locally that the organiser never sees. Two players can each hold 'the last seat'.	In mod/673.js, distinguish first-boot probe failure from mid-sessio a mid-session fetch rejection, surface a localized error and let the rather than silently invoking the local fallback <code>`l()`</code> . Reset <code>`o`</code> to <code>`n`</code> cooldown so the next call re-probes, and expose the mode so the show an offline banner.
P-CTEST-2 Critical	tools/*.mjs (all seven regression suites)	Test coverage / reproducibility	68 of 68 consumer routes have no flow coverage, and the suites cannot be run on a clean checkout Nothing verifies that a player can sign up, join a match, pay, cancel, get a refund, top up or withdraw from the wallet, block or report another player, send a message, redeem a reward, scan a check-in QR, or use an invite code. <code>tools/bundle.py build</code> (line 54, 70) runs only <code>`node --check`</code> on each module — syntax, not behaviour — and CLAUDE.md leaves running the seven suites to whoever remembers. Evidence: Every <code>`openApp`</code> call across all seven suites: <code>flows-admin.mjs</code> opens only <code>/admin/*</code>, <code>flows-org1.mjs</code> and <code>flows-organizer.mjs</code> only <code>/organizer/*</code>, <code>flows-kids.mjs</code> only <code>/kids</code>, <code>flows-search.mjs</code>:7 and <code>flows-xcut.mjs</code>:42,57 and <code>rules.mjs</code>:7 open only <code>`/`</code>. The consumer manifest lists 68 routes; exactly one (<code>./(tabs)/index.tsx</code>, via <code>`/`</code>) is ever opened. mod/671.js exports 374 API functions; the 20 exercised anywhere in tools/ are <code>cancelMatch</code>, <code>createMatch</code>, <code>fetchAdminAuditLog</code>, <code>fetchBIDashboard</code>, <code>fetchBooking</code>, <code>fetchDemandModelStats</code>, <code>fetchOrganizerApplications/Matches/Ratings/ReferralStats/Series/Stats</code>, <code>fetchPendingVenues</code>, <code>fetchSeriesAnalytics</code>, <code>fetchVenues</code>, <code>issueSanction</code>, <code>resetFeedWeights</code>, <code>restoreUser</code>, <code>reviewVenue</code>, <code>setDemandWeights</code> — all admin or organizer. <code>tools/smoke.mjs</code>:7 — <code>`import { chromium } from 'opt/node22/lib/node_modules/playwright/index.mjs';`</code> — an absolute path outside the repo; playwright appears nowhere in package.json, which has no <code>`test`</code> script, and the repo has no CI workflow of any kind.	Any bundle edit can break the entire money and safety surface and every gate will still pass. And because playwright is loaded from a hardcoded machine-local path and is not a declared dependency, the suites cannot run in CI or on a new laptop at all, so the manual discipline is the only line of defence and it is not enforceable.	Add playwright to devDependencies, replace the absolute import <code>tools/smoke.mjs</code> :7 with a bare <code>`playwright`</code> import, add an <code>`npm test`</code> that runs all seven suites plus <code>`bundle.py build`</code> , and wire it to CI. flow suites for the money path <code>./pay/[id].tsx</code> , <code>./wallet/index.tsx</code> , <code>./n bookings.tsx</code> , <code>./refunds.tsx</code> , the join path <code>./game/[id].tsx</code> , <code>./join-co auth`</code> <code>./(auth)/sign-in.tsx</code> , <code>./(auth)/sign-up.tsx`</code> — ranked in that or

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-1 High	./{auth}/sign-in.tsx (reset flow) — mod/631.js	Error handling / account recovery dead end	Password reset reports success after the code delivery failed, and deletes the code it just stored The catch swallows the delivery failure, removes the pending code from `ka`, and the function still returns `sent: true` unconditionally. The caller has no way to distinguish a delivered code from a discarded one. Evidence: mod/631.js:920-936 — `r.mockRequestPasswordReset = async (e) => { ... if (Qt.users.some(...) && (ka.set(t, {code: a, ...}), a !== sa)) try { await ca(t, a); } catch (e) { (ka.delete(t), noteWriteFailure("sms.reset", e)); } return (await logAudit("auth.password_reset_requested", ...), { sent: !0, demo_code: a === sa ? a : "" }); }`. Verified with demo:false: the call returns `ok:{"sent":true,"demo_code":""}` while the console logs `[persist] write failed for sms.reset (total 1): SMS_NOT_CONFIGURED`.	A locked-out user is told their reset code is on its way, waits, never receives it, and then cannot even enter a code because `mockResetPassword` will throw <code>E_CODE_EXPIRED_REQUEST_A_NEW_ONE</code> for a key that was deleted. Repeating the request repeats the same lie. Account recovery is a closed loop with no exit and no error message anywhere in the UI.	In mod/631.js:920-936, rethrow (or return `{sent:false, reason}`) v throws instead of swallowing it, so <code>./{auth}/sign-in.tsx</code> can show a and leave the code in `ka` only when the send succeeded.
P-CQUAL-2 High	./{auth}/sign-in.tsx, ./{auth}/sign-up.tsx — mod/913.js	Error handling / dead end	Remote auth calls bypass the timeout wrapper, so a hung API leaves the sign-in button spinning forever Four of the eight remote auth entry points — the whole phone and password-reset surface — have no bound on how long they wait, and parse the body before checking the status, so an HTML 502 page becomes a JSON SyntaxError rather than a handled error. Evidence: mod/913.js:22-37 defines `authFetch` with an AbortController and a 10s timeout that converts failures to `NETWORK_TIMEOUT`. But mod/913.js:75-84 (`remoteOtpRequest`), 85-94 (`remoteOtpCheck`), 95-104 (`remoteResetRequest`) and 105-113 (`remoteResetConfirm`) each call bare `fetch(...)` directly with no signal, no timeout, and no `response.ok` check before `await o.json()`. Only `remoteSignIn` / `remoteSignUp` / `remoteOtpSignIn` / `remoteOtpVerify` route through `authFetch` via `h`.	With a real backendUrl configured and the API slow or wedged, tapping 'Send code' on <code>./{auth}/sign-in.tsx</code> leaves `Q(!0)` set (mod/613.js:37,47) with the button in its loading state indefinitely — the `finally` never runs because the promise never settles. The user's only escape is to reload the page. On a 502 they instead see a raw `Unexpected token '<'...` message.	Route `remoteOtpRequest`, `remoteOtpCheck`, `remoteResetRe` `remoteResetConfirm` through `authFetch` (mod/913.js:22) and r helper at mod/913.js:39-49, which already handles 429 and non-. bodies.
P-CSEC-23 High	./{auth}/sign-up.tsx (defect in mod/631.js)	Authentication bypass / account takeover	A verified OTP stays usable for 30 minutes by any caller, because the code record is keyed only by phone number with no binding to the checking client There is no nonce, no per-attempt token and no session binding tied to the successful check. The only key is the canonical phone number, so the "verified" state is global to that number for 1 800 000 ms. Any subsequent verify/sign-in call for that phone with an arbitrary code succeeds. This rule is missing in 631 itself, so it survives verbatim if this code is lifted onto a server. Evidence: mod/631.js:856-861, `mockCheckOtp` marks the record verified and extends its life: ... 856 return (857 (i.verified = !0), 858 (i.expires = Date.now() + 18e5), 859 await _a.set(a, i), 860 { verified: !0, existing: Qt.users.some((e) => e.email === `\${a}@otp.playora.app` }) ... mod/631.js:885-888, `mockVerifyOtp` then skips the code comparison entirely for a verified record: ... 885 if (!s.verified) { 886 if (((s.attempts += 1), s.attempts > 5)) throw (_a.delete(o), new Error("OTP_ATTEMPTS")); 887 if (t.trim() !== s.code) throw new Error("OTP_WRONG"); 888 } ... The same two-call shape is the remote contract: mod/913.js:514 `remoteOtpCheck` posts to /auth/otp/check and mod/913.js:543 `remoteOtpSignIn` posts phone+code to /auth/otp/signin.	Once a legitimate user completes the OTP check step, anyone who knows their 8-digit Kuwaiti mobile number can call otp/signin with any code within 30 minutes and receive a full session for that account. The 5-attempt cap at mod/631.js:886 is bypassed too, since it sits inside the branch that is skipped.	Have mockCheckOtp return a single-use, high-entropy continuati bound to the phone and to the caller, store its hash on the record it (not just `verified`) in `ya`. Do not extend `expires` past the orig lifetime, and clear the record on first successful verify.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-6 High	./(tabs)/_layout.tsx (dock, mod/1620.js)	Backend availability and data loss	A single failed request permanently downgrades the session to the in-browser store and latches the offline banner on `backendMode` is a one-way latch with no recovery path, no retry timer and no way for a screen to re-probe. It should re-run the `health` check after a backoff, and `rpc` should distinguish a transport failure (retry) from an application error. It should also reject a non-JSON response instead of calling `y.json()` on it. Evidence: mod/673.js:65-67 — ` } catch { return ((o = "mock"), console.warn("[[backend]] unreachable, falling back to local store"), l()); }` and mod/673.js:26-34 — `k = () => null !== o ? Promise.resolve() : (l    (l = (async () => { c = n(); o = c && (await u(c, 900)) ? "remote" : "mock"; }) ()), l)` — once `o` is non-null the probe never runs again. mod/673.js:68-70 — `const f = await y.json(); if (!f.ok) throw new Error(f.error ?? "Server error.");` never checks `y.ok`. The banner in scope reads that latched value: mod/1620.js:102-112 — `!globalThis.__PLAYORA_CONFIG__?.backendUrl && "mock" === (0, b.backendMode)() && ... children: L("offlineBanner")`.	One transient network blip — a tunnel, a lift, a 502 from the proxy — flips the whole session to the local store for as long as the tab is open. Every subsequent write (a court reservation, a review, a follow) is written only to localStorage and reported as success, then silently diverges from the server. The dock's offline banner never clears even after connectivity returns, so the user is told the app is offline while it appears to work. A gateway error page also surfaces as a raw `SyntaxError` string through `localizeStoreError` in mod/672.js:44.	In mod/673.js make the fallback time-boxed: record the failure tir reset `o` to null after a backoff so `k()` re-probes, and check `y.ok y.json()`. Have the dock banner at mod/1620.js:102 reflect a live rather than the latched `backendMode()`.
P-CTEST-3 High	./(tabs)/feed, ./chat/[gameId], ./friends/index, ./messages/index	Error handling	Every screen in this group has no error branch: one failed backend call leaves a permanent loading spinner with no retry Each loader should catch, render a failure state and offer a retry — the shared helper for exactly this already exists in 9001 and is used by other pairs of the app. Here a rejection is an unhandled promise rejection and the screen is stuck. Evidence: mod/1622.js:15-23 — `U = useCallback(async () => { if (e && (v(await fetchFeed(e.id)), H(await fetchStoryTray(e.id)), "female" === c??.audience)) ... )}` — no try/catch around the two loads; on rejection `T` stays `null` and mod/1622.js:150 `null === T ? jsx(ActivityIndicator) : ...` spins forever. mod/2456.js:11-13 — `L = useCallback(async () => { t && M(await fetchConversations(t.id)) }, [t])` with mod/2456.js:61 `null === z ? jsx(ActivityIndicator)` . mod/2378.js:15-17 — `V = useCallback(async () => { t && (q(await listFriends(t.id)), D(await fetchSuggestedFollows(t.id)).catch(() => [])) }` — the suggestions call is guarded, `listFriends` is not, and mod/2378.js:134 `null === l ? jsx(ActivityIndicator)` . mod/1843.js:17-22 — `K = useCallback(async () => { if (!e) return; await ensureChatSeed(e); const t = await fetchChatMessages(e, E, s?.id); (O(t), L(1), ...) )}` — `L(1)` is unreachable if either await rejects, and mod/1843.js:70 `V ? jsx(ActivityIndicator) : ...` spins forever. None of the seven in-scope modules contains the string `retry`, and none imports the shared `GateScreen`/`RetryButton` from mod/9001.js:65-123 (only mod/2503.js does).	Any backend failure — a write that throws `E_STORAGE_FULL` or `E_COULD_NOT_SAVE_CHANGES` from `Za` (mod/631.js:1772-1773), a hydration failure, a `forbidden` from a stale id, or a remote /rpc error once `backendUrl` is configured — leaves the user staring at a spinner on the feed, their message list, their friends list or a match chat, with no message, no retry and no way forward except force-quitting. Nothing is logged to the user and nothing is recoverable in-session.	Wrap each loader in try/catch, set an error state, and render `Gat kind: "error", onRetry })` from 9001 (or an inline `RetryButton`) in: bare `ActivityIndicator`; make sure the `loading` flag is cleared in `L(1)`/`E(1)` run on the failure path too. Follow the pattern alread mod/2379.js:33-38.
P-CSEC-24 High	./(tabs)/feed.tsx	Data exposure	The feed silently falls back to every post in the system, ignoring the author's profile visibility The feed should only ever show posts the viewer is entitled to see. Instead, whenever the viewer's followee set yields no post by another user — which is the state of every brand-new account and of anyone whose followees have not posted — the code discards the follow graph and returns `Qt.posts`, the entire global post table, filtered only by audience partition, mute and block. Evidence: mod/631.js:14483-14484 — `n = Qt.posts.filter((e) => t.has(e.author_id)).filter(i); return (n.some((t) => t.author_id !== e) ? n : Qt.posts.filter(i))` where `t = new Set([...\$d(e), e])` (accepted followees plus self) and the filter `i` at 631.js:14480 checks only audience partition, mute and block: `(t.author_id === e    ia(a, aa(t.author_id))) && (t.author_id === e    (!l(e, t.author_id) && !Xd(e, t.author_id)))`. There is no `profile_visibility` check, and the fallback is not gated on the demo flag — compare the story tray, whose analogous fallback is explicitly gated: mod/631.js:14878 `if (Fa()) && ! [...t].some(...))`. `profile_visibility` is a real access control everywhere else: mod/631.js:14121 (`mockGetFollowList` throws `followers_only`, 14152 (`mockGetSuggestedFollows` skips private), 14202 (`mockSearchPlayers` skips private). Female-audience accounts default to restricted: mod/631.js:562 `_t = { profile_visibility: "followers", show_online: !1, allow_messages: "followers" }`.	A freshly registered account that follows nobody is shown the posts of every user in its audience partition, including users whose `profile_visibility` is `followers` or `private`. Because female-audience accounts default to `followers`, the سيادات partition's most privacy-sensitive users have their captions and media shown by default to any stranger in that partition who has not yet followed anyone. The check is absent in 631 itself, so it would still be a hole behind a real server.	In `mockGetFeed`, add the visibility predicate to filter `i`: for `t.aut e`, require `""public" === al(t.author_id).profile_visibility    Vd(e, t.a Qd(e).has(t.author_id)`. Either delete the `Qt.posts` fallback entir 1622's `emptyFeed` empty state with its `find players to follow` bu job, or gate the fallback on `Fa()` the way the story tray does.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-25 High	./(tabs)/feed.tsx	Data retention / right to erasure	Account deletion leaves the user's posts, comments, direct messages and social graph in the store Deleting an account should remove or irreversibly anonymise the content the user authored and the relationships they formed. Here it removes the identity rows but leaves every piece of authored social content and the whole follow/friend/block graph behind. Evidence: mod/631.js:2424-2442 — `r.mockDeleteAccount = async (e) => { ... Qt.users = Qt.users.filter(...); Qt.profiles = Qt.profiles.filter(...); Qt.bookings = ...; Qt.reviews = ...; Qt.notifications = ...; Qt.gamePlayers = ...; Qt.chatMessages = Qt.chatMessages.filter((e) => e.author_id !== `self-\${t}`), await Promise.all(([Za(q, ...), Za(Y, ...), Za(W, ...), Za(K, ...), Za(\$, ...), Za(X, ...), Za(ee, ...)]) ...)`; `Qt.posts`, `Qt.postComments`, `Qt.postLikes`, `Qt.postSaves`, `Qt.postShares`, `Qt.postReports`, `Qt.dmConversations`, `Qt.dmMessages`, `Qt.follows`, `Qt.friendships`, `Qt.blocks` and `Qt.mutes` are all untouched. Those rows keep rendering: mod/631.js:14363 `author_name: Td(e.author_id)` feeds the feed card at mod/1622.js:271, mod/631.js:14617 `other_name: Td(a)` feeds the DM header at mod/2455.js:107, and mod/631.js:14131 `name: Td(e)` feeds the follow list at mod/2376.js:113. With the profile row gone, `Td` (mod/631.js:13080) degrades to `bd(e)    "Member"`.	After a user deletes their account, their feed posts and captions stay published, their direct messages stay readable in the other party's thread, and they remain a row in other users' follower and following lists — now rendered as the generic name 'Member' with a working Follow button. The deletion is advertised as removing the account but the user's words remain in public view, which fails the erasure promise and leaves orphaned graph rows that inflate follower counts forever.	Extend `mockDeleteAccount` to purge or tombstone `Qt.posts`, `Qt.postComments`, `Qt.postLikes`, `Qt.postSaves`, `Qt.postSha` `Qt.postReports` by the user id; delete `Qt.dmMessages` they se `Qt.dmConversations` whose `participant_ids` include them; and rows from `Qt.follows`, `Qt.friendships`, `Qt.blocks` and `Qt.mute` each of those collections in the same `Promise.all`.
P-CARCH-7 High	./(tabs)/feed.tsx	Read amplification / missing pagination	The feed builds every post in the database and decorates each one with five full-table scans `mockGetFeed` accepts no cursor, offset or limit, and the fallback branch at mod/631.js:14484 deliberately widens to `Qt.posts` (every post ever written) whenever the viewer follows nobody who has posted — which is the state of every brand-new account. Counts are recomputed per post instead of being aggregated once into maps. Evidence: mod/631.js:14483-14486 — `n = Qt.posts.filter((e) => t.has(e.author_id)).filter(i); return (n.some((t) => t.author_id !== e) ? n : Qt.posts.filter(i)).sort(...).map((t) => sl(t, e));`. mod/631.js:14360-14373 — `sl` computes `author_name: Td(e.author_id)` (profiles scan, mod/631.js:13080), `author_username: Qt.profiles.find(...)` , `like_count: Qt.postLikes.filter(...)` , `comment_count: Qt.postComments.filter(...)` , `share_count: Qt.postShares.filter(...)` , `liked: Qt.postLikes.some(...)` , `saved: Qt.postSaves.some(...)` — seven scans per post over five different tables.	A new user's very first feed load is the worst case in the app: it materialises the entire global post table and runs ~7 full-table scans per post. At 10,000 posts and 50,000 likes that is roughly 350 M comparisons in one synchronous pass — the tab freezes. Existing users pay the same cost proportional to how much their followees have posted, with no ceiling.	Add `{limit, cursor}` to `mockGetFeed` (mod/631.js:14476) and sl calling `sl`; precompute `Map<post_id, count>` for likes, commen and saves once per call and have `sl` read from them; replace thn unbounded `Qt.posts.filter(i)` fallback with a bounded `popular rec query`.
P-CARCH-8 High	./(tabs)/index.tsx	Read amplification / N+1	The upcoming-games list is O(games × bookings) and the home screen runs it twice on every focus There is no index from game_id to its bookings. Each of the five per-game derived counts re-scans the global bookings table, and the expiry/waitlist sweep runs once per game rather than once per pass. Nothing memoises the result between the two `Mi` invocations a single home-screen focus triggers. Evidence: mod/631.js:2688 — `await Promise.all(n.map((e) => O(e.id)))` inside `Mi`. mod/631.js:2654-2657 — `Oi = (e) => { const t = hi(e); return t ? Xt(e, () => Di(t)) : Promise.resolve(!1); }`; `hi` is `gi().find(...)` (mod/631.js:2503) and `gi()` is `Fa() ? [...La, ...Qt.games] : Qt.games` (mod/631.js:2502), so demo mode reallocates the whole games array per lookup. `Di` (mod/631.js:2613+) iterates `for (const i of Qt.bookings)` and then loops `while (ki(e.id, t) < e.max_players)` . mod/631.js:2668-2677 — `li` adds `bookings_count: ki(e.id)`, `waitlist_count: vi(e.id).length`, `pending_count: Qt.bookings.filter(...)` , plus `Ri(e.id, a)` — four more full scans of `Qt.bookings` per game (`ki` mod/631.js:2507, `vi` mod/631.js:2508-2511). mod/1677.js:42 calls `fetchDiscover` (which reaches `Mi` via `Gi`, mod/631.js:15399) and mod/1677.js:60 calls `fetchUpcomingGames` — `Mi` again — in the same pass; mod/1677.js:66-71 fires that pass on every `useFocusEffect` and again on every `useLiveRefresh` tick.	At 50 open games and 5,000 bookings that is ~1.25 M row comparisons per `Mi` call, doubled per home focus, on the main thread. Kuwait-scale traffic (a few hundred games, tens of thousands of bookings) puts the home tab into multi-second jank every time the user switches back to it, and the same `Mi` runs again from `discover` and the recommender.	Build a `Map<game_id, booking[]>` once at the top of `Mi` (mod/631.js:15399) and pass it into `Di`, `li`, `ki`, `vi` and `Ri` instead of re-filtering `Qt.games` hoist `gi()` out of `hi` so the demo-mode concat happens once pe than per lookup; and have `mockGetDiscover` reuse the `Mi` resu computes in `Gi` instead of letting the screen ask for it a second t

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-26 High	./(tabs)/profile.tsx	Input validation	mockUpdateProfile writes full_name and bio with no sanitiser and no length cap, and the name input has no maxLength Profile editing is the one place a display name is user-controlled after account creation, and it is the only name path with zero normalisation. A name can carry control characters, newlines, bidi overrides and unbounded length; a bio is capped only by a client-side <code>.slice(0,160)</code> that the remote path never reaches. Evidence: mod/631.js:2384-2388 is the entire write: <code>`const i = t, n = (0, d.default)(i, B); ((Qt.profiles[a] = Object.assign({}, Qt.profiles[a], n)), await Za(Y, Qt.profiles), ...)`</code>. <code>`B`</code> is a deny-list, not an allow-list — mod/631.js:465 <code>const B = ["role", "consent", "consent_updated_at", "id", "audience"];</code> — so every other field, <code>full_name</code> and <code>bio</code> included, is merged verbatim. No <code>sanitizeName</code>, no <code>sanitizeText</code>, no <code>slice</code> appears anywhere in the function (mod/631.js:2366-2390). The caller passes raw state: mod/1805.js:552-556 <code>`full_name: Y, bio: ee.slice(0, 160),`</code> and the name field has no length attribute at all: mod/1805.js:326 <code>`(0, L.jsx)(j.Input, { label: X("fullName"), value: Y, onChangeText: Z },)`</code>. The creation paths do sanitise, which shows the omission is an oversight: mod/631.js:968 <code>`full_name: (0, v.sanitizeName)(a ?? t.split("@")[0])    t.split("@")[0];`</code> and mod/631.js:4771 <code>`const a = (0, v.sanitizeName)(t.full_legal_name);`</code>. The plain sign-up path is no better: mod/631.js:2292 <code>`full_name: a    n.split("@")[0];`</code>.	<code>full_name</code> is the most widely rendered user string in the app — feed post headers, story tray, chat, notifications, lineups, leaderboards, player cards. A user signs up with a sanitised name and then edits it to anything: a multi-kilobyte name, a name containing newlines, or a bidi-spoofed name that renders as someone else's. Because <code>Qt.profiles</code> is serialised as one JSON row per write (mod/631.js:2386 <code>`await Za(Y, Qt.profiles)`</code>), an oversized name also inflates every subsequent profile write.	In <code>mockUpdateProfile</code> (mod/631.js:2366) apply <code>`sanitizeName(t.full_name, t.bio, 500)`</code> with a non-empty check and <code>`sanitizeText(t.bio, 500)`</code> before the write. Add <code>denyList</code> to <code>denyList</code> to reject unknown keys with an allow-list instead of the deny-list at mod/631.js:465, and add <code>`maxLength: 80`</code> to the <code>fullName</code> Input at mod/1805.js:326.
P-CSEC-27 High	./_layout.tsx (root route table, module 18)	Information disclosure / broken 404	expo-router's development Unmatched-Route and /_sitemap screens ship in the production bundle These are expo-router's development affordances. They are outside the app's auth gate and outside module 9001's role checks, they are English-only with no Arabic, and their bundled PNGs are blocked by the app's own CSP so they render broken. Evidence: Booting the shipped index.html at <code>`/no-such-page`</code> renders <code>`Unmatched Route   Page could not be found.   http://localhost/no-such-page   Go back   •   Sitemap`</code>, plus <code>`Refused to load the image 'http://localhost/assets/node_modules/expo-router/assets/unmatched.20e71bdf79e3a97bf55fd9e164041578.png' because it violates ... img-src`</code>. Booting at <code>`/_sitemap`</code> — with <code>`role: 'guest`</code>, i.e. signed out — renders a file browser of the route table: <code>`index.tsx   {auth}/_layout.tsx   sign-in.tsx   sign-up.tsx   (tabs)/_layout.tsx   feed.tsx   ...`</code>. index.html contains three <code>`+not-found`</code> references.	Any visitor — signed out — can enumerate the complete route table at <code>`/_sitemap`</code>, including the <code>`/admin/*`</code> and <code>`/organizer/*`</code> screens, turning route discovery into a single URL. Any user who mistypes a URL or follows a stale share link lands on a debug screen showing their raw URL and broken images instead of a Playora 404. No suite opens either route.	Add a real <code>`+not-found`</code> route to the route table in module 18 that product's own <code>EmptyState</code> in both locales, and strip the <code>`/_sitemap`</code> from the compiled route table before shipping.
P-CQUAL-3 High	./awards/[gameId].tsx	Date/time handling	The awards header always reads "Voting closes Just now" because formatRelative cannot express a future time <code>`formatRelative`</code> is a past-only "x minutes ago" formatter (its other callers, mod/2458.js:198 and mod/2372.js:116, pass <code>`created_at`</code>). The awards screen is the only caller that passes a future instant, and the clamp turns the entire 48-hour window into "Just now". Evidence: mod/1835.js:56 <code>`? H("voteClosesLabel", { when: (0, B.formatRelative)(N.closes_at) })`</code> (copy: <code>"Voting closes %(when)%"</code>). mod/1626.js:8-10 <code>`e.formatRelative = (n) => { const u = Date.now() - new Date(n).getTime(), s = Math.max(0, Math.round(u / 6e4)); if (s < 1) return (0, t.t)("JustNow"); ...`</code> — <code>`Math.max(0, ...)`</code> clamps every future timestamp to zero. <code>`closes_at`</code> is always in the future while the ballot is open: mod/631.js:14959 <code>`closes_at: new Date(new Date(a).getTime() + S.DEFAULT_VOTING_WINDOW_MS).toISOString()`</code> with <code>`DEFAULT_VOTING_WINDOW_MS = 1728e5`</code> (48 hours).	For the whole voting window the subtitle says "Voting closes Just now" (Arabic: "يُغلق التصويت الآن"). Users are told the ballot is about to shut from the moment it opens, so the deadline is worthless — some will not bother voting, and nobody can tell how long they actually have. There is no other deadline affordance on this screen.	Add a future-aware branch to <code>`formatRelative`</code> in mod/1626.js (sig) and use an "in %(n)" set of keys, or render <code>`closes_at`</code> on mod/1626.js:8-10 with the countdown component already used by the MVP panel (<code>`CountdownTimer`</code>, referenced from the <code>`/game/[id]`</code> panel) instead of <code>`formatRelative`</code>.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-4 High	/awards/[gameld].tsx	Error handling	Award votes fail silently: no error is caught, shown, or even mapped to a message Every write path on the awards screen assumes success. On rejection the sheet is already closed, `await G()` never runs so the UI is not refreshed, and the promise rejects unhandled. The project's own convention (CLAUDE.md: throw `E_SOME_CODE`, map it in 674, add `seSomeCode` to both locales) is not followed by any of the five award error codes. Evidence: mod/1835.js:24-28 `J` = async (a) => { if (!T !t !D) return; const s = D.key; (O(null), await (0, R.castAwardVote)(T.id, t, s, a), await G()); } — `O(null)` closes the sheet first, then the await, with no try/catch. The backend throws bare lowercase codes: mod/631.js:15143-15148 `if (!r    !O(r)) return o("voting_closed"); ... if (!s.includes(e)) return o("not_participant"); if (i === e) return o("no_self_vote"); if (!s.includes(i)) return o("invalid_nominee"); if (!T(t).includes(a)) return o("invalid_award");`. None of `voting_closed`, `not_participant`, `no_self_vote`, `invalid_nominee`, `invalid_award` appears in the error map in mod/674.js, so `storeErrorText` (mod/674.js:305-313) would return the raw code even if something rendered it. The same pattern repeats for the other four mutations on this screen: revoke at mod/1835.js:124-126 has no try at all, `createCustomAward` at mod/1835.js:220-232 and `publishAwards` at mod/1835.js:245-254 use try/finally with no catch.	The realistic case is mundane and common: the 48-hour window lapses while the nominee sheet is open, or the organizer publishes early (mod/631.js:15171). The user taps a player, the sheet closes, and absolutely nothing happens — no toast, no error, no state change. They will tap again and again believing the app is broken. `voting_closed` is by far the most likely rejection and it is the one with the least feedback.	Wrap `J` (mod/1835.js:24) and the revoke/create/publish handler: try/catch, surface `classifyError`/`RetryButton` from mod/9001.js, the five codes in mod/631.js:15143-15148 to `E_*` form with entries in mod/674.js and both locales in 909.
P-CARCH-9 High	/awards/[gameld].tsx	Background work in a read path	The whole award lifecycle only advances when somebody happens to open the ballot screen Opening voting, sending the "voting is open" fan-out, sending the "closing soon" nudge, tallying and publishing are all side effects of a GET. There is no scheduler, no cron, and no write-path trigger at match completion. Evidence: mod/631.js:14951-14975 `Dl` = async (e) => { if (!"completed" !== Ji(e)) return null; let t = Qt.awardVoting.find((t) => t.match_id === e.id); if (!t) { ... Qt.awardVoting.push(t), await Za(Qe, Qt.awardVoting)); ... for (const t of Si(e.id)) await bi({ ... type: "award_voting_open", ... }); } return t; }`. mod/631.js:15025-15048 `Il` = async (e) => { const t = await Dl(e); if (!t && !t.published && Date.now() >= new Date(t.closes_at).getTime()) && (await Rl(e, t), t && !t.published && !t.close_nudged)) { ... type: "award_voting_closing" ... } return t; }`. Both are invoked only from read paths: mod/631.js:15054 inside `mockGetAwardBallot`, mod/631.js:15209 inside `mockGetMatchAwards`, and mod/631.js:15133 inside `mockCastAwardVote`.	If no participant opens `/awards/<id>` after the match ends, nobody is ever told voting opened, so nobody votes; and if nobody opens it after `closes_at`, results are never tallied, `award_results` is never sent, and the winners never reach the leaderboard or passport — the ballot sits in `voting` forever. Whoever does open it first silently pays the cost of an N-participant notification fan-out inside their page load (`for (const t of Si(e.id)) await bi(...)`), each `bi` re-serialising the entire notification array via `Za`). This does not become correct by moving to a server; it becomes a GET that writes.	Move `Dl`'s open-and-announce and `Il`'s close/tally/publish into completion write path and a scheduled job, and reduce `mockGetAwardBallot` (mod/631.js:15050) to a pure read.
P-CSEC-28 High	/awards/leaderboard.tsx	Audience partition / cross-partition exposure	Award leaderboard screen omits viewerId, so the partition defaults to male and female users are shown male players A partition key that defaults to a real partition rather than failing closed, combined with one screen that forgets to pass it. The two call sites of the same API disagree. Evidence: The backend partitions on a caller-supplied `viewerId` and silently defaults it: mod/631.js:15286-15289 ``` const a = aa(e.viewerId); return [...t.entries()] .map([(e, t)] => ({ user_id: e, name: Td(e), total: t.total, mvp: t.mvp }))) .filter((e) => ia(aa(e.user_id), a)) ``` and `aa` returns `""male""` for a missing id: mod/631.js:750-757 `const aa = (e) => e ? ... : "male"`. The leaderboard screen never passes it — mod/1836.js:17 `M(await (0, S.fetchAwardLeaderboard)({ sport: H ?? void 0, season: R ?? void 0 })))` — and the module imports no auth hook (`useAuth` does not appear in mod/1836.js). The home tab gets it right: mod/1677.js:57 `fetchAwardLeaderboard({ viewerId: e.id })`.	A female user opening /awards/leaderboard sees the top 50 male players by name — a direct cross-partition identity leak in an app whose core safety model is that the two audiences never see each other. She also never sees her own partition's leaderboard or her own standing, so the screen is simultaneously broken for her. Because `name` comes from `Td` (mod/631.js:13080), any of those users with a blank profile name is listed by email address.	Pass the viewer in mod/1836.js:17 (`fetchAwardLeaderboard({ sport: H ?? void 0, season: R ?? void 0 })))`, and make the default fail closed in `mockGetAwardLeaderboard` (mod/631.js:15286) — require `viewerId` to be passed, rather than letting `aa(undefined)` resolve to the male partition.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-29 High	<code>./booking/[id].tsx</code>	Refund handling / inconsistent policy	Organizer cancellation refunds seat fees in full but forfeits split-plan contributions from the same players Two player-money paths for the same event resolve oppositely. Seat fees are refunded unconditionally because the organizer cancelled; court split-plan shares created by <code>`mockCreatePaymentPlan`</code> are routed through <code>`Mr`</code> , which was written for an organizer voluntarily cancelling a court and therefore charges the venue's late-cancellation penalty — to the players, not the organizer who cancelled. Evidence: <code>mod/631.js:3775</code> — match cancellation forces eligibility for seat payments: <code>`for (const e of o) await Br(n, e, lO);`</code> (the third argument is the <code>`refundEligible`</code> flag, hardcoded true). <code>mod/631.js:3777-3781</code> — court payments are delegated: <code>`if (n.court_booking_id) try { await Mr(t, n.court_booking_id, `Match cancelled: \${r}`); } catch (a) { ... }`</code> . <code>mod/631.js:7465-7474</code> — <code>`Mr`</code> applies the venue's cutoff to the players: <code>`r = (0, H.freeCancellationAllowed)(l.starts_at, n?.cancellation_cutoff_hours ?? 6); ... for (const e of o) { const t = "paid" === e.status; ((lr && t)    ((e.status = t ? "refunded" : "expired"), t && (... await ac(e.payer_id, e.amount_kwd, e.id))), ...`</code> — when <code>`lr`</code> and the row is paid, the short-circuit skips the refund entirely and only a <code>`payment_forfeited`</code> notice is sent (7479).	A player who paid their share of the court through the split plan loses it when the organizer cancels inside the venue's cutoff (default 6 hours), through no act of their own, while a player in the same match who paid a seat fee is made whole. The forfeited player sees only a <code>`payment_forfeited`</code> notification with no recourse, and <code>./refunds.tsx</code> shows the row as kept.	Pass the cancellation cause into <code>`Mr`</code> and skip the forfeit branch if cancellation originated from <code>`cancelGameLocked`</code> (<code>mod/631.js:37</code>) organizer-initiated match cancellation should refund the players a the penalty to the organizer, not to the payers.
P-CSEC-30 High	<code>./booking/venue/[id].tsx</code>	Commercial and partial financial data exposure	mockGetVenueBookingDetail returns the full venue profile — commission terms, IBAN digits and staff roster — to any consumer The consumer booking endpoint returns the unredacted <code>`venueProfiles`</code> row while the privileged admin endpoint returns a hand-built slim DTO. The redaction rule exists in the codebase and is simply not applied on the path with the widest audience. Evidence: <code>mod/631.js:6971</code> — <code>`r.mockGetVenueBookingDetail = async (e) => { / `await ei(), await Dr();` / `const t = fi().find((t) => t.id === e);` / <code>mod/631.js:6975</code> <code>`? { venue: t, profile: hr(e) ?? null, courts: Qt.courts.filter((t) => t.venue_id === e && t.active) }`</code>. <code>`hr`</code> is the raw record: <code>mod/631.js:6817</code> <code>`hr = (e) => Qt.venueProfiles.find((t) => t.venue_id === e);`</code> and that record carries <code>`owner_id`</code>, <code>`commission_type`</code>, <code>`commission_value`</code>, <code>`payout_iban_last4`</code> and <code>`staff`</code> (created at <code>mod/631.js:7010-7025</code>). No caller argument exists — <code>mod/671.js:512</code> <code>`e.fetchVenueBookingDetail = (o) => t.store.mockGetVenueBookingDetail(o);`</code> — and the consumer screen calls it directly: <code>mod/1842.js:22</code> <code>`const t = await (0, S.fetchVenueBookingDetail)(e);`</code>. The admin list of the very same records is deliberately redacted, with the reason written down at <code>mod/631.js:7095</code>: <code>`// ADM1 (F-ADM1-22/31): returns every venue registration ... as a slim DTO — no staff list, IBAN digits or policy text for a list view.`</code></code>	Every visitor to a venue booking page receives the venue's platform commission type and value, the last four digits of its payout IBAN, the owner's user id and the full staff roster with roles and join dates. Commission terms are commercially sensitive between the platform and its venues, and the IBAN fragment plus owner identity is useful material for payout-redirection social engineering.	Build an explicit public DTO in <code>`mockGetVenueBookingDetail`</code> (<code>mod/631.js:6971</code>) exposing only <code>`cancellation_policy`</code> , <code>`cancellation_cutoff_hours`</code> , <code>`amenities`</code> , <code>`photos`</code> and <code>`auto_accept`</code> mirroring the slim shape already written for <code>`mockGetPendingVer`</code> the full record behind <code>`Sr(caller, venueId)`</code> .
P-CSEC-31 High	<code>./chat/[gameId].tsx</code>	Impersonation and message disclosure	Match chat has no membership check on read and takes the sender's identity from the request body <code>`mockGetChatMessages`</code> should verify the caller holds a live booking for the game (the helper <code>`Fi`/`Ri`</code> already computes that) before returning the transcript, and <code>`mockSendChatMessage`</code> should derive <code>`author_id`</code> from the caller rather than from <code>`payload.user_id`</code> , then apply the same membership test. Evidence: Read: <code>mod/631.js:2855</code> — <code>`r.mockGetChatMessages = async (e, t, a) => { / `await ei(),` / <code>`Qt.chatMessages` / `filter((a) => a.game_id === e && a.channel === t)`</code> — the third argument <code>`a`</code> is used only to recompute <code>`is_self`</code>, never to authorize. Write: <code>mod/631.js:2862</code> <code>`const Gi = async (e) => {` with <code>mod/631.js:2868</code> <code>`author_id: `self-\${e.user_id}\`,`</code> and <code>mod/631.js:2869</code> <code>`author_name: (0, v.sanitizeName)(e.user_name);`</code> — there is no caller parameter at all; the author is whatever the payload says. The facade passes the payload straight through: <code>mod/671.js:415</code> <code>`e.sendMessage = (o) => t.store.mockSendMessage(o);`</code>. Team chat, by contrast, gates both sides on membership: <code>mod/631.js:9055</code> <code>`r.mockGetTeamChat = async (e, t) => (await ei(), await lo(e, t),` and <code>mod/631.js:9063</code> <code>`if ((await ei(), await lo(t, e), "system" === a)) throw new Error("E_NOT_ALLOWED");`</code>.</code></code></code>	Anyone with a game id reads the full private chat of a match they never joined. Worse, anyone can post into any match's chat under any <code>`user_id`</code> and any <code>`user_name`</code> , so messages can be attributed to another player — including the organizer — with no way for readers to tell. <code>`mockShareLineupToChat`</code> (<code>mod/631.js:6781</code>) already routes through <code>`Gi`</code> with a caller-supplied display name, showing how easily forged posts enter the same stream.	Add a caller parameter to <code>`mockSendChatMessage`/`Gi`</code> (<code>mod/631.js:2855</code>) and build <code>`author_id`/`author_name`</code> from the session profile, ignoring <code>`e.user_id`/`e.user_name`</code> . Add a participant assertion to both <code>`mockGetChatMessages`</code> (<code>mod/631.js:2855</code>) and the send path, including <code>`lo`</code> in the team-chat code.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-32 High	./chat/[gameId].tsx	Data exposure / private-group membership	Match group chat can be read and written by any signed-in user who knows the game id Match chat is the app's private group conversation, including the `team_a` / `team_b` channels, and it has no participation check on read or write. It is also the one game-scoped read that skips the audience partition every other game path enforces. Evidence: mod/631.js:2855-2861 — the entire function, with no caller check of any kind: ``` r.mockGetChatMessages = async (e, t, a) => (await ei(), Qt.chatMessages .filter((a) => a.game_id === e && a.channel === t) .map((e) => Object.assign({}, e, { is_self: a ? e.author_id === `self-\${a}` : e.is_self }))) ``` No booking lookup, no `ra(a, game.audience)` partition check (contrast `Ni` at mod/631.js:2701 which does call `ra`), no block check. The screen passes the raw route param: mod/1843.js:20 `const t = await fetchChatMessages(e, E, s?.id)`. Sending is equally open — `Gi` (mod/631.js:2862-2876) writes a message for any `game_id` and takes the author's name from the client: mod/631.js:2869 `author_name: (0, v.sanitizeName)(e.user_name)`.	Anyone with a game id — shared in an invite link, a referral, or a screenshot — reads the whole conversation of a match they are not in, across the gender partition, including messages from users who blocked them. They can also post into it under any display name they choose, since the author name is client-supplied. Moving this code to a server changes nothing: the authorization is absent from the function, not merely bypassable.	In `mockGetChatMessages` (mod/631.js:2855) and `Gi` (mod/631.js:2862-2876) load the game with `hi(gameId)`, call `ra(callerId, game.audience)` before returning or appending. Derive `author_name` from `Qt.profiles` using the caller id instead of trusting `e.user_name`.
P-CSEC-33 High	./chat/[gameId].tsx	Attribution integrity / authorization	Game chat lets the caller dictate the author name attached to a message and never checks that the sender is in the game Unlike team chat and the feed, game chat trusts the client for both the audience check and the identity label. This is not the browser-backend caveat: the function itself contains no membership assertion and no profile lookup, so the same hole exists verbatim after this code moves to a server. Evidence: mod/631.js:2862-2875 is the whole write: `const Gi = async (e) => { await ei(); const t = { id: ea(), game_id: e.game_id, channel: e.channel, author_id: `self-\${e.user_id}`, author_name: (0, v.sanitizeName)(e.user_name), avatar_seed: 99, is_self: !0, body: (0, v.sanitizeText)(e.body, 1e3), created_at: new Date().toISOString(), }; return (Qt.chatMessages.push(t), await Za(ee, Qt.chatMessages), t); }` — there is no membership or booking check on `e.game_id`, and `author_name` comes from the argument rather than from the profile. The screen supplies it from its own state: mod/1843.js:31-36 `const t = await (0, _sendChatMessage)({ game_id: e, channel: E, user_id: s.id, user_name: l?.full_name    "You", body: n, });`. The facade is a straight pass-through: mod/671.js:415 `e.sendChatMessage = (o) => t.store.mockSendChatMessage(o);`. Every comparable surface does it correctly — mod/631.js:9062-9075 `r.mockPostTeamChat = async (e, t, a, i, n) => { if ((await ei(), await lo(t, e), ...) resolves the name server-side with `author_name: or(e),` and the feed derives it with `author_name: Td(e.author_id),` (mod/631.js:14362).	A caller can post into any game's chat without being a participant, and can attach any display name to the message. Because author_name is frozen into the stored row rather than resolved at read time, the impersonating label persists for every reader of that chat and survives the real user later changing their name.	In mockSendChatMessage (mod/631.js:2862) assert participation game the way mockPostTeamChat does (`await lo(...)`) equivalent membership, reusing the confirmed-booking lookup already present in mod/631.js:2890), and derive `author_name` from the store with `Td(e.user_id)` instead of accepting `e.user_name`; drop user_name from the facade signature at mod/671.js:415 and the call at mod/1843.js:31-36.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-34 High	/clubs/[id].tsx	Access control / privilege escalation	Club invitations are unbound bearer tokens: the invited email is never checked when the token is redeemed The invite record carries an `email` (mod/631.js:13215) and the member row is created with `bd(e)    a.email` (13244), so the design clearly intends the invite to be bound to one recipient. Redemption ignores that binding entirely, and `mockAcceptClubInvitation` also bypasses `canTransitionMembership` (module 659) — a `revoked` member is set straight back to `active` with the invited role. Evidence: mod/631.js:13230-13248 — redemption matches on the token alone and never compares the caller to `a.email`: <pre>... 13230 r.mockAcceptClubInvitation = async (e, t) => { 13231 await ei(); 13232 const a = Qt.clubInvites.find((e) => e.token === t.trim()); 13233 if (!a !(0, I.isInviteRedeemable)(a, Date.now())) throw new Error("invitation_invalid"); ... 13238 ? ((n.status = "active"), (n.role = "owner" === n.role ? "owner" : a.role)) ... Module 659 shows the redeemability test has no email component: `e.isInviteRedeemable=(n,s)=>'pending'===n.status&&s<new Date(n.expires_at).getTime();` mod/1847.js:339-342 makes it worse by inventing a recipient when the field is blank: ... 339 const e = await (0, B.inviteClubMember)(n.id, Z.id, { 340 email: W.trim() "invitee@club.app", 341 role: K, 342 }); ... and mod/1847.js:354-358 renders the raw token on screen for copying.</pre>	An invite intended for one person's email can be redeemed by anyone who sees the token — it is displayed in plaintext in the club screen, shared over WhatsApp, and returned inside `pending_invites` to every member holding `members:invite` (coach and captain included, mod/631.js:13159-13161). Because `a.role` can be `admin` or `manager`, a forwarded or shoulder-surfed token hands an outsider org-level permissions including `roles:manage` and `members:remove`. A previously revoked member can also re-enter at their old role.	In mod/631.js:13230 compare the caller's email (`bd(e)`) against case-insensitively and throw `invitation_invalid` on mismatch; route acceptance through `canTransitionMembership` before flipping `s` mod/1847.js:339 require a non-empty, validated email instead of `invitee@club.app`.
P-CARCH-10 High	/clubs/[id].tsx	Payment integrity / revenue leakage	Club subscriptions activate with no payment step and free trials can be restarted without limit There is no charge, no payment-method capture, no invoice, and no record of trials already consumed. `Qt.clubSubs` is filtered by `club_id` before the push, so the previous subscription row — the only evidence a trial was already taken — is destroyed on every call. `price_minor` is copied into the record and fed straight into the MRR figures shown at mod/1847.js:197-198. Evidence: mod/631.js:13256-13279 — a paid plan becomes `active` (or `trialing`) purely on request, and any prior subscription for the club is deleted first: <pre>... 13256 r.mockStartClubSubscription = async (e, t, a, i) => { 13257 (await ei(), Od(t, e, "subscriptions:manage")); 13258 const n = ft.find((e) => e.id === a); 13259 if (!n) throw new Error("plan_not_found"); ... 13261 o = i.trial && n.trial_days > 0, 13263 d = o ? "trialing" : "active", ... 13277 (Qt.clubSubs = Qt.clubSubs.filter((e) => e.club_id !== t)), 13278 Qt.clubSubs.push(f), ... mod/1847.js:225-229 — the trial flag is client-supplied and the button is unconditional: ... 225 onPress: () => 226 Y(`plan-\${e.id}`, () => 227 (0, B.startClubSubscription)(n.id, Z.id, e.id, { trial: e.trial_days > 0 })), 228), ... </pre>	A club owner taps "Start trial", waits for `trial_end` to pass, taps it again, and is back in a fresh trial indefinitely — the paid tier is effectively free forever. Meanwhile the dashboard reports MRR and monthly value for subscriptions that were never billed, so any revenue reporting built on `mrr_minor` is fiction.	Persist a per-club `trial_used_at` (or keep cancelled subscription instead of deleting them at mod/631.js:13277) and reject `i.trial` if it has already been consumed; derive `o` from server state rather than client's `{trial:...}` argument. Gate the transition to `active` on a confirmed payment authorisation before writing the row.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-35 High	./clubs/[id].tsx	Payment integrity / revenue loss	Club paid plans activate without any charge, and the trial can be restarted indefinitely A 25.000 KWD/month plan is displayed to the user, activated on tap, and never billed — not at activation, not at `trial_end`, and not at `next_billing`, which no code path reads. Line 13277 deletes any existing subscription for the club before inserting the new one, so pressing "Start trial" again issues a fresh 14-day trial with no memory of the previous one. Evidence: mod/631.js:13256-13286 — `mockStartClubSubscription` runs no gateway call, no `tc`, no `Vr` and writes no ledger row; it only builds and stores the subscription: `d = o ? "trialing" : "active", ... l = { id: ea(), club_id: t, plan_id: n.id, status: d, price_minor: n.price_minor, ... next_billing: o ? s : (0, M.nextBillingDate)(r, n.interval, 1) }` then `(Qt.clubSubs = Qt.clubSubs.filter((e) => e.club_id !== t)), Qt.clubSubs.push(t)`. The plans are priced: mod/631.js:567-571 — `{ id: "pro", price_minor: 25e3, currency: "KWD", interval: "month", trial_days: 14 }, { id: "elite", price_minor: 6e4, ... }`. The screen advertises the price and starts the plan with one tap: mod/1847.js:219 — `0 === e.price_minor ? D("freePlan") : `\${z(e.price_minor)}\$D("perMonth")}` and mod/1847.js:227 — `onPress: () => Y(`plan-\${e.id}`, () => (0, B.startClubSubscription)(n.id, Z.id, e.id, { trial: e.trial_days > 0 })))`.	Every paid club tier is free forever, and the 14-day trial is unlimited. Any club manager can hold Elite entitlements permanently. The subscription table shows plans as "active"/"trialing" with future `next_billing` dates, so reporting reflects subscription revenue that will never arrive.	In `mockStartClubSubscription` (mod/631.js:13256), require a full payment for non-zero `price_minor` — run it through `tc` or the gateway before inserting the row — and record prior trial usage per club so the flag at 13261 is honoured only once. Add a billing sweep that acts on `next_billing` and moves the subscription to past_due when the current trial ends.
P-CSEC-36 High	./clubs/[id].tsx	Data exposure / PII	Club dashboard returns every member's email address to every member regardless of role Membership alone should not confer access to other members' login emails. The DTO has a per-field permission pattern and simply skips it for the one field that is PII. Evidence: Member rows are created carrying the raw email: mod/631.js:13113-13118 `Qt.clubMembers.push({ ... name: Td(e), email: bd(e), ... })` and mod/631.js:13243-13244 `name: Td(e), email: bd(e)    a.email`. `mockGetClubDashboard` gates only on being an active member of any role — mod/631.js:13144-13147 `const i = Dd(t, e) ... if (li && ln) throw new Error("forbidden")` — then returns the rows verbatim at mod/631.js:13149/13158: ... s = Qt.clubMembers.filter((e) => e.club_id === t && "active" === e.status), ... members: s, ... Every other sensitive branch in the same DTO is permission-checked (`pending_invites` on `members:invite`, `subscription`/`mrr_minor` on `reports:view`, mod/631.js:13159-13163) — `members` is not. The screen prints it: mod/1847.js:267-272 `!e.email && ... children: e.email`.	Joining a club — which any user can do with an invite token, mod/631.js:13230 — hands over a harvestable list of every other member's email address, rendered as a caption under each name. For a Kuwait sports club with mixed-audience membership this is a bulk PII disclosure and a ready-made spam/phishing list.	In `mockGetClubDashboard` (mod/631.js:13140), map `members` to a list of email addresses, dropping the projection that drops `email` unless `o.includes("members:invite")` is the caller's own, and remove the unconditional `e.email` capture at mod/1847.js:271.
P-CSEC-37 High	./clubs/create.tsx	Input validation / storage exhaustion	Club names are stored with no sanitiser and no length limit whatsoever, and the raw name is copied into the audit log mockCreateClub is the only user-facing create path in the backend with no cap and no sanitiser on its name field, so it accepts a string of any length containing any character. Evidence: mod/631.js:13093-13096: `r.mockCreateClub = async (e, t) => { await ei(); const a = t.name.trim(); if (!a) throw new Error("name_required");` — `t.name` is the only processing; no sanitization, no slice. The name is then persisted (`name: a,` mod/631.js:13100) and copied verbatim into the audit record: mod/631.js:13124 `await (0, w.logAudit)("club.created", (0, w.actorRef)(e), { club: n.id, name: a }`,`). The screen imposes no limit either — mod/2370.js:36-41 `(0, l.jsx)(x.Input, { label: M("clubName"), placeholder: M("clubNamePlaceholder"), value: P, onChangeText: W, })` with no maxLength, and the shared Input component (module 625) sets no default maxLength and renders no character counter. Every sibling create path does cap: mod/631.js:8538 `const a = (0, v.sanitizeText)(t.name, 60);` (teams), mod/631.js:7226 `const i = (0, v.sanitizeText)(a.name, 60);` (venues).	A single oversized club name is written into playora.mock.clubs.v1 and then again into playora.audit.v1. The audit writer swallows the resulting quota failure — mod/643.js:42-45 `await t.default.setItem(e, JSON.stringify(u.slice(0, a)));` } catch (t) { y(e, t); }` — so the general audit log silently stops recording for that device while the console shows only '[persist] write failed'. Control characters and bidi overrides in the name then render on every club card and club detail header.	In mockCreateClub (mod/631.js:13095) replace `t.name.trim()` with `v.sanitizeText(t.name, 60)` and keep the empty check, add `max` to the Input at mod/2370.js:36, and truncate the `name` value passed to logAudit at mod/631.js:13124.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-5 High	/contact.tsx	Dead-end flow	The contact form tells the user support will reply, but the message is written to a store nothing ever reads Submission writes a record to the submitting browser's own localStorage and stops. There is no admin inbox screen, no export, no transport, and no ticket reference returned to the user. The record also stores no `user_id`, so even if a reader existed the message could not be tied to an account, and `topic` is written through unvalidated against the six options the UI offers (mod/2373.js:19-26). Evidence: mod/631.js:2794-2807 `r.mockSubmitContact = async (e) => { const t = await xa(Ci, []), a = { id: ea(), first_name: ..., message: (0, v.sanitizeText)(e.message, 2e3), created_at: ... }; return (await Za(Ci, [a, ...t]), a); };` with `const Ci = "playora.mock.contact.v1"` at mod/631.js:2793. That key occurs exactly once in the entire shipped bundle — `grep -c "playora.mock.contact.v1" index.html` returns 1 — and `mockSubmitContact` has no reader anywhere: the only three occurrences of the name in index.html are its declaration, its export and the facade line mod/671.js:416. The success card at mod/2373.js:86-92 renders `contactSuccessBody` = "Thanks for reaching out — we'll get back to you soon."	Every support request, facility enquiry and corporate enquiry submitted through the app is silently discarded while the user is explicitly told a human will respond. The user has no ticket number, no copy of what they sent, and no other in-app support channel — mod/1805.js:864 is the only entry point. Clearing site data destroys even the local copy.	Route `mockSubmitContact` (mod/631.js:2794) through the real t return a reference id and show it on the success card at mod/237 there is a recipient, replace `contactSuccessBody` with copy that promise a reply and offer a mailto/WhatsApp fallback. Also recor in `user_id` and validate `topic` against the allowed set server-sic
P-CARCH-11 High	/discover.tsx	Read amplification / fake pagination	Player search materialises every profile with a full bookings scan each, then throws the result away to return 20 rows The `offset`/`limit` parameters give the appearance of pagination but are applied after the entire candidate set has been scored. The per-candidate work includes a nested scan of the global bookings table, so the cost is O(profiles × bookings) regardless of page size. Evidence: mod/631.js:14197 — `for (const u of Qt.profiles) {`. Inside that loop: mod/631.js:14202 `al(u.id)` (privacySettings scan), 14221-14222 `Qd(u.id)` (friendships scan) and `Zd(u.id)` (teamMembers scan), 14227-14231 `for (const e of Qt.bookings) ...` — a full bookings scan **per profile** — and 14232 `Qt.dmConversations.find(...)`. The slice happens only at the very end: mod/631.js:14267-14269 `m = Math.max(0, t.offset ?? 0), w = Math.min(20, ...); return u.slice(m, m + w);`. The screen calls it on a 300 ms debounce (mod/2374.js:38-40) and again on focus (mod/2374.js:49-53), and `Load more` re-runs the whole thing with a larger offset (mod/2374.js:360).	At 5,000 profiles and 20,000 bookings, one keystroke-debounce costs ~100 M iterations; `Load more` costs the same again for each additional page, so page 5 has done the full computation five times. The search box becomes unusable well before Playora reaches a realistic Kuwait user base, and because both a debounced effect and a focus effect fire on mount, every visit pays it at least twice.	Filter and sort on cheap fields first, slice to `offset+limit`, and only compute `mutuals`, `mutual_games`, `dm_state` and `online` for surviving page. Hoist `Qd`, `Zd` and the `Qt.bookings` pass into r once before the loop at mod/631.js:14197.
P-CSEC-38 High	/feed/compose.tsx	User control over own content	Feed posts and comments can never be deleted or edited by their author An author should be able to remove or correct their own post and their own comments. The compose screen writes an immutable record: there is no delete, no edit, and no author-only menu on the rendered card. Evidence: `Qt.posts` is only ever appended to or replaced wholesale by hydration/seeding — mod/631.js:2067 (`Qt.posts = la`, hydrate), 14393 (`Qt.posts.unshift(i)`, create), 14464 (`Qt.posts.push(...t)`, demo seed) — and is never filtered by author on any user action. `Qt.postComments` is the same: 2068 (hydrate), 14548 (`Qt.postComments.push(n)`). Greps for `mockDeletePost`, `mockDeleteComment`, `deletePost`, `deleteComment` return nothing in mod/631.js or mod/671.js. The feed card offers no author menu — the only control in the header row is the report flag, mod/1622.js:291-300 — even when `e.author_id === t` (the viewer's own post). Contrast the DM path, which does support deletion: mod/631.js:14746 `r.mockDeleteDirectMessage` plus the long-press affordance at mod/2455.js:199.	A user who posts a typo, the wrong photo, a phone number, or something they regret has no way to take it down — not from the feed, not from their profile, not even by deleting their account (see the erasure finding). The only mechanism the UI offers against any post, including your own, is the report flag, which does nothing. For a consumer social app this is both a user-trust failure and a data-protection problem, since users cannot withdraw content they published.	Add `mockDeletePost(userId, postId)` and `mockDeleteComment(commentId)` to 631 that assert `author_id === userId`, remove th its dependent likes/saves/shares/comments, and persist; export t 671; and add an author-only overflow menu to the feed card in 16 when `e.author_id === t`) with Delete behind a confirm, plus a de in the comments sheet.
P-CTEST-4 High	/feed/compose.tsx	Error handling	Compose silently discards the post when the write fails A failed post should tell the user why and keep their draft. Instead the rejection is swallowed, the loading spinner just stops, the screen does not navigate, and no message appears — the user is left looking at their own text with no idea whether it posted. Evidence: mod/2375.js:56-71 — `onPress: async () => { if (t && U && !S) { O(!0); try { (await createPost(t.id, { kind: A ? "highlight" : "text", caption: H, clipId: A ? cover_clip_id ?? null, mediaUri: W ? uri ?? null, mediaType: V ? type ?? null })), _replace(`/feed`)); } finally { O(!1); } } }` — `try/finally` with no `catch`, and the module's dependency list (mod/2375.js:345) contains no 674, so it has no way to render a store error at all. The call can throw on several ordinary paths: mod/631.js:14377 `if (!a    t.clipId    t.achievementKey    t.mediaUri)) throw new Error("empty_post")`, mod/631.js:1732 `if (Math.floor((3 * i.length) / 4) > 7e5) throw new Error("E_IMAGE_TOO_LARGE_PICK_A_SMALLER")` inside `Ya`, and mod/631.js:1772-1773 `throw new Error(/quotaQuotaExceeded[NS_ERROR_DOM_QUOTA]/i.test(a) ? "E_STORAGE_FULL" : "E_COULD_NOT_SAVE_CHANGES")` inside `Za`. The media picker is equally unguarded: mod/2375.js:31-34 `q = async (t) => { const l = await pickMedia(t); l && (D(l), F(null)) }`.	Attaching a photo larger than ~700KB, or posting once device storage is near its quota — both routine on a phone — silently fails. The user taps Post, nothing visible happens, and they tap again, which either fails again or produces a duplicate once space frees up. All three of these errors already have translated strings (`seEmptyPost`, `selImageTooLargePickASmaller`, `seStorageFull` at mod/674.js:28, 105, 277) that this screen can never display.	Add `catch (err) { setError(storeErrorText(err?.message ?? ""))    t to the post handler, import 674 into 2375, and render the messag composer the way mod/2455.js:361-370 does. Wrap `pickMedia` handler so a cancelled or failed picker is reported rather than dro

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-6 High	<code>./feed/compose.tsx</code> , <code>./stories/compose.tsx</code>	Media handling / data loss	Photos attached to posts and stories are persisted as blob: URLs, so every uploaded image is permanently broken after the first reload A blob: URL is valid only for the document that created it. Persisting one to storage stores a pointer that is guaranteed dead on the next page load, for the author and for everyone else. Evidence: The picker returns a session-scoped object URL — module 1808: <pre>`\$`({uri:URL.createObjectURL(t),type:t.type.startsWith("video")?"video":"image",name:t.name}})`.</pre> Both composers hand that straight to the backend: <code>mod/2375.js:32 `const l = await (0, v.pickMedia)(t);`</code> then <code>mod/2375.js:64 `mediaUri: W?.uri ?? null;`,`</code> mod/2488.js:25 <code>`const t = await (0, w.pickMedia)("image");`</code> then <code>mod/2488.js:62 `mediaUri: V?.uri ?? null;`,`</code> . The backend's only media handler passes anything that is not a base64 data-image through untouched — <code>mod/631.js:1727-1731 `Ya = async (e) => { if (!e) return null; const t = Ba.exec(e); if (!t) return e;`</code> where <code>`Ba = /^data:(image\/(?:png jpe?g webp gif));base64,([A-Za-z0-9+\/=]+)\$/`</code> (mod/631.js:1724) — so <code>`media_uri: await Ya(t.mediaUri)`</code> (mod/631.js:14385) stores the literal blob: string into <code>playora.mock.posts.v1</code> . The avatar path avoids this by converting through a canvas first (module 1807 <code>`o.toDataURL("image/jpeg",.82)`</code>), used at <code>mod/1805.js:313</code>), which shows the correct approach exists and was simply not applied here.	Every photo post and photo story looks correct while composing and in the session that created it, then renders as a broken image forever — for the author and for every other viewer, who never had access to that blob in the first place. The post row is kept, so the feed fills with permanently empty media cards.	Convert the picked file to a data: URL before calling the facade — <code>canvas.encode</code> already in module 1807 (<code>pickAvatarImage</code>) as a <code>s</code> helper — at <code>mod/2375.js:32</code> and <code>mod/2488.js:25</code> , and defensively <code>data/non-/media/ URIs</code> in <code>Ya</code> (mod/631.js:1727) instead of <code>returnii</code> unchanged.
P-CSEC-39 High	<code>./friends/index.tsx</code>	Harassment / block evasion	Blocking does not stop friend requests: a blocked user can find the blocker in search and keep re-sending requests A block should be symmetric across every social channel. DMs and follows honour it; friend requests do not — the blocked party can still search for the blocker by name, send a request, and re-send it without limit after each decline. Evidence: <code>mod/631.js:13533</code> — <code>`mockSearchUsers`</code> filters on <code>`t.id !== e`</code> & <code>`admin`</code> <code>!`=== t.role`</code> & <code>`ia(i, t.audience ?? "male")`</code> & <code>`!Ho(t.id)`</code> & <code>`(t.full_name ?? "")`</code>.<code>toLowerCase()</code>.<code>includes(a)`</code> — no <code>`Xd`/`el`</code> block filter and no <code>`profile_visibility`</code> filter, unlike the sibling <code>`mockSearchPlayers`</code> at <code>mod/631.js:14201-14202</code> <code>`if (Xd(e, u.id) tl(e, u.id)) continue;`</code> if <code>`("private" === al(u.id).profile_visibility) continue;`,`</code> <code>mod/631.js:13561</code> — <code>`mockSendFriendRequest = async (e, t) => { if ((await ei(), oa(e, t), e === t)) throw new Error("invalid");`</code> if <code>`(!Qt.profiles.some(...))`</code> throw new <code>Error("not_found")`</code>; if <code>`(zd(e, t))`</code> throw new <code>Error("already_exists")`</code>; ...} — <code>`oa`</code> is only the audience assert; there is no block check, unlike <code>`mockFollow`</code> (<code>631.js:14049</code> <code>`if ((oa(e, t), el(e, t))) throw new Error("blocked")`</code>) and <code>`gl`</code> (<code>631.js:14651</code> <code>`if ((oa(e, t), Xd(e, t))) throw new Error("blocked")`</code>). The incoming list has no block filter either: <code>mod/631.js:13594</code> <code>`"pending" === n.status`</code> & <code>`n.addressee_id === e`</code> ? <code>`a.push({ id: n.requester_id, name: Td(n.requester_id) })`,`</code> rendered at <code>mod/2378.js:148-171</code>. Declining deletes the row (<code>mod/631.js:13583</code> <code>`Qt.friendships = Qt.friendships.filter((e) => e.id !== i.id)`</code>), so the same request can be sent again immediately, and the only rate limit in the entire backend is on club invites (<code>mod/631.js:13202-13206</code>).	A user who blocks a harasser still receives that person's name and request card on <code>/friends</code> , repeatedly, with no way to stop it short of deleting the account. The block feature appears to work (the harasser vanishes from DMs and follows) while leaving an open channel, which is worse than no block at all. Accepting such a request also silently grants messaging capability, since friendship bypasses the <code>`allow_messages: "followers"`</code> gate (mod/631.js:14654) and auto-accepts new conversations (mod/631.js:14635).	In 631: add <code>`if (Xd(e, t)) throw new Error("blocked")`</code> to <code>`mockSendFriendRequest`</code> , add <code>`!Xd(e, t.id)`</code> & <code>`!tl(e, t.id)`</code> & <code>`"private(t.id).profile_visibility`</code> to the <code>`mockSearchUsers`</code> predicate, and <code>`n.requester_id`</code> out of the <code>`incoming`</code> array in <code>`Hd`</code> (<code>631.js:13587</code> , a per-sender daily cap on friend requests while you are there.
P-CSEC-40 High	<code>./game/[id]</code>	Data exposure / access control	Private matches are not private: the invite code is handed to any caller <code>`visibility: "private"`</code> is used only to hide a match from listings (mod/631.js:2685) and to mint a code at creation (mod/631.js:3529). Nothing verifies that a reader or joiner of a private match holds that code, and the code itself is readable by anyone who can name the game. The organizer-side code was already hardened for exactly this (mod/631.js:4183-4188, 'Private invite codes are never returned to anyone else') — the consumer read was not. Evidence: <code>mod/631.js:6314-6318</code> <code>`r.mockGetMatchInvite = async (e) => { await ei(); const t = await Xn(e); return { code: t.code, link: `https://playora.app/m/\${t.code}`, expires_at: t.expires_at; }`,`</code> — the function takes only a game id; there is no viewer argument to check against, and the facade passes none (mod/671.js:505 <code>`e.fetchMatchInvite = (o) => t.store.mockGetMatchInvite(o)`</code>). The game screen calls it for every viewer on load: <code>mod/2379.js:42-44</code> <code>`fetchMatchInvite(e).then((e) => he(e?.link ?? null))`</code>. Reading a match enforces only the audience partition, never visibility: <code>mod/631.js:2696-2703</code> <code>`const Ni = async (e, t) => { ... ra(t, a.audience); ... return li(a, i, t);`,`</code> and <code>`ji`</code> (mod/631.js:2988-3102) never inspects <code>`visibility`</code> either.	Anyone with a match id — from a shared link, a screenshot, a chat, or an enumerated notification payload — can open a private match, see its roster, join it outright, and lift the invite code to pass on. The private/invite-only product promise (badge rendered at <code>mod/2379.js:284-285</code>) is decorative.	Give <code>`mockGetMatchInvite`</code> a caller argument and return the code organizer or an accepted participant, and add a visibility gate to <code>`that`</code> that requires a valid invite/referral for <code>`visibility === "private"`</code> matches

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-41 High	/game/[id]	Authorization	Any signed-in user can reshuffle any match's teams and overwrite any roster row Every sibling mutation is gated — kick uses `zi` (mod/631.js:3307), all lineup writes use `dc`/`sc` (mod/631.js:16387-16397) — but team allocation and the self-player upsert have no rule to move to a server. Evidence: mod/631.js:2911-2920 `r.mockSetPlayerTeam = async (e, t, a) => { (await ei(t), md(e)); const i = Qt.gamePlayers.findIndex((e) => e.id === t); if (i >= 0) return ((Qt.gamePlayers[i] = Object.assign({}, Qt.gamePlayers[i], { team: a })), void (await Za(X, Qt.gamePlayers)));; ... }` — `md` only rejects `guest:` ids (mod/631.js:12394-12396); there is no organizer check, no participant check, not even a game id parameter. mod/631.js:2921-2925 `r.mockUpsertSelfPlayer = async (e) => { await ei(t); const t = Qt.gamePlayers.findIndex((t) => t.id === e.id); (t >= 0 ? (Qt.gamePlayers[t] = e) : Qt.gamePlayers.push(e), ...) }` takes a whole client-supplied row and replaces any matching record with no ownership check at all. The screen wires both to every viewer: mod/2379.js:85-91 and mod/2379.js:406-410 `TeamAllocation, { players: U, onMove: ze, ... }` rendered whenever `U.length > 0`.	Any account can rewrite the team split of any match it can open, and `mockUpsertSelfPlayer` lets a caller replace another player's roster row wholesale (display name, avatar, user_id) because the record is keyed only by the id the client sends. Sabotaging a rival's line-up, or renaming another player, needs one API call.	Add a game id plus an organizer-or-participant assertion to `mockSetPlayerTeam`, and in `mockUpsertSelfPlayer` take the caller's id and assert it equals the target row's `user_id` (or `self-<callerId>`); gate the `onMove` prop in mod/2379.js on organizer/participant.
P-CARCH-12 High	/game/[id]	Business-rule consistency	Waitlist promotion has no match-status or kick-off guard — players are promoted into matches that already started Promotion is the one way into a match that ignores kick-off, registration closure and match state. The hold it writes is also inconsistent with the payment it creates — the booking is held 15 minutes flat (`t + 9e5`) while `Hr` computes `Math.min(Jt, Math.max(startsAt - now, 12e4))` (mod/631.js:7665-7666), i.e. as little as 2 minutes, and the notification always claims 15. Evidence: mod/631.js:2628-2646 `for (; ki(e.id, t) < e.max_players; ) { const i = vi(e.id)[0]; if (!ti) break; ((i.status = "reserved"), (i.reserved_until = new Date(t + 9e5).toISOString()), ... await Hr(e, i.user_id, i.id), await bi(({ ... type: "waitlist_promoted", ... minutes: Math.round(15), ... })))` — no check on `e.status`, `e.starts_at` or `e.registration_closed_at`. It runs on every read of any match: mod/631.js:2697 `const Ni = async (e, t) => { (await ei(t), await tn(t), await Oi(e)); ...` and `Oi` (mod/631.js:2654-2657). The join path it shadows does check: mod/631.js:2993 `if (new Date(a.starts_at).getTime() < Date.now()) throw new Error("E_THIS_MATCH_HAS_ALREADY_STARTED");` and mod/631.js:3026 `if (a.registration_closed_at) throw new Error("E_REGISTRATION_IS_CLOSED");`.	Anyone opening the page of a match that has already kicked off (or whose registration the organizer closed) promotes the waitlist behind the scenes: waitlisted players get a 'You're in — pay in 15 minutes' notification and a payment request for a match they cannot attend, and the hold expires minutes later. Users who accept are billed for a finished game.	Guard the promotion loop in `Di` (mod/631.js:2628) with `!e.status && !e.registration_closed_at && new Date(e.starts_at).getTime() < Date.now()`, and derive the notification's `minutes` and the booking's `reser` from the same value `Hr` uses.
P-CSEC-42 High	/game/[id].tsx	Broken access control (private match bypass)	mockGetMatchInvite requires no authorization and mints a fresh invite code as a side effect of a read Two independent invite secrets for a match — `Qt.invites[]` and `games.invite_code` — are both returned to any caller who has the game id, while the organizer-facing list goes to the trouble of redacting one of them. A read endpoint also performs a persistent write. Evidence: mod/631.js:6314 — `r.mockGetMatchInvite = async (e) => { (await ei(t)); / `const t = await Xn(e);` / `return { code: t.code, link: `https://playora.app/m/\${t.code}`, expires_at: t.expires_at };`. One argument, a game id; no caller and no organizer check. `Xn` (mod/631.js:6303) is not a pure read — when no live invite exists it generates one and persists it: `let n = Zn(i.sport); ... Qt.invites.push(r), await Za(zt, Qt.invites), r`. The same module states the opposite rule for the organizer surface at mod/631.js:4201: `// The private invite code belongs to the organizer; an admin reading the list never sees it.` followed by `return own9 ? row9 : Object.assign(row9, { invite_code: null });`. The game row's own `invite_code` (set at mod/631.js:3529 `invite_code: "private" === t.visibility ? Hi() : null;`) also leaks, because `li` spreads the whole row: mod/631.js:2665 `li = (e, t, a) => { ... `return Object.assign({}, e, {` — and `mockGetGame`/`mockGetGameScreen` return that object to every viewer (mod/631.js:2702 `return li(a, i, t);`). The screen calls it unconditionally: mod/2379.js:42 `(0, t.fetchMatchInvite)(e)`.	The private-match invite link can be harvested for any match by anyone, defeating the point of `visibility: "private"`, and shared onward outside the organizer's control. Because `Xn` mints a code when none exists, an attacker can also create invite records for matches that never had one, and the unauthenticated write is a cheap way to grow `Qt.invites` without limit.	Add a caller argument to `r.mockGetMatchInvite` and assert `zi(g)` before returning. Split `Xn` into a pure lookup for reads and an `ensureInvite` used only by the organizer's share action. Strip `in` `li` (mod/631.js:2665) for anyone who is not the organizer or an admin, mirroring the redaction already written at mod/631.js:4202.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-43 High	/game/[id].tsx	Unauthorized write / data tampering	mockUpsertSelfPlayer and mockSetPlayerTeam take no caller and let any client overwrite any player row Neither function establishes who is calling or that the target row belongs to them. <code>`mockUpsertSelfPlayer`</code> in particular accepts <code>`id`</code> , <code>`user_id`</code> , <code>`display_name`</code> , <code>`is_self`</code> and <code>`team`</code> from the client and writes them unvalidated, so it is a blind row-overwrite primitive over <code>`Qt.gamePlayers`</code> . Evidence: <code>mod/631.js:2921</code> — <code>`r.mockUpsertSelfPlayer = async (e) => { / `await ei(); / `const t = Qt.gamePlayers.findIndex((t) => t.id === e.id); / `t >= 0 ? (Qt.gamePlayers[t] = e) : Qt.gamePlayers.push(e), await Za(X, Qt.gamePlayers));`</code> — the entire row, keyed by a client-supplied <code>`id`</code> , replaces the stored row verbatim. <code>mod/671.js:412</code> <code>`e.upsertSelfPlayer = (o) => t.store.mockUpsertSelfPlayer(o);`</code> carries no caller. <code>mod/631.js:2911</code> <code>`r.mockSetPlayerTeam = async (e, t, a) => { / mod/631.js:2912 `(await ei(), md(e));`</code> — <code>`md`</code> only rejects guest sessions (<code>mod/631.js:12394</code> <code>`if (e.startsWith("guest:")) throw new Error("E_SIGN_UP_TO_JOIN_BROWSING_IS");`</code>) and <code>`e`</code> here is the game id, not a user; the row is then located purely by <code>`t`</code> . Row ids are handed to every viewer by <code>mod/631.js:2910</code> <code>`r.mockGetGamePlayers = Ui;`</code> .	Any user can rename another player in a match, move them between teams, repoint a roster row at a different <code>`user_id`</code> , or mark an arbitrary row <code>`is_self: true`</code> — which then feeds the lineup claim logic and the game screen's <code>`You`</code> rendering (<code>mod/2379.js:248</code> , <code>mod/2379.js:266</code>). Team assignment drives lineup auto-balance and the squad UI, so a griefer can scramble any match's roster from outside the match.	Give both functions an explicit caller argument. In <code>`mockUpsertSelfPlayer`</code> accept only the mutable fields (<code>`team`</code> , <code>`avatar_seed`</code>) and force <code>`\`self-\${caller}\`</code> , <code>`user_id = caller`</code> , <code>`is_self = true`</code> . In <code>`mockSetPlayerTeam`</code> assert <code>`zi(hi(gameld), caller)`</code> so only the organizer reassigns other rows.
P-CSEC-44 High	/group/[gameld].tsx	Unauthorized obligation / seat squatting	mockCreateGroupBooking's "not_friends" check only verifies the profile exists, so anyone can be booked and billed The guard that was meant to restrict a group booking to the leader's friends degenerated into an existence check. Nothing verifies consent, friendship, or even that the target has heard of the match. Evidence: <code>mod/631.js:13668</code> — <code>`for (const t of r) { / mod/631.js:13669 `if (!Qt.profiles.some((e) => e.id === t)) throw new Error("not_friends"); / mod/631.js:13670 `oa(e, t);`</code> . The error name says friendship, but the predicate only asserts the id resolves to a profile; <code>`oa`</code> is the audience-partition check (<code>mod/631.js:766</code> <code>`oa = (e, t) => { ra(e, aa(t)); }`</code>), not a relationship check. The real friend set is computed elsewhere and is available in this file — <code>mod/631.js:14019</code> <code>`Qd = (e) => { ... "accepted" === a.status ...}`</code> — and the config endpoint even returns it to the screen: <code>mod/631.js:13640</code> <code>`const i = Fd(a), { friends: n } = await Hd(e);`</code> . Each named id then gets a reserved booking and a money obligation: <code>mod/631.js:13697</code> <code>`status: "reserved",`</code> and <code>mod/631.js:13726</code> <code>`amount_kwd: i.amount_kwd,`</code> with <code>mod/631.js:13727</code> <code>`paid_by_leader: n,`</code> .	A leader can add any user in the same audience partition to a group booking. Each victim gets a <code>`reserved`</code> booking row consuming a seat in a match they never joined and a <code>`pending_payment`</code> group member row with an amount due, plus a <code>`payment_request`</code> notification. Repeated across matches this both squats free seats (blocking genuine joiners, since <code>`ki`</code> counts reserved seats) and generates bogus payment demands against unrelated accounts.	Replace the predicate at <code>mod/631.js:13669</code> with a real relationship (! <code>Qd(e).has(t)</code>) throw new <code>Error("not_friends");`</code> — using the same <code>`mockGetGroupConfig`</code> already surfaces, and keep the <code>`oa`</code> partition as an additional condition.
P-CSEC-45 High	/group/[gameld].tsx	Payment integrity / funds to wrong party	Group booking attributes the leader's payments to each member, so refunds pay the wrong person The money leaves the leader's card or wallet, but the ledgered obligation is recorded against each member. When a member later leaves the match, <code>`mockLeaveMatch`</code> → <code>`Br`</code> → <code>`ac`</code> credits that member's wallet with money the leader paid. Nothing records who actually funded the row. Evidence: <code>mod/631.js:13800-13819</code> — one gateway capture is taken from the caller, then a payment row per member is written against the member: <code>`d = await Yr(e, s, "match_payment", t, a); for (const i of r) Qt.payments.push({ id: ea(), kind: "seat", booking_id: i.booking_id ?? t, game_id: n.id, payer_id: i.user_id ?? e, payer_name: or(i.user_id ?? e), ... amount_kwd: (0, z.roundKwd)(Number(i.amount_kwd ?? 0)), status: "paid", method: a, gateway_ref: d, ... })`</code> . <code>`e`</code> is the leader who was charged <code>`s`</code> ; <code>`i.user_id`</code> is the member. <code>mod/631.js:13742-13745</code> — <code>`jd`</code> selects exactly the members the leader pays for: <code>`t === a ? e.filter((e) => "leader" === e.kind e.paid_by_leader) : ...`</code> . Refunds follow <code>`payer_id`</code> : <code>mod/631.js:7715-7717</code> — <code>`(i.status = "refunded"), (i.refunded_at = ...), await ac(t, i.amount_kwd, i.id)`</code> where <code>`t`</code> is the seat holder.	A player who was invited into a group and never paid a file receives a wallet refund when they drop out, while the leader who paid is out of pocket with no refund path and no record tying the charge to them. Repeatable: join a friend's group, let them pay, leave before the cutoff, keep the cash. It also corrupts <code>`mockGetMyPaymentRequests`</code> , which shows members payments they never made.	Add a <code>`funded_by`</code> field set to the charging user in the <code>`mockPayment`</code> (<code>mod/631.js:13807</code>) and have <code>`Br`/`ac`</code> credit <code>`funded_by ?? payer`</code> <code>`payer_id`</code> for seat-entitlement lookups only.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-46 High	./group/[gameId].tsx	Refund handling / money destroyed	Cancelling a group member never touches their payment — paid seats are voided with no refund Group members are confirmed by payment (<code>`mockPayGroup`</code> writes rows with <code>`status: "paid"``</code>), but the group's own cancellation entry point drops the seat without consulting the payment at all — not even to check the 2-hour refund cutoff that the equivalent solo path honours. Anyone in <code>`e === i.leader_id    e === n.user_id    e === r.organizer_id    Id(e)`</code> can trigger it. Evidence: <code>mod/631.js:13852-13866</code> — the whole body of <code>`mockCancelGroupMember`</code>'s locked section: <code>`n.status = "cancelled"; const o = n.booking_id ? Qt.bookings.find((e) => e.id === n.booking_id) : null; o && "cancelled" !== o.status && ((o.status = "cancelled"), (o.reserved_until = null), (o.updated_at = new Date().toISOString()));; (0 === Qt.groupMembers.filter((e) => e.group_id === t && "cancelled" !== e.status && "declined" !== e.status).length && (i.status = "cancelled"), await Za(W, Qt.bookings), await xd(), await Ud(), await (0, w.logAudit)("group.member_cancelled", ...), await Di(r), await rd(r));`</code> — <code>`Qt.payments`</code> is never read or written, and neither <code>`Br`</code> nor <code>`ac`</code> is called. Contrast <code>mod/631.js:3130-3132</code> in <code>`mockLeaveMatch`</code>, which does run the refund: <code>`s = !r (0, F.isRefundEligible)(r.starts_at, o), d = r ? await Br(r, t, s) : { paid_kwd: 0, refund_kwd: 0 }`</code>.	Money is destroyed silently. A leader who paid for four seats and then removes one — or a member who drops out through the group screen instead of the match screen — loses that seat's fee with no refund, no <code>`refund_issued`</code> notification, no ledger row and no entry on <code>./refunds.tsx</code> . The same act via <code>./game/[id].tsx</code> would have been refunded in full, so the outcome depends purely on which screen the user happened to use.	In <code>`mockCancelGroupMember`</code> (<code>mod/631.js:13852</code>), mirror <code>`mockLeaveMatch`</code> : compute <code>`isRefundEligible(game.starts_at, C</code> call <code>`Br(game, seatHolder, eligible)`</code> for the member's paid seat (c funding party per the group-attribution fix), and push a <code>`seatCanc</code> row so the cancellation appears on <code>./refunds.tsx</code> .
P-CSEC-47 High	./group/[gameId].tsx	Refund handling / money destroyed	When a match is cancelled, only one of the leader's guest-seat payments is refunded <code>`Gr`</code> was written for the one-seat-per-user case and uses <code>`.find`</code>. Group bookings break that invariant: the leader legitimately holds several paid seat rows for the same game. The cancellation loop iterates seat holders, not payments, so the extra rows are never reached from either direction. Evidence: <code>mod/631.js:7650-7654</code> — the seat-payment lookup returns a single row: <code>`const Gr = (e, t) => Qt.payments.find((a) => "seat" === a.kind && a.game_id === e && a.payer_id === t && ("pending" === a.status    "paid" === a.status))`</code>. <code>mod/631.js:3775</code> — cancellation refunds once per distinct seat holder: <code>`for (const e of o) await Br(n, e, l0);`</code> where <code>`o`</code> is built at 3765-3772 from <code>`t.user_id`</code> of each booking. <code>mod/631.js:13707</code> — guest bookings carry a synthetic holder: <code>`user_id: e.user_id ?? `guest:\${m.id}:\${a}``,`</code> while <code>mod/631.js:13807</code> writes the matching payment as <code>`payer_id: i.user_id ?? e`</code> — undefined for a guest, so it falls back to the leader. Multiple paid rows therefore share one <code>`payer_id`</code>, and <code>`Br`</code> → <code>`Gr`</code> refunds only the first; <code>`Br(n, "guest:...", l0)`</code> matches nothing.	A leader who books four guest seats at 5 KWD and has the organizer cancel the match gets 5 KWD back and loses 15 KWD. The rows stay <code>`status: "paid"``</code> forever, so they show as kept on <code>./refunds.tsx</code> and are still counted as revenue by <code>`mockGetVenueRevenue`</code> . The larger the group, the more is destroyed.	Change <code>`Br`</code> to operate over all matching rows — replace the <code>`Gr`</code> with a filter over every <code>`"seat"``</code> payment for the game whose fund the user, and refund each — or drive <code>`cancelGameLocked`</code> (<code>mod/631.js:3775</code>) from <code>`Qt.payments`</code> for the game rather than fr booking list.
P-CSEC-48 High	./group/[gameId].tsx	Data exposure / payment detail	Group booking detail returns every member's name and amount owed to anyone holding the group id A read of someone else's group booking succeeds. The function already has the caller id in hand and already knows who the members are, so the membership check is a one-line omission rather than a design gap. Evidence: <code>mod/631.js:13748-13770</code> — <code>`qd`</code> takes the caller <code>`e`</code> and a group id <code>`t`</code>, and uses <code>`e`</code> only to compute the caller's own total: <pre>... const a = Qt.groupBookings.find((e) => e.id === t); if (!a) throw new Error("not_found"); ... r = Qt.groupMembers.filter((e) => e.group_id === t), o = n.leader_id === e, s = jd(r, e, n.leader_id).filter(...).reduce(...), ... return { group: n, members: r, is_leader: o, my_due_kwd: ..., seconds_left: ... }; ...</pre> There is no <code>`members.some(m => m.user_id === e)`</code> guard. The partitioning helper <code>`jd`</code> (<code>mod/631.js:13743-13746</code>) exists precisely to split "what I owe" from "what others owe", yet <code>`members: r`</code> ships the unfiltered rows, each carrying <code>`user_id`</code>, <code>`name`</code>, <code>`kind`</code>, <code>`status`</code>, <code>`amount_kwd`</code>, <code>`paid_by_leader`</code>, <code>`booking_id`</code> (<code>mod/631.js:13716-13727</code>). Facade: <code>mod/671.js:642</code> <code>`e.fetchGroupBooking = (o, c) => t.store.mockGetGroupBooking(o, c)`</code>; screen: <code>mod/2444.js:52</code>.	Any user with a group id learns the full guest list of a private booking — who is playing with whom, including named guests who have no account — plus each person's outstanding balance in KWD and who is paying for whom. That is social-graph plus payment data about third parties.	In <code>`qd`</code> (<code>mod/631.js:13748</code>) throw <code>`not_found`</code> unless <code>`n.leader_id r.some(m => m.user_id === e)`</code> , and for non-leaders return <code>`mem</code> reduced to the caller's own row plus name-only entries for the res

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-13 High	/group/[gameld].tsx	Error handling / dead end	/group/[gameld] captures the load error into state that is only rendered inside the branch the loading gate blocks, leaving a spinner that never resolves `A` is the setter for `L`. When `fetchGroupConfig` throws, `L` stays `null`, so the component returns the spinner before it ever reaches the `FormError` that would have shown `ce`. The error is captured and then made unreachable. Evidence: mod/2444.js:27-40 — `pe = useCallback(async () => { if (M && e) try { A(await fetchGroupConfig(M.id, e)); ... } catch (e) { ue(e.message); } }, [M, e])`. mod/2444.js:272-276 — `if (!L) return <SafeAreaView style={[z.center, ...]}><ActivityIndicator/></SafeAreaView>`. The error state `ce` is rendered only at mod/2444.js:222 and mod/2444.js:588, both of which are inside the JSX that follows the `!L` early return.	Any failure on the group-booking screen — a storage write failure, a hydration failure, a stale `gameld` — renders an infinite spinner with no message, no retry and no back button (this branch renders a bare `SafeAreaView`, not the header). The user is stuck mid-payment-split with no way forward except the browser back button.	Change the gate at mod/2444.js:272 to `if (!L) return ce ? <ErrorScreen> : <Spinner>`, or adopt the `GateScreen` from mod/9001.js:72 which already renders title, bc and back.
P-CSEC-49 High	/media/[id].tsx	Broken authorization / signed URL issuance	mockGetClipSignedUrl fails open when the parent upload row is missing, and only ever blocks 'private' The guard is written as 'throw only if private'. When the parent upload row cannot be found, i is undefined, `private` === undefined is false, and the guard never fires - an unauthenticated-by-ownership caller receives a signed CDN token. Even when the row is present, the check permits any signed-in user to mint a token for someone else's 'unlisted' upload, while the far less sensitive share operation requires 'public'. The strongest control sits on the weakest action and the weakest control sits on the action that actually hands out media bytes. It should be allowlist-shaped: permit only owner, admin, or visibility === 'public'. Evidence: mod/631.js:13447-13461: r.mockGetClipSignedUrl = async (e, t) => { const a = Qt.mediaClips.find((e) => e.id === t); if (!a) throw new Error("not_found"); const i = Qt.mediaUploads.find((e) => e.id === a.upload_id); if (!(i.owner_id === e !d(e)) && "private" === i?.visibility) throw new Error("forbidden"); ... url: `https://cdn.playora.app/\${i?.storage_key} ?? "media"/\${a.id}?token=\${t}&exp=...` Compare the sibling operation at mod/631.js:13431 (mockShareClip), which is fail-closed and stricter: if (!(i.owner_id === e !d(e)) && "public" !== n?.visibility) throw new Error("forbidden");	Any signed-in user who learns a clip id - trivially, since mockGetMediaUpload (mod/631.js:13417-13419) returns every clip row of an unlisted upload to any caller, and stories expose clip ids directly - can mint a 10-minute CDN URL for another user's unlisted match footage. Orphaned clips (which the concurrent-tab overwrite described in the storage finding produces routinely) can be signed by literally anyone, with the URL falling back to the generic `media` path. Each issuance is audit-logged as media.signed_url with the requester's id, so the logs will show the breach after the fact but nothing prevents it.	Rewrite the guard in mockGetClipSignedUrl (mod/631.js:13452) to allow: `if (!(i.owner_id === e    !d(e))    i?.visibility === "public")) throw Error("forbidden");` so a missing upload row denies rather than permits drop the `i?.storage_key ?? "media"` fallback at mod/631.js:1345 of throwing 'not_found'. Align mockGetMediaUpload (mod/631.js: the same rule so unlisted uploads are not enumerable by non-owner
P-CARCH-14 High	/media/[id].tsx	Stuck state / infinite loading	An upload can get permanently stuck in 'processing', with the detail screen polling it every 1.5s forever Clips are persisted before the upload's status is persisted. If the second write throws E_STORAGE_FULL, or the tab is closed between the two awaits, localStorage ends up holding clips for an upload whose row still says 'queued'. On the next load Ld calls Pd, Pd sees clips already exist and returns immediately without ever setting status to 'ready', and no other code path in 631 can repair it. Pd should set and persist the status even on the early-return branch, and the two writes should be ordered status-last-but-idempotent or wrapped so a partial write self-heals. Evidence: mod/631.js:13297 in Pd short-circuits before the status transition: if (Qt.mediaClips.some((t) => t.upload_id === e.id)) return; and the two persists that must both succeed are sequential and unordered against each other - mod/631.js:13339-13344: (Qt.mediaClips.push(...n), (e.status = "ready"), (e.progress = 100), ... await Za(mt, Qt.mediaClips), await Nd(), Za throws on quota - mod/631.js:1772-1774: throw new Error(/quotaQuotaExceeded[NS_ERROR_DOM_QUOTA]/i.test(a) ? "E_STORAGE_FULL" : "E_COULD_NOT_SAVE_CHANGES"); Pd is the only writer of status 'ready', and Ld (mod/631.js:13358-13370) is the only caller of Pd. The screen polls on that status forever - mod/2451.js:27-34: Y = W?.upload.status, Z = "queued" === Y "processing" === Y; ... if ((X(), !Z)) return; const e = setInterval(X, 1500);	The user's video is stuck on the 'AI is detecting key moments...' spinner forever. /media/[id] re-fetches every 1.5 seconds with no timeout, no retry ceiling and no error path, each tick re-reading the store and potentially rewriting the whole uploads array (mod/631.js:13366). The library row at mod/2453.js:100 shows 'processingHint' indefinitely, and mod/2453.js:24 runs the same 1.5s interval for as long as any one upload is stuck - so a single wedged upload permanently busy-loops both screens. The 'failed' status that exists in the UI (mod/2453.js:151) and in the dashboard counters (mod/631.js:13493) is never assigned anywhere in 631, and mod/2451.js:99 maps every non-'processing' status to the 'Queued' label, so even an explicit failure would render as the same infinite spinner.	In Pd (mod/631.js:13293) move the status/progress/processed_assignment and Nd() persist above the `Qt.mediaClips.some(...)` so an upload whose clips already exist is always marked ready. C (mod/631.js:13358) a wall-clock ceiling that flips the upload to 'failed' bounded interval, and make /media/[id] stop polling and render a affordance for 'failed' rather than falling through to the 'Queued' label at mod/2451.js:99.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-7 High	/media/[id].tsx	Dead end / false confirmation	The clip link button claims 'Secure link copied' but nothing is ever copied The screen calls an API that exists solely to produce a shareable URL, throws that URL away, and then tells the user it is on their clipboard. It should write the returned url to the clipboard (or surface it in a selectable field) and only then show the confirmation - and the confirmation should be conditional on the write succeeding. Evidence: mod/2451.js:52-59 awaits the call purely for the side effect and discards the result: <pre>oe = async (e) => { if (c) try { (await (0, S.fetchClipSignedUrl)(c.id, e.id), ee(D("signedUrlReady"))); } catch { ee(D("errForbidden")); } }; The backend does return a usable URL - mod/631.js:13457-13460: url: `https://cdn.playora.app/\${!?.storage_key ?? "media"}/\${a.id}?   token=\${r}&exp=\${encodeURIComponent(n)}`, expires_at: n, The toast string asserts a clipboard write (bundle-src/909.js): signedUrlReady: "Secure link copied (expires in 10 min)" and the control is labelled as a copy action - mod/2451.js:196: accessibilityLabel: D("copyLinkA11y"), No clipboard or Linking module appears in the screen's dependency list (mod/2451.js:439-442).</pre>	A user taps the link button, reads 'Secure link copied (expires in 10 min)', pastes into WhatsApp and gets whatever was on their clipboard before - or nothing. There is no other route to the clip URL anywhere in the six media screens, so the app's central 'share my highlight' action is unreachable while affirmatively telling the user it succeeded. The failed attempt is still audit-logged as media.signed_url (mod/631.js:13456), so the logs record link generation that never reached a user.	Capture the response in oe (mod/2451.js:52) and write `res.url` to clipboard via expo-clipboard/navigator.clipboard, showing signedUrl only after the write resolves and a distinct failure toast otherwise. access is unavailable in the target browsers, render the url in a separate Input inside the existing sheet instead of claiming a copy.
P-CQUAL-8 High	/media/[id].tsx	Dead end / metric integrity	The share sheet shares nothing - it only increments a counter and fabricates the dashboard's share metrics Tapping a platform in the share sheet performs no share: no deep link is opened, no native share sheet is invoked, no caption or hashtags reach the clipboard. The screen imports neither Linking nor a share API (mod/2451.js:439-442). It should consume the returned payload - open deep_link or hand caption/hashtags/deep_link to the platform - and only count a share once one actually left the app. Evidence: mod/2451.js:42-51 discards the share payload: <pre>se = async (e, t) => { if (c) { \$(null); try { (await (0, S.shareClip)(c.id, e.id, t), ee(D("sharedToast", { p: D("plat_\$(t)) })), X()); } catch { ee(D("errForbidden")); } } }; The backend builds a complete, ready-to-use payload - mod/631.js:13444: (0, N.sharePayload)(a, n?.title ?? "Match highlight", t, n?.watermark ?? 10) which (module 661) returns { platform, aspect, caption, hashtags, deep_link: `https://playora.app/clip/\${a}?utm_source=-...`, watermark }. The counter is bumped regardless - mod/631.js:13433: (i.shares += 1), and that counter is what the dashboard reports - mod/631.js:13496 and mod/631.js:13519: s = n.reduce((e, t) => e + t.shares, 0), engagement: n.reduce((e, t) => e + (0, N.engagementScore)(t.views, t.shares), 0),</pre>	Users are told 'Shared to Instagram' when nothing was shared, so highlights the product exists to distribute never leave the device. Because shares is incremented unconditionally, /media/dashboard's dashShares, dashEngagement, sharesByPlatform bars and topClips ranking are all built on events that did not happen - an organizer making decisions from that dashboard is reading pure fiction. The deep_link the backend generates also points at playora.app/clip/<id>, a host and route that do not exist in this app (routes are /media/[id]), so wiring it up naively would produce dead links.	In se (mod/2451.js:42) use the resolved payload: open payload.d Linking (or navigator.share where available), and move the `i.share increment in mockShareClip (mod/631.js:13433) behind a confirm the client that the share was dispatched. Correct sharePayload's host/route in module 661 to the app's real origin and /media/[id] to enabling it.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-15 High	/media/upload.tsx	Role gating / dead end	/media/upload and /media/dashboard are offered to every user but require an approved organizer, and the gate fires only after the whole form is filled The media library loads for every role, advertises two actions, and both are reserved for approved organizers, venue owners and admins. Nothing on /media, /media/upload or /media/dashboard consults the role before rendering - the repo's own shared role-gate helpers (module 9001) are not imported by any of these screens. Worse, the authorization is not a deliberate assert: it is an incidental side effect of computing owner_kind, a field that is written and never read, so it would disappear the moment someone 'cleans up' that dead field. Evidence: mod/631.js:13374 in mockUploadMatchVideo: const a = await Cd(e); mod/631.js:13291-13292: Cd = async (e) => Id(e) ? "admin" : Qt.venueProfiles.some((t) => t.owner_id === e) ? "venue" : (await on(e), "organizer"), mod/631.js:4504-4511, on() throws for anyone whose organizer application is not approved: async function on(e) { if ((await ei(), !rn(e))) throw (... new Error("E_ORGANIZER_APPROVAL_REQUIRED_YOUR_ORGANIZER_APPLICA")); } The sole consumer of Cd's return value is a field nothing ever reads - mod/631.js:13379 `owner_kind: a,` is the only occurrence of owner_kind in all 17k lines. mod/631.js:13488 gates the dashboard the same way: t (await on(e)); Yet the entry points are unconditional. mod/2453.js:64-68: (0, B.jsx)(b.Button, { title: P("uploadVideo"), onPress: () => M.push("/media/upload"), ... }) mod/2453.js:52-53: onPress: () => M.push("/media/dashboard"), and the library itself has no gate at all (mod/631.js:13403 mockGetMediaLibrary).	A plain user opens /media, sees an empty library telling them to 'upload your first' video, taps Upload, picks a sport, two durations, a visibility, toggles watermark and consent, taps Generate - and only then is told their organizer application must be approved. The dashboard button is worse: mod/2452.js:15-18 catches every error into a single boolean and mod/2452.js:47-54 renders a lock EmptyState with no retry, no explanation and no link to apply as an organizer, so the user is stranded on a screen whose only exit is the back chevron. Both are pure dead ends for the largest user segment.	Gate the entry points, not the submit: hide or disable the Upload i dashboard buttons in mod/2453.js:52 and mod/2453.js:64 unless is an approved organizer, venue owner or admin, using the modu helpers, and render an explicit 'apply to become an organizer' CT In the backend, replace the incidental `await Cd(e)` authorization mockUploadMatchVideo (mod/631.js:13374) with an explicit asse not depend on a dead field.
P-CARCH-16 High	/messages/[id].tsx	Dead end	Declining a message request permanently and irreversibly kills the conversation for both people Declining a message request should be reversible — the recipient changed their mind, or tapped the wrong button. Instead it writes a permanent tombstone that silently makes the pair unable to ever message each other again, with no undo, no 'undecline', and no explanation that the decision is final. Evidence: `declined_at` is written in exactly one place — mod/631.js:14742 `((a.declined_at = new Date().toISOString(), await Za(He, Qt.dmConversations), await logAudit("dm.request_declined", ...))` — and is never cleared anywhere in the bundle; the only other occurrences read it (mod/631.js:14613, 14706, 14740, 14761, 14234). Both sides are then locked out: mod/631.js:14706 `if (i.declined_at) throw new Error("request_declined")` blocks sending, mod/631.js:14761 `if (a.declined_at) throw new Error("not_a_request")` blocks accepting, and the screen hides the composer for both — mod/2455.js:78 `se = !!q && (q.declined    q.blocked_by_me    q.blocked_me)` gating mod/2455.js:371 `se && jsxs(Div, { style: [S.composer, ...] })`. Restarting from a profile does not help: `fi` (mod/631.js:14631) reuses the existing conversation id `dm:a:b` and returns it with `declined: !!e.declined_at`. The blocked-user equivalent is recoverable via /blocked (mod/1837.js:19-21 `unblockUser`); declining has no equivalent screen and no undo button anywhere.	One mis-tap on the small 'Decline' button — which sits directly beside 'Accept' in the request bar (mod/2455.js:259-274) and only asks for a single confirm — permanently severs a conversation. The user who declined sees a dead thread stuck in their Requests section forever; the sender sees a 'declined' pill in Primary and can never reach that person again through the app, even after both become friends. This is unrecoverable state a real user reaches by accident.	Add `mockUndoDeclineMessageRequest(user, convId)` in 631 th `declined_at` when the caller is the decliner (and persists `He`), € 671, and render an 'Undo' button in the `q?.declined` bar in 2455 (mod/2455.js:302-312) for the party where `!initiated_by_me`. Alt clear `declined_at` when `mockStartConversation` is called again decliner.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-9 High	./notifications.tsx	Notification correctness	A recommended ban notifies every admin with "your sanction was overturned" and notifies the sanctioned player with nothing The pending-approval branch reuses the `sanction_reviewed` payload shape — which means "a decision was made about *your* sanction" — but sends it to the admin reviewers, with `upheld:false` standing in for "not yet decided". The only branch that notifies the target user (mod/631.js:12483 `type: "sanction_issued"`) is the `else`, so a player recommended for a ban or suspension is told nothing at all. Evidence: mod/631.js:12448 `const r = (0, D.needsAdminApproval)(a.type, i)` (true for `ban` and `suspension`, bundle-src/656.js:100). mod/631.js:12468-12478 `r ? (await br((e) => ({ id: ea(), user_id: e, type: "sanction_reviewed", sanction_id: o.id, sanction_type: o.type, upheld: !1, read: !1, created_at: ... })), ...)`. `br` is the admin fan-out: mod/631.js:6842 `br = async (e) => { for (const t of Qt.profiles.filter((e) => "admin" === e.role)) await bi(e(t.id)); }`. The renderer at mod/2458.js:399-402 reads `upheld` literally: `if ("sanction_reviewed" === t.type) return t.upheld ? { title: n("notifSanctionUpheldTitle"), ... } : { title: n("notifSanctionLiftedTitle"), body: n("notifSanctionLiftedBody") }`; where `notifSanctionLiftedBody` is "An admin has overturned a disciplinary action. Your account standing has improved."	Every admin receives "Disciplinary action overturned — your account standing has improved" whenever an organizer recommends a ban on someone else, and tapping it deep-links to `/passport/<the admin's own id>` (mod/2458.js:62-63). Meanwhile the recommended-for-ban player gets no notification; the only way they discover it is by visiting /conduct, where a `pending_approval` sanction is rendered identically to an overturned one (mod/2372.js:118-122). Reviewers are trained to ignore a notification that is always wrong, and the player is denied notice of an action against them.	In mod/631.js:12468 emit a distinct admin-queue type (e.g. `sanction_pending_review`) with its own copy and a deep link to it, and send the target user a real notice of the pending recommendation. Keep `sanction_reviewed`/`upheld` for actual d from `mockReviewSanction` (mod/631.js:12530).
P-CQUAL-10 High	./notifications.tsx	Error handling	Tapping an unread notification does nothing when the read-marking call fails Marking-as-read is bookkeeping; it should never gate navigation. It is sequenced ahead of routing with no catch, so a bookkeeping failure swallows the user's tap entirely and produces an unhandled rejection instead of an error. Evidence: mod/2458.js:31-35 `const H = async (n) => { (ln.read && t && (await (0, T.markNotificationRead)(t.id, n.id), z((t) => L.map(...))), "squad_confirm_needed" !== n.type && ... )`; — the entire navigation ternary (mod/2458.js:36-79) is the second operand of the same comma expression, so it is unreachable if the awaited call rejects. That call can reject: `mockMarkNotificationRead` persists via `Za` (mod/631.js:2938), and `Za` throws on a storage failure — mod/631.js:1769-1772 `throw new Error(/quotdQuotaExceededdNS_ERROR_DOM_QUOTA/i.test(a) ? "E_STORAGE_FULL" : "E_COULD_NOT_SAVE_CHANGES")`.	On a device whose localStorage is full — which this app makes likely, since notifications are never pruned (mod/631.js:2926) and every write re-serialises the whole array — every unread notification becomes untappable. A `payment_request`, `replacement_offer` or `booking_decision` can no longer be opened, and the user gets no explanation. Read notifications still navigate, so the failure looks random.	In mod/2458.js:31, navigate first (or fire the read-marking without `markNotificationRead(...).catch(() => {})` and update local state optimistically, so routing never depends on the persistence call.
P-CQUAL-11 High	./notifications.tsx	Dead-end navigation	Sanction notifications deep-link to the passport, which shows nothing about sanctions The deep link for the two most consequential conduct notifications points at the stats-and-trophies passport rather than at the disciplinary screen. Both notification bodies reference a specific action (`body: t.reason` at mod/2458.js:398), and the destination shows none of it. Evidence: mod/2458.js:62-63 `: "sanction_issued" === n.type    "sanction_reviewed" === n.type ? t && B.push(`/passport/\${t.id}`)`. mod/2473.js `./passport/[id].tsx` contains no occurrence of `sanction`, `standing` or `conduct`; its only data calls are mod/2473.js:17 `V(await (0, k.fetchPassport)(t, o?.id)), \$(await (0, k.fetchPlayerAwards)(t))`. The screen that actually renders the sanction is `./conduct/index.tsx`, which loads it at mod/2372.js:19 `L(await (0, B.fetchUserSanctions)(t.id, t.id).catch(() => []))`, and the route is already wired for notifications two lines below at mod/2458.js:64-65 `("coc_update" === n.type ? B.push("/conduct"))`.	A player told "Disciplinary action: red card" taps through and lands on their own achievements page — no reason, no issuer, no status, no standing, no way to reach the conduct screen from there. The single moment where the platform owes the sanctioned user an explanation ends in a dead end, and the same wrong link is what every admin receives from the mis-targeted pending-ban notification (mod/631.js:12468).	Change mod/2458.js:62-63 to `B.push("/conduct")`, matching the `coc_update` branch immediately below it.
P-CTEST-5 High	./notifications.tsx — mod/631.js	Silent feature no-op	Push notifications are blocked by the app's own CSP and the failure is swallowed Token registration (mod/631.js:2515-2518) succeeds and stores tokens, so the product reports push as enabled, but every send is refused at the network layer and `.catch(() => {})` discards the rejection without logging. Evidence: mod/631.js:2569-2578 — `Ei = (e) => { const t = Qt.pushTokens.filter(...); if (0 === t.length) return; const {title:a, body:i} = Si(e); fetch("https://exp.host/-/api/v2/push/send", {method:"POST", ...}).catch(() => {}); }`, called from mod/631.js:2589 on every new notification. index.html line 6 CSP is `connect-src 'self' https://api.rush-x.xyz` — exp.host is not listed. Verified in the shipped bundle: the fetch is refused with `Refused to connect to 'https://exp.host/-/api/v2/push/send' because it violates the following Content Security Policy directive: "connect-src 'self' https://api.rush-x.xyz"`.	Every push a player has opted into — match starting soon, squad window closing, payment due, replacement found — is never delivered, and nothing anywhere reports it. For a pickup-sports app, 'your match is in an hour' silently not arriving is a no-show problem, not a cosmetic one. No suite opens ./notifications.tsx, so the blocked request has never appeared in a test run.	Add `https://exp.host` to the CSP connect-src in index.html, and replace empty `.catch(() => {})` at mod/631.js:2577 with `noteWriteFailure('push.send', e)` so a blocked or failing send is at least recorded.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-50 High	/passport/[id].tsx	Privacy controls / data exposure	mockGetPlayerAwards takes no viewer and no gate, so /passport/[id] leaks award history even when "Show achievements" is off The passport privacy toggle `achievements` empties `H.achievements`/`H.badges` (mod/631.js:6662-6663 `achievements: l ? d(g.ACHIEVEMENTS) : []`), but the awards payload is fetched from a completely separate, ungated function and rendered alongside it. Evidence: mod/631.js:15227 `r.mockGetPlayerAwards = async (e) => {` — a single subject parameter, no viewer, no audience/block/visibility check, returning `{ total,.mvp, by_award, recent, seasonal, ... }` where `recent` carries `date: a?.starts_at ?? e.created_at, venue: a ? Ai(a.venue_id) : ""` (mod/631.js:15249-15250). The facade passes it straight through: mod/671.js:699 `e.fetchPlayerAwards = (o) => t.store.mockGetPlayerAwards(o);`. The screen calls it for any id: mod/2473.js:17 `t && (V(await (0,k.fetchPassport)(t, o?.id)), \$(await (0,k.fetchPlayerAwards)(t)), O(1));` and renders the breakdown at mod/2473.js:203-204 `!!N && N.by_award.length > 0 &&` and the count at mod/2473.js:184 `n: K + (N?.total ?? 0)`,`.	A user who turns off "Show achievements" on their passport still has their full match-award record — every award key, its count, and the ten most recent awards with venue and date — served to any viewer. The toggle visibly changes one grid on the screen while the data it is supposed to hide is displayed directly above it.	Give `mockGetPlayerAwards` a viewer parameter and apply the `dr` should use (audience, block, profile_visibility, plus the subject `achievements` passport toggle), or have `dr` return the award subject itself so there is one gated read instead of two.
P-CARCH-17 High	/pay/[id].tsx	Error recovery / dead end	An interrupted payment leaves the charge in_flight forever and the payment can never be completed `in_flight` has no timeout, no expiry sweep and no operator reset. The catch at mod/631.js:7850 only clears it when the gateway call rejects in-process; if the tab is closed, the device sleeps, or the process is killed during the await, the row is stranded permanently. Evidence: mod/631.js:7839 the charge row is written `status: "in_flight"` and persisted before the gateway call: mod/631.js:7846 `(r ? Object.assign(r, t) : Qt.paymentCharges.push(t), await Za(Q, Qt.paymentCharges));` then mod/631.js:7848 `o = await Yr(e, i.amount_kwd, ...)` . Every later attempt hits mod/631.js:7834 `else if (r && "in_flight" === r.status) throw new Error("E_PAYMENT_IS_ALREADY_BEING_PROCESSED");`. Nothing ever ages an in_flight row out — the only other reader deliberately refuses to touch it: mod/631.js:8073 `const t = Qt.paymentCharges.find((t) => t.key === Ur(e)); if (t && "failed" !== t.status) continue;`. The screen's only response is mod/631.js:7834's message shown at mod/2474.js:188 `n.default.alert(H("error"), (0, T.storeErrorText)(e?.message ?? ""))    H("error"))`.	A player whose browser is closed mid-payment can never pay for that seat again. Every retry alerts "payment is already being processed", the seat hold expires, the booking is forfeited, and there is no button anywhere in the consumer app that clears the state. This is a permanent dead end reachable from one lost network connection.	Give the charge row a lease: record `created_at` (already present mod/631.js:7842) and in `\$r` (mod/631.js:7834) treat an `in_flight` than a short TTL as `failed` and re-enter the capture block; additior `Jr` (mod/631.js:8073) expire stale in_flight rows rather than skip
P-CSEC-51 High	/pay/[id].tsx	Payment authorization / disclosure	Choosing the Wallet payment method silently charges a card for any shortfall The user selected "pay from my wallet". The code converts that into "pay what the wallet covers and put the rest on a card" without a second confirmation, without showing the split, and without the user having chosen KNET. `Yr` should surface the shortfall to the caller and let the screen confirm, not substitute a payment instrument. Evidence: mod/631.js:7752 `if ("wallet" === n) { const n = await tc(e, (0, A.toFils)(t), a, i, "knet"); return `wallet_\${await (0, c.randomToken)(8)}\${n.external_fils > 0 ? "+knet" : ""}`; }` — the fallback method is hard-coded to "knet". Inside `tc`, mod/631.js:16248 `if (r.external_fils > 0) { if (!n    "wallet" === n    "cash" === n) throw new Error("E_INSUFFICIENT_WALLET_BALANCE_ADD_FUNDS_OR"); const e = `gw_\${n}...`; if (! (await Vr(e, r.external_fils / 1e3, n))) throw ... }` — because `n` arrives as "knet", the guard never fires and the card is captured.	A player with 2.000 KWD in the wallet taps Pay on a 6.000 KWD seat with Wallet selected and is charged 4.000 KWD on a card they never picked for this transaction. The success alert at mod/2474.js:181 says only "payment succeeded" — the split is never shown, and the ledger records only the wallet leg (mod/631.js:16258 `s > 0 && (...)` records `credits_fils + wallet_fils` only), so the card leg is invisible in wallet history too.	In `Yr` (mod/631.js:7751), pass `n` (the user's chosen method) then instead of the literal "knet", so the existing `E_INSUFFICIENT_WALLET_BALANCE_ADD_FUNDS_OR` guard combined payment is intended, return the planned split from `planCombinedPayment` to /pay/[id].tsx and require an explicit capture before capture.
P-CSEC-52 High	/pay/[id].tsx	Payment data exposure (IDOR)	fetchPayment accepts the caller id and then discards it — any payment record is readable by id mockGetPayment looks up a payment purely by its id and returns the whole row decorated with the venue name (`Lr`). It should compare the caller against `payment.payer_id` (and allow the payee venue staff / an admin), exactly as mockGetBookingPayments and mockPayRequest already do. Evidence: mod/631.js:7645 — `r.mockGetPayment = async (e, t) => {` / `await ei();` / `const a = Qt.payments.find((t) => t.id === e);` / `return a ? Lr(a) : null;`. The second parameter `t` is the caller and is never referenced in the body. mod/671.js:538 `e.fetchPayment = (o, c) => t.store.mockGetPayment(o, c);` and mod/2474.js:20 `A(await (0, S.fetchPayment)(e, c?.id))` both dutifully pass the session id. Every sibling in the same block does check: mod/631.js:7560 `if (t && a.organizer_id !== t && !ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");` (mockGetBooking), mod/631.js:7633 `Qt.payments.filter((a) => a.booking_id === t && (i    a.payer_id === e))` (mockGetBookingPayments), and the pay path itself at mod/631.js:7819 `if (i.payer_id !== e) throw new Error("E_THIS_PAYMENT_REQUEST_IS_NOT_YOURS");`.	Any signed-in user who obtains or guesses a payment id can open /pay/<id> and read another player's payment record: payer_id, payer_name, amount_kwd, payee_venue_id, method, gateway_ref, paid_at and status. Payment ids are handed out in notifications (type: "payment_request" carries `payment_id`, mod/631.js:7608) and in group-booking payloads, so they circulate outside the payer. This is a financial-privacy breach that survives a move to a real server, because the server would read `args[1]` the same way the mock does — from the request, not the session.	In `r.mockGetPayment` (mod/631.js:7645) reject when the caller the payer nor authorized for the payee venue: `if (t && a.payer_id !==vr(t, a.payee_venue_id) && !ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");`, and make argument mandatory rather than optional.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-12 High	./pay/[id].tsx	Error handling / unrecoverable state	A charge stuck in_flight permanently blocks the payment and the seat, with no recovery path `in_flight` has no timeout and no expiry sweep. Because the row is persisted to localStorage before the await, any interruption between 7846 and 7852 — tab closed during 3-D Secure, browser kill, crash, network drop — leaves it in_flight permanently. `paymentCharges` rows carry `created_at` but nothing reads it. Evidence: mod/631.js:7834 — `else if (r && "in_flight" === r.status) throw new Error("E_PAYMENT_IS_ALREADY_BEING_PROCESSED");`. The state is persisted before the gateway call at mod/631.js:7846 — `(r ? Object.assign(r, t) : Qt.paymentCharges.push(t), await Za(Q, Qt.paymentCharges));` — and only the in-process catch clears it (mod/631.js:7850): `throw ((Qt.paymentCharges.find((e => e.key === n).status = "failed"), await Za(Q, Qt.paymentCharges), e);`. A grep for `in_flight` across mod/631.js returns only 7834, 7839, 8114 — and 8114 is the read-only counter in `mockCountCharges`. No sweep expires it: `Zr` (mod/631.js:8262) only touches `Qt.payments`, and the reconciler refuses to act while the charge is live (mod/631.js:8073-8074): `const t = Qt.paymentCharges.find((t => t.key === Ur(e)); if (t && "failed" !== t.status) continue;`.	The player can never pay that bill: every retry on ./pay/[id].tsx throws "payment is already being processed" with no way out, and the Pay button is the only control on the screen. The seat is equally stuck — `mockReconcileSeatPayments` skips it because the charge is not "failed" — so the hold is held against capacity, the organizer sees an unpaid confirmed player they cannot clear, and nobody can reclaim or resell the seat.	Give `in_flight` charges a lease: stamp `created_at` (already present in mod/631.js:7843) and treat a row older than the gateway timeout when read at 7832, or add an expiry pass in `Zr`. Surface a "retry affordance on ./pay/[id].tsx for `E_PAYMENT_IS_ALREADY_BEING_PROCESSED` rather than alert.
P-CSEC-53 High	./pay/[id].tsx	Payment integrity / data exposure	Payment lookup ignores the viewer argument, so any payment record is readable by id The ownership check the neighbouring function performs is simply missing here, even though the caller id is already threaded through the facade for this purpose. Evidence: mod/631.js:7645-7649 — the second parameter `t` is the caller and is never referenced: ... <pre>r.mockGetPayment = async (e, t) => { await ei(); const a = Qt.payments.find((t) => t.id === e); return a ? Lr(a) : null; }; ...</pre> `Lr` (mod/631.js:7629) returns the raw payment row plus the venue name; the row carries `payer_id`, `payer_name` or(t), `payee_venue_id`, `amount_kwd`, `status`, `method`, `gateway_ref` (mod/631.js:7663-7683). The screen passes the caller and the route id: mod/2474.js:19 `A(await (0, S.fetchPayment)(e, c?.id))`, facade mod/671.js:538. Compare the sibling `mockGetBooking`, which does enforce it: mod/631.js:7559 `if (t && a.organizer_id !== t && !ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");`.	Anyone who obtains or enumerates a payment id reads another user's payment record: who paid, their name, how much, by which method, and the gateway reference. The same screen then offers to pay it, so the exposure sits directly on a money path.	In `mockGetPayment` (mod/631.js:7645) add `if (t && a.payer_id !== ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");` before returning, mirroring `mockGetBooking` at mod/631.js:7559.
P-CSEC-54 High	./pay/[id].tsx — mod/631.js (gateway module 641)	Secret management	The gateway webhook secret is a literal in the public bundle and cannot be changed by the documented config block The only signature-verified entry point in the payment system checks against a constant that is readable in plain text at rush-x.xyz, and the `process.env` fallback that was meant to override it cannot be populated in a static browser build. The config block at index.html lines 7-16, which the comment says is the only thing that needs editing, has no field for it. Evidence: mod/631.js:7934-7940 — `r.mockGatewayWebhook = async (e, t) => { ... if (!await u.sandboxProvider.verifySignature(e, t, u.GATEWAY_WEBHOOK_SECRET)) throw ... new Error("E_INVALID_WEBHOOK_SIGNATURE") ... }`. Module 641 in index.html defines `_e.GATEWAY_WEBHOOK_SECRET=undefined!=typeof process&&process.env?.GATEWAY_WEBHOOK_SECRET  "whsec_playora_sandbox"`. Verified at runtime in the shipped bundle: `__r(641).GATEWAY_WEBHOOK_SECRET` === "whsec_playora_sandbox", because `process.env` is `{"NODE_ENV":"production"}`.	Anyone who opens the bundle can compute `sha256("whsec_playora_sandbox:" + payload)` and forge a webhook that marks any payment succeeded — and if module 641 is lifted to the server as-is, the same literal default silently ships there too the first time GATEWAY_WEBHOOK_SECRET is unset. .env.example documents JWT_SECRET but never mentions this one.	Remove the `   "whsec_playora_sandbox"` fallback in module 641 secret fails loudly, keep the secret entirely server-side, and add it .env.example alongside JWT_SECRET.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-13 High	./privacy-controls.tsx	Error handling / dead end	/privacy-controls is a permanent dead end for guests and after any load error: full-screen spinner with no header, no error, no way back The screen conflates "still loading" with "failed", and the failure branch has no affordance. A shared `GateScreen` with retry and back already exists for exactly this (mod/9001.js:72-123) and is not imported here (mod/2477.js dependency list line 271 contains no 9001). Evidence: mod/2477.js:12-19 `(0, f.useFocusEffect)((0, a.useCallback)() => { t && (0, C.fetchPrivacy)(t.id).then(R).catch(() => R(null)); }, [t]);` and mod/2477.js:30-34 `if (!P) return (0, w.jsx)(p.SafeAreaView, { style: { flex: 1, backgroundColor: T.bg, alignItems: "center", justifyContent: "center" }, children: (0, w.jsx)(o.default, { color: T.accentText }) });` — the failure state and the loading state are the same `null`, and that branch renders only an ActivityIndicator: no header, no back button, no retry. The backend throws for any id without a profile row: mod/631.js:988-990 `const t = Qt.profiles.find((t => t.id === e); if (!t) throw new Error("E_PROFILE_NOT_FOUND");`. Guest sessions have ids like `guest:male` with no profile row (mod/630.js:74 `const s = { user: { id: `guest:\${t}`, email: "guest@playora.app" }, token: "guest" };`) and the root layout lets guests stay in the tabs (mod/1809.js:52-54). The entry point is unconditional: mod/1805.js:662-668 renders the Privacy tile with `onPress: () => J.push("/privacy-controls")` for every user.	Every guest who taps Privacy on the profile tab lands on a spinner that never resolves, with no in-app way out — only the browser back button. The same happens to a signed-in user whose profile row is missing or whose transport errors. `E_PROFILE_NOT_FOUND` is already mapped to a localised string (mod/674.js:167 `E_PROFILE_NOT_FOUND: "seProfileNotFound"`), so the message exists and is simply thrown away.	Track loading and error separately in mod/2477.js:10-11, and ren `GateScreen` from mod/9001.js with `kind: "error"`, the localised i from `classifyError`, `onRetry` re-running `fetchPrivacy`, and `onE or disable the Privacy tile at mod/1805.js:662 for guest sessions.
P-CSEC-55 High	./privacy-controls.tsx	Consent / biometric-adjacent data	The "Media consent" toggle is never consulted when clips are cut for a tagged player The toggle claims to control whether the user appears in other people's clips, but the clip generator tags and cuts a per-player reel for anyone the uploader names, with zero lookup of that player's `media_consent`. Uploads default to `visibility: "unlisted"` (mod/631.js:13385), and `mockGetMediaUpload` (mod/631.js:13417-13419) only blocks "private", so any user holding the upload id can fetch the clip list including the `player_id` tags. Evidence: mod/2477.js:210-212 writes it — `value: P.media_consent, onChange: (t) => z({ media_consent: t })`, labelled mediaConsentSub = "Allow appearing in others' clips and photos". The only consent check in the media pipeline is the uploader's own checkbox: mod/631.js:13374 `if (!t.consent) throw new Error("consent_required");`. Tagged players are accepted unfiltered at mod/631.js:13390 `player_tags: (t.player_tags ?? []).slice(0, 8)` and a personal reel is minted per tag at mod/631.js:13318-13336: <pre>... for (const a of e.player_tags) { const r = (0, N.buildClip)(t, "player_reel", e.clip_length, e.duration_seconds); r && n.push({ ... kind: "player_reel", player_id: a, ... }); } ...</pre> No occurrence of `media_consent` in mod/631.js (lines 974, 993, 1004, 1009, 1014, 1990, 2164, 2298, 5094, 5113, 5132, 5155, 6877, 9331, 9515) is a read on the media path.	A user who turned media consent off — the default for female-audience accounts, mod/631.js:2298 `media_consent: id` — still has a named highlight reel of herself produced and shareable by any uploader. This is a broken consent record for footage of identifiable people, exactly the thing a consent toggle exists to evidence.	In `mockUploadMatchVideo` (mod/631.js:13372), filter `t.player_t` `Qt.profiles.find(p => p.id === tag)?.media_consent` before storin check in the clip loop at mod/631.js:13318 so tags added later ca produce a reel; drop clips whose `player_id` has since revoked ca `mockGetClipSignedUrl`.
P-CARCH-18 High	./privacy-controls.tsx	Duplicated state / privacy enforcement	Two independent stores hold 'who can see my profile', and the one the /privacy-controls screen writes is never consulted by any visibility check Two screens both titled around profile visibility write to two different tables with two different value vocabularies, and only one of the two tables is honoured by the code that decides whether a profile appears in discover, search or the feed. Evidence: Store A: mod/631.js:997-1016 — `mockUpdatePrivacy` writes `a.privacy_visibility` onto the profile row `everyone` `same_audience` `connections` and persists `Za(Y, Qt.profiles)`. Store B: mod/631.js:14802-14812 — `mockSetPrivacySettings` writes `profile_visibility` (`public` `followers`) into `Qt.privacySettings` and persists `Za(qe, ...)`. Every enforcement site reads store B via `al` (mod/631.js:14031-14032), e.g. mod/631.js:16824 `public" === al(t).profile_visibility    Vd(e, t)` in `mockGetDiscover` and mod/631.js:14202 `if ("private" === al(u.id).profile_visibility) continue;` in `mockSearchPlayers`. Grepping `privacy_visibility` across mod/631.js returns only mod/631.js:991, 1002, 1007, 1012 (the read/write pair itself) plus seed defaults at 972, 1987-1988, 2162, 2296, 5092, 5111, 5130, 5153, 6875, 9329, 9513 — no enforcement site reads it. `/privacy-controls` (mod/2477.js:15, 24) uses store A; `/privacy` (mod/2478.js:16, 20) uses store B.	A user who sets 'Connections only' on `/privacy-controls` gets a preview that visibly hides their name (mod/2477.js:98) and a persisted value — but they remain fully visible in `/discover`, `/friends` search and the story tray, because those read store B, which still says `public`. Since the two screens can also be set to contradictory values, there is no single answer to 'is this profile public'. For a product whose female-audience partition depends on visibility controls, this is a silent failure of a user-facing privacy promise.	Pick one store. Migrate `privacy_visibility` into `Qt.privacySettings.profile_visibility` with a three-value vocabulary: `same_audience` `followers`, have `mockGetPrivacy` / `mockUpdatePrivacy` (mod/631.js:986, 997) d `al` / `mockSetPrivacySettings`, and add the `same_audience` cas `al(...)` checks at mod/631.js:14202 and 16824.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-56 High	/privacy.tsx	Consent integrity	/privacy seeds consent state from a profile that is still null on first render, silently revoking the untouched consent On a cold load of /privacy (deep link, refresh, or any navigation that mounts the screen before the profile resolves) both switches render off regardless of what the user actually consented to, and the first toggle the user flips writes `{analytics:false, marketing:false}` plus their one change — clearing the other consent they never touched. Evidence: mod/2478.js:10 `[E, R] = (0, l.useState)(u?.consent ?? { analytics: !1, marketing: !1 }`,` — a `useState` initializer with no effect anywhere in the module that re-syncs `E` when `u` (profile) arrives; the only focus effect (mod/2478.js:14-18) sets the privacy-settings and blocked-count state only. The profile is loaded asynchronously one tick after the session: mod/630.js:34-40 `(0, u.fetchProfile)(v.user.id).then((t => { P(t); ... })).catch(() => P(null))`. The write then sends the whole object: mod/2478.js:24-25 `const s = Object.assign({}, E, { [l]: o });` (R(s), await (0, B.updateConsent)(t.id, s), await M());` and mod/631.js:2391-2396 replaces the stored value wholesale: `Qt.profiles[a] = Object.assign({}, Qt.profiles[a], { consent: t, consent_updated_at: new Date()}).toISOString()`)`.	A user who previously granted analytics consent opens /privacy, sees both switches off (wrong), turns marketing on, and the stored consent record is rewritten with analytics revoked. The consent audit trail at mod/631.js:2397-2400 records the revocation as a deliberate user action. The same mechanism can silently re-grant nothing but silently revokes, which is the direction that creates both a compliance record mismatch and a support burden.	Replace the `useState` initializer at mod/2478.js:10 with state sync `u` (a `useEffect` on `u?.consent`, or render from `u.consent` dire hold the screen in a loading state until `profile` is non-null. Additive `mockUpdateConsent` merge the patch into the existing consent rather than replacing it.
P-CSEC-57 High	/privacy.tsx	Right to erasure	mockDeleteAccount leaves the user's messages, posts, follows, blocks, privacy settings and achievements in place "Delete account" is advertised under a Your Data heading with a destructive confirm, but performs a partial purge. Nothing anonymises or reassigns the residue either, so direct messages, feed posts, stories, team memberships and the social graph survive with the deleted user's id attached. Evidence: mod/631.js:2424-2445: `r.mockDeleteAccount = async (e) => { ... Qt.users = ..., Qt.profiles = ..., Qt.bookings = ..., Qt.reviews = ..., Qt.notifications = ..., Qt.gamePlayers = ..., Qt.chatMessages = Qt.chatMessages.filter((e) => e.author_id !== `self-\${t}`), await Promise.all([(...)], await (0, w.logAudit)("data.deleted", ...), await li(null)); };` — seven collections. The store holds many more that key on the same user id and are untouched: `Qt.dmMessages` (mod/631.js:2075), `Qt.blocks` (2076), `Qt.privacySettings` (2078), `Qt.presence` (2079), `Qt.stories` (2080), `Qt.follows` (2066), `Qt.posts` (2067), `Qt.friendships` (2061), `Qt.teamMembers` (2031), `Qt.passportPrivacy` (2021), `Qt.achievements` (2020). The screen presents this as full deletion: mod/2478.js:233-235 `onPress: async () => { (await (0, B.deleteAccount)(t.id), await H()); };`.	A user exercises deletion and their private messages and posts remain readable to their counterparties and teammates; follower counts on other accounts stay inflated by an account that no longer exists; their block list is dropped (so anyone they blocked is effectively unblocked) while their DMs are kept — the exact inverse of what the user asked for. Under Kuwait DPPR and GDPR-style expectations this is an incomplete erasure.	Extend `mockDeleteAccount` (mod/631.js:2424) to purge or tomb collection keyed on the user id — enumerate `Qt` rather than listing hand — and decide explicitly per collection between hard delete and anonymisation (DMs and posts usually need tombstoning so the thread stays coherent). Add a test that asserts no collection still exists id after deletion.
P-CSEC-58 High	/refunds.tsx	Refund handling / consumer protection	Consumer refunds are paid into a wallet balance that a plain user can never withdraw The refund path converts a card payment into closed-loop in-app credit for a class of user who has no exit. Nothing in the refund UI says so — mod/2479.js:253-259 labels the amount `youReceiveRow` ("You receive") and mod/2479.js:300 explains `refundedInFullWhy` ("Cancelled in time — the whole fee came back"), both of which read as money returned to the original instrument. Evidence: Every refund posts to the user's spendable wallet account: mod/631.js:16276 `postings: (0, A.transferPostings)(A.acct.platform("refunds"), A.acct.available(e, i)` in `ac`, called from the seat refund at mod/631.js:7715 and the admin refund at mod/631.js:7994. Withdrawal is then closed to that same user: mod/631.js:16058 `if (!((0, A.withdrawEligible)(a?.role ?? "user", ki(e).length > 0)) throw ... new Error("E_WITHDRAWALS_ARE_AVAILABLE_TO_VERIFIED_ORGANIZERS")`, where `withdrawEligible = (t,a) => `organizer`===t    `admin`===t    a` (module 655 in index.html). The UI hides the control entirely: mod/2506.js:139 `F.withdraw_eligible && (0, W.jsx)(v, { ... label: M("walletWithdraw") ... })`.	A player who pays 6.000 KWD by KNET and cancels in time is told the whole fee came back, but the money is now app credit they can only spend on Playora and can never withdraw. There is no disclosure at the point of refund and no route out of the balance.	Either refund to the original instrument (reverse the capture via the in `ac`, mod/631.js:16270) or, if closed-loop credit is intended, post `A.acct.credits(e)` rather than `available`, and state plainly in the copy (`youReceiveRow`, `refundedInFullWhy` in module 909) that amount is returned as Playora balance.
P-CSEC-59 High	/replacement/[id]	Authorization / sanctions	Accepting a replacement offer bypasses the suspension, guest and audience checks every other join enforces The replacement flow writes a booking directly instead of routing through the guarded join, so `E_ACCOUNT_BANNED_FROM_MATCHES` / `E_ACCOUNT_SUSPENDED` are unreachable on this path. Evidence: mod/631.js:12191-12239 `r.mockAcceptReplacement = async (e, t) => { await ei(); const a = ...; if (a.candidate_id !== e) throw ...; return Xt(a.game_id, async () => { ... Qt.bookings.push(l) ... }) }` — the whole function contains no `md(e)` (guest gate), no `await wd(e)` (ban/suspension gate) and no `ra(e, n.audience)` (audience gate). The normal join path applies all three: mod/631.js:2992 `ra(t, a.audience)` and mod/631.js:2996-2997 `md(t), await wd(t)`. The candidate shortlist does not filter sanctioned users either — mod/631.js:5257-5266 filters only on organizer, admin role and audience.	A player banned or suspended by the conduct system keeps receiving auto-generated replacement offers and can put themselves back into matches by tapping Accept in a push notification — the sanction is enforced on one of the two ways into a match. An organizer-created manual offer (mod/631.js:12318) can also seat a candidate across the audience partition.	In `mockAcceptReplacement` call `md(e)`, `await wd(e)` and `ra(e, n.audience)` before creating the booking (mod/631.js:12214), and sanction users from `Sn`/`ed` when shortlisting candidates.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CTEST-6 High	./replacement/[id]	Error handling	Replacement offer screen spins forever when the fetch fails and swallows decline failures Accept has a catch that alerts and refetches (mod/2480.js:110-115); load and decline do not. Rejections here are ordinary: the store proxy rethrows every backend error (mod/672.js:41-45) and the transport throws on a non-ok RPC body (mod/673.js:69). Evidence: mod/2480.js:14-16 `N = useCallback(async () => { e && (P(await fetchReplacementOffer(e, c?.id)), V(!1)); }, [e, c?.id]);` — `V(!1)` only runs after a successful await, and there is no try/catch, so any rejection leaves `E === true` and mod/2480.js:22-26 `if (E) return ... ActivityIndicator` renders indefinitely, with no error text, no retry and no back affordance. Decline is likewise unguarded: mod/2480.js:131-140 `onPress: async () => { if (c && O) { F(!0); try { (await declineReplacement(c.id, O.id), R.back()); } finally { F(!1); } } }` — a `finally` with no `catch`.	This screen is reached from a push notification with a minutes-long expiry. A single failed read — offline, a slow network, a server hiccup — traps the user on a spinner until the offer expires, and a failed decline leaves the screen in place with no message, so they tap again or give up while the seat stays reserved to them.	Wrap the body of `N` in try/catch, set an error state and always `finally`, render an EmptyState with a retry and back button for that `./game/[id]` does at mod/2379.js:103-122), and add a catch to the handler.
P-CARCH-19 High	./rewards/index.tsx	Reward redemption / dead end	Redeemed reward vouchers can never be used — nothing in the app consumes the code The redemption flow has a debit and no credit. There is no venue-side lookup, no `markVoucherUsed` mutation, and no field in the payment path that accepts a voucher — the separate promo mechanism (`mockValidateVenuePromo`, mod/631.js:15904) works on venue promo codes, not on loyalty redemptions. Evidence: Redemption destroys points and mints a code: mod/631.js:11602 `Qt.loyaltyLedger.push({ ... type: "redeem", points: -a.cost_points, ... })` and mod/631.js:11620 `code: `PLR-\${Hi()}`, status: "active"`. `Qt.loyaltyRedemptions` is then only ever written or listed — the complete set of references in the backend is mod/631.js:671 (init), 2040 (hydrate), 11626 (push), 11637 (`mockGetMyRedemptions`, list for display) and 11719 (admin count). No code path sets `status` to anything other than `active`, and no checkout, venue-scan or payment function reads a `PLR-` code. The screen can therefore only ever show expiry: mod/2481.js:332 `const l = "active" !== e.status    Date.parse(e.expires_at) < Date.now();`	A player spends 700 points on the "Nike 15% voucher" and receives a code that no screen in the app — and no staff tool in it — can accept or mark used. The points are gone, the voucher expires in 60 days (mod/631.js:11623), and the user has no way to realise the value or get the points back. The success copy ("Your code is below. It is also saved under My vouchers.") implies a redemption flow that does not exist.	Add a redemption-consumption mutation in mod/631.js that looks code up in `Qt.loyaltyRedemptions`, checks `status === "active"` expiry, sets `status = "used"` with a `used_at`, and wire it into `./venue/skan.tsx` and/or the discount step of the payment flow; unnot sell categories with no fulfilment path.
P-CSEC-60 High	./settings/region.tsx	Payment integrity	Region settings relabel the currency symbol without converting the amount; no exchange rate exists anywhere `formatMoney` is presentation-only but is wired to a user-facing currency selector, so the app claims a price is denominated in a currency it is not. The inconsistency is visible inside the app: the profile wallet chip hardcodes the real currency instead (mod/1805.js:208 `children: [(We / 1e3).toFixed(3), " KWD"],`), so the same balance reads "12.00 AED" on one screen and "12.000 KWD" on another. Evidence: mod/2485.js:117-128 offers every currency key: `Object.keys(f.CURRENCIES).map((l) => ... onPress: () => l({ currency: l })),`. The formatter only swaps symbol and decimal count: mod/912.js:138-142 `e.formatMoney = (n, r) => { const s = t[o.currency] ?? t.KWD, l = "ar" === r ? s.symbolAr : s.symbolEn; return `\$(f(n.toFixed(s.decimals), r)) \${l}`;`. There is no rate table and no conversion — `grep -rn "exchangeRate fxRate convertCurrency"` over the whole module dump returns nothing, and 631 stores every amount in KWD (mod/631.js:568-570 `price_minor: 25e3, currency: "KWD"`, mod/631.js:15961/16007 `currency: A.WALLET_CURRENCY`). The screen demonstrates the bug in its own preview: mod/2485.js:47 `children: (0, p.formatPrice)(12),`.	A user who selects AED, SAR or GBP sees every game fee, refund quote and wallet amount in the app relabelled with that symbol while the actual charge stays in KWD — roughly a 10x understatement for SAR/AED and a 2.6x overstatement for GBP. The decimal count also changes (KWD 3 → GBP 2), so 12.345 KWD renders as "12.35 £". This is a consumer-pricing misstatement reachable in two taps from settings.	Either restrict the currency list in mod/2485.js:117 to the currencies the backend actually settles in (KWD only today), or add a rate source to convert in `formatMoney` (mod/912.js:138) while keeping the stored units canonical. Until a rate source exists, remove the currency selector from the region screen.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-61 High	/squad/[gameId].tsx	Private-group membership exposure	Private matches have no membership gate: any same-audience user with the game id gets the full roster, squad list and lineup `visibility: "private"` is enforced when listing games and nowhere when reading one. No `game.visibility !== "private"    hasBooking(caller)    caller === organizer_id` check exists in 631 on the detail, roster, squad or lineup paths. Evidence: `mockGetGame` checks the audience partition but never the game's `visibility`: mod/631.js:2696-2703 ``` const Ni = async (e, t) => { (await ei(), await tn(), await Oi(e)); const a = hi(e); if (!a) return null; ra(t, a.audience); ``` Everything downstream inherits that. `mockGetSquadState` gates only on `Ni` and then returns names and photos for every booking: mod/631.js:4045-4052 / 4106-4107 `members: s, waitlist: d`. `mockGetGamePlayers` (mod/631.js:2878) has no gate at all, and `mockGetLineup` (mod/631.js:2620-2625 → 16420-16425) calls it with no additional check. By contrast the listing path does filter: mod/631.js:2685 `(e) => "scheduled" === e.status && "public" === e.visibility && ...`, and the game screen itself renders a Private badge (mod/2379.js:284-285), so the visibility flag is real and load-bearing everywhere except the read gate.	A private match — the mechanism a closed group uses to keep a session to itself — leaks its entire membership to any same-partition user who obtains the id from an invite link, a referral row or a shared screenshot: who is confirmed, who is waitlisted, their names and photos, and the lineup positions. The private-group boundary the product sells does not exist on the read path.	Add a single guard in `Ni` (mod/631.js:2696) — after `ra`, if `a.vis "private"` require `t === a.organizer_id` or a non-cancelled bookin and apply the same guard directly in `Ui` (mod/631.js:2878), `mockGetSquadState` (mod/631.js:4043) and `uc` (mod/631.js:16 those are reachable without going through `Ni`.
P-CQUAL-14 High	/stories/compose.tsx	Data persistence / broken media	Photo stories are stored as blob: object URLs, so every photo story is permanently blank for all viewers An object URL is valid only inside the document that created it and dies on reload. Ya is the sanitiser that is supposed to move an image into blob storage and return a durable `/media/` path, but it silently passes through anything that is not a base64 data URI instead of rejecting it. The compose screen should convert the picked file to a data URI (or call putBlob directly) before handing it to createStory, and Ya should reject any uri that is not a data: URI or an existing `/media/` path. Evidence: pickMedia (module 1808, inlined in index.html) returns a tab-scoped object URL: s{uri:URL.createObjectURL(t), type:..., name:t.name}) mod/2488.js:25 and mod/2488.js:62 pass it straight through: const t = await (0, w.pickMedia)("image"); mediaUri: V?.uri ?? null, mod/631.js:14831 stores it via Ya: media_uri: await Ya(t.mediaUri), and Ya only understands data: URIs - mod/631.js:1729-1731: if (!e) return null; const t = Ba.exec(e); if (!t) return e; with Ba at mod/631.js:1722: const Ba = /^(data:(imageV(?:png jpe?g webp gif));base64,([A-Za-z0-9+/=]+)\$)/, A blob: URL misses the regex and is written verbatim into playora.mock.stories.v1. The viewer renders it unconditionally - mod/2487.js:99-105 passes it to ApplImage, and mod/2487.js:95 blacks out the background whenever it is truthy: { style: [v.full, { backgroundColor: Z.media_uri ? "#000" : (Z.bg_color ?? "#0f172a")}] }, ApplImage (module 1632) has no onError and mediaUrl (module 1660) passes non-/media/ URIs through untouched.	Every photo story posted from the web app is broken. The author sees it once in the tab that created it; after a reload, and for every other viewer, the image 404s, ApplImage renders nothing, and because media_uri is truthy the background is forced to #000 - so the story is a pure black screen with, at best, the caption text (which is optional: mod/2488.js:22 allows a photo with no text). Nothing in the UI signals a failure. As a side effect the 700 KB size cap at mod/631.js:1733 and the image-type allowlist in Ba are unreachable on this path, so story images have no validation at all, and URL.createObjectURL is never revoked so each pick leaks its blob for the tab's lifetime.	In mod/2488.js:24-27, read the picked File into a base64 data UF (FileReader) before setting state, and call URL.revokeObjectURL object URL once consumed. Harden Ya (mod/631.js:1727) to thr E_UNSUPPORTED_IMAGE instead of `return e` when the input neither Ba nor the MEDIA_PATH prefix, so no non-durable uri cai playora.mock.stories.v1.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-62 High	/teams/[id].tsx	Membership rules / moderation bypass	Joining an invite-only clan by code reinstates banned and removed members and ignores clan suspension <code>`mockJoinTeamByCode`</code> duplicates the membership-creation logic of <code>`ho`</code> instead of delegating to it, and in doing so drops the team-status check and the ban check. A member whose status is <code>`banned`</code>, <code>`removed`</code> or <code>`rejected`</code> is silently promoted back to <code>`active`</code>. Evidence: <code>mod/631.js:8686-8689</code>, the normal join path, blocks suspended clans and banned users: <pre>... 8686 if (!a "active" !== a.status) throw new Error("E_TEAM_NOT_FOUND"); 8687 const i = Qt.teamMembers.find((a => a.team_id === t && a.user_id === e); 8688 if (i && ("active" === i.status "pending" === i.status)) return i; 8689 if ("banned" === i?.status) throw new Error("E_YOU_CAN_NO_LONGER_JOIN_THIS"); ...</pre> <code>mod/631.js:8734-8751</code>, the code path, does neither — it reuses the existing row whatever its status and forces it active: <pre>... 8734 "invite_only" === i.privacy 8735 ? eo(i.id, async () => { 8736 const t = Qt.teamMembers.find((t => t.team_id === i.id && t.user_id === e); 8737 if (t && "active" === t.status) return t; 8738 const a = t ?? { ... status: "active", ... }; 8749 (a.status = "active"), ...</pre> The only checks <code>`mockJoinTeamByCode`</code> performs are the code lookup (8730-8731) and the audience partition (8733).	A captain removes or bans a disruptive player; that player re-enters the clan in one tap using a code they already have (or that any visitor can read — see the invite-code finding). Moderation decisions are not durable. The same call also lets anyone join a clan that an admin has suspended via <code>`mockAdminSuspendTeam`</code> , defeating the suspension.	In <code>mod/631.js:8735</code> , before mutating, add <code>`if (!i    "active" !== i.status) throw new Error("E_TEAM_NOT_FOUND")`</code> and <code>`if ("banned" === t?.status) throw new Error("E_YOU_CAN_NO_LONGER_JOIN_THIS")`</code> , or refactor <code>bx</code> to share one helper so the rules cannot drift apart again.
P-CQUAL-15 High	/teams/[id].tsx	Dead end / unreachable state	A clan owner can never leave or delete their clan — the error tells them to transfer ownership, which the consumer UI cannot do The backend precondition and the UI capability set disagree. The only escape route the error names (<code>`transfer ownership`</code>) has no button anywhere in the consumer app, and there is no <code>`deleteTeam`</code> in <code>mod/671.js</code> either. Evidence: <code>mod/631.js:8795</code> blocks the owner: <code>`if ("owner" === a.role) throw new Error("E_TRANSFER_OWNERSHIP_BEFORE_LEAVING_THE_TEAM");`</code> <code>mod/2489.js:366-376</code> therefore renders nothing at all for an owner — no leave, no delete: <pre>... 366 H.viewer_member 367 ? "owner" !== Je 368 ? (0, F.jsx)(w.Button, { title: C("leaveTeam"), ... }) 376 : null ...</pre> <code>mod/2489.js:787-791</code> is the only role control the screen offers, and it only promotes a player to captain: <pre>... 787 Ye && 788 "owner" === Je && 789 "player" === e.role && 791 onPress: () => Ge() => (0, L.changeMemberRole)(r.id, H.id, e.id, "captain"), ...</pre> The backend supports the transfer (<code>mod/631.js:8825-8828</code> handles <code>`i`</code> === <code>"owner"</code> by demoting the old owner), and <code>mod/671.js:564</code> exports <code>`changeMemberRole`</code> — but no consumer screen ever passes <code>"owner"</code>.	Every user who creates a clan — including one created by accident, with a typo'd name, or abandoned — is bound to it permanently. They remain the accountable owner of a roster and a ladder position they cannot walk away from, and support has no self-service remedy to offer. The clan cannot be wound down, so abandoned clans accumulate in rankings and rival lists forever.	Extend the role menu in <code>mod/2489.js:787</code> with a "Make owner" at <code>`owner`</code> viewers that calls <code>`changeMemberRole(r.id, H.id, e.id, "owner")`</code> (already handled at <code>mod/631.js:8825</code>), and add a delete/archive button to the menu. Until then, change the message message to <code>`E_TRANSFER_OWNERSHIP_BEFORE_LEAVING_THE_TEAM`</code> 674 so it does not instruct the user to do something the app cannot do.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-16 High	/teams/[id].tsx	Dead end / user feedback	Requesting to join a private or invite-only clan produces no feedback and no visible pending state `Ge` (mod/2489.js:90-99) only surfaces failures; the success path just calls `Ue()`. Because `viewer_member` stays false and `viewer_role` stays null for a pending member, the page re-renders byte-identical to before the tap. The screen has a `Ke` pending list (line 126) but renders it only for managers (line 693). Evidence: mod/2489.js:377-386 sends the request and then reloads, with no success handling: <pre>... 377 : (0, F.jsx)(w.Button, { 378 title: "public" === H.privacy ? C("joinTeam") : C("requestToJoin"), 381 onPress: () => 382 Ge(async () => { 383 await (0, L.joinTeam)(r.id, H.id); 384 })), ...</pre> mod/631.js:8704-8706 creates the row as `pending` for any non-public clan: <pre>... 8704 (r.status = n ? "active" : "pending"), 8705 (r.joined_at = n ? new Date().toISOString() : null), 8706 (r.requested_at = n ? null : new Date().toISOString()), ...</pre> mod/631.js:8423-8425 means a pending member is invisible to the viewer-state helpers, so the reload changes nothing: <pre>... 8423 ao = (e, t) => Qt.teamMembers.find((a) => a.team_id === e && a.user_id === t && "active" === a.status), 8425 no = (e, t) => !t && !ao(e, t), ...</pre> Re-tapping returns the existing row silently (mod/631.js:8688) — no error, no change.	A user taps "Request to join", the button spins, and the screen comes back exactly as it was. They cannot tell whether the request was sent, so they tap again and again — each tap is a silent no-op. There is nowhere in the app that shows their own outstanding request or lets them cancel it, so they are left with no signal until a captain happens to approve.	Have `fo()` in mod/631.js:8603-8609 also expose `viewer_status` (active/pending/rejected), and in mod/2489.js:377 swap the button disabled "Request pending" chip plus a cancel action when that view is `pending`. At minimum, show a confirmation toast in `Ge` when the toast resolves with `status === "pending"`.
P-CSEC-63 High	/teams/[id].tsx	Broken access control (private group bypass)	mockGetTeamInvite hands any caller a private team's invite code, and joinTeamByCode then grants instant membership The invite code is the only secret protecting an `invite_only` team, and a read endpoint with no authorization gives it away. `mockGetTeam` deliberately blocks `private` teams but not `invite_only` ones, so a non-member can load /teams/<id>, receive the code in the same render pass, and redeem it. Evidence: mod/631.js:9199 — `r.mockGetTeamInvite = async (e) => {` / `await ei();` / `const t = to(e);` / `return t ? { code: t.invite_code, link: `https://playora.app/t/\${t.invite_code}` } : null;`. It takes only a team id: no caller, no membership test. Every other team read is gated — mod/631.js:8636 `return a ? (ra(t, a.audience), "private" !== a.privacy    no(e, t)    (t && ro(t)) ? fo(a, t) : null);` and mod/631.js:8659 `"private" !== a.privacy    no(e, t)    (t && ro(t))`. Redemption then bypasses approval entirely: mod/631.js:8734 `invite_only` === i.privacy ? eo(i.id, async () => {` ... mod/631.js:8744 `status: "active";` / mod/631.js:8745 `joined_at: new Date().toISOString();` — whereas the normal path mod/631.js:8690 `const n = "public" === a.privacy;` leaves non-public joins with `pending`. The screen fetches it for every viewer: mod/2489.js:68 `(0, L.fetchTeamInvite)(e).catch(() => null);`.	Any user who knows a team id joins any `invite_only` team as an active member with no owner approval and no notification. Active membership unlocks the team chat (`lo` gate at mod/631.js:9056/9063), the member roster with usernames, the calendar and the event history. The team privacy setting offered in the creation UI (mod/2498.js:407 `_ = ["public", "private", "invite_only"]`) is therefore not enforced.	Give `r.mockGetTeamInvite` a caller argument and require manager approval (e.g. `await so(teamId, caller)`), matching `mockPinTeamChat` / `mockSetTeamBadge`. Separately, make the `invite_only` branch of `mockJoinTeamByCode` (mod/631.js:8734) require approval to grant `pending` membership that an owner or captain approves through `mockDecideJoinRequest`.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-17 High	/teams/battle/[battleId].tsx	Dead end / error handling	The battle screen is a permanent spinner with no way out when the battle id does not exist `null` and "still loading" are represented by the same state, so a legitimately missing battle is indistinguishable from an in-flight request. The screen already knows how to handle the audience case (`if (\$) return <NotAvailable/>` at line 65) but has no equivalent for not-found. Evidence: mod/631.js:10047-10050 returns null rather than throwing for an unknown id: <pre>... 10047 r.mockGetClanBattle = async (e, t) => { 10048 (await ei(), await Bo()); 10049 const a = Qt.clanBattles.find((t) => t.id === e); 10050 if (!a) return null; ... mod/2495.js:66-71 renders a bare spinner for that case, with no header and no back control — the back button at line 117 sits *after* this early return: ... 66 if (!U) 67 return (0, v.jsx)(m.SafeAreaView, { 68 edges: ["top"], 69 style: { flex: 1, backgroundColor: W.bg, alignItems: "center", justifyContent: "center" }, 70 children: (0, v.jsx)(s.default, { color: W.accentText }), 71 }); ... The loader at mod/2495.js:17-28 has no `finally` and no null branch: `H(await (0, k.fetchClanBattle)(e, c?.id))`.</pre>	Battle links are shared — mod/2496.js:364-367 and 386 explicitly push them into WhatsApp, and mod/2489.js:908/936 links to them from the clan page. A recipient who opens a link to a battle that was deleted, or who lands on a stale link after a store reset, gets a spinning indicator forever with no back button, no message and no navigation. The only escape is the browser chrome or reinstalling the app.	In mod/2495.js:17-28 add a `notFound` state set when `fetchClan` resolves to null, and render `NotAvailable` (module 2385, already as `A`) for it; separately track `loading` so the spinner clears. Wrap in try/finally so a thrown non-audience error also terminates the s
P-CARCH-20 High	/teams/battle/[battleId].tsx	Concurrency / data integrity	Battle accept and result submission are not serialized, so simultaneous captains create duplicate games and duplicate result rows Every team-membership mutation is wrapped in `eo(teamId, ...)` , but the battle lifecycle — accept/decline, submit, counter, confirm — is not, even though a clan legitimately has several co-captains who can all act (`canManageTeam` admits owner, captain and co_captain, module 667). Evidence: mod/631.js:8413-8421 is the per-team serialization helper the codebase already has: <pre>... 8413 async function eo(e, t) { 8414 const a = (Xr.get(e) ?? Promise.resolve()).catch(() => {}).then(t); 8415 Xr.set(e, a); ... It is applied only at 8684, 8735, 8767, 8792, 8805 and 8819 — all membership calls. The battle path never uses it. mod/631.js:10725 checks the status, then awaits repeatedly before writing, leaving a wide window: `if ("pending" !== i.status) throw new Error("E_THIS_CHALLENGE_WAS_ALREADY_ANSWERED");` ... then at 10776 `Qt.games.unshift(r, { game_id: r.id }, await Za(te, Qt.games),` mod/631.js:10241 and 10284-10299 do the same for results — a read-then-push with no lock: ... 10241 const d = Qo(t); 10242 if (d) { ... } 10284 const l = { battle_id: t, ... cycle: 1, ... }; 10298 return (10299 Qt.battleResults.push(l), ... `Qo(e)` (mod/631.js:10075) is `Qt.battleResults.find((t) => t.battle_id === e)` — it returns only the first row, so a second row is orphaned but still persisted.</pre>	Two co-captains accepting the same gauntlet at once both pass the pending check and both run the game-creation block at mod/631.js:10738-10782: two paid games (price_kwd 3.5, full player caps) appear in the games list, only one is linked to the battle, and both clans get duplicate confirmation notifications and duplicate system chat lines. On the result path, both sides submitting at the same moment produce two `battleResults` rows for one battle; the orphan is invisible to the UI but is swept by `Xo()` (mod/631.js:10098-10125), generating spurious escalation notifications and a duplicate entry in the admin escalation queue.	Wrap the mutating bodies of `mockRespondClanChallenge` (mod/631.js:10721), `mockSubmitBattleResult` (10224), `is` (104 (10313) in `eo(battle.id, ...)` so all writes for one battle are serialized inside the critical section rather than before

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-21 High	/teams/battle/[battleId].tsx	Error handling / dead end	/teams/battle/[battleId] rethrows inside a focus effect and shows a permanent spinner for any battle id the store does not know Two separate paths land on the same non-terminal state. A store error rejects an unhandled promise (‘throw e’ at mod/2495.js:25 inside an effect callback) and leaves ‘U’ null; a missing battle returns null and also leaves ‘U’ null. Both render the same spinner, which has no header, no back control and no retry. Evidence: mod/2495.js:17-28 — ‘Z = useCallback(async () => { ... try { (H(await fetchClanBattle(e, c?.id)), ...) } catch (e) { if (isAudienceError(e)) return void O(!0); throw e; } }, [e, c?.id])` and mod/2495.js:29-33 `useFocusEffect(useCallback(() => { Z(); }, [Z]))` — the promise is never awaited or caught. mod/631.js:9975-9980 — `mockGetClanBattle` returns `null` for an unknown id: `const a = Qt.clanBattles.find((t) => t.id === e); if (!a) return null;`. mod/2495.js:66-71 — `if (!U) return <SafeAreaView edges={[{“top”}] style={{flex:1, ..., justifyContent:“center”}}><ActivityIndicator/></SafeAreaView>`.	Battle links are shared into chat and expire. Opening a stale `teams/battle/<id>` deep link — or hitting any store error on this screen — gives an eternal spinner on a screen with no navigation affordance, plus an unhandled rejection in the console. The user’s only escape is the browser back button, which the in-app dock does not provide on this route.	Replace `throw e` at mod/2495.js:25 with a setter for an error state to distinguish `null` (not found) from `still loading` with an explicit `loading` boolean, and render `GateScreen` (mod/9001.js:72) with retry in both cases.
P-CSEC-64 High	/venue/[id].tsx	Reputation integrity / attribution forgery	Venue reviews and organizer ratings take the author id from the request payload Both functions use a client-supplied author id as the dedupe key and as the stored attribution, and neither checks that the author ever booked the venue or played the match being rated. Evidence: mod/631.js:2948 — `r.mockUpsertReview = async (e) => {` where `e` is the whole row and the identity comes from it: `const ... i = Qt.reviews.findIndex((t) => t.venue_id === e.venue_id && t.user_id === e.user_id);` then `Qt.reviews.push({ id: ea(), venue_id: e.venue_id, user_id: e.user_id, rating: t, comment: a, ...})`. There is no caller parameter — mod/671.js:400 `e.upsertReview = (o) => t.store.mockUpsertReview(o);` — and the screen simply supplies its own id: mod/2502.js:130 `await (0, v.upsertReview)({ venue_id: L.id, user_id: L.id, rating: O, comment: N })`. The same shape recurs at mod/631.js:4297 `r.mockRateOrganizer = async (e) => {` with mod/631.js:4300 `a = Qt.orgRatings.findIndex((t) => t.game_id === e.game_id && t.rater_id === e.rater_id);` and mod/631.js:4305 `organizer_id: e.organizer_id,` — `rater_id`, `game_id` and `organizer_id` all arrive from the client. Those rows feed the public trust tier: mod/631.js:4265 `c = Qt.orgRatings.filter((t) => t.organizer_id === e),` inside `mockGetOrganizerReputation`.	A review or rating can be attributed to a user who never wrote it, and because the uniqueness key is `(venue_id, user_id)` / `(game_id, rater_id)`, a single actor can fabricate an unlimited number of distinct ratings by varying the id fields. `mockGetOrganizerReputation` averages those rows into the score and tier (‘elite’/‘trusted’/‘rising’) shown to players choosing a match, so organizer reputation and venue star ratings can be manufactured or sabotaged at will.	Change both signatures to take the caller explicitly — `mockUpsertReview(callerId, payload)` and `mockRateOrganizer(payload)` — set `user_id`/`rater_id` from the caller only, and require participation (a `confirmed` booking at that venue / in that game) before `mockSubmitSkillEvaluation` does at mod/631.js:5591).
P-CSEC-65 High	/venue/[id].tsx	Audience partition / data exposure	Venue reviews expose the reviewer’s real name and photo with no viewer, partition, or block filter Every other identity-bearing read in 631 takes the caller and applies `ra`/`ia` for the audience partition, `Xd` for blocks, and `avatar_mode` for the photo setting. This one takes no caller at all, so all three are structurally impossible. Evidence: mod/631.js:2781-2790 — the function takes only a venue id, so no caller-scoped filtering is possible: <pre>... r.mockGetReviews = async (e) => (await ei(), Qt.reviews.filter((t) => t.venue_id === e) map((e) => { const t = Qt.profiles.find((t) => t.id === e.user_id); return Object.assign({}, e, { author: t ? { full_name: t.full_name, avatar_url: t.avatar_url } : void 0, }); }) ... `Object.assign({}, e, ...)` also spreads the raw review row, which carries `user_id` (mod/631.js:2956-2962). Facade `e.fetchReviews = (o) => t.store.mockGetReviews(o)` (mod/671.js:399); rendered at mod/2502.js:165 `children: e.author?.full_name    V("player")`.</pre>	A female user who reviews a venue has her full name, her avatar photo and her user id published on a page every male user can open, defeating the partition. It also exposes her to a user she has blocked, and it ignores her `avatar_mode: "initials"` choice. The reviewer’s `user_id` in the payload lets a viewer pivot to other per-id endpoints.	Change the signature to `mockGetReviews(venueId, viewerId)` (at mod/671.js:399 to pass it), filter out reviews whose author fails `ia(aa(viewerId), aa(review.user_id))` or `Xd(viewerId, review.user_id)` to respect `avatar_mode`, and project the row instead of spreading `user_id` stays server-side.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-66 High	/venue/apply.tsx	Content review bypass	A venue registered from /venue/apply is published to the public directory immediately, before any admin review Two code paths create a venue pending review and only one of them withholds it from the directory. The /venue/apply path omits the `listed: false` / `review_status: "pending"` pair that the organizer custom-venue path sets, and `mockGetVenues` treats a missing flag as listed. Evidence: mod/631.js:6993-7008 — the venue row mockApplyVenue pushes: `d = { id: r, name: e, city: "Kuwait City", area: i, sports: ..., custom: l0, created_at: new Date().toISOString() }` — no `listed` and no `review_status`, mod/631.js:2490 — `r.mockGetVenues = async () => (await pi()).filter((e) => !1 !== e.listed);` — `false !== undefined` is true, so the new row passes. The organizer path does it correctly, mod/631.js:3426-3450 — `// ORG1 (F-ORG1-19): a venue an organizer types in ... stays out of the public directory (listed: false) until an admin reviews` ... `custom: l0, listed: l1, review_status: "pending";`. Admin review is what is supposed to set the flag, mod/631.js:7149-7155 — `const lst9 = "approved" === s; ... Object.assign({}, Qt.venues[vi9], { listed: lst9, review_status: s })`. Public consumer of the unfiltered list: mod/2450.js:14 — `(0, j.fetchVenues)()` on /map.tsx.	Any signed-in user (and, per the guest finding, any guest) can publish an arbitrary venue name onto the public map instantly, with no review. The app's own copy promises the opposite — venueApplicationHint reads "An admin reviews every application before you go live." Rejection later sets listed:false, but the venue is public for the whole review window, and if no admin ever reviews it, permanently.	In mockApplyVenue (mod/631.js:6993) add `listed: !1, review_sta` "pending", created_by: e` to the venue object so it matches the o path at mod/631.js:3449-3451, and make `mockGetVenues` requ e.listed` rather than `!1 !== e.listed` so a future path that forgets t closed.
P-CSEC-67 High	/venue/apply.tsx	KYC / onboarding fraud	Venue registration collects no identity, contact or duplicate checks, while the organizer application in the same backend demands full KYC The venue role carries booking acceptance, price setting, commission liability, revenue reporting and (per the scanner) attendance authority, yet its application has none of the identity verification, duplicate-document detection, submission-velocity cap, re-apply cooldown or already-applied guard that the organizer application enforces. There is also no check that the caller already has a pending or approved venue profile, so the same user can submit unlimited applications, each creating a new public venue row. Evidence: mod/631.js:6978-7032 — the entire validation in mockApplyVenue: `if (ro(e)) throw new Error("E_ADMINS_CANNOT_REGISTER_VENUES"); ... const e = (0, v.sanitizeText)(t.name ?? "", 80); if (le) throw new Error("E_ADD_YOUR_VENUE_NAME");`; then it pushes the venue and a profile with `status: "pending", commission_type: he, commission_value: l0`. No owner-identity fields, no duplicate check, no velocity limit, no status guard against an existing profile. The organizer application, which grants strictly less financial authority, is far stricter — mod/631.js:4771-4801: `const a = sanitizeName(t.full_legal_name); if (la) throw new Error("E_ENTER_YOUR_FULL_LEGAL_NAME"); ... if (!/^data:imageV(?:png jpe? glwebp);base64/,.test(img9)) throw new Error("E_UPLOAD_YOUR_IDENTITY_DOCUMENT"); ... if ("under_review" === d?.status) throw new Error("E_YOUR_APPLICATION_IS_ALREADY_UNDER_REVIEW"); if ("rejected" === d?.status && d.reapply_after && ...) throw new Error(`E_REAPPLY_IN_DAYS:\${e}`); ... Qt.applications.some((t) => t.user_id !== e && t.id_doc_ref === _) && u.add("duplicate_document");length >= 2 && u.add("velocity");`. The form matches: mod/2503.js:100 — `disabled: !V.trim() 0 === _length` — a name and one sport chip is the complete submission.	Anyone can flood the admin review queue and the public venue directory with venue registrations under fabricated names, at zero cost and with no traceable identity behind them. A rejected applicant can resubmit the same venue immediately under a new name. Admins reviewing a registration have nothing to review — mockGetPendingVenues (mod/631.js:7104-7116) can only show them a name, area and sports, because that is all that was ever collected.	Bring mockApplyVenue (mod/631.js:6978) to parity with mockApp (mod/631.js:4770): require owner legal name, verified mobile and commercial-licence reference, and a real address/location; reject caller already has a pending or approved venue profile; apply the hour submission-velocity cap and post-rejection cooldown; and e /venue/apply form (mod/2503.js:53-95) to collect them.
P-CTEST-7 High	/venue/portal.tsx	Error handling / dead end	Portal and scanner hang on a permanent spinner with no error state and no retry when the first fetch fails or the user is momentarily null An `await` that rejects skips the loading-flag reset, so the rejection becomes an unhandled promise and the screen stays in its indeterminate spinner forever. The same happens whenever `user` is null. There is no error UI, no retry control and no way back other than the browser. Evidence: mod/2504.js:19-29 — `le = useCallback(async () => { if (le) return; const t = (await S.fetchMyVenues(e.id))[0] ?? null; if ((N(t), t) { ... ..catch() => []} ... } J(!1); }, [e]);` — `J(!1)` (clear loading) is reached only on the success path, and `fetchMyVenues` is the one call with no `catch`. mod/2504.js:45-49 — `if (q) return SafeAreaView({ ... children: ActivityIndicator })` is the entire loading state: no timeout, no error branch, no retry. Identical shape in mod/2505.js:15-34 — `if (le) return; const t = await _fetchMyVenues(e.id); ... E(!1);`. Neither module imports the shared gate helpers: the dependency lists at mod/2504.js:620-623 and mod/2505.js:298-301 contain no 9001, so `GateScreen` / `RetryButton` (mod/9001.js:65-123) are unused here.	With `window.__PLAYORA_CONFIG__backendUrl` set, any 5xx, timeout or dropped connection on the first request leaves the venue owner staring at a spinner for the whole session — the exact behaviour a venue hits on a flaky mobile connection at the front desk while a player waits to be checked in. Reloading repeats it. This is a hard dead end on both revenue-bearing screens.	Wrap the body of `le` (mod/2504.js:19) and `G` (mod/2505.js:15) try/catch/finally so the loading flag always clears, hold the caught state, and render `GateScreen` from mod/9001.js with `kind: "err` `onRetry` that re-invokes the loader. Add 9001 to both dependenc

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-68 High	/venue/portal.tsx	Authorization	Every venue management API is authorized on ownership alone and never on approval status Approval status gates only the creation of *new* bookings (via `kr` inside mockReserveCourt, mod/631.js:7342). Nothing gates management. A venue whose registration is pending, rejected or suspended keeps full write access to courts, prices, auto-accept, staff, existing bookings, revenue reads and attendance. Evidence: mod/631.js:6822-6837 — `const vr = (e, t) => { const a = hr(t); return !a && (a.owner_id === e !a.staff.some((t) => t.user_id === e) "admin" === Qt.profiles.find((t) => t.id === e)?.role); }; async function Sr(e, t) { if (!vr(e, t)) throw (await logAudit("venue.access_denied", ...), new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_MANAGE_4")); }` — `a.status` is never consulted. An approved-only helper exists right above it and is not used here: mod/631.js:6818-6821 — `yr = (e) => "approved" === hr(e)?.status; async function kr(e) { if (!yr(e)) throw new Error("E_THIS_VENUE_IS_NOT_CURRENTLY_ACCEPTING"); };` `Sr` is the sole gate on mockUpdateVenueProfile (7057), mockAddVenueStaff (7072), mockRemoveVenueStaff (7090), mockUpsertCourt (7225), mockSetCourtActive (7269), mockVenueDecideBooking (7424), mockGetVenueBookings (7548) and mockGetVenueRevenue (8364). The screen gates nothing either — mod/2504.js:100-107 only swaps a hint card: `~approved~` != ae.status && Card({ ... children: "pending" === ae.status ? W("venuePendingHint") : W("venueSuspendedHint") })`, while the Accept/Reject buttons (178-189), court editor (211-361), auto-accept switch (385-388) and staff controls (451-482) all render unconditionally.	An admin who suspends a venue for fraud or a safety complaint does not actually stop it: the operator can still confirm or reject the reservations already in flight (creating settlements and charging players), change court prices, add staff and mark attendance. A rejected applicant retains the same powers indefinitely. The suspension is cosmetic, which defeats the purpose of the review workflow this is the consumer half of.	Add an approved-status assertion — reuse `kr`/`yr` from mod/631.js:7090, 7225, 7269, 7424, 8330), allowing read-only calls (mockGetVenueBookings, mockGetVenueRevenue) through for a venue. In the screen, render the management sections only when `~approved~` === ae.status`.
P-CARCH-22 High	/venue/portal.tsx	Payment integrity / stale data	venueDecideBooking can confirm a hold that has already expired, because it never runs the expiry sweep `status` is only lazily reconciled with `reserved_until`, and the decision endpoint reads `status` without first reconciling. A booking whose 30-minute hold lapsed while the portal screen sat open still reads `reserved` in memory, so Accept succeeds. Evidence: mod/631.js:7420-7425 — `r.mockVenueDecideBooking = async (e, t, a) => { await ei(); const i = gr(t); if (!i) throw new Error("E_BOOKING_NOT_FOUND"); if ((await Sr(e, i.venue_id), "reserved" != i.status)) throw new Error("E_THIS_BOOKING_IS_NO_LONGER_PENDING"); return pr(i.court_id, async () => {` — no call to `Cr`. The sweep that retires stale holds, mod/631.js:7513-7526 — `Cr = async () => { const e = Date.now(); ... ("reserved" === a.status && a.reserved_until && new Date(a.reserved_until).getTime() < e && ((a.status = "expired"), ...`. Every read path calls it — mod/631.js:7549 (`mockGetVenueBookings`), 7540 (`mockGetOrganizerBookings`) — but the write path does not. The hold is 30 minutes: mod/631.js:7386 — `reserved_until: _ ? null : new Date(Date.now() + 18e5).toISOString()`. The portal loads once per focus and never refreshes: mod/2504.js:30-34 — `useFocusEffect(useCallback(() => { le(); }, [le]))`.	The player has already been told the hold expired and the slot was released back to the market. The venue then taps Accept on the stale row: the booking flips to `confirmed`, `lr(i)` writes a settlement with commission (mod/631.js:7433), and a `booking_decision` notification tells the organizer their reservation was approved. The organizer is now on the hook for a court they gave up on, and the slot can end up double-sold against a newer reservation for the same window.	Call `await Cr()` at the top of mockVenueDecideBooking (mod/631.js:7420) alongside `ei()`, and move the `~reserved~` != i.status` test inside `pr(i.court_id, ...)` lock at 7425 so the status check and the confirm are atomic. Re-check `reserved_until` against `Date.now()` before calling `Cr`.
P-CARCH-23 High	/venue/portal.tsx	Payments / dead end	A venue can never set a payout account from the consumer surface, so net payouts accrue with nowhere to go The portal shows a payout row it can never populate and offers no editor for it; no other consumer route calls updateVenueProfile with payout_iban_last4. Cancellation policy and cutoff hours are in the same position — displayed at mod/2504.js:391-403, supported at mod/631.js:7060-7062, never editable. Evidence: mod/2504.js:404-417 — `ae.payout_iban_last4 && Box({ ... Text({ children: W("payoutAccount") }) }, Text({ children: ["**** ", ae.payout_iban_last4] }) })` — read-only, and rendered only when the value already exists. mod/2504.js:387 is the only call to the update API in the whole screen — `onValueChange: (t) => oe() => (t, S.updateVenueProfile)(e.id, re.id, { auto_accept: t })`. The backend supports the field, mod/631.js:7065 — `null != a.payout_iban_last4 && (!a.payout_iban_last4 = a.payout_iban_last4.replace(/D/g, "").slice(-4)    null)`, but every real registration initialises it null: mod/631.js:7017 — `payout_iban_last4: null,`. A grep of 631 shows the only non-null write is the demo seed, mod/631.js:6895 — `payout_iban_last4: "4417",`. Meanwhile payouts accumulate: mod/631.js:8373-8379 — `netKwd: roundKwd(a.reduce((e, t) => e + t.net_to_venue_kwd, 0)), settledKwd: ..., pendingKwd: ...`.	Every venue onboarded through this surface reaches approved status with no payout destination. The portal reports a growing Net payout and Settled figure while the platform holds money it has no instruction to send. Onboarding cannot be completed without an out-of-band process that does not exist in the app, and the venue has no way to correct or update bank details after a change.	Add an editable payout-account field (and cancellation policy / cutoff hours) to the venue settings card at mod/2504.js:366-419, wired to the `updateVenueProfile` parameters at mod/631.js:7060-7065, and wired to the venue from reaching `approved` in mockAdminReviewVenue until `payout_iban_last4` is set.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-69 High	/wallet/index.tsx	KYC / AML compliance	KYC is self-attested and auto-approved — typing any 12 digits marks the account verified There is no `pending` status, no reviewer identity, no document capture and no checksum on the civil ID. The only consumer of the result is the withdrawal gate at <code>mod/631.js:16063</code> `if ("verified" !== (V(e)?.status ?? "unverified")) throw ... E_IDENTITY_VERIFICATION_KYC_IS_REQUIRED_BEFORE`, so the control it is supposed to enforce is defeated by the same call that creates it. Evidence: <code>mod/631.js:16087</code> `r.mockSubmitWalletKyc = async (e, t, a) => { await ei(); const i = t.trim(); if (!i) throw new Error("E_ENTER_YOUR_LEGAL_NAME"); if (!(!0, A.looksLikeCivilId)(a)) throw new Error("E_ENTER_A_VALID_12_DIGIT_CIVIL"); ... const n = { user_id: e, status: "verified", full_name: i, id_masked: (0, A.maskIdNumber)(a), submitted_at: new Date().toISOString(), reviewed_at: new Date().toISOString() };` — `status` is set to `“verified”` and `reviewed_at` stamped in the same breath as submission, with no reviewer and no pending state. `looksLikeCivilId` is `t => /\d{12}\$/.test(t.replace(/s/g, ""))` (module 655 in index.html). The screen gates only on length: <code>mod/2506.js:350</code> `disabled: !oe 12 !== ne.length`.	Any user who qualifies for withdrawals (organizer, admin, or venue owner) clears identity verification by typing a name and twelve arbitrary digits, then withdraws to the payout rail. The audit trail records `wallet.kyc_verified` (<code>mod/631.js:16104</code>) as if a review happened. For a Kuwait payments product this is a regulatory exposure, not just a product gap.	Make `mockSubmitWalletKyc` write `status: "pending"` with `revi null`, add a separate admin-only review mutation that sets `verified`/`rejected`, and validate the civil ID beyond its length (Ku IDs carry a check digit and an encoded birth date).
P-CSEC-70 High	/wallet/index.tsx	Identity / funds custody	Guest sessions can hold, top up and spend wallet money under a shared pseudo-account id Wallet mutations take the actor id on trust. `mockWalletAddFunds`, `mockWalletWithdraw` and `mockWalletTransfer` never assert `Qt.profiles.some(p => p.id === e)`; `mockWalletTransfer` checks only the recipient (<code>mod/631.js:16110</code>). Evidence: Guests are deliberately not redirected to sign-in: <code>mod/1809.js:52</code> `const n = "(auth)" === c[0], t = o?.user.id.startsWith("guest:") ?? !1; o    n ? o && !t && n && S.replace("(tabs") : S.replace("(auth/sign-in");` and the guest identity is a fixed string shared by every guest on the device: <code>mod/630.js:74</code> `const s = { user: { id: `guest:\${t}`, email: "guest@playora.app" }, token: "guest" };`. The wallet entry point is ungated: <code>mod/1805.js:194</code> `onPress: () => J.push("/wallet")`. `mockWalletAddFunds` never checks that the actor is a real profile: <code>mod/631.js:16021-16053</code> validates only the amount, the method and the velocity before posting to `A.acct.available(e)`. Compare <code>mod/631.js:12395</code> `if (e.startsWith("guest:")) throw new Error("E_SIGN_UP_TO_JOIN_BROWSING_IS");` — the codebase already knows guests must be blocked from state-changing flows.	A browsing guest reaches /wallet from the profile tab, tops up 50.000 KWD by card, and the balance is booked to `owner:guest:male:available` — an id shared by every guest who has ever picked that audience on the device and wiped by `clearGuestSession` on sign-out. The money belongs to no account, is visible to the next guest, and is unrecoverable by the person who paid.	Add an actor guard at the top of the wallet mutations in <code>mod/631.js:16055, 16108, 16140, 16173, 16289</code> : reject ids starting with `gue` with no row in `Qt.profiles`, reusing the `E_SIGN_UP_TO_JOIN_BROWSING_IS` pattern; and hide the v <code>mod/1805.js:194</code> for guest sessions.
P-CARCH-24 High	/wallet/index.tsx	Double-spend / concurrency	A money request can be paid twice — the pending-status check sits outside the serialization queue `ql` (<code>mod/631.js:15915</code>) is the module's mutual-exclusion primitive, keyed per user. Two invocations that both get past `await ei()` before either body runs will both observe `pending`, both enqueue, and both post a transfer. The status check must be re-evaluated inside the critical section, exactly as the balance check is. Evidence: <code>mod/631.js:16173</code> `r.mockRespondWalletRequest = async (e, t, a) => { await ei(); const i = Qt.walletRequests.find((e) => e.id === t); if (!i i.from_user_id !== e) throw new Error("E_REQUEST_NOT_FOUND"); if ("pending" !== i.status) throw new Error("E_THIS_REQUEST_WAS_ALREADY_RESOLVED"); return a ? (await ql(e, async () => { if (W(A.acct.available(e)) < i.amount_fil) throw ...; (await Yl({ kind: "request_payment", ... })), (i.status = "paid"), ...; }));` — the `pending` guard runs before `ql`, and the body inside `ql` never re-reads the status. The balance guard, by contrast, is correctly inside the queue.	One 20.000 KWD request gets settled twice: the payer is debited 40.000 KWD and the requester credited 40.000 KWD, with two `request_payment` ledger entries carrying the same `ref: i.id`. Moving this code to a server makes it worse, not better — concurrent HTTP requests are the normal case there, while the UI's shared `loading` flag (<code>mod/2506.js:437</code>) is the only thing narrowing the window today.	Move the lookup and the `“pending” !== i.status` check inside the callback in <code>mod/631.js:16179</code> , alongside the existing balance che read the request from `Qt.walletRequests` there.
P-CARCH-25 High	/wallet/index.tsx	Double-spend / concurrency	Venue payouts can be claimed twice — the claimable set is computed before the lock is taken The dedupe relies on a read that happens before the write it is meant to exclude. Two overlapping claims capture the identical settlement list and both post payouts for it. The claim button in <code>mod/2506.js:390-397</code> passes no `disabled` guard, only `loading: L`. Evidence: <code>mod/631.js:16289</code> `r.mockClaimVenuePayout = async (e) => { await ei(); const t = \$(e); if (0 === t.length) throw new Error("E_NO_SETTLED_PAYOUTS_TO_CLAIM"); await ql(e, async () => { for (const a of t) { const t = (0, A.toFils)(a.net_to_venue_kwd); t <= 0 (await Yl({ kind: "venue_payout", postings: (0, A.transferPostings)(A.acct.platform("payouts"), A.acct.available(e), t), ..., ref: a.id, ... }));` — `\$(e)` is evaluated outside `ql`, and the loop inside `ql` never re-checks. `\$(` dedupes purely by scanning already-written ledger refs: <code>mod/631.js:15947</code> `const a = new Set(Qt.walletLedger.filter((e) => "venue_payout" === e.kind && e.ref).map((e) => e.ref)); return Qt.settlements.filter((e) => t.has(e.venue_id) && "settled" === e.status && !a.has(e.id));`	A venue owner double-taps Claim and the platform pays the same settled settlements twice out of `platform:payouts` into their available balance — real money leaving the platform, with two ledger entries sharing the same `ref`. There is no reversal path in the consumer app.	Move `const t = \$(e)` inside the `ql(e, ...)` callback in <code>mod/631.js</code> : the claimable set is computed under the lock, and add a uniqueness assertion on `(kind: "venue_payout", ref)` inside `Yl` (<code>mod/631.js</code> : before appending.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-26 High	./wallet/index.tsx	Half-succeeded payment / durability	A ledger write that fails after the gateway has captured loses the money with no compensating action The capture and the book-keeping are not a unit and there is no compensating transaction. `Sr` gets this right for seat payments — the captured charge row is persisted (mod/631.js:7856) so a retry short-circuits and self-heals — but the wallet top-up has no such record, and `Yl` leaves the in-memory array mutated even when the persist fails. Evidence: In `mockWalletAddFunds` the card is captured first: mod/631.js:16034 `if (!await Vr(n, t / 1e3, a))) throw ... new Error("E_PAYMENT_COULD_NOT_BE_VERIFIED_PLEASE");` and only then is the ledger written, mod/631.js:16040-16049 `await q(e, () => Yl({ kind: "add_funds", postings: (0, A.transferPostings)(A.acct.external(a), A.acct.available(e), t, ... )))`. `Yl` mutates memory before persisting and propagates any persistence failure: mod/631.js:15934 `return (Qt.walletLedger.unshift(t), await Za(at, Qt.walletLedger), t);` and `Za` converts a full store into a throw, mod/631.js:1772 `throw new Error(/quotaQuotaExceeded NS_ERROR_DOM_QUOTA/i.test(a) ? "E_STORAGE_FULL" : "E_COULD_NOT_SAVE_CHANGES");`. There is no `catch` around the capture and no reversal call anywhere in the function.	A top-up captured at the gateway when storage is full throws `E_STORAGE_FULL` at the user, while the credit exists only in memory: the balance appears on screen until reload and is then gone permanently. The user has been charged and has nothing. The same shape applies to the refund poster `ac` (mod/631.js:16270) and the transfer (mod/631.js:16123), where memory and storage also diverge on a failed write.	Record the gateway reference before capture (as `paymentCharge` mod/631.js:7846) so a retry is idempotent; in `Yl` (mod/631.js:15934) `unshift` back in a `catch` before rethrowing so memory never diverges; and add a reversal call on the capture when the ledger v
P-CSEC-71 High	./wallet/index.tsx	Payment integrity / credit laundering	Promo credits become withdrawable cash through a pay-then-cancel cycle The three-account design deliberately separates `credits` (promo/prize/referral money, usable only inside the app) from `available` (real money that can be transferred to another user or withdrawn). Payment drains `credits` first; refund returns everything to `available`. Nothing tags a refund with the funding mix, so the separation is one-way and leaks. Evidence: mod/631.js:16247 — `const r = (0, A.planCombinedPayment)(t, Wl(A.acct.credits(e)), Wl(A.acct.available(e)));` and the helper spends credits first: `e.planCombinedPayment = (t, a, n) => { const l = Math.max(0, Math.round(t)), o = Math.min(l, Math.max(0, a)), s = Math.min(l - o, Math.max(0, n)); return { credits_fils: o, wallet_fils: s, external_fils: l - o - s }; }` (module 655). mod/631.js:16276 — the refund lands in a different account: `postings: (0, A.transferPostings)(A.acct.platform("refunds"), A.acct.available(e), i)`. mod/631.js:16125 and 16075 — only `available` is spendable outward: `mockWalletTransfer` posts from `A.acct.available(e)` and `mockWalletWithdraw` checks `if (Wl(A.acct.available(e)) < t) throw new Error("E_INSUFFICIENT_WALLET_BALANCE")`. Credits are granted at 16210 (`mockAdminGrantCredit` → `A.acct.credits(t)`) and 16231 (referral reward → `A.acct.credits(e)`).	Any holder of promo credits can convert them to cash: join a paid match, pay (credits are consumed first), cancel before the 2-hour cutoff, and the same value reappears in `available`. From there it can be transferred to any other user (`mockWalletTransfer`) or, for an organizer or venue owner, withdrawn to a bank rail. Referral bonuses (1.000 KWD each at mod/631.js:16231) and admin prize grants become a cash-out channel, and marketing spend converts directly into payout liability.	Record the funding split on the payment row in `tc` and have `ac` (mod/631.js:16269) refund each portion to its originating account. `credits_fils` to `A.acct.credits(e)`. Alternatively make `planCombinedPayment` spend `available` before `credits` so credits are never the refundable leg.
P-CARCH-27 High	./wallet/index.tsx	Financial read path	Wallet balances are recomputed by scanning a never-compacted global ledger, five or more times per wallet open There is no balance snapshot or running total per account. Every read replays the full double-entry history of every user in the system, and the ledger grows monotonically with every top-up, transfer, seat payment, refund and payout. Evidence: mod/631.js:15936-15940 — `function Wl(e) { let t = 0; for (const a of Qt.walletLedger) for (const i of a.postings) i.account === e && (t += i.delta_fils); return t }`. `Zl` calls it three times at mod/631.js:16008-16010 (`available_fils`, `pending_fils`, `credits_fils`), loops the whole ledger again at mod/631.js:15982-16000 to build `transactions`, and scans it once more inside `\$l` (mod/631.js:15944-15951). mod/631.js:15934 — `Qt.walletLedger.unshift(t), await Za(at, Qt.walletLedger)` is the only write path; nothing ever removes or checkpoints entries. `Zl` is also the return value of every mutation (e.g. mod/631.js:16019 `Zl(e)` at the end of `mockWalletAddFunds`), and balance guards re-scan too (mod/631.js:16072, 16122, 16180, 16247).	After a year of real traffic the ledger is the largest table in the store; opening `./wallet` replays it five times and `mockPayGroup`/`mockPayRequest` replay it again on each balance check, so paying for a game gets slower every month. The same array is also whole-array-serialised on every append (mod/631.js:15934), so writes degrade in lockstep with reads.	Maintain a materialised `Map<account, balance_fils>` updated in (mod/631.js:15920) when postings are appended, and have `Wl` a periodic checkpoint entry so `transactions` can be built from the ledger rather than the whole of it. Add `{limit, cursor}` to `Zl` so the 20-row page (mod/2506.js:517) does not require materialising every transaction.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-72 High	631 (shared name helper Td)	Data exposure / PII	Display-name helper falls back to the account's login email, and nothing stops a user saving a blank name Clearing the Full name field and pressing Save stores `full_name: ""`, after which `Td` returns the falsy-fallback `bd(e)` — the complete login email address — and renders it as that person's public display name everywhere. There is no server-side minimum-length check and no client-side one. Evidence: mod/631.js:13079-13080: ``` const bd = (e) => Qt.users.find((t) => t.id === e)?.email ?? "", Td = (e) => Qt.profiles.find((t) => t.id === e)?.full_name bd(e) "Member", ``` `Td` is the name renderer for 44 call sites across the consumer surface — follow requests (14112), friends and group members (13697-13698, 13951), DM conversation headers (14617), story trays (14892), award ballots and leaderboards (15096, 15124, 15288), wallet counterparties (16136, 16169, 16242), venue CRM customers (15799), club members (13117). The profile editor sends the raw text-input state with no trim and no emptiness check: mod/1805.js:553 `full_name: Y`, where `[Y, Z] = useState(M?.full_name ?? "")` (mod/1805.js:11) and the only other uses are display (307, 326). `mockUpdateProfile`'s denylist is `B = ["role","consent","consent_updated_at","id","audience"]` (mod/631.js:465) — `full_name` passes through unvalidated (mod/631.js:2385).	A user's email address is published to strangers across the whole app: to anyone who sends them a follow request, to every member of a shared match group, on the award leaderboard, in wallet transfer receipts. Email is the account identifier and the password-reset channel (mod/631.js:920), so this also widens the account-takeover surface. It is triggered by an ordinary mistake — clearing a field — not by an attack.	Validate in `mockUpdateProfile` (mod/631.js:2366): reject or ignore `full_name` that is empty after trim, and change `Td` (mod/631.js: fall back to a non-PII placeholder ("Player") rather than `bd(e)`. for the places that genuinely need the email (club invite matching for a display name.
P-CSEC-73 High	mod/631.js (conduct backend, reached from ./conduct/index.tsx)	Data exposure	A user's full disciplinary record can be read for any account id with no authorization check `mockGetUserSanctions` takes `(target, viewer)` and enforces self-or-admin; `mockGetDisciplinaryStanding` takes only `(target)` and enforces nothing. The facade mirrors that: mod/671.js:764 `e.fetchDisciplinaryStanding = (o) => t.store.mockGetDisciplinaryStanding(o);` has no viewer argument to check against. Evidence: mod/631.js:12607 `r.mockGetDisciplinaryStanding = async (e) => (await ei(), _d(e));` — the function takes only a target id and performs no caller check. mod/631.js:12389 `_d = (e) => (0, D.computeStanding)(cd(e))`, which returns `{ key, warnings, yellowCards, redCards, isSuspended, isBanned }` (bundle-src/656.js:69-86). The guarded twin sits seven lines above: mod/631.js:12600-12601 `r.mockGetUserSanctions = async (e, t) => { if ((await ei(), t !== e && !ro(t))) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW_2"); ...`.	This is a hole that survives moving the code to a server: the RPC transport forwards any `mock*` name verbatim (mod/672.js:36-47, mod/673.js:60-64 `body: JSON.stringify({ fn: t, args: n })`), so `POST /rpc {fn:"mockGetDisciplinaryStanding", args: {<anyUserId>}}` returns whether that player is banned or suspended and their warning/yellow/red counts. User ids are freely discoverable — the leaderboard returns `user_id` for the top 50 (mod/631.js:15288) and every player card links `player/<id>`. Conduct history is exactly the category of personal data that must not be enumerable.	Change `mockGetDisciplinaryStanding` to `(target, viewer)` and use the same guard as `mockGetUserSanctions` (`if (viewer !== target && !ro(viewer)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW_2")`), update mod/671.js:764 to forward the viewer, and update the one call site mod/2372.js:18.
P-CARCH-28 High	mod/631.js (notification fan-out, surfaced on ./notifications.tsx)	Notification correctness	The notification de-duplicator collapses unrelated notifications that share a type within 60 seconds The guard is written as "same event about the same match", but for every payload without a `game_id` it degenerates into "same type, any subject". The discriminating field for these types is `follower_id`, `team_id`, `sanction_id` or the wallet entry id, none of which is compared. Evidence: mod/631.js:2579-2589 `bi = async (e) => { e.audience = e.audience ?? aa(e.user_id); const t = Date.now() - 6e4; Qt.notifications.some((a) => a.user_id === e.user_id && a.type === e.type && a.game_id === e.game_id && !a.read && new Date(a.created_at).getTime() > t) ((Qt.notifications = [e, ...Qt.notifications]), await Za(\$, Qt.notifications), Ei(e)); }`. Many notification payloads carry no `game_id` at all, so `a.game_id === e.game_id` reduces to `undefined === undefined`: wallet activity at mod/631.js:15957-15964 (`type: "wallet_activity", kind_key, direction, amount_fils, counterparty_name`), follows at mod/631.js:14066-14074 (`type: n ? "new_follower" : "follow_request", follower_id, follower_name`), sanctions at mod/631.js:12483-12492 (`type: "sanction_issued", sanction_id, sanction_type, reason`), and team notices at mod/631.js:8714 and 8777 (`team_id` only).	Two people follow you in the same minute and you are told about one. Two wallet movements in the same minute and the second — a real money event with an amount and a counterparty — is dropped and never recoverable, since there is no other wallet notification path. Two sanctions issued in the same incident and the player is notified of one. Two invitations from different teams and one vanishes. Nothing is logged; the notification simply never exists.	Give `bi` an explicit dedupe key per type (match on `sanction_id`, `follower_id`, `team_id`, the ledger entry id, and `game_id` only when actually set) rather than the implicit `game_id` comparison at mod/631.js:2586, and skip de-duplication entirely for types that carry no monetary amount.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CTEST-8 High	mod/631.js (push delivery, surfaced on ./notifications.tsx)	Deployment configuration	Push notifications can never be delivered from the deployed web build — the CSP blocks the send and the failure is swallowed <code>https://exp.host</code> is not in <code>connect-src</code>, so the browser refuses the request before it leaves the page. The <code>.catch()</code> discards the resulting <code>TypeError</code>, so nothing in the app, and nothing in the smoke harness that checks for console/page errors, distinguishes "push sent" from "push structurally impossible". The app continues to accept and store push tokens as if delivery worked. Evidence: <code>index.html:5`<meta http-equiv="Content-Security-Policy" content="default-src 'none'; ... connect-src 'self' https://api.rush-x.xyz; ..." />`. mod/631.js:2569-2577`Ei = (e) => { const t = Qt.pushTokens.filter((t) => t.user_id === e.user_id).map((e) => e.token); if (0 === t.length) return; const { title: a, body: i } = Si(e); fetch("https://exp.host/~api/v2/push/send", { method: "POST", headers: ..., body: JSON.stringify(t.map((e) => ({ to: e, title: a, body: i, sound: "default" }))) }).catch(() => {}); }`. `Ei` is called on every notification creation at mod/631.js:2589. Token registration still succeeds: mod/631.js:2512-2521`r.mockRegisterPushToken`.</code>	Every out-of-app notification the product depends on — match reminders, squad confirmation deadlines, replacement offers, payment requests — is silently dead on rush-x.xyz, while a CSP violation is reported on every single notification write. Users who grant push permission and register a token receive nothing and have no way to tell.	Either add the push host to <code>connect-src</code> in <code>index.html:5</code> and send server rather than the client (client-side sending also exposes every recipient's push token to the sending browser), or gate <code>Ei</code> (mod/ behind a capability check so the code path is not attempted on web dead registration UI is hidden.
P-CSEC-74 High	mod/631.js (shared name resolver Td)	PII disclosure	A user who clears their display name has their email address rendered as their name across the feed, DMs, friends and follows A missing display name should fall back to a non-identifying placeholder such as <code>'Member'</code>. Instead it falls back to the raw account email, and the write path does not stop the name from being cleared in the first place. Evidence: mod/631.js:13079-13080 — <code>`const bd = (e) => Qt.users.find((t) => t.id === e)?.email ?? ""</code>, <code>Td = (e) => Qt.profiles.find((t) => t.id === e)?.full_name    bd(e)    "Member"</code> — when <code>'full_name'</code> is falsy the fallback is the account email. Nothing prevents an empty name: mod/631.js:2366 <code>`r.mockUpdateProfile = async (e, t) => { ... const i = t, n = (0, d.default)(i, B); Qt.profiles[a] = Object.assign({}, Qt.profiles[a], n ...)`</code> validates only <code>'audience'</code> and <code>'username'</code>; <code>'B'</code> at mod/631.js:465 is <code>["role", "consent", "consent_updated_at", "id", "audience"]</code>, so <code>'full_name'</code> is written through verbatim with no <code>'sanitizeName'</code>, no trim and no non-empty check — and the editor sends it raw: mod/1805.js:553 <code>'full_name: Y'</code>. <code>'Td'</code> is the display name on every in-scope surface: feed author mod/631.js:14363 rendered at mod/1622.js:271, DM peer name mod/631.js:14617 rendered at mod/2455.js:107 and mod/2456.js:165, friends list mod/631.js:13595/13601 rendered at mod/2378.js:109, follow list mod/631.js:14131 rendered at mod/2376.js:113.	A user who clears the name field in the profile editor and saves immediately starts publishing their email address as their public display name — on every feed post they have written, in the header of every DM thread, in other people's friend and follower lists, and in the story tray. Email addresses are handed to strangers with no consent step and no indication anything happened, and the user has no way to delete the posts that now carry it (see the post-deletion finding).	In 631, drop <code>'bd(e)'</code> from the <code>'Td'</code> fallback chain so it reads <code>`.full`</code> , <code>"Member"</code> , and add validation to <code>`mockUpdateProfile`</code> : <code>`if (“full_name” const n = sanitizeName(t.full_name); if (!n) throw new Error(“name_required”); t = { ...t, full_name: n }; )`</code> — <code>'name_required'</code> already mapped in mod/674.js:43. Mirror the guard in the 1805 series so the error surfaces before the round trip.
P-CARCH-29 High	mod/631.js (shared store — affects every route)	Data loss / concurrency	Nothing coordinates browser tabs, so a second tab's whole-array write silently destroys the first tab's data Each tab hydrates its own copy of <code>'Qt'</code> at boot and thereafter writes whole tables from that copy. There is no version/etag check in <code>'Za'</code>, no storage-event listener to re-hydrate, and no merge. Evidence: mod/631.js:1780-1789 — hydration runs once and is latched: <code>`ei = () => Qt.hydrated ? Promise.resolve() : ...`</code> and <code>`ti = async () => { if (Qt.hydrated) return; ... }`</code>. mod/631.js:1768 — <code>`await l.default.setItem(e, JSON.stringify(t))`</code> writes the entire in-memory array. Grepping the shipped <code>index.html</code> for <code>`addEventListener("storage" / "storage"</code> and for <code>'BroadcastChannel'</code> returns zero hits, so no tab ever learns that another tab changed a key.	Playora is a web SPA where opening a game in a second tab is normal. Tab A loads with 10 bookings. Tab B joins a game and writes 11. Tab A then joins a different game and writes its stale <code>10 + 1 = 11</code> , permanently erasing Tab B's booking — the user paid for a seat that no longer exists. The same clobber applies to messages, wallet ledger entries and notifications, and it is completely silent.	Add a <code>'storage'</code> event listener that re-runs the affected <code>'xa'</code> read and latches <code>'Qt'</code> for that key, and stamp each persisted table with a monotonically increasing revision that <code>'Za'</code> verifies before overwriting (re-hydrate and re-append mismatch). A <code>'BroadcastChannel'</code> ping after each <code>'Za'</code> gives the client an immediate, ordered signal.
P-CTEST-9 High	mod/673.js (backend transport) + index.html config block	Deployment configuration	A 900ms health probe, and a CSP connect-src the operator must remember to edit, decide silently whether the app talks to the server at all Both the timeout and the allowlist mismatch fail into the identical silent outcome: mode <code>'mock'</code>. A backendUrl pointing at a Railway-generated domain, or an API that answers <code>/health</code> in 1.2s on a Kuwait mobile connection, produces an app that boots, looks completely normal, and uses the in-browser store. Evidence: mod/673.js:16-24 — <code>`u = async (t, n) => { try { const o = new AbortController(), c = setTimeout(() => o.abort(), n), l = await fetch(`\${t}/health`, { signal:o.signal}); return (clearTimeout(c), l.ok); } catch { return !1 } }`</code> — called at mod/673.js:32 as <code>`o = c && (await u(c, 900)) ? "remote" : "mock"</code>. <code>index.html</code> line 6 hardcodes <code>connect-src 'self' https://api.rush-x.xyz`</code>; the config comment at <code>index.html</code> line ~13 says the backendUrl <code>'Must also be listed in the Content-Security-Policy connect-src above'</code>, but nothing enforces it — a CSP refusal lands in the same <code>'catch'</code> and returns false.	A deployment can be signed off as working while no request has ever reached the API. Users create accounts, book and pay entirely in their own browser; support sees an empty database. The only trace is a console warning nobody reads. <code>`remoteEnabled()`</code> is exported (mod/672.js:4-9) but no screen renders it, so there is no indicator to check.	Raise the probe timeout to a realistic mobile value and retry once deciding; derive the CSP <code>connect-src</code> from the same configured constant and validate at boot that backendUri is allowed and fail loudly if not); and the resolved <code>'backendMode()'</code> somewhere visible in a non-production build so a misconfigured deploy is obvious.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-30 High	mod/673.js (backend transport, shared by every screen)	Systemic authorization bypass	Viewer-identity arguments are optional, and the RPC transport strips them when omitted Authorization depends on an argument the caller chooses whether to send, rather than on the session. The server would read the viewer from `body.args[1]`, which the client controls and can simply omit — and the transport is built to omit it. Evidence: Guards across 631 are written as `if (viewer && ...)` , so a falsy viewer skips the check entirely: mod/631.js:4186 `if (t && t !== e && !ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");` followed by `return !t    t === e;` — `orgSelf(org, undefined)` returns `true`, i.e. 'this is the owner'. Same shape at mod/631.js:7560 `if (t && a.organizer_id !== t && !ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");` (mockGetBooking), mod/631.js:12283 `if (t && a.candidate_id !== t && !ro(t)) {` (mockGetReplacementOffer), and mod/631.js:8659 `private` !== a.privacy no(e, t) (t && ro(t))` (mockGetTeamMembers). The transport actively removes omitted trailing arguments before sending: mod/673.js:53 `e.rpc = async (t, n, l) => {` / mod/673.js:55 `for (; n.length && void 0 === n[n.length - 1];) n = n.slice(0, -1);`, and mod/672.js:25 proxies any `mock*` name through it: `return "function" == typeof c && /*mock[A-Z]*/.test(String(n)) ? async (...e) => { ... await (0, t.rpc)(String(n), e, () => c(...e));`.	This is the pattern that makes the individual holes above reachable rather than theoretical. A caller who posts `{fn: "mockGetBooking", args: ["<id>"]}` reads any court booking; `{fn: "mockGetOrganizerMatches", args: ["<organizerId>"]}` returns the organizer's matches with `invite_code` intact because `own9` is true; `{fn: "mockGetReplacementOffer", args: ["<id>"]}` returns any offer. Every `if (viewer && ...)` guard in 631 is opt-in, so the whole authorization model degrades to `checked only when the client asks to be checked`.	Take the acting identity from the authenticated session at the RP rather than from `args`: have the server resolve the bearer token mod/673.js's `/rpc` handler and inject the caller as the first argum rejecting any client-supplied value. Until then, make every viewer mandatory in 631 — turn each `if (t && ...)` into `if (!t) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW");` if `(t !==` . starting with `orgSelf` (mod/631.js:4185), `mockGetBooking` (mod/631.js:7560) and `mockGetReplacementOffer` (mod/631.js:
P-CSEC-75 High	serve.json / railway.json (deployment config)	Source and configuration disclosure	The static host publishes the entire repository, including the prettified source of the whole backend The build step is `npm install --omit=dev` with no copy/prune into a dist directory, so every working file in the repo becomes a public URL on rush-x.xyz. That includes the internal audit report directory (docs/audit/), the route → module map that names every admin route, and the architecture notes in CLAUDE.md. Evidence: serve.json line 2 — `public`: `.` — combined with railway.json `startCommand`: `npm start` and package.json `start`: `serve -s . -l \${PORT:-3000} -c serve.json`, which serves the repository root. Running exactly this command against the repo and fetching paths returns HTTP 200 for: /bundle-src/631.js (656856 bytes — the full prettified mock backend with every business rule), /tools/smoke.mjs, /CLAUDE.md, /env.example, /auth/jwt.ts, /rpc.ts, /package.json, /railway.json, /nixpacks.toml, /docs/audit/findings/route-module-map.txt. `directoryListing`: false` blocks browsing but not direct fetch.	An attacker gets a readable, prettified copy of every authorization rule, error code, seed fixture and localStorage key, plus the repo's own audit findings naming known weaknesses — without having to unminify anything. The route map and CLAUDE.md hand them the admin surface and the config block's structure directly.	Build into a `dist/` directory containing only index.html and set `p` in serve.json (or add a serve.json `rewrites`/deny rule for bundle-: docs, auth, *.ts, *.md). While editing serve.json, add X-Content-T: Referrer-Policy and X-Frame-Options headers — it currently sets Cache-Control.
P-CARCH-31 High	shared — mod/673.js (transport) and mod/672.js (store proxy)	Input validation coverage	All input sanitisation and length capping lives in the fallback mock backend, so the remote RPC path ships raw user text to the server The 60/80/200/500/1000/2000-character caps and the control-character strip are implemented inside mod/631.js, which by design is the offline fallback. Once window.__PLAYORA_CONFIG__.backendUrl points at a live server and /health answers, none of that code executes. Validation belongs either in a shared layer the facade always runs (671) or on the server; today it exists in neither. Evidence: mod/673.js:53-70: `e.rpc = async (t, n, l) => { if ((await k(), "remote" !== o)) return l(); ... y = await fetch(`\${c}/rpc`, { method: "POST", headers: ..., body: JSON.stringify({ fn: t, args: n }) });` — the arguments are the screen's raw props, serialised straight to the wire. `l()` (the local implementation in 631, the only place sanitizeText/sanitizeName are called) runs `*only*` when the backend is absent or unreachable: mod/672.js:40-46 `async (...e) => { try { return await (0, t.rpc)(String(n), e, () => c(...e)); }`. Across the entire consumer surface exactly one screen sanitises before calling the facade — mod/917.js:134 `(0, T.sanitizeName)(ke)`. Every other producer passes the raw state: mod/2373.js:171-177 `first_name: M.trim(), ... message: F.trim(),`; mod/1843.js:31-36 `body: n;`; mod/2375.js:60-65 `caption: H`,`.	In the configured production deployment every cap in this audit is dead code. A client can POST an arbitrarily long chat body, team name or contact message and nothing in the shipped bundle stops it. It also means the mock and the real backend will disagree about what is stored, so anything QA validates against the fallback is not what production does.	Move sanitizeText/sanitizeName/sanitizePhone and the per-field out of mod/631.js into a shared validator that mod/671.js applies ` arguments of every write facade (createPost, sendChatMessage, postTeamChat, sendDirectMessage, submitContact, createTeam, updateProfile) before handing them to store, so both transports g normalisation — and mirror it server-side in rpc.ts.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-20 Medium	./(tabs)/feed.tsx	Crash	Feed renderItem dereferences the user object without the null guard used everywhere else in the same component <code>`renderItem`</code> should use the same <code>`e?.id ?? ""`</code> guard the rest of the component uses. The inconsistency is the bug: the author clearly anticipated a null user in four other places in this file and missed this one. Evidence: mod/1622.js:173-186 — <code>`renderItem: ({ item: t }) => jsx(Div, { ..., children: jsx(B, { item: t, userId: e.id, colors: u, t: k, onChange: U, onToast: q, onOpenAuthor: ..., onOpenComments: ... }) })`</code> reads <code>`e.id`</code> unconditionally, where <code>`e`</code> is <code>`const { user: e, profile: c } = useAuth()`</code> (mod/1622.js:6). Every sibling use in the same component guards it: mod/1622.js:83 <code>`userId: e?.id ?? ""`</code> for the story tray, mod/1622.js:30 <code>`e`</code> && <code>`s.default.settle(...)`</code>, mod/1622.js:188 <code>`e`</code> && <code>`jsx(CommentsSheet, { ..., userId: e.id, ... })`</code>, and the loader itself mod/1622.js:16 <code>`if (e && ...)`</code>. <code>`user`</code> does become null while the component is mounted: mod/630.js:61-67 <code>`signOut: async () => { ... w(null), P(null) }`</code> sets it synchronously, and the redirect that follows is an effect in the root layout (mod/1809.js:50-55) that only runs after the render; the tab navigator (mod/1619.js:6-15) keeps the feed screen mounted while the user is on the profile tab.	Signing out while the feed screen is still mounted with posts in state re-renders the visible feed cells with <code>`e === null`</code> , throwing <code>`TypeError: Cannot read properties of null (reading 'id')`</code> inside <code>FlatList</code> 's cell renderer and taking down the React tree with a blank screen instead of a clean redirect to sign-in. The same happens on any other transition that clears the session with feed data present.	Change <code>`userId: e.id`</code> at mod/1622.js:178 to <code>`userId: e?.id ?? ""`</code> , mod/1622.js:83; or short-circuit the whole list with <code>`data: e ? (T ? :</code>
P-CSEC-76 Medium	./(tabs)/feed.tsx	Harassment control bypass	The shared post guard checks the audience partition but not blocks, so a blocked user can still comment and like Reads filter blocked users out; writes do not check blocks at all. A block is meant to be symmetric and is enforced on DMs (mod/631.js:14707 <code>`if (Xd(i.participant_ids[0], i.participant_ids[1])) throw new Error("blocked");`</code>), on wallet transfers (mod/631.js:16111) and on stories, but not on feed interactions. Evidence: mod/631.js:14537 — <code>`cl = (e, t) => {`</code> / mod/631.js:14538 <code>`const a = Qt.posts.find((e) => e.id === t);`</code> / mod/631.js:14539 <code>`if (!a) throw new Error("not_found");`</code> / mod/631.js:14540 <code>`return (oa(e, a.author_id), a);`</code> — <code>`oa`</code> is only the audience-partition assertion. Every caller inherits the gap: mod/631.js:14542 <code>`r.mockAddComment = async (e, t, a) => {`</code> / mod/631.js:14543 <code>`(await ei(), cl(e, t));`</code>, and likewise <code>`mockToggleLike`</code> (mod/631.js:14496) and <code>`mockToggleCommentLike`</code> (mod/631.js:14566). The block predicate is defined and used everywhere else — mod/631.js:14027 <code>`Xd = (e, t) => Qt.blocks.some((a) => (a.blocker_id === e && a.blocked_id === t)    (a.blocker_id === t && a.blocked_id === e));`</code> — including in the reader immediately below: mod/631.js:14556 <code>`filter((a) => a.post_id === t && !Xd(e, a.author_id) && !(e, a.author_id))`</code>, and in the feed itself at mod/631.js:14482.	A user who has been blocked can still comment on and like the blocker's posts. The blocker's own feed hides those comments, but everyone else sees them, so the block gives the victim a false sense of protection while the harasser keeps a public channel under their content.	Add the block test to <code>`cl`</code> (mod/631.js:14537): <code>`if (e !== a.author_i`</code>
P-CARCH-33 Medium	./(tabs)/feed.tsx	Error handling / refetch storm	The feed screen has no error path and refetches the entire feed after every like, comment and save If <code>`fetchFeed`</code> or <code>`fetchStoryTray`</code> rejects, <code>`T`</code> is never assigned, so the list renders the <code>`null === T`</code> spinner permanently and the rejection is unhandled. Separately, every single interaction — a like, a save, a comment, a share — calls <code>`U`</code>, which re-runs the full <code>`mockGetFeed`</code> (every post, seven scans each) plus <code>`mockGetStoryTray`</code>. Evidence: mod/1622.js:15-23 — <code>`U = useCallback(async () => {`</code> if (e && (v(await fetchFeed(e.id)), H(await fetchStoryTray(e.id), ...)) ... }, [e, c?.audience])` — the two awaits that matter sit outside the inner <code>`try`</code>, which only guards the privacy-card lookup. mod/1622.js:24-28 wires <code>`U`</code> to <code>`useFocusEffect`</code> with no <code>`.catch`</code>, mod/1622.js:147-151 — <code>`ListEmptyComponent: ... null === T ? <ActivityIndicator> : <EmptyState .../>`</code>. mod/1622.js:181 — <code>`onChange: U`</code> is handed to every post card, and mod/1622.js:192 hands it to the comments sheet.	Tapping the heart on a post rebuilds the entire feed from scratch, including the story tray, and re-renders the whole list; on a feed of a few hundred posts that is a visible freeze per tap. And any store failure on the home feed leaves the user staring at a spinner with no retry — the most-visited screen in the app has no recovery path.	Wrap the two awaits at mod/1622.js:16 in try/catch, add an error : alongside the <code>`null === T`</code> gate, and give the empty component a For interactions, apply the returned <code>`{liked, count}`</code> from <code>`toggleLil`</code> (mod/631.js:14509) to the single item in local state instead of call

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-21 Medium	./(tabs)/feed.tsx, ./messages/index.tsx, ./messages/[id].tsx, ./group/[gameId].tsx, ./discover.tsx, ./player/[id].tsx	Rendering correctness	Fourteen avatar-initial sites take name.slice(0,1), producing a broken-glyph replacement character for any emoji-leading name Initial extraction assumes one character equals one JS index. It does not handle surrogate pairs, combining marks, or a name that starts with a zero-width character (which yields a blank circle). Evidence: mod/1622.js:260 `children: e.author_name.slice(0, 1).toUpperCase(),`; mod/2456.js:153 `children: l.other_name.slice(0, 1).toUpperCase(),`; mod/2455.js:101 `children: (q?.other_name ?? "")?.slice(0, 1).toUpperCase(),`; mod/2444.js:179 `children: e.name.slice(0, 1).toUpperCase(),`; mod/2374.js:205 `children: t.name.slice(0, 1).toUpperCase(),`; mod/2475.js:143 `children: (F.display_name    "")?.slice(0, 1).toUpperCase(),`; also mod/1835.js:300, mod/1836.js:103, mod/1837.js:66, mod/1847.js:256, mod/2378.js:323, mod/2487.js:145, mod/1622.js:427. `_.slice(0,1)` returns one UTF-16 code unit, so for a name beginning with an astral character it returns a lone high surrogate: "u{1F525}Ali".slice(0,1) === "ud83d" and "u{1F3C6} Rockets".slice(0,1) === "ud83c". Emoji-leading names are reachable because mockUpdateProfile applies no sanitiser (mod/631.js:2384-2386) and team names keep emoji through sanitizeText (mod/631.js:8538 `const a = (0, v.sanitizeText)(t.name, 60);`).	Any player or team whose name starts with an emoji — extremely common in this product, which itself seeds emoji-bearing content — shows a  replacement box instead of an initial in their avatar across the feed, the messages list, the DM header, the group screen and discovery. It is visible to every other user, not just the owner.	Add a shared `initialOf(name)` helper that does `[...String(name ? "").trim()].find(ch => /\p{L}\p{N}/u.test(ch)) ?? '?'` and replace all fr `.slice(0, 1).toUpperCase()` call sites with it.
P-CSEC-77 Medium	./(tabs)/feed.tsx, ./stories/[id].tsx	Unvalidated URL handling	media_uri reaches a raw /<video src> with no scheme or origin allow-list anywhere in the path There is no allow-list of permitted schemes or origins between the client-supplied mediaUri and the DOM. mockCreatePost (mod/631.js:14385) and mockCreateStory (mod/631.js:14828) accept whatever string arrives, and with the remote RPC path active mod/631.js never runs at all, so the only filter that exists today is bypassed in production. Evidence: React Native Web escapes text children, so every user string rendered as a Text child in this app is safe — I found no innerHTML, dangerouslySetInnerHTML, document.write or user-controlled href across the consumer modules. The one exception is media URLs, which land in literal DOM host elements. FeedMedia (module 1628) is raw JSX host markup: `return`video`===s?(0,o.jsx)("video",{src:t.style:n.controls:10,playsinline:!0,preload:"metadata"}):(0,o.jsx)("img",{src:t.style:n.alt:""})`, fed at mod/1622.js:326-328 `(0,A.jsx)(k.FeedMedia,{uri:e.media_uri,type:e.media_type,...})`. Stories use ApplImage, whose resolver is also a pass-through — module 1660: `e.mediaUri=n=>n?.startsWith(t.MEDIA_PATH)?`\${(0,l.backendBase())}\${n}`:n:null` — fed at mod/2487.js:100-101 `(0,C.jsx)(u.ApplImage,{uri:Z.media_uri,...})`. The value itself is never validated: mod/631.js:1729-1730 `const t = Ba.exec(e); if (!t) return e;` returns any non-data-image string verbatim, and the blob-store branch that would rewrite it is unreachable in this build because the storage adapter (module 619) defines no putBlob — `if (!n) return e;` at mod/631.js:1733 always takes the early return.	An arbitrary absolute URL in media_uri causes every viewer's browser to issue a request to that host the moment the post scrolls into view, leaking each viewer's IP, User-Agent and a precise view-time signal to a third party the viewer never chose — and giving the poster a read receipt for a public feed item. It also lets a post point at content hosted anywhere, outside any moderation the platform applies to its own storage.	Validate in Ya (mod/631.js:1727): accept only the data:image form matched by Ba and paths beginning with MEDIA_PATH, and thro E_UNSUPPORTED_IMAGE for anything else — the check mockSetTeamBadge already performs at mod/631.js:9094 `if ((av await so(t, e), null != a && !Ba.test(a) && !a.startsWith(ja))) throw Error("E_UNSUPPORTED_IMAGE");`. Mirror it in mediaUrl (mod a stored bad value cannot render.
P-CQUAL-22 Medium	./(tabs)/profile.tsx	Correctness / audit-log integrity	The profile tab fetches its own passport without a viewer id, so self-view is treated as a stranger and every focus writes a spurious audit entry The passport is refetched on every focus of the profile tab (mod/1805.js:81-85 `useFocusEffect(useCallback(() => { Ge(); }, [Ge]))`), and the general audit log is a 500-entry ring buffer (mod/643.js:20 `o = 500` with `u.unshift(...)` and `u.slice(0, a)` at mod/643.js:36-42). Evidence: mod/1805.js:43-45 `(0,l.fetchPassport)(e.id).then(Pe).catch(() => Pe(null)),` — one argument. The facade forwards both (mod/671.js:501 `e.fetchPassport = (o, c) => t.store.mockGetPassport(o, c);`), so the viewer arrives as `undefined` and mod/631.js:6613 `i = t === e` evaluates false for the user's own passport. Consequences inside the same function: mod/631.js:6615 `r = i    "admin" === n` is false, so mod/631.js:6656/6658/6667 apply the subject's own privacy toggles to the subject; and mod/631.js:6645-6648 `i    (await (0, w.logAudit)("passport.accessed", (0, w.actorRef)(t ?? null), { subject: e.slice(-6) })))` fires with a null actor. The comparison caller gets it right: mod/2473.js:17 `fetchPassport(t, o?.id)`. The transport also strips the trailing undefined before an RPC (mod/673.js:55 `for (; n.length && void 0 === n[n.length - 1]; ) n = n.slice(0, -1);`), so a real server sees the same anonymous read.	Two effects. First, a user who turned off "Show sports played" on their own passport sees their own profile card silently fall back to preferred_sports (mod/1805.js:94 `qe = ve?.sports_played?.length ? ve.sports_played : (M?.preferred_sports ?? [])`) — the toggle applies to the owner. Second, every visit to the profile tab appends a `passport.accessed` entry with `actorRef: null`, so normal navigation floods the 500-entry audit log and evicts real security events (blocks, consent changes, privilege actions) within a few dozen taps.	Pass the viewer at mod/1805.js:43: `fetchPassport(e.id, e.id)`. Se make the self-check in `dr` robust by treating a missing viewer as rather than as a mismatched stranger, and consider not audit-log that carry a null actor at mod/631.js:6645.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-23 Medium	/awards/[gameId].tsx	Error handling	/awards/[gameId] spins forever for an unknown or stale game id `N` is initialised to `null` (mod/1835.js:11) and the null branch is an indefinite `ActivityIndicator` with no timeout, error branch or retry. The route is reachable with an arbitrary id — directly by URL, since this is a single-page web app, and from an `award_results`/`award_voting_open`/`award_voting_closing` notification at mod/2458.js:70-73 `B.push(`/awards/\${n.game_id}`)`, which can reference a match that no longer resolves. Evidence: mod/1835.js:16-18 `G = (0, a.useCallback)(async () => { T && t && W(await (0, R.fetchAwardBallot)(T.id, t)); }, [T, t])` — no catch. mod/631.js:15050-15053 `r.mockGetAwardBallot = async (e, t) => { await ei(); const a = hi(t); if (!a) throw new Error("not_found"); ...`. The render falls through to a bare spinner: mod/1835.js:262 `: (0, z.jsx)(s.default, { color: L.accentText, style: { marginTop: k.spacing.xl } })`.	A mistyped or stale `/awards/<id>` URL, or a notification for a match that has since been removed, produces a screen that loads forever under a header that reads "Match awards" with no badge and no content. The user is given no error and no signal that retrying will not help.	Catch in `G` (mod/1835.js:16), distinguish `not_found` from a trar via `classifyError` (mod/9001.js:59), and render `EmptyState`/`G` with a retry instead of the fallback spinner at mod/1835.js:262.
P-CQUAL-24 Medium	/awards/[gameId].tsx	Rendering correctness	Tied award winners render with duplicate React keys and no indication that the award was shared `award_key` is not unique across winners once a tie exists, so React receives duplicate keys in the same list and reconciles rows by a key that identifies two different people. Evidence: `tallyVotes` returns every tied nominee (module 651 in index.html): `const n=[0][1],l=t.filter(([,a])=>a===n).map(([,a])=>a);s.push({award_key:a,winner_ids:l.votes.n,tied:l.length>1})`. mod/631.js:14990-15003 then pushes one `awardWins` row per winner, all sharing `award_key`: `t.award_key` and `votes: t.votes`. `mockGetAwardBallot` returns them as separate entries (mod/631.js:15058-15073). The screen keys the list on the award, not the winner: mod/1835.js:90 `N.winners.map((t) => (0, z.jsx)(b.Card, { ... }, t.award_key))` — the key argument is on mod/1835.js:140. The MVP panel does consume the tie flag `(l.tied ? M('mvpCoWinnersTitle') : M('mvpWinnerTitle'))`, and `mockGetMvpPanel` exposes `tied: s.length > 1` at mod/631.js:15126, but the ballot response has no equivalent field.	On any tie — routine in a pickup match where three or four votes decide an award — React emits a duplicate-key warning (which the smoke harness is supposed to treat as a failure) and can reuse the wrong card's state when the list re-renders after a vote or a revoke. Visually, two winners appear as two independent awards of the same name each showing the full vote count, with nothing telling the reader it was shared.	Key on the win id, which the payload already carries: change mod/1835.js:140 to `t.id` (set at mod/631.js:15064). Add a `tied` f ballot winners in mod/631.js:15058 and render a co-winner label, `mvpCoWinnersTitle`.
P-CSEC-78 Medium	/awards/[gameId].tsx	Destructive action safeguards	Revoking an award from the consumer screen is a single unconfirmed tap that is permanent and tells nobody An irreversible, reputation-affecting action is exposed as a bare trash icon inside a consumer screen, with a `hitSlop` of 8 that enlarges the touch target, sitting immediately beside the nominee row. Unlike every other destructive action in this codebase it requires no reason, and unlike sanctions it produces no notification to the affected player. Evidence: mod/1835.js:122-136 `N.is_admin && (0, z.jsx)(l.default, { onPress: async () => { T && (await (0, R.revokeAward)(T.id, t.id), await G()); }, accessibilityRole: "button", accessibilityLabel: H("revokeBtn"), hitSlop: 8, style: { marginStart: k.spacing.sm }, children: (0, z.jsx)(p.Ionicons, { name: "trash-outline", size: 18, color: L.danger }) })` — no confirmation, no reason prompt, no catch. mod/631.js:15297-15308 `r.mockRevokeAward = async (e, t) => { (await ei(), await sn(e)); const a = Qt.awardWins.find((e) => e.id === t); if (!a) throw new Error("not_found"); ((a.revoked = !0), (a.revoked_by = e), await Za(Je, Qt.awardWins), await (0, w.logAdminAudit))("award.revoked", e, a.winner_id, {...}); };` — `revoked` is only ever set to `!0` here and to `!1` at creation (mod/631.js:15001); no un-revoke exists anywhere. Compare the conduct actions, which all demand a reason: mod/631.js:12520 and 12559 `if (!r) throw new Error("E_A_REASON_IS_REQUIRED")`.	One mis-tap permanently strips a player's award. It disappears from the ballot (mod/631.js:15060 filters `!e.revoked`), from the leaderboard (mod/631.js:15280) and from their passport, the player is never told, and there is no code path to restore it — only the admin audit entry records that it happened. The reason field the audit log would want is never captured.	Add a confirmation dialog and a required reason in mod/1835.js:1 the call in try/catch, accept and persist that reason in `mockRevo` (mod/631.js:15297) as `mockReviewSanction` does, notify the af player, and add an un-revoke path.
P-CQUAL-25 Medium	/awards/leaderboard.tsx	Error handling / request races	The awards leaderboard has no error state and races its own filter changes Neither load handles rejection, so a failure leaves `E` at `null` and the screen at a permanent spinner. And because every chip tap recreates `N` and fires a new request without cancelling the previous one, responses are applied in completion order, not request order — `M` simply overwrites with whatever lands last. Evidence: mod/1836.js:13-15 `(0, l.useEffect)(() => { (0, S.fetchAwardSeasons)().then(W); }, []);` — `then` with no `.catch`. mod/1836.js:16-18 `const N = (0, l.useCallback)(async () => { M(await (0, S.fetchAwardLeaderboard)({ sport: H ?? void 0, season: R ?? void 0 })); }, [H, R]);` and mod/1836.js:20-22 `(0, l.useEffect)(() => { N(); }, [N]);` — no catch, no abort, no request sequencing. mod/1836.js:77-78 `null === E ? (0, z.jsx)(o.default, { color: t.accentText })`.	Tapping quickly through the sport and season chips can leave the board showing results for a filter that is no longer selected, with the chips rendering the other selection — the user reads the wrong leaderboard with no indication anything is stale. On a slow connection this is the normal case, not the edge case. A failed load is indistinguishable from a slow one and never resolves.	In mod/1836.js:16 track a request sequence number (or an Abort and discard responses that are not the latest, add `.catch` on bot an error state and a retry, and reset `E` to `null` on filter change s list is not shown under new chips.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-34 Medium	./booking/[id].tsx	Financial reporting / unaccounted funds	Money taken on a late-cancelled booking is forfeited while the venue settlement row is deleted, so it appears in no revenue figure Forfeited money lives only as a “paid” payment row attached to a “cancelled” booking. It is not in “gross” (settlements were deleted), not in “refunds” (status is not “refunded”), and there is no ledger posting recording that the platform kept it. “paidPlayers” counts the row but no KWD figure includes it. Evidence: mod/631.js:7472-7474 — the forfeit short-circuit leaves the payment at “paid”: `((!r && t)    ((e.status = t ? "refunded" : "expired"), t && ((e.refunded_at = new Date().toISOString()), await ac(e.payer_id, e.amount_kwd, e.id))), ...`. mod/631.js:7489 — the venue's claim is then destroyed unconditionally: `Qt.settlements = Qt.settlements.filter((e) => !(e.booking_id === i.id && "pending" === e.status))`. Revenue is read only from settlements and refunds, mod/631.js:8371-8381: `grossKwd: (0, z.roundKwd)(a.reduce((e, t) => e + t.gross_kwd, 0)), ... refundsKwd: (0, z.roundKwd)(n.filter((e) => "refunded" === e.status).reduce((e, t) => e + t.amount_kwd, 0))` where `a = Qt.settlements.filter(...)` . Same shape in `mockGetAdminFinancials`, mod/631.js:8392-8394.	Real money collected from players disappears from every financial view the venue and the admin have. Neither party can tell how much was forfeited, the venue loses both the slot and the settlement with no record of why, and there is no audit trail to answer a player asking where their money went.	In `Mr` (mod/631.js:7469-7489), post the forfeited amount explicit ledger row from `external:clearing` to `platform:forfeits` per payment mark the payment “forfeited” rather than leaving it “paid”. Settlement then delete the venue's settlement row, or record a cancellation-f settlement, and add forfeits as a line in `mockGetVenueRevenue` `mockGetAdminFinancials` .
P-CQUAL-26 Medium	./chat/[gameId].tsx, ./messages/[id].tsx, ./teams/chat/[teamId].tsx, ./contact.tsx, ./venue/[id].tsx	Silent data loss	Long text is truncated silently at an arbitrary UTF-16 boundary, with no maxLength, no counter and no error The backend enforces limits the UI never communicates and the user never consents to. Truncation is applied after the fact rather than prevented at the keyboard, and it is byte-naïve so it can corrupt the final character rather than merely dropping it. Evidence: Every cap is a bare `.slice(n)` that neither warns nor errors: mod/631.js:2872 `body: (0, v.sanitizeText)(e.body, 1e3),`, mod/631.js:14707 `const n = (0, v.sanitizeText)(a, 1e3),`, mod/631.js:2803 `message: (0, v.sanitizeText)(e.message, 2e3),`, mod/631.js:14376 `const a = t.caption.trim().slice(0, 2e3),`, mod/631.js:14821 `const a = (t.text ?? "").trim().slice(0, 280),`. The matching composers set no client limit — mod/1843.js:100-108 (`multiline: !0,` with no `maxLength`, mod/2497.js:207-215, mod/2455.js:383, mod/2373.js:155-162, mod/2502.js:116-123 — and the shared Input component (module 625) renders no character counter and applies no default `maxLength: `(0,p.jsx)(c.default, Object.assign({placeholderTextColor:v.textMuted,...},j))`. Because `String.prototype.slice` operates on UTF-16 code units, a cut landing between a surrogate pair leaves a lone surrogate: "u{1F525}".slice(0,1) === "ud83d", which renders as U+FFFD.	A long chat message, DM or support request is accepted, the composer clears, and the recipient sees a message that stops mid-sentence — the sender has no way to know. A story caption cut at exactly 280 code units can end in a broken-glyph replacement character where an emoji was.	Add `maxLength` to each composer (1000 for mod/1843.js:100 and mod/2497.js:207 and mod/2455.js:383, 2000 for mod/2373.js:155 and mod/2502.js:116), show a counter in the shared Input when `max` set, and make the backend caps reject-with-error rather than truncate minimum slice on code points via [...s].slice(0,n).join("").
P-CQUAL-27 Medium	./clubs/[id].tsx	Error handling / dead end	The club screen's error state is a dead end that prints raw backend error codes The shared `GateScreen` in mod/9001.js:72-123 exists precisely for this — it renders a header with a back control, an optional retry and a localised title — but module 1847 does not import it (module 9001 is absent from its dependency list at mod/1847.js:497-500). The error branch also has no retry affordance even though `J` is a stable callback. Evidence: mod/1847.js:51-57 — when the dashboard fails there is no header, no back button, no retry, just centred text: <pre>... 51 if (!F) 52 return (0, k.jsx)(c.SafeAreaView, { 53 style: [V.center, { backgroundColor: y.bg }], 54 children: E 55 ? (0, k.jsx)(s.default, { style: [_typography.body, { color: y.danger }], children: X(E) }) 56 : (0, k.jsx)(i.default, { color: y.accentText }), 57 }); ...</pre> mod/1847.js:31-40 — `X` maps only four codes and returns the raw message otherwise: <pre>... 31 const X = (e) => 32 "org_unverified" === e ? D("errUnverified") 34 : "rate_limited" === e ? D("errRateLimited") 36 : "quota_exceeded" === e ? D("errQuota") 38 : "forbidden" === e ? D("errForbidden") 40 : e, ...</pre> mod/631.js:13143 throws a code `X` does not cover: `if (!a) throw new Error("club_not_found");`	A user following a stale or shared club link sees a full-screen page containing nothing but the untranslated English token “club_not_found” in red, with no back button, no retry and no navigation of any kind. In the app shell there is no way off that screen. Arabic users get an English internal identifier. The same dead end occurs for any member removed from a club, whose “forbidden” at mod/631.js:13146 at least maps to a string but still offers no exit.	Render `GateScreen` from module 9001 in mod/1847.js:51 (`kind` `onRetry: J`), which supplies the back control and the localised code “club_not_found” to the `X` map in mod/1847.js:31 and to the error table in module 674 with “en”/“ar” strings in 909.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-79 Medium	/clubs/[id].tsx	Privilege escalation via bearer token	Club invitation tokens are not bound to the invited email, so any holder can redeem them The token is treated as a pure bearer credential with no binding to the identity it was issued for, and the invited role is applied to whoever redeems it. Evidence: mod/631.js:13230 — `r.mockAcceptClubInvitation = async (e, t) => {` / mod/631.js:13232 `const a = Qt.clubInvites.find((e) => e.token === t.trim());` / mod/631.js:13233 `if (!a    !(!0, l.isInviteRedeemable)(a, Date.now())) throw new Error("invitation_invalid");` — the invite's `email` field is never compared to the caller. The invite is created with a target address at mod/631.js:13215 `email: a.email.trim().toLowerCase(),` and a role at mod/631.js:13216 `role: a.role,`. The caller's own address is right there in the redemption body — mod/631.js:13243 `email: bd(e)    a.email,` (with `bd` defined at mod/631.js:13079 `const bd = (e) => Qt.users.find((t) => t.id === e)?.email ?? "",`) — so the comparison was available and skipped. Redemption grants the invited role outright: mod/631.js:13245 `role: a.role,` / mod/631.js:13246 `status: "active",`.	An invitation intended for a specific person — potentially at `manager` or `admin` rank, since `mockInviteClubMember` only forbids inviting at or above the inviter's own rank (mod/631.js:13199) — grants that role to anyone who obtains the token: from a forwarded email, a shared screenshot, or the `pending_invites` array that `mockGetClubDashboard` returns to members holding `members:invite` (mod/631.js:13157), which includes the raw `token`.	In `mockAcceptClubInvitation` (mod/631.js:13230) require the cal address to match: `if (bd(e).toLowerCase() !== a.email) throw new Error("invitation_invalid");`. Also strip `token` from the invite object by `mockGetClubDashboard` at mod/631.js:13157.
P-CQUAL-28 Medium	/clubs/create.tsx	Error handling / input validation	Club creation swallows every failure and stores an unbounded, unsanitized club name Two divergences from the clan equivalent that was clearly the template: no error surfacing (mod/2498.js:346 has `catch (e) { r.default.alert(...) }`) and no `sanitizeText(name, 60)`. `sanitizeText` is what strips control characters (`[^\u0000-\u001f]`) and enforces a length cap across the rest of the backend. Evidence: mod/2370.js:86-96 — there is a `try` / `finally` but no `catch`: ... 86 onPress: async () => { 87 if (t && P.trim()) { 88 A(!0); 89 try { 90 const l = await (0, T.createClub)(t.id, { name: P.trim(), type: H }); 91 B.replace(`/clubs/\${l.id}`); 92 } finally { 93 A(!1); 94 } 95 } 96 }, ... mod/2370.js:36-41 sets no `maxLength` on the name field, unlike the clan form (mod/2498.js:146: `maxLength: 60`). mod/631.js:13093-13096 does not sanitize or cap either, unlike `mockCreateTeam` (mod/631.js:8538 `const a = (0, v.sanitizeText)(t.name, 60);`): ... 13093 r.mockCreateClub = async (e, t) => { 13094 await ei(); 13095 const a = t.name.trim(); 13096 if (!a) throw new Error("name_required"); ... 	If `createClub` rejects — a store write failure, a future quota rule, an unreachable remote backend — the spinner stops and absolutely nothing else happens: no alert, no message, no navigation. The user taps again and again with no explanation, and the rejection surfaces only as an unhandled promise in the console. Separately, a pasted name of arbitrary length with embedded control characters is persisted verbatim into `playora.mock.*.v1` and rendered in the club list (mod/2371.js:88) and header (mod/1847.js:86), bloating localStorage toward its quota.	Add a `catch` in mod/2370.js:89 that alerts with `storeErrorText(e exactly as mod/2498.js:346 does; add `maxLength: 60` to the Inp mod/2370.js:36; and replace `t.name.trim()` at mod/631.js:13095 v.sanitizeText)(t.name, 60)`.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-35 Medium	/clubs/index.tsx	Unreachable screen / broken flow	The clubs hub and the rankings screen ship in the bundle but nothing in the app navigates to them, stranding the club invite flow Three shipped screens (2371, 2370, 2501) have no entry point. The clubs hub is load-bearing: it is the sole owner of both "create a club" and "accept a club invitation", so removing its entry point severs the club onboarding loop while the invite-sending half (mod/1847.js:332-358) still works and still hands out tokens. Evidence: A whole-bundle scan of index.html finds exactly one occurrence of each route string, and in both cases it is the Expo Router route-table entry, not a navigation call: <code>"/clubs/index.tsx"; { enumerable: !0, get: () => r(d[...]) }</code> and <code>"/teams/rankings.tsx"; { enumerable: !0, get: () => r(d[83]) }</code>. No <code>"push("/clubs")</code>, <code>"href:/clubs"</code> or <code>"/teams/rankings"</code> exists anywhere in index.html. The only in-app door into clubs is mod/1805.js:683, which requires an existing membership: <code>"onPress: () => J.push(Oe ? "/venue/portal" : Ee ? "/clubs/\${Ee.club.id}" : "/venue/apply");"</code> <code>"clubs/create"</code> is reachable only from mod/2371.js:53 — inside the unreachable hub: <code>"onPress: () => _.push("/clubs/create");"</code> and the <code>"only"</code> place an invitee can redeem a club token is also inside it, mod/2371.js:124-137: <code>"placeholder: L("pasteToken")" ... "const { club_id: t } = await (0, k.acceptClubInvitation)(e.id, A.trim());"</code>	A club admin sends an invitation and reads the token to a colleague; the colleague opens the app and finds no screen anywhere that accepts it — the invite is undeliverable by design. No user can create a club either, so the only clubs that exist are ones seeded or created before the entry point was lost. <code>/teams/rankings</code> , the only view of the global ladder that battles pay into, is equally invisible; users see their rank number on the clan page with no way to see the table it comes from.	Add a "Clubs" tile to the profile tools section in mod/1805.js:682 to <code>"clubs"</code> unconditionally (falling back to the hub when <code>"Ee"</code> is null), <code>"teams/rankings"</code> from the clan page rank tile (mod/2489.js:241-242) to the league screen. If the screens are genuinely retired, delete mod/2371/2370/2501 and the club invite UI in mod/1847.js:332 rather than a flow that dead-ends.
P-CQUAL-29 Medium	/conduct/index.tsx	Error handling	/conduct spins forever if the code-of-conduct status call fails The only unguarded call is the one that gates rendering. On rejection <code>"O"</code> stays <code>"null"</code> and <code>"N"</code> stays <code>"true"</code>, so both halves of the render guard hold forever; there is no catch, no error state and no retry, even though mod/9001.js ships <code>"GateScreen"</code> and <code>"RetryButton"</code> for exactly this. Evidence: mod/2372.js:15-21 <code>"K = (0, s.useCallback)(async () => { t && (R(await (0, B.fetchCoCStatus)(t.id)), E(await (0, B.fetchDisciplinaryStanding)(t.id).catch(() => null)), L(await (0, B.fetchUserSanctions)(t.id, t.id).catch(() => [])), P(1)); }, [t]);"</code> — the two later calls carry <code>"catch"</code>, the first does not. mod/2372.js:27-31 <code>"if (N !O) return (0, T.jsx)(p.SafeAreaView, { style: [_.center, ...], children: (0, T.jsx)(l.default, { color: o.accentText }) })";</code>. The sequence is a comma expression, so a rejection from <code>"fetchCoCStatus"</code> also skips <code>"P(1)"</code> and both remaining loads.	A transient failure leaves the code-of-conduct screen as a permanent spinner. The user cannot read the rules, cannot see their standing, cannot see their sanction history, and — critically — cannot accept the code of conduct, which is the screen's only action. Their only exit is the OS back gesture; the in-screen back button is not rendered in the loading branch.	Wrap the body of <code>"K"</code> (mod/2372.js:15) in try/catch, set an error state, and render <code>"GateScreen"</code> with <code>"kind: "error"</code> and <code>"onRetry: K"</code> from mod/9001.js:72 instead of an unbounded spinner.
P-CQUAL-30 Medium	/conduct/index.tsx	Sanction visibility	A sanction's status is communicated to the sanctioned user as an unlabelled green middot, and history is silently truncated <code>"mockGetUserSanctions"</code> returns four distinct statuses — <code>"active"</code>, <code>"pending_approval"</code>, <code>"rejected"</code>, <code>"overturned"</code> — and the screen collapses all three non-active ones into the same unlabelled dot, so a pending ban recommendation looks identical to one that was thrown out. The decision note the reviewer was forced to write (<code>"E_A_REASON_IS_REQUIRED"</code>, mod/631.js:12520) is fetched and discarded. Evidence: mod/2372.js:118-122 <code>"active" !== t.status && (0, T.jsx)(u.default, { style: [S.typography.caption, { color: o.success, marginStart: 6 }], children: "xb7" })"</code> — a period, in green, is the entire status indicator. mod/2372.js:98 <code>"H.slice(0, 6).map((t) => { " with no counter and no "show all" control. The row renders only "sanctionMeta(t.type).emoji", "A(s.labelKey)", "reason" and "formatRelative(t.created_at)" (mod/2372.js:105-117). The backend returns far more: "status", "category", "issuer_name", "reviewed_at", "review_note" and "evidence_url" (mod/631.js:12595-12605, fields created at mod/631.js:12449-12464).</code>	This is the only screen where a sanctioned player can see what was done to them, and it shows neither who issued it, nor the violation category, nor whether it is still in force, nor the reviewer's reasoning. A player with more than six entries cannot reach the older ones at all — there is no pagination, no count, and no other route to them. The rendering also crashes the screen if an unknown type ever appears, since <code>"sanctionMeta"</code> returns <code>"undefined"</code> (bundle-src/656.js:40) and mod/2372.js:99 immediately reads <code>"s.emoji"</code> .	Render <code>"t.status"</code> as a labelled Badge in mod/2372.js:118 using the <code>"Badge"</code> component, surface <code>"issuer_name"</code> , the category label as <code>"review_note"</code> , replace the <code>"slice(0, 6)"</code> at mod/2372.js:98 with a <code>"show all"</code> affordance, and guard the <code>"sanctionMeta"</code> result before dereferencing it.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-36 Medium	/conduct/index.tsx	Conduct lifecycle	Sanctions have no expiry and no appeal path, so a suspension is a permanent ban in practice A `suspension` is modelled identically to a `ban`: both stay `active` until an administrator manually calls `mockRestoreUser` (mod/631.js:12553). Nothing decays, nothing expires, and warnings and yellow cards accumulate against `computeStanding` forever, so a warning from two years ago still darkens a player's standing badge. Evidence: mod/631.js:12449-12464 builds the sanction record — `{ id, user_id, type, category, reason, evidence_url, game_id, issued_by, issuer_role, status, created_at, reviewed_by, reviewed_at, review_note }` — with no duration or expiry field of any kind. mod/631.js:12388-12392 `const cd = (e) => Qt.sanctions.filter((t) => t.user_id === e && "active" === t.status), _d = (e) => (0, D.computeStanding)(cd(e)), ud = (e) => { const t = _d(e); return t.isBanned    t.isSuspended; }` — membership is by status alone, never by time. mod/631.js:12397-12402 `async function wd(e) { if (!ud(e)) return; await (0, w.logAudit)("conduct.join_blocked", ...); const t = _d(e); throw new Error(!t.isBanned ? "E_ACCOUNT_BANNED_FROM_MATCHES" : "E_ACCOUNT_SUSPENDED"); }`. Grepping the backend for `appeal` returns nothing, and /conduct offers no action but `cocAccept` (mod/2372.js:209).	A player suspended for lateness is locked out of joining any match indefinitely, sees `standing_suspended` on /conduct with no end date, and has no in-app way to contest it or ask for review — the only support channel, /contact, writes to a store nobody reads. This is the kind of open-ended penalty with no notice period and no appeal that a consumer platform cannot defend.	Add `expires_at` to the sanction record at mod/631.js:12449 and (mod/631.js:12388) exclude expired entries, decay warnings and cards after a defined period, display the expiry on the /conduct hi and add an appeal action that opens a reviewable case rather than and-forget contact form.
P-CSEC-80 Medium	/conduct/index.tsx	Sanction history exposure	Disciplinary standing is fetched by subject id with no caller, so it can never be authorized Two functions read the same sanction records; one takes a viewer and enforces self-or-admin, the other takes only the subject. The second cannot be secured without a signature change, so the hole persists after this code moves to a server. Evidence: mod/631.js:12607: <pre>... r.mockGetDisciplinaryStanding = async (e) => (await ei(), _d(e)); ... where `_d = (e) => (0, D.computeStanding)(cd(e))` and `cd = (e) => Qt.sanctions.filter((t) => t.user_id === e && "active" === t.status)` (mod/631.js:12388-12389) — the subject's live sanction set, including whether they are banned or suspended (`ud` at mod/631.js:12391-12393 reads `isBanned` / `isSuspended` off it). The sibling function on the same data does gate correctly: mod/631.js:12601 `if ((await ei(), t) !== e && !ro(t)) throw new Error("E_YOU_ARE_NOT_AUTHORIZED_TO_VIEW_2");`. The facade mirrors the single argument: mod/671.js:764 `e.fetchDisciplinaryStanding = (o) => t.store.mockGetDisciplinaryStanding(o);`.</pre>	Any caller can look up whether an arbitrary user is currently banned, suspended, or carrying active cards. Conduct records are the most stigmatising data the app holds about a person and the app's own sibling endpoint treats them as self-or-admin only.	Change to `mockGetDisciplinaryStanding(viewerId, subjectId)` with guard as `mockGetUserSanctions` (mod/631.js:12601), update mod/671.js:764, and have /conduct/index.tsx (mod/2372.js:18) pass as it already does for `fetchUserSanctions` on the next line.
P-CQUAL-31 Medium	/contact.tsx	Error handling	The contact form has no catch, so a failed submission discards the typed message with no error shown The only branch in the handler is success. There is no catch, no error state bound to the form, and no retry — the failure surfaces only as an unhandled promise rejection in the console. Evidence: mod/2373.js:167-180: `onPress: async () => { if (ee()) { Q(!0); try { (await (0, S.submitContact)({ first_name: M.trim(), ... message: F.trim(), }, Y(!0))); } finally { Q(!1); } } }` — try/finally with no catch and no error state. The error does reach this point: mod/672.js:41-45 rethrows `(`) catch (e) { throw (0, r.localizeStoreError)(e); }` and mod/673.js:68-69 throws on any non-ok server response (`const f = await y.json(); if (!f.ok) throw new Error(f.error ?? "Server error:");`) — and `y.json()` itself is outside the fetch try/catch at mod/673.js:65, so a non-JSON error page from a proxy also throws here. Sibling screens handle this correctly, e.g. mod/2497.js:225-226 `}) catch (e) { n.default.alert(B("error")), (0, W.storeErrorText)(e?.message ?? "") B("error");`. The same pattern appears in game chat: mod/1843.js:30-42 `try { const t = await (0, _sendChatMessage)({...}); (O(...), A(""), ...) } finally { q(!1); }`.	A user writes a support request, presses Submit, the spinner stops and nothing happens — no success screen, no error, no indication the message was lost. Because this is the app's only support channel, the failure mode is a customer who believes they have contacted the business and has not. The same shape in game chat leaves a failed send looking identical to a successful one until the message fails to appear.	Add a catch around the await at mod/2373.js:169 that surfaces `storeErrorText(e?.message)` through the existing error state (`J` matching mod/2497.js:225, and keep the composed text in state; can retry; apply the same at mod/1843.js:37.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-81 Medium	/feed/compose.tsx	Input sanitisation	Feed captions and comments bypass sanitizeText entirely, and sanitizeText itself strips only C0 control characters All user-authored text should pass through one sanitiser before storage, and that sanitiser should strip bidi and zero-width format characters — the app ships both `en` (LTR) and `ar` (RTL) locales in the same views, so directional overrides change what a reader sees. Today the feed path applies no sanitiser at all, and the messaging path applies an incomplete one. Evidence: mod/631.js:14376 — `const a = t.caption.trim().slice(0, 2e3);` — the caption is stored raw; `sanitizeText` is never called on it, and the raw value is also what feeds hashtag extraction (mod/631.js:14387 `hashtags: ol(a)`). mod/631.js:14544 — `const i = a.trim().slice(0, 1e3);` — comments likewise. The messaging paths do sanitise: mod/631.js:14707 `const n = (0, v.sanitizeText)(a, 1e3);` for DMs and mod/631.js:2872 `body: (0, v.sanitizeText)(e.body, 1e3);` for match chat. `sanitizeText` itself (module 650 in index.html) is `function(s,n=2e3){return s.replace(t,"").slice(0,n).trim()}` with `const t=new RegExp("[\u0000-\u001F\u007F]","g")` — C0 controls and DEL only; it leaves U+202A-202E and U+2066-2069 bidi overrides, U+200B/U+200E/U+200F zero-width and directional marks, and unbounded combining marks intact. Post captions render straight into ` <text>` at mod/1622.js:303-310 `children: x.caption`, comments via `li` (mod/631.js:14532 `text: e.text`).</text>	Raw control characters, unbounded zero-width padding and bidi overrides survive into stored captions, comments, DM bodies and match-chat bodies. In the mixed LTR/RTL feed a caption containing U+202E renders reversed and can be made to display text different from what is stored — enough to misattribute a quote or disguise a link's visible text next to the author's own name. Zero-width runs also let a user pad a caption past the visible length with no rendered content. The inconsistency between the feed and messaging paths means a fix applied to one will not cover the other.	Call `sanitizeText(t.caption, 2e3)` in `mockCreatePost` and `sanit1e3)` in `mockAddComment` instead of the bare `trim().slice()`. E regex in module 650 to strip format characters as well — e.g. `/[ ‮yp.eldnub/sloot` aiv dliuber dna ,stifeneb rellac yreve os
P-CQUAL-32 Medium	/feed/compose.tsx	Misleading UI	The compose screen's character counter and hashtag hint both describe behaviour the code does not implement The counter should state the limit actually enforced (2000), and the hint should not advertise @-mentions when neither the screen nor the backend does anything with them. Three different numbers — 450, 500, 2000 — are in play for one field. Evidence: mod/2375.js:154 `maxLength: 2e3` on the caption input, mod/2375.js:163 `{ color: H.length > 450 ? z.warning : z.textMuted, ... }`, mod/2375.js:165 `children: B("charCounter", { n: H.length, m: 500 })` — the counter template is `"%{n} / %{m}"` (index.html, both `en` and `ar`), so a 1200-character caption renders as "1200 / 500" in warning colour and posts successfully, because the backend caps at 2000: mod/631.js:14376 `const a = t.caption.trim().slice(0, 2e3);`. Separately mod/2375.js:167-170 renders `B("hashtagHint")` = "Tip: add #hashtags and @tag players to reach more people." but there is no mention handling anywhere: the preview extractor at mod/2375.js:21 matches only `/#[p{L}0-9_]+/gu`, the create call at mod/2375.js:60-66 passes only `kind`, `caption`, `clipId`, `mediaUri`, `mediaType`, and `mockCreatePost` therefore always stores `tagged_player_ids: (t.taggedPlayerIds ?? []).slice(0, 20)` as an empty array (mod/631.js:14388).	A user writing a long caption sees a red over-limit counter and either self-censors a post that would have been accepted or believes the post will be rejected. A user who types `@someone` expecting the tagged player to be notified gets nothing — no tag is stored, no notification is sent, and the mention renders as plain text. Both are promises the product does not keep, on the one screen where users create content.	Pass `m: 2000` to `charCounter` and move the warning threshold proportion of the real cap (e.g. `H.length > 1800`), or lower `maxLength` to 500 if 500 is the intended product limit — p number and use it in both places. Either implement @-mentions (alongside `ol`, resolve to ids, pass `taggedPlayerIds` to `createPr via `bi`) or drop the "@tag players" clause from `hashtagHint` in t and `ar` in module 909.
P-CQUAL-33 Medium	/feed/compose.tsx, /stories/compose.tsx, /(tabs)/feed.tsx	Inconsistent input handling	Feed captions, story text, comments and post reports bypass sanitizeText entirely while chat and DMs use it There is no single chokepoint for `text` another user will see'. Whether a control character survives depends on which of four hand-written expressions a given screen happens to hit, and the most public surface in the app (the feed) is the least filtered. Evidence: Four public-text write paths use a bare trim/slice instead of the shared sanitiser. mod/631.js:14376 `const a = t.caption.trim().slice(0, 2e3);` (feed post); mod/631.js:14821 `const a = (t.text ?? "").trim().slice(0, 280);` (story); mod/631.js:14544 `const i = a.trim().slice(0, 1e3);` (post comment); mod/631.js:14577 `reason: a.trim().slice(0, 500);` (post report). The neighbouring message paths do sanitise — mod/631.js:14707 `const n = (0, v.sanitizeText)(a, 1e3);` and mod/631.js:2872 `body: (0, v.sanitizeText)(e.body, 1e3);` — so the same product ends up with two different definitions of acceptable text. The caption is additionally re-parsed for hashtags without normalisation: mod/631.js:14387 `hashtags: ol(a),` where `ol` is mod/631.js:14356 `const t = e.match(/#[p{L}0-9_]+/gu) ?? [];`.	Control characters a moderator cannot see survive into feed captions, story text and comments but not into DMs. Feed captions keep newlines while DMs lose them, so the same pasted text behaves differently depending on where it is posted — and neither behaviour is the intended one. The inconsistency also means any future hardening of sanitizeText (for bidi characters, say) leaves the feed unprotected.	Route all four through the shared helper: `(0, v.sanitizeText)(t.cap 2000)` at mod/631.js:14376, `(0, v.sanitizeText)(t.text ?? "", 280)` mod/631.js:14821, `(0, v.sanitizeText)(a ?? "", 1000)` at mod/631. and `(0, v.sanitizeText)(a ?? "", 500)` at mod/631.js:14577, and di hashtags from the sanitised value.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-34 Medium	/follows/[id].tsx	Error handling	/follows[id] rethrows the cross-audience error into nothing, hanging the screen forever This module copied the audience-error convention from its siblings but kept only the rethrow, without the 'other world' state the siblings render. The rethrow was presumably meant to reach a boundary that does not exist in this app. Evidence: mod/2376.js:15-23 — `M = useCallback(async () => { if ( _ && e) try { (E(await fetchFollowList(e, _id, A)), H(!1)); } catch (e) { if ((0, S.isAudienceError)(e)) throw e; (H(!0), E(!)); } }; [_, e, A])`. The rethrow has no consumer: there is no error boundary in the root layout (mod/1809.js:45-143 is a plain `Stack` with no boundary), so the promise rejects unhandled, `R` stays `null`, `N` stays `null` and mod/2376.js:66-67 `null === N ? jsx(ActivityIndicator)` spins forever. Every other screen that calls `isAudienceError` handles it locally instead: mod/2379.js:36-38 `return isAudienceError(e) ? (Se(!0), void X(!1)) : ...`, mod/2489.js:57 `if (isAudienceError(e)) return (we(!0), void be(!1));`, mod/2495.js:24 `if (isAudienceError(e)) return void O(!0);`, mod/2496.js:29 same. The throw comes from mod/631.js:14120 `if ((await ei(), t !== e && oa(t, e), t !== e))` via `ra`/`oa` at mod/631.js:763-768.	Opening `/follows/<id>` for a user in the other audience partition — reachable from any pasted, bookmarked or shared deep link, since this is a URL-addressed SPA — leaves the screen on a spinner permanently, with an unhandled promise rejection in the console. The user gets no message, no back-off, and no indication that the list is simply not theirs to see; the `privateAccount`/`followListLocked` empty state at mod/2376.js:69-73 is right there but never used for this case.	Replace the rethrow with a local state flag as the four sibling scre `const [otherWorld, setOtherWorld] = useState(false)` and `if (isAudienceError(e)) return (setOtherWorld(true), void H(!1));`, th the existing `EmptyState` (or `GateScreen({kind:"denied"})` from that flag. Also clear the spinner by calling `H(!1)` on every exit pa
P-CQUAL-35 Medium	/friends/index.tsx	Error handling / concurrency	Every mutation on /friends fails silently, and the search fires one unordered request per keystroke Each mutation should catch, surface the mapped error and leave the list in a truthful state; the search should debounce and ignore responses that arrive after a newer query. Neither is done. Evidence: mod/2378.js:26-35 — `Y = async (n) => { if (t) { U(n); try { (await sendFriendRequest(t.id, n), await W(H)); } finally { U(null); } } }`; mod/2378.js:36-45 — `\$ = async (n, s) => { ... try { (await respondFriendRequest(t.id, n, s), await V(), await W(H)); } finally { U(null); } } }`; mod/2378.js:46-55 — `G = async (n) => { ... try { (await followPlayer(t.id, n), await V()); } finally { U(null); } } }`. All three are `try/finally` with no `catch`, and the module's dependency list (mod/2378.js:346) contains no 674, so it cannot render a store error at all. These calls do throw: mod/631.js:13564 `if (zd(e, t)) throw new Error("already_exists")`, mod/631.js:13578 `if (ti) throw new Error("not_found")` in `mockRespondFriendRequest`, mod/631.js:14047-14049 `throw new Error("invalid")` / `throw new Error("not_found")` / `throw new Error("blocked")` in `mockFollow`. The search is unguarded and unsequenced: mod/2378.js:23-25 `W = async (n) => { ( _ (n), lt    n.trim().length < 1 ? O(!) : O(await searchUsers(t.id, n))) }` — called from `onChangeText` (mod/2378.js:90) on every character, with no debounce and no request-ordering guard.	Accept a friend request in one tab and then accept the same one in a second tab that still shows it: `mockRespondFriendRequest` throws `not_found`, the rejection is swallowed, the spinner on the button stops, and the card stays on screen as if nothing happened — the user taps again and again with no feedback. The same silence hides `already_exists` on a duplicate add and `blocked` on a follow. Separately, typing quickly issues one search per character with no ordering guarantee, so a slower response for an earlier prefix can overwrite the results for what the user has actually typed.	Import 674 into 2378, add an `error` state, and give `Y`, `\$` and `i` that sets `storeErrorText(err.message)    t("error")` rendered near (mirror mod/2455.js:361-370). Debounce `W` by ~250ms and gua request-sequence ref so only the latest response is applied.
P-CSEC-82 Medium	/friends/index.tsx	Privacy controls / data exposure	Friend search skips the private-profile and block filters that the player search applies Two name-search endpoints over the same profile table disagree on which privacy rules apply. The one used by /friends omits the block check and the private-profile check entirely. Evidence: mod/631.js:13538-13546 — the entire filter: ... return Qt.profiles.filter((t) => t.id !== e && "admin" !== t.role && "analyst" !== t.role && ia(!, t.audience ?? "male") && !Ho(t.id) && (t.full_name ?? "").toLowerCase().includes(a),) ... The equivalent loop in `mockSearchPlayers` applies two more checks — mod/631.js:14201-14202: ... if (Xd(e, u.id) tl(e, u.id)) continue; if ("private" === al(u.id).profile_visibility) continue; ... Both are reachable from the same UI surface: mod/2378.js:24 calls `searchUsers`, mod/2374.js:34 calls `searchPlayers`.	A user who set their profile to private is still returned by friend search, by name, with their relation status — the setting that is actually enforced elsewhere is bypassed on this path. A user who blocked someone still appears in that person's search results and can be sent a friend request, since `mockSendFriendRequest` (mod/631.js:13561) checks `oa` but not `Xd`. Blocking is supposed to be the one control that always holds.	Add `if (Xd(e, t.id)    tl(e, t.id)) return false;` and `if ("private" === al(t.id).profile_visibility) return false;` to the predicate at mod/631. and add an `Xd` guard to `mockSendFriendRequest` at mod/631 match `mockFollow` (mod/631.js:14049).

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-36 Medium	/game/[id]	Silent failure after payment	Paying to take a specific position silently loses the position if someone else claims it first The user paid specifically to occupy that slot (the flow is entered at mod/2379.js:988-992 `if (Ne && !Le && de) { const e = { side: de.side, slotId: de.slotId }; (ce(null), je({ thenClaim: e })); }` and the checkout summary is labelled `positionLabel`, mod/2379.js:1017). The only failure that matters is discarded. Evidence: mod/2379.js:1020-1027 `onDone: async (e) => { const t = be?.thenClaim; if ((je(null), "confirmed" === e.status)) { if (t && A && H) try { await claimLineupSlot(A.id, H.id, t.side, t.slotId); } catch {} Alert.alert(F("seatPaidTitle"), ...)` — the claim is wrapped in a bare `catch {}`. The claim genuinely fails under a race: mod/631.js:16729 `if (o.player_id && o.player_id !== r.id) throw new Error("E_THAT_POSITION_IS_ALREADY_TAKEN");` and it also throws `E_THE_LINEUP_IS_LOCKED` (mod/631.js:16718) if the organizer locked the board while the checkout sheet was open.	Two players tapping the same pitch position at once both pay; one silently ends up on the bench and is told 'Seat paid' with no mention that the position went to someone else. They discover it later on the line-up board with no explanation and no refund path.	Catch the claim error in mod/2379.js:1024, and alert the paid-but-case explicitly (reusing the `joinPosTakenBody` copy already use mod/2379.js:67), then refresh.
P-CQUAL-37 Medium	/game/[id]	Error handling	Team moves apply optimistically and are never rolled back or reported when the write fails No rollback, no error surface, and no refetch after the write, unlike every other mutation on this screen which alerts and calls `Pe()` . Evidence: mod/2379.js:85-92 `ze = useCallback(async (e, t) => { K((t => i.map((t) => (t.id === e ? Object.assign({}, i, { team: t }) : t))) ; const i = U.find((t) => t.id === e); if (!i.is_self ? await upsertSelfPlayer(Object.assign({}, i, { team: t })) : await setPlayerTeam(A.id, e, t); }, [U, A]);` — local state is mutated first and neither await has a catch, so a rejection is an unhandled promise rejection and the optimistic edit stays on screen. `setPlayerTeam` rejects for guest accounts (mod/631.js:2912 `md(e)` → `E_SIGN_UP_TO_JOIN_BROWSING_IS`, mod/631.js:12394-12396) and both reject on any transport error (mod/673.js:69).	A guest (or anyone offline) drags a player between teams, sees the move stick, and plans around a split the backend never recorded; the truth only reappears on a hard refresh. It also produces console-visible unhandled rejections in production.	Snapshot `U` before the optimistic update in mod/2379.js:87, rest catch, surface `storeErrorText(e?.message)`, and `await Pe()` on
P-CQUAL-38 Medium	/game/[id]	Data consistency	The game screen counts capacity two different ways and can show 'no spots left' next to an enabled Join button One screen, two capacity denominators. Whenever the roster is larger than the booking count — which is the normal case for seeded matches, where `Ui` returns players with no corresponding bookings — the ticket and the CTA disagree by construction. Evidence: The CTA and the 'full' badge use bookings: mod/2379.js:133 `const We = Math.max(0, H.max_players - H.bookings_count);` (badge at mod/2379.js:283 `!De && !Se && 0 === We && Badge "full"`, join button at mod/2379.js:830 `We > 0 ? Button {joinGame}`). The ticket uses the roster instead: mod/2379.js:204 `going: Math.max(U.length, H.bookings_count);` and mod/2379.js:1173 `h = Math.max(0, e.max_players - i);` rendered as `\${formatNumber(h)} \${s("spotsLeft")}` at mod/2379.js:1263. `U` comes from `mockGetGamePlayers` (roster rows, mod/631.js:2878-2908) while `bookings_count` is `ki` (mod/631.js:2670, booking rows) — two different populations.	The ticket can read '10/10 · 0 spots left' while the button below it says 'Join game', or the 'Full' badge can be absent on a match the header calls full. Players do not know whether a seat exists until they tap, and the mismatch reads as a broken or dishonest listing.	Derive one occupancy number in mod/2379.js (use `H.bookings_` both, or have the backend return a single `going` on the game pa pass it to both the ticket and the CTA.
P-CSEC-83 Medium	/game/[id].tsx	Identity confusion	mockGetGamePlayers returns the stored is_self flag instead of computing it for the caller `is_self` is a per-viewer projection being stored and replayed. Once player A's row is persisted with `is_self: true`, player B's copy of the roster contains two rows claiming to be B. Evidence: mod/631.js:2882 — `const i = Qt.gamePlayers.filter((t) => t.game_id === e);` and mod/631.js:2909 `return i.map((e) => Object.assign({}, e, { avatar_url: e.user_id ? (o.get(e.user_id) ?? null) : null }));` — `is_self` is copied straight out of storage and never re-evaluated against `t`. `mockUpsertSelfPlayer` persists rows carrying `is_self: true` for whoever called it (mod/631.js:2924 `Qt.gamePlayers.push(e)`), so those rows are returned to every other viewer still flagged as 'self'. The sibling chat reader does it correctly: mod/631.js:2859 `.map((e) => Object.assign({}, e, { is_self: a ? e.author_id === `self-\${a}` : e.is_self })))`. The flag is load-bearing for writes: mod/2379.js:89 `!?.is_self` / mod/2379.js:90 `? await (0, T.upsertSelfPlayer)(Object.assign({}, i, { team: t })))`, and for slot claiming: mod/631.js:16717 `const r = (await Ui(t, e)).find((t) => t.user_id === e    t.is_self);`.	On the game screen B sees another player labelled 'You' (mod/2379.js:266 `e.is_self ? F("you") : e.display_name.split(" ")[0]`) and, when B changes 'their' team, the client takes the `upsertSelfPlayer` branch and overwrites A's row. In `mockClaimLineupSlot` the `   t.is_self` fallback can resolve to A's row first, so B's tap places A into the slot and clears A from their previous position via `_c(n, r.id)`.	In `Ui` (mod/631.js:2878) drop the stored flag and derive it per ca `is_self: !!t && (e.user_id === t    e.id === `self-\${t}`)` — in the fi mod/631.js:2909, and remove the `   t.is_self` fallback from `pc` (mod/631.js:16717) so a slot claim only ever matches `t.user_id =

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-84 Medium	/game/[id].tsx	Broken access control	A match's private visibility is never checked when the match is read or joined `visibility: "private"` is implemented purely as an omission from the listing query. Nothing in `mockGetGame`, `mockGetGameScreen` or `mockJoinMatch` requires an invite code or any relationship to the organizer. Evidence: Matches carry the setting — <code>mod/631.js:3528 `visibility: t.visibility,`</code> with the enum validated at <code>mod/631.js:3365 `visibility: ["public", "private"];`</code> — and the browse list honours it: <code>mod/631.js:2685 `(e) => "scheduled" === e.status && "public" === e.visibility && new Date(e.starts_at).getTime() >= i,`</code>. Neither of the two id-addressed paths does. Read: <code>mod/631.js:2696 `const Ni = async (e, t) => {` / mod/631.js:2699 `if (la) return null;` / mod/631.js:2700 `ra(t, a.audience);` / mod/631.js:2702 `return li(a, i, t);`</code> — only the audience partition. Join: <code>mod/631.js:2988 `ji = async (e, t) => {` / mod/631.js:2992 `if ((ra(t, a.audience), "scheduled" !== a.status)) throw new Error("E_THIS_MATCH_IS_NO_LONGER_OPEN");`</code> — the subsequent checks cover start time, organizer self-join, registration closure, skill policy and approval mode, but `a.visibility` is never read.	A private match is fully readable and directly joinable by anyone who has the game id — and game ids leak through shared links, referral rows (`Qt.referrals.game_id`), group bookings and notification payloads. Combined with the unauthorized `mockGetMatchInvite`, a private match offers no practical protection over a public one, which matters most for the female-audience partition where private hosting is the point.	Add a visibility gate to `Ni` (mod/631.js:2696) and `ji` (mod/631.js:2992) to check `a.visibility === "private"` and the caller is not the organizer, an already holding a booking or a resolved invite for that game, refusing to call `mockJoinMatch` through a flag that marks the invite as present. This ensures only holders still join.
P-CSEC-85 Medium	/game/[id].tsx	Access control / invite integrity	Any viewer of a match can mint and read its shareable invite code, including for private matches Issuing a share credential is an organizer action, but the function takes only a game id. Combined with the missing visibility gate on `mockGetGame`, it turns a private match into a publicly joinable one for anyone who can open its detail page. Evidence: <code>mod/631.js:6314-6318</code> — no caller parameter at all, so the function cannot be authorized: <pre> ... r.mockGetMatchInvite = async (e) => { await ei(); const t = await Xn(e); return { code: t.code, link: `https://playora.app/m/\${t.code}`, expires_at: t.expires_at }; }; ... `Xn` (mod/631.js:6303-6313) creates the invite row if none exists, so a mere read mints a live code. The game screen calls it for every viewer: <code>mod/2379.js:42 `(0, T.fetchMatchInvite)(e)`</code>, facade <code>mod/671.js:505 `e.fetchMatchInvite = (o) => t.store.mockGetMatchInvite(o)`</code>. `mockJoinMatch` then admits the holder: <code>mod/631.js:6367-6372</code>.</pre>	A non-participant who reaches /game/<id> for a private match walks away with a working invite link they can forward to anyone, and the recipients can join. The organizer has no signal that a code was minted — `mockLogShare` audits deliberate shares (mod/631.js:6360) but `Xn`'s implicit creation writes no audit entry.	Give `mockGetMatchInvite` a caller parameter and require `zi(gai)` (mod/631.js:2967) — organizer only — before minting or returning. Then, only the organizer can call /game/[id].tsx to mint or read an invite.
P-CARCH-37 Medium	/game/[id].tsx	Dead end	/game/[id] renders the bare word 'error' with no retry and no back button when the game id no longer resolves The screen has a well-built error branch with retry and back at <code>mod/2379.js:104-122</code>, but it is gated on `!Q && ke && !H` — it requires a thrown error to have populated `ke`. The 'resolved successfully but game is null' case falls through to a naked `Text` with no controls. Evidence: <code>mod/631.js:16963-16979</code> — `mockGetGameScreen` returns `{ game: a, ... }` where `a = await Ni(t, e)`, and `Ni` returns null for an unknown id (<code>mod/631.js:2696-2700: `const a = hi(e); if (la) return null;`</code>) — it does not throw. <code>mod/2379.js:128-131</code> — `if (!H) return <SafeAreaView style={{flex:1, alignItems:"center", justifyContent:"center", ...}}><Text style={{typography:body, {color: q.text}}>[F("error")]</Text></SafeAreaView>;`.	Game links are the main share surface (`https://playora.app/...` invites, chat links, notifications at <code>mod/2458.js:48</code>). Opening a cancelled, deleted or simply stale game link shows a blank screen containing one word and no navigation — the dock is not rendered on this route, so the user is stranded.	Treat a null `game` the same as an error: at <code>mod/2379.js:44</code> , after <code>fetchGameScreen(...)</code> , do `if (!t.game) { Ce(F("notFound")); X(!1) }` so control reaches the existing retry/back branch at <code>mod/2379.js:104</code> .
P-CTEST-10 Medium	/group/[gameId]	Error handling	Unknown or unreadable gameId leaves the group screen on a permanent spinner The screen has an error state but renders the loading state in front of it, and offers no retry or back control in the loading branch (the header with the back button is only rendered in the two later branches). Evidence: <code>mod/2444.js:27-39 `pe = useCallback(async () => { if (M && e) try { A(await fetchGroupConfig(M.id, e)); ... } catch (e) { ue(e.message); } }, [M, e]);`</code> — the error goes to `ce`, but the render gate above every error surface is <code>mod/2444.js:272-276 `if (!L) return <SafeAreaView style={z.center}><ActivityIndicator/></SafeAreaView>;`</code>. `L` stays null on failure, so the `FormError` at <code>mod/2444.js:588</code> that would show `ce` is never reached. `mockGetGroupConfig` throws for any bad id: <code>mod/631.js:13636 `if (la) throw new Error("not_found");`</code>.	A stale link, a deleted match or any transient store error gives an infinite spinner with no header, no back button and no message; on web the only escape is the browser's back button.	Split loading from error in <code>mod/2444.js</code> : track a `loading` flag, and render the header plus an `EmptyState` with retry and back controls.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-86 Medium	/group/[gameId]	Data exposure	Group and replacement reads authorize only when the caller volunteers their own id Both reads are identity-optional: pass no caller and the check disappears; pass any caller and `qd` returns everything anyway. The rule is missing in the store itself, so moving this code to a server keeps the hole. Evidence: mod/631.js:13748-13770 `const qd = async (e, t) => { await ei(); const a = Qt.groupBookings.find((e) => e.id === t); if (!a) throw new Error("not_found"); ... const n = ..., r = Qt.groupMembers.filter((e) => e.group_id === t), o = n.leader_id === e, ... return { group: n, members: r, is_leader: o, my_due_kwd: ..., seconds_left: ... } }` — membership is computed (`o`) but never enforced; any id returns the full member list and amounts. mod/631.js:12279-12287 `r.mockGetReplacementOffer = async (e, t) => { ... if (t && a.candidate_id !== t && !ro(t)) { const e = hi(a.game_id); if (!e e.organizer_id !== t) return null; } return dd(a); }` — the guard is skipped entirely when `t` is falsy, and the facade makes the argument optional (mod/671.js:750 `e.fetchReplacementOffer = (o, c) => t.store.mockGetReplacementOffer(o, c)` , called as `fetchReplacementOffer(e, c?.id)` at mod/2480.js:15).	Holding a group id exposes who is in that booking, what each person owes and who the leader is; calling the replacement read without a user id exposes the candidate's name and the match they were offered. Both are ordinary RPC calls once the backend is real.	Make the caller id required and assert it in `qd` (return only for the member, the organizer or an admin) and in `mockGetReplacementOffer` (reject a missing caller rather than treating it as 'unchecked').
P-CARCH-38 Medium	/group/[gameId]	Request volume / race conditions	Player search fires on every keystroke with no debounce, cancellation or error handling Results are applied in completion order, not request order, and a rejection is an unhandled promise rejection. Every other list on this screen is loaded once in `pe()`. Evidence: mod/2444.js:414-421 `Input, { placeholder: W("searchPlayers"), value: q, onChangeText: async (e) => { if (!!U(e), !M    e.trim().length < 1)) return void J([]); const t = await searchUsers(M.id, e); J(t.map((e) => ({ id: e.id, name: e.name }))); } }` — one request per character from a single-character prefix, no debounce timer, no AbortController, no request-sequence check before `J(...)` , and no try/catch.	Typing a six-letter name issues six searches, the slowest of which wins and repopulates the list with results for a stale prefix, so the user taps the wrong person into their group; against a real backend the whole user table is scanned from a one-character prefix by every keystroke of every user.	Debounce to ~250 ms and require 2+ characters in mod/2444.js: request id (or AbortController) and drop late responses, and catch into the existing `ce` error state.
P-CSEC-87 Medium	/group/[gameId].tsx	Data exposure (IDOR)	mockGetGroupBooking returns the whole group roster and every member's amount due to any caller The caller is threaded in far enough to compute `my_due_kwd` and `is_leader`, then ignored for the authorization decision on the roster itself. Evidence: mod/631.js:13748 — `qd = async (e, t) => { / mod/631.js:13749 `await ei(); / mod/631.js:13750 `const a = Qt.groupBookings.find((e) => e.id === t);` / `if (!a) throw new Error("not_found");` — no membership test on `e`. The return at mod/631.js:13766 is `members: r,` where `r = Qt.groupMembers.filter((e) => e.group_id === t)` — the complete list. The caller `*is*` used, but only to scope the money figure: mod/631.js:13756 `s = jd(r, e, n.leader_id)` with `jd` at mod/631.js:13744 partitioning dues by leader vs member. The mutating siblings do check membership: mod/631.js:13851 `if (!(e === i.leader_id    e === n.user_id    e === r.organizer_id    Id(e))) throw new Error("forbidden");` (mockCancelGroupMember), and the list view scopes properly at mod/631.js:13775 `Qt.groupMembers.filter((t) => t.user_id === e)`.	Any user with a group id reads the group's full membership: every member's `user_id`, display name, `kind` (leader/friend/guest), `amount_kwd`, `paid_by_leader` flag, per-member payment status and `booking_id`. That exposes who is playing with whom and exactly how much each person is paying, and the leaked `booking_id` values feed other id-addressed endpoints.	At the top of `qd` (mod/631.js:13748) require the caller to be the member, the match organizer or an admin — reuse the predicate written at mod/631.js:13851 — and otherwise throw `forbidden`.
P-CTEST-11 Medium	/join-code	Error handling	Invite-code lookup has no error handling and the Join button is dead while the session loads Every other join surface reports failures (mod/2379.js:155-156 alerts with `storeErrorText`); this one reports only the 'no such code' case and never the 'lookup failed' case. And it renders an enabled primary button whose handler is a no-op while `user` is null. Evidence: mod/2447.js:52-63 `onPress: async () => { if (W.trim()) { A(!0), R(null)); try { const t = await resolveInviteCode(W, e?.id); if (!t) return void Alert.alert(P("invalidInviteCode"), ""); } finally { A(!1); } } }` — `try/finally` with no `catch`, so any rejection from the store proxy (mod/672.js:41-45) surfaces as an unhandled promise rejection and the button simply stops spinning. The join handler is gated on a user that may not exist yet: mod/2447.js:96-100 `onPress: async () => { if (e && M) { ... } }` where `e` is `useAuth().user`, which is null until the session resolves (mod/630.js:15-25, `loading` is never consulted by this screen).	A user pasting a code on a flaky connection taps 'Find match', sees the spinner stop and nothing else — no error, no result — and has no way to tell whether the code or the network is at fault. On a cold deep link into /join-code the 'Join game' button silently does nothing on first tap.	Add a catch that alerts `storeErrorText(e?.message)` in both handlers at mod/2447.js, and disable the buttons (or show the spinner) while `useAuth().loading` is true.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-88 Medium	./join-code.tsx	Audience partition / cross-partition exposure	Invite-code resolution skips the audience-partition check every other game read enforces The partition check is present on the join action ("ji" at mod/631.js:2810) but absent on the read that precedes it, so the data is disclosed and only the write is refused. Evidence: mod/631.js:6320-6335 — `er` looks the code up and returns a full game DTO with no `ra(t, r.audience)`: ... const n = Qt.invites.find((e) => e.code === a && new Date(e.expires_at).getTime() > i); if (!n) return null; const r = hi(n.game_id); if (!r "scheduled" !== r.status) return null; ... return li(r, s, t); ... `li` (mod/631.js:2665-2677) includes `venue`, `bookings_count`, `waitlist_count`, `pending_count` and `organizer_name`: ... or(e.organizer_id)`. Every sibling read does call `ra`: mod/631.js:2701 ("Ni"), 2810 ("ji"), 8636 ("mockGetTeam"), 8733 ("mockJoinTeamByCode"). The screen renders the returned card before any join attempt: mod/2447.js:56-58 `const t = await resolveInviteCode(W, e?.id); if (!t) return ...; R(t);`.	A male-partition user who is given a female-partition invite code sees the match card — title, venue and address, start time, how many are booked, and the organizer's real name — before being told he cannot join. One shared code is enough to pull identity and location data across the partition boundary.	Add `ra(t, r.audience)` in `er` immediately after `const r = hi(n.gar(mod/631.js:6325), so the lookup fails with the standard audience instead of returning the DTO; ./join-code.tsx already handles thro this path.
P-CQUAL-39 Medium	./lineup/[gameId]	State handling	A locked lineup still renders every editing control; each tap fails with an error banner `te` is the intended gate and was applied to exactly one control. The lock badge is shown in the header (mod/2448.js:67-77) but nothing is disabled. Evidence: mod/2448.js:45 computes the correct predicate `te = !ID?.can_edit && !D.locked,` but it is used only for the captain/jersey card (mod/2448.js:160-162 `te && ae?.player_id && "slot" === H?.type &&`). Every other affordance checks only `can_edit`: mod/2448.js:135 `if (!D.can_edit) { ... }` in `onSlotPress`, mod/2448.js:248 `D.can_edit ? O(...)` for bench chips, and the whole editor block at mod/2448.js:286 `D.can_edit && jsxs(Fragment, { — formation chips, auto-balance, randomize, share, template and editor grants. The backend rejects all of them while locked: mod/631.js:16471, 16494, 16515, 16621, 16670 `if (n.locked) throw new Error("E_THE_LINEUP_IS_LOCKED");`.	After an organizer locks the line-up, captains and organizers still see a fully interactive board: every drag, formation chip and auto-balance press produces a red 'The line-up is locked' banner. Users conclude the app is broken rather than that the line-up is final, and the lock reads as advisory.	Replace `D.can_edit` with `te` at mod/2448.js:135, 248 and 286, a 'locked' hint in place of the editor block so the state is explained merely refused.
P-CQUAL-40 Medium	./lineup/[gameId]	Sport-specific correctness	Padel and tennis matches are given a football formation board with a goalkeeper Team size 1 has no entry in `suggestFormationKey`, so it falls through to `custom-1`, which the helper widens to two slots; nothing suppresses the pitch/goalkeeper model for racket sports. Evidence: mod/631.js:16318-16334 `function nc(e) { switch (e.format) { ... case "padel_2": case "tennis_singles": return 1; case "padel_4": case "tennis_doubles": return 2; ...` and mod/631.js:16355-16360 `const n = nc(i), r = suggestFormationKey(n); a = { game_id: e, a: rc(r), b: rc(r), ...` — a lineup is created for every match regardless of sport, and `rc` builds slots from the shared football formation helper (module 654), whose `slotsForShape` always begins `[id: 'gk', role: 'GK', x: 5, y: 08]]` and whose `formationSlots('custom-1')` clamps to `Math.max(2, Math.min(11, 1))`, i.e. two slots for a one-player side. The board is then rendered for every viewer of the match: mod/2379.js:413-460 `we && ... FormationBoard, { a: we.a.slots, b: we.b.slots, ...` and the join-at-position dialog labels the slot `F('pitchRole_\${de.role}')` (mod/2379.js:964), whose only defined values are GK/DF/MF/FW.	Two of the app's three sports get a football pitch: a singles tennis match shows two positions per side, one of them a goalkeeper, and the 'Take this position' dialog offers to seat a tennis player as GK. Padel and tennis organisers see the product as built for football only.	Return early from `oc` (mod/631.js:16350) for padel/tennis format lineup is created, and gate the FormationBoard in mod/2379.js:4: ./lineup route on football formats — or add racket-specific slot sha GK role.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-89 Medium	/media/[id].tsx	Metric integrity / unattributed writes	recordClipView takes no actor, performs no authorization, and rewrites the whole clips table on each call The view counter accepts a bare clip id from anyone, for any clip including clips of private uploads, with no viewer identity, no per-viewer dedupe, no rate limit and no audit trail. The dedupe lives in a useRef that is discarded on unmount, so it is not a control at all. A server-side implementation of exactly this code would still count anonymous, unbounded, unattributable views. Evidence: The facade drops the caller entirely - mod/671.js:631: e.recordClipView = (o) => t.store.mockRecordClipView(o); The backend performs no check and no dedupe - mod/631.js:13421-13425: r.mockRecordClipView = async (e) => { await ei(); const t = Qt.mediaClips.find((t) => t.id === e); t && ((t.views += 1), await Za(mt, Qt.mediaClips)); }; It is also the only media mutation with no audit entry - compare mod/631.js:13443 (media.shared), mod/631.js:13456 (media.signed_url), mod/631.js:13481 (media.copyright_reported). The only dedupe is a per-mount client Set - mod/2451.js:40: K.current.has(e.id) (K.current.add(e.id), await (0, S.recordClipView)(e.id, X())); and views drives the dashboard's ranking - mod/631.js:13509: engagement: (0, N.engagementScore)(e.views, e.shares),	View counts - and therefore engagementScore, dashViews, dashEngagement and the topClips leaderboard on /media/dashboard - can be inflated arbitrarily against any clip, including a competitor organizer's. Even with no bad actor, a user tabbing back and forth between /media and /media/[id] re-counts every clip on each visit, so the numbers organizers see are systematically inflated. Because there is no audit entry, there is no forensic record of who did it, unlike every other media mutation.	Change the facade signature at mod/671.js:631 to pass the view mockRecordClipView (mod/631.js:13421) record a (clip_id, viewe row and increment only on first sight, refusing views on clips who upload is private and the viewer is not the owner. Add a logAudit matching the other media mutations.
P-CSEC-90 Medium	/media/[id].tsx	Moderation / takedown dead end	Copyright reports are write-only: the flagged bit and the mediaReports table are never read by anything A copyright report flips a boolean nobody reads and appends a row to a table nobody queries. No screen, no admin surface and no read path in 631 filters on flagged, so the reported clip stays fully visible, viewable, shareable and signable. There is also no authorization (a user can report their own clip, or any clip), and no dedupe or rate limit, so the same clip can be reported unboundedly. Evidence: mod/631.js:13463-13483, mockReportClipCopyright has no authorization and no dedupe: r.mockReportClipCopyright = async (e, t, a) => { await ei(); const i = Qt.mediaClips.find((e) => e.id === t); if (!i) throw new Error("not_found"); ... Qt.mediaReports.push(n); const r = Qt.mediaUploads.find((e) => e.id === i.upload_id); return (r && ((r.flagged = !0), await Nd()), ...); "flagged" occurs exactly three times in 17k lines: the initialiser at mod/631.js:13392 ("flagged: !1,"), the setter at mod/631.js:13479, and nothing else. `Qt.mediaReports` occurs only at mod/631.js:693 (init), mod/631.js:2060 (hydrate) and mod/631.js:13476/13480 (push/persist) - it is never queried, filtered or listed. The screen confirms success regardless - mod/2451.js:306-308: (await (0, S.reportClipCopyright)(c.id, e.id, G "Copyright"), J(""), ee(D("reportSubmitted")));	A rights-holder reports infringing footage, sees 'Report submitted', and nothing whatsoever happens - the clip remains live and shareable indefinitely. Combined with the total absence of any delete path for media, the product has no mechanism to honour a takedown at all, which is a direct legal exposure for a service that ingests match footage of identifiable people. The unbounded report table is also a free write amplifier: each report rewrites the whole mediaReports array and the whole mediaUploads array.	Make flagged load-bearing: filter flagged uploads out of mockGetMediaUpload (mod/631.js:13412), mockShareClip (mod/631.js:13426) and mockGetClipSignedUrl (mod/631.js:1344 review, and expose the mediaReports queue on the existing /admin surface. Add a one-report-per-(reporter, clip) constraint in mockReportClipCopyright (mod/631.js:13463).

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CTEST-12 Medium	/media/index.tsx	Error handling / misleading empty state	/media reports a failed library load as 'you have no videos' The screen cannot distinguish 'the library is empty' from 'the library failed to load'. A hydration failure, an E_STORAGE_FULL, or an audience-partition error all produce the reassuring 'Upload your first video' empty state. Neither screen offers a retry, so the user's only recovery is a full page reload. Evidence: mod/2453.js:11-18 collapses every failure into an empty array: <pre>_ = (0, s.useCallback)(async () => { if (t) try { T(await (0, l.fetchMediaLibrary)(t.id)); } catch { T([]); } }, [t]), and mod/2453.js:72-73 renders that as a success state: : 0 === S.length ? (0, B.jsx)(m.EmptyState, { icon: "film-outline", title: P("noVideos"), body: P("uploadFirst") }) The sibling dashboard has the mirror-image defect - mod/2452.js:15-18 maps every error to a single boolean: D(await (0, C.fetchMediaDashboard)(t.id)); } catch { A(!0); } which renders an 'errForbidden' lock panel (mod/2452.js:49-53) for storage errors and network errors alike, with no retry control.</pre>	A user whose store failed to hydrate is told, with a friendly illustration, that the videos they uploaded do not exist - and is pushed toward uploading again, which will either duplicate their work or fail for the same underlying reason. On /media/dashboard the same class of transient error is reported as a permanent permissions problem ('errForbidden'), sending the user to support for an access issue that does not exist.	Track an error state separately from the empty state in mod/2453 S as null on failure, set an error message, and render a retry butt mod/2452.js:15, distinguish the organizer-approval error code (E_ORGANIZER_APPROVAL_REQUIRED_YOUR_ORGANIZER from all other failures and show a retry for the latter.
P-CSEC-91 Medium	/messages/[id].tsx	Harassment / abuse	A stranger can push unlimited messages into an unaccepted request thread; the backend has no rate limit on any social action A message request should carry at most one message (or a small handful) until the recipient accepts, and the social write paths should be rate limited per sender. Neither constraint exists. Evidence: mod/631.js:14709-14721 — `i.accepted    i.initiator_id === e    ((i.accepted = !0), await logAudit("dm.request_accepted", ...)); const r = { id: ea(), conversation_id: t, sender_id: e, body: n, read: !1, created_at: ... }; return (Qt.dmMessages.push(r), ...)` — when the sender is the initiator of an unaccepted conversation both short-circuit operands are satisfied and the message is appended anyway; there is no cap on how many messages a pending request may carry and no time window. The content reaches the recipient's list without them opening anything: mod/631.js:14618 `last_body: n && !n.deleted_at ? n.body : ""` is rendered at mod/2456.js:185 `children: t.pending_sent ? l("requestSentLabel") : t.last_deleted ? l("messageDeleted") : t.last_body`. The only rate limit in the whole 17k-line backend is on club invites — mod/631.js:13202-13206 `if (Qt.clubInvites.filter(...).length >= l.DEFAULT_QUOTA.free.invitesPerDay) throw new Error("rate_limited");` nothing comparable guards `mockSendDirectMessage`, `mockSendFriendRequest`, `mockFollow`, `mockCreatePost`, `mockAddComment` or `mockReportPost`.	Anyone whose target has the default `allow_messages: "everyone"` can open a request thread and then deliver an unbounded stream of abusive text into it. The recipient sees the latest line as the preview on /messages before they have accepted anything, and the whole backlog the moment they open the thread. Declining is their only defence and, per the decline finding, it is irreversible. Combined with the absent rate limit, the same is true of friend requests, follows, posts and comments.	In `mockSendDirectMessage`, reject with `rate_limited` when `!i.i.initiator_id === e && w(t).filter(m => m.sender_id === e).length` shared per-actor sliding-window counter in 631 and apply it to `mockSendDirectMessage`, `mockSendFriendRequest`, `mockFollow`, `mockCreatePost`, `mockAddComment` and `mockReportPost`; `rate_limited` is already mapped in mod/674.js:52.
P-CTEST-13 Medium	/messages/[id].tsx	Invalid route parameter	/messages/<unknown-id> renders a believable empty conversation with a working composer instead of a not-found state An unknown, malformed or non-participating conversation id should produce a clear 'conversation not found' state with a way back. Instead the error is discarded and the screen presents a functioning-looking chat with an unnamed peer. Evidence: mod/2455.js:21-29 — `X = useCallback(async () => { if (v && e) { try { const { conversation: t, messages: s } = await fetchThread(v.id, e); (M(t), W(s)); } catch {} (E(!1), requestAnimationFrame(...)); }, [v, e])` — the `catch {}` swallows the 'forbidden' thrown by mod/631.js:14680 `if (!a    !m!(a, e)) throw new Error("forbidden")`, then clears the loading flag regardless. With `q` still `null`, mod/2455.js:78 computes `se = !!q && (...)` as `false`, so the composer at mod/2455.js:371 `!se && !jsx(Div, { style: [S.composer, ...] })` is rendered; the header shows an empty name and a `?` avatar (mod/2455.js:101 `(q?.other_name ?? "?").slice(0, 1).toUpperCase()`, mod/2455.js:107 `children: q?.other_name ?? ""`); and the list shows the ordinary `noConversations` empty state (mod/2455.js:155-159).	Following a stale or mistyped /messages link — from a bookmark, a shared URL, or a conversation deleted on another device — gives the user a blank thread with an active send button and a nameless avatar. They type a message, tap send, and only then get an error (mod/2455.js:46), with no explanation of whose conversation this was supposed to be. The `catch {}` also hides genuine transport failures behind the same misleading empty state.	Replace `catch {}` with a `notFound` state, and render `GateScreen.error`, title: t("noConversations"), onBack })` from 9001 when the or `q` is still null after loading; keep the composer mounted only v non-null, i.e. change the guard at mod/2455.js:371 to `!!q && !se`

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-39 Medium	/messages/index.tsx	Performance	The messaging data path does full scans of the global message table per conversation and per render Conversation listing and thread loading should be indexed and paginated; the read-receipt anchor should be memoised on the message array, not recomputed per render. None of that is in place, and there is no `onEndReached`, cursor or limit anywhere in the messaging path. Evidence: mod/631.js:14605-14608 — `wl = (e) => Qt.dmMessages.filter((t) => t.conversation_id === e).sort((e, t) => new Date(e.created_at).getTime() - new Date(t.created_at).getTime())` scans and sorts the entire global message array for one conversation. `mockGetConversations` calls it once per conversation, twice: mod/631.js:14666 `.filter((t) => ml(t, e) && wl(t.id).length > 0 && !Xd(...))` and then `pl` at mod/631.js:14611 `i = wl(e.id)` — so listing N conversations is O(N × total messages) with 2N sorts. `mockGetThread` walks every message in the database to mark reads: mod/631.js:14683 `for (const a of Qt.dmMessages) a.conversation_id !== t    a.sender_id === e    a.read    ((a.read = !0), (a.read_at = n), (i = !0));` and then returns the whole thread with no limit or cursor. On the client, the conversation list is rendered in a plain ScrollView with no virtualisation (mod/2456.js:58 `jsx(ScrollView, { contentContainerStyle: {...}, children: ... D.map(...) } )`), and the thread screen recomputes the read-receipt anchor by copying and reversing the entire message array on every render, including every keystroke in the composer: mod/2455.js:77 `re = q?.accepted ? ([...P].reverse().find((e) => e.is_self && e.read && !e.deleted)? : id ?? null) : null`.	A user with a few hundred conversations and a few thousand messages pays a quadratic cost every time /messages gains focus, and the list is built as one unvirtualised ScrollView. In a long thread, every character typed into the composer allocates and reverses a copy of the full message array, so the composer visibly lags. Nothing here degrades gracefully — the cost grows with total account history, not with what is on screen.	Build a `conversation_id -> messages` index in `Qt` (or memoise `mockGetConversations` does one pass instead of 2N; have `pl` already-computed last message; give `mockGetThread` a `limit` / cursor and mark reads through the same index rather than scanning `Qt.dmMessages`. On the client, swap 2456's ScrollView for a FlatList, and wrap the 2455 read-receipt computation in `useMemo(..., [P, q]? : a`
P-CARCH-40 Medium	/messages/index.tsx	N+1	The DM list and unread badge are both N+1 over the global message table, and the conversation list scans it twice per row No `Map<conversation_id, messages[]>` is ever built, and the same per-conversation message scan is duplicated between the filter predicate and the projection. Evidence: mod/631.js:14605-14609 — `wl = (e) => Qt.dmMessages.filter((t) => t.conversation_id === e).sort(...)`; mod/631.js:14665-14668 — `Qt.dmConversations.filter((t) => ml(t, e) && wl(t.id).length > 0 && ...)`.map((t) => pl(t, e));` `pl` calls `wl(e.id)` again (mod/631.js:14612-14615), so each conversation scans and sorts the entire message table twice. mod/631.js:14670-14676 — `mockGetUnreadDMCount` does `Qt.dmMessages.filter((t) => { const a = Qt.dmConversations.find((e) => e.id === t.conversation_id); ... })` — a find over all conversations inside a filter over all messages.	With 100 conversations and 10,000 messages, opening `/messages` performs 200 full scans plus 200 sorts of the message table — roughly 2 M comparisons and 200 O(n log n) sorts — before a single row paints. The unread badge is O(messages × conversations) and is fetched independently. Both grow with total message volume in the store, not with the user's own volume.	Build `const byConv = new Map()` from a single pass over `Qt.dmConversations` at the top of `mockGetConversations` (mod/631.js:14662) and parallelise the unread badge calculation. Rewrite `mockGetUnreadDMCount` to build a `Set` of the viewer's conversation ids once and then make one pass over the message table.
P-CTEST-14 Medium	/my-bookings.tsx	Error handling	My Bookings has no error path — a failed load is rendered as "you have no bookings" A rejected `fetchMyBookings` produces an unhandled promise rejection, leaves `z` at its initial `[]`, and clears the spinner — which the UI reads as "no data" rather than "load failed". There is no retry and no way to distinguish the two. Evidence: mod/2457.js:16 `try { const [t, s] = await Promise.all([(0, T.fetchMyCourtBookings)(e.id), (0, T.fetchMyCourtBookings)(e.id).catch(() => [])]); (W(t), D(s)); } finally { H(!1); }` — a `try/finally` with no `catch`. The secondary call is defended with `.catch(() => [])`; the primary one is not. The screen carries no error state at all (no `useState` for an error, no retry control) and falls through to mod/2457.js:63 `ListEmptyComponent: ... (0, L.jsx)(k.EmptyState, { icon: "bookmark-outline", title: S("noBookingsTitle"), body: S("noBookingsBody") } )`. Every sibling screen in this group does handle it — mod/2479.js:24, mod/2481.js:29, mod/2483.js:19, mod/2506.js:29 all set an error and render a retry.	On a slow or failing network a player who paid for a game opens My Bookings and is told they have no bookings. The natural reaction is to book and pay again. The console also carries an unhandled rejection, which the project's own smoke harness treats as a failure condition.	Add a `catch` in mod/2457.js:22 that stores `storeErrorText(e?.message)` and render the same EmptyState + Retry pair the other screens use (mod/2479.js:76-94).

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CTEST-15 Medium	/notifications.tsx	Dead code / unshipped feature	Two notification types are fully wired in the UI but never produced; a code-of-conduct version bump notifies nobody Bumping `CURRENT_COC_VERSION` flips `accepted` to false for the entire user base with no notification, no interstitial and no gate on any action — the sole consequence is that a button reappears on a screen the user has no reason to revisit. The notification type built to announce it is never emitted, and `team_invite` is likewise a permanently dark branch. Evidence: `coc_update` is handled end to end on the client — routing at mod/2458.js:64-65 `: "coc_update" === n.type ? B.push("/conduct")`, copy at mod/2458.js:403 `if ("coc_update" === t.type) return { title: n("notifCocUpdateTitle"), body: n("notifCocUpdateBody") }`; , icon at mod/2458.js:515 — but `grep -rn ""coc_update""` across the whole module dump matches only mod/2458.js. `team_invite` is the same: mod/2458.js:57, 388, 507, and nowhere else. Meanwhile the acceptance check is purely version-comparative: mod/631.js:12403-12409 `r.mockGetCoCStatus = async (e) => { await ei(); const t = Qt.cocAcceptances.filter((t) => t.user_id === e).sort((e, t) => t.version - e.version)[0]; return { version: D.CURRENT_COC_VERSION, accepted: !!t && t.version >= D.CURRENT_COC_VERSION, accepted_version: t?.version ?? null }; }` with `CURRENT_COC_VERSION = 1` (bundle-src/656.js:15).	The first CoC revision will go unaccepted by almost everyone, because the only signal is a button on /conduct. The platform then cannot claim users agreed to the current rules — which is the entire point of versioning acceptance — while carrying shipped, translated, tested-looking code that implies the announcement exists. The regression suites cannot cover either branch because nothing can generate the input.	Emit `coc_update` to all users when `CURRENT_COC_VERSION` is bumped (or delete the branch at mod/2458.js:64/403/515), gate match join current acceptance the way `wd` gates on standing (mod/631.js:12403-12409) either implement the `team_invite` producer or remove its three c branches.
P-CARCH-41 Medium	/notifications.tsx	Unbounded growth	Notifications are returned and rendered in full with no limit, cursor or pruning The notification store is append-only and read whole. Cost per write and per page load grows linearly with a user's lifetime notification count, and the persisted row grows without bound against a hard localStorage quota. Evidence: mod/631.js:2926-2931 `r.mockGetNotifications = async (e) => (await ei(), Qt.notifications.filter((t) => t.user_id === e).sort((e, t) => new Date(t.created_at).getTime() - new Date(e.created_at).getTime()));` — no limit, no offset, no cursor. mod/671.js:406 `e.fetchNotifications = (o) => t.store.mockGetNotifications(o);` offers no pagination argument. The screen loads the lot in one shot (mod/2458.js:13-20 `z(await (0, T.fetchNotifications)(t.id))`), regroups all of it on every change (mod/2458.js:82-97 `\$\$.forEach((t) => o[n(t.created_at)].push(t))`) and feeds it to a single SectionList (mod/2458.js:131-134). Nothing ever deletes: the only removals in mod/631.js are account deletion at 2431 and the de-dupe guard at 2582; there is no `mockDeleteNotification` and no retention sweep. Every write re-serialises the entire array — mod/631.js:2589 `Qt.notifications = [e, ...Qt.notifications]), await Za(\$, Qt.notifications)`.	An active player accumulates match reminders, join requests, wallet activity and squad notices indefinitely. Page load eventually parses and groups thousands of records before first paint, and each new notification rewrites the whole blob — until `Za` throws `E_STORAGE_FULL` (mod/631.js:1769-1772), at which point the failure propagates into whatever action triggered the notification, and unread notifications become untappable as described in the read-marking finding. The user has no delete, no archive and no filter.	Add `limit`/`cursor` parameters to `mockGetNotifications` (mod/631.js:2926-2931) and the facade, page the SectionList in mod/2458.js:131 with `onEndReached`, and add a retention sweep that drops read notifications past a cutoff.
P-CARCH-42 Medium	/notifications.tsx	Unbounded growth	Notifications, stories, story views, chat and DM tables grow without bound and are read without pagination Only `feedSignals` (mod/631.js:15620) and the audit log (mod/643.js:20-22, caps of 500/5000) have retention limits. Every other append-only table grows forever, and because writes are whole-array (see the storage finding), each append re-serialises the full history. Evidence: mod/631.js:2589 — `Qt.notifications = [e, ...Qt.notifications)` on every insert; grepping `Qt.notifications` across mod/631.js shows the only removal is account deletion (mod/631.js:2431 `Qt.notifications.filter((e) => e.user_id !== t)`). mod/631.js:2926-2931 — `mockGetNotifications` returns every row for the user with no limit. mod/631.js:14827-14828 — `Qt.stories.unshift(n)` with expiry enforced only by a read-time filter `hl = () => Qt.stories.filter((e) => new Date(e.expires_at).getTime() > Date.now())` (mod/631.js:14817); expired rows are never deleted. mod/631.js:14914 — `Qt.storyViews.push(...)` with no pruning. mod/631.js:2874 `Qt.chatMessages.push(t)` and mod/631.js:2855-2861 `mockGetChatMessages` returns the whole channel history. mod/631.js:12737-12745 `Qt.demandLog.push(...)` and mod/631.js:12965-12974 `Qt.optimizerLog.push(...)` have no cap, unlike `Qt.feedSignals` which is trimmed to 2000 at mod/631.js:15620.	An active user accumulates thousands of notifications; `mockGetNotifications` returns all of them and the badge count at mod/631.js:2932 re-scans the whole table. Expired stories are never reclaimed, so `hl()` — called once plus once per story inside `mockGetStoryTray` (mod/631.js:14875-14898) — filters an ever-growing array on every feed load. The per-write serialisation cost grows linearly with account age until the quota ceiling ends it.	Cap `Qt.notifications` at a few hundred per user inside `bi` (mod/631.js:2589) the way `feedSignals` is capped; delete expired rows in `hl` (mod/631.js:14817) rather than filtering them, and cascade-delete `storyViews`; add `{limit, before}` to `mockGetNotifications`, `mockGetChatMessages` (mod/631.js:2855), `mockGetThread` (mod/631.js:14677) and `mockGetTeamChat` (mod/631.js:9055); `demandLog` and `optimizerLog`.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-41 Medium	/passport/[id].tsx	Error handling / dead end	/passport/[id] has no error path and fabricates a passport for ids that do not exist The screen has exactly two states, loading and loaded, and the backend never distinguishes "no such user" from "a user with no activity". The shared `GateScreen` with retry/back (mod/9001.js:72-123) is not imported (dependency list at the end of mod/2473.js contains no 9001). Evidence: mod/2473.js:16-18 `Y = (0, a.useCallback)(async () => { t && (V(await (0, k.fetchPassport)(t, o?.id)), \$(await (0, k.fetchPlayerAwards)(t)), O(!1)); }, [t, o?.id]);` — no try/catch, and `O(!1)` (clear loading) is sequenced after both awaits, so a rejection leaves the screen on mod/2473.js:30-34 `if (E    !H) return ... ActivityIndicator` forever, with no header and no back control. For a nonexistent id there is no rejection either, because `dr` never checks existence: mod/631.js:6438-6441 `ar = (e) => { const t = Qt.profiles.find((t) => t.id === e), a = t?.created_at ?? new Date().toISOString(),` and mod/631.js:6548-6551 `or = (e) => Qt.profiles.find((t) => t.id === e)?.full_name ?? Qt.bookings.find((t) => t.user_id === e)?.display_name ?? "Player",`.	/passport/<garbage-uuid> renders a complete, plausible passport — display name "Player", member-since today, 0 matches, a computed attendance class and reputation class — instead of a not-found state, which makes id-probing indistinguishable from real accounts. And any genuine failure (transport error, a future authorization throw) strands the user on an ActivityIndicator with no in-app exit.	Throw `not_found` from `dr` (mod/631.js:6612) when `Qt.profiles` for the subject, and wrap mod/2473.js:17 in try/catch that clears it `finally` and renders `GateScreen` from mod/9001.js with retry and back.
P-CQUAL-42 Medium	/pay/[id].tsx	Payment integrity / idempotency	The payment idempotency key identifies a payer-and-booking pair, not a payment, so a second request for the same pair settles without a charge An idempotency key should identify the charge attempt, i.e. include the payment id and amount. As written, any two payment rows sharing (game, payer, booking) are treated as the same charge, whatever their amounts. The only thing preventing a re-split today is a UI condition — mod/1838.js:258 `0 === L.length ? ...` hides the split controls once any payment row exists — not a backend rule. Evidence: mod/631.js:7655 `Ur = (e) => `seat:\$(e.game_id ?? "court");\$(e.payer_id):\$(e.booking_id ?? e.id)` — the payment's own id is used only as a fallback. `\$r` short-circuits the entire capture block on a hit: mod/631.js:7833 `if (r && "captured" === r.status) i.gateway_ref = r.gateway_ref;` and mod/631.js:7835 `if (!r "failed" === r.status) { ...capture... }`, then unconditionally marks the row paid at mod/631.js:7860 `(i.status = "paid"), (i.method = a), (i.paid_at = new Date().toISOString(), ...)` . `mockCreatePaymentPlan` can mint a second row for the same pair because it clears only pending ones: mod/631.js:7583 `(Qt.payments = Qt.payments.filter((e) => !(e.booking_id === n.id && "pending" === e.status)))`, then pushes a fresh payment per payer with the same `booking_id: n.id` and `game_id: n.game_id` (mod/631.js:7589-7590).	If a payment plan is ever re-issued for a booking (a supported backend operation, exposed through the facade at mod/671.js:535), the already-paid participant is billed again and, when they pay, `\$r` reuses the old captured charge: the row is marked `paid` and the venue collects nothing. A raised amount is likewise settled for free. The safety of this depends entirely on one render-time condition in a screen.	Include the payment id and amount in `Ur` (mod/631.js:7655) so that each payment row gets its own idempotency key, and add a backend guard `mockCreatePaymentPlan` (mod/631.js:7563) that refuses to re-issue when the booking already has non-pending payments, or reconciles them.
P-CQUAL-43 Medium	/player/[id].tsx	Error handling	/player/[id] reports every failure — not-found, audience mismatch, network — as "This account is private" Three materially different conditions collapse into one message. There is also no retry: once `O` is set the only exit is Back, and the callback never re-runs unless the screen is refocused. Evidence: mod/2475.js:20-28 `K = (0, t.useCallback)(async () => { if (!i && e) try { const t = await (0, _fetchPlayerProfile)(i.id, e); (M(t), E(!1), H(Limited ? [] : await (0, _fetchHighlightCollections)(e))); } catch { E(!0); } }, [i, e]);` — a bare catch setting the single `O` flag, whose render branch is mod/2475.js:53-88, a lock icon over `A("privateAccount")` / `A("privateAccountBody")` with a back button and no retry. The backend distinguishes the causes — mod/631.js:14274 `if (!a) throw new Error("not_found");` and mod/631.js:14275 `i    oa(e, t);` which throws `AUDIENCE_MISMATCH` (mod/631.js:748 `ta = "AUDIENCE_MISMATCH"`) — and `AUDIENCE_MISMATCH` has no entry in the error map (mod/674.js has `AUDIENCE_IMMUTABLE` at line 53 but no `AUDIENCE_MISMATCH`), so `localizeStoreError` passes the raw code through.	A user who follows a stale or mistyped link is told the account is private rather than that it does not exist. A user whose network blips while opening a profile is told the account is private and has to navigate away and back to try again. Support cannot distinguish these reports from genuine privacy behaviour, and the real private-account case (`limited` with `limited_reason`, mod/631.js:14314-14315) is already handled separately at mod/2475.js:278-337 — so the error branch is exclusively wrong-message territory.	Use `classifyError` from mod/9001.js:59 in the catch at mod/2475.js:53 to distinguish the error, render `GateScreen` with `kind: "error"` plus `onRetry: K` for any further attempt, not a genuine privacy outcome. Add `AUDIENCE_MISMATCH` to the error map at mod/674.js with its own locale key in both `en` and `ar`.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-92 Medium	/player/[id].tsx	Privacy control bypass	profile_visibility is enforced on the profile read but not on the posts, stories or highlights behind it `profile_visibility` is checked once, on the profile envelope, and then the endpoints that return the actual content behind that envelope re-fetch it without re-checking. For highlights the check exists only in the React component. Evidence: The gate exists and is applied correctly in two places: mod/631.js:14281 `s = al(t).profile_visibility,` / mod/631.js:14282 `d = n    (!h && ("private" === s    ("followers" === s && !o && !r))),` (mockGetPlayerProfile), and mod/631.js:14121 `if (!("public" === al(e).profile_visibility    Vd(t, e)    Qd(t).has(e)))    el(e, t)) throw new Error("followers_only");` (mockGetFollowList). It is absent from the content readers: mod/631.js:14488 `r.mockGetUserPosts = async (e, t) => {` / mod/631.js:14489 `await el(),` / mod/631.js:14490 `e !== t && oa(e, t),` — `oa` is only the audience partition; mod/631.js:14905 `r.mockGetAuthorStories = async (e, t) => {` / `if ((await el(), t !== e && oa(e, t), t !== e && Xd(e, t))) throw new Error("blocked");` — blocks but no visibility test; and mod/631.js:14587 `r.mockGetHighlightCollections = async (e) => {` / mod/631.js:14589 `const t = Qt.mediaClips.filter((t) => t.owner_id === e),` — no viewer argument at all. The only thing stopping the last one today is a client-side conditional: mod/2475.js:24 `(M(t), E(!1), H(t.limited ? [] : await (0, _fetchHighlightCollections)(e)))`.	A user who sets their profile to `private` or `followers` still has their posts, their stories and their highlight-clip collections readable by any stranger who knows their user id — including the default for female-audience accounts, which is `profile_visibility: "followers"` (mod/631.js:562 `_t = { profile_visibility: "followers", show_online: !1, allow_messages: "followers" }`). The privacy toggle therefore promises more than the backend delivers.	Extract the visibility predicate used at mod/631.js:14282 into a sh and call it at the top of `mockGetUserPosts` (mod/631.js:14488) and `mockGetAuthorStories` (mod/631.js:14905). Add a viewer param to `mockGetHighlightCollections` (mod/631.js:14587) and apply the helper there instead of relying on mod/2475.js:24.
P-CQUAL-44 Medium	/preferences.tsx	Data integrity / silent no-op	/preferences cannot clear any field: deselecting everything saves nothing and reports success Empty and absent are conflated. The guards turn a deliberate "none" into "do not send this key", so a cleared value is never persisted, while the save path still navigates back as though it succeeded. Evidence: mod/2476.js:110-118 `await (0, j.updateProfile)(e.id, Object.assign({}, z.length ? { preferred_sports: z } : {}, A ? { skill_level: A } : {}, D ? { home_area: D } : {}), await T(), M.back());`. The UI explicitly supports deselection: mod/2476.js:74 `onPress: () => L((t) => (t.includes(e) ? t.filter((t) => t !== e) : [...t, e])),` and mod/2476.js:90 `onPress: () => W((t) => (t === e ? null : e))`. On return the focus effect re-seeds from the unchanged profile: mod/2476.js:16-19 `o && (L(o.preferred_sports ?? []), W(o.skill_level && "all" !== o.skill_level ? o.skill_level : null), E(o.home_area ?? null));`.	A user who deselects all sports, or clears their skill level, taps Save, sees the screen dismiss, returns to /preferences and finds the old values back. There is no error and no indication the change was discarded. These fields drive matchmaking and recommendations (mod/631.js:14152, 14202), so the user is stuck with stale preferences they believe they removed.	Send the keys unconditionally in mod/2476.js:112-117 `( { preferred_sports: A ?? "all", home_area: D } )` and let the backend distinguish between undefined, or add explicit "clear" semantics. The same fix is the sports/level chips in the profile-tab editor at mod/1805.js:551- does send unconditionally and therefore behaves differently from for the same data.
P-CARCH-43 Medium	/privacy-controls.tsx	Error handling / dead end	/privacy-controls maps its load error onto the loading sentinel, turning every failure into an eternal spinner The catch handler explicitly writes `null`, which is the exact value the render gate treats as 'still loading'. There is no separate error state and no retry path. Evidence: mod/2477.js:12-19 — `useFocusEffect(useCallback(() => { t && fetchPrivacy(t.id).then(R).catch(() => R(null)); }, [t])`. mod/2477.js:30-34 — `if (!P) return <SafeAreaView style={{flex:1, ..., alignItems:"center", justifyContent:"center"}}><ActivityIndicator/></SafeAreaView>;` where `P`/`R` are the same state pair.	`mockGetPrivacy` throws `E_PROFILE_NOT_FOUND` (mod/631.js:989) whenever the profile row is missing — which is the state of a guest session and of any account whose profile write failed. Those users see a spinner forever on a privacy screen, with no header and no way back.	Store the error separately (`catch (err) => setError(err)`) and render a `GateScreen` from mod/9001.js:72 with `kind: "error"` and `onRender` prop. The same one-line fix applies to `/passport/[id]` (mod/2473.js:16-22) and to the profile screen (mod/2473.js:30-34 renders the same bare spinner) and to the teams/league screen (mod/2500.js:12-29).
P-CQUAL-45 Medium	/privacy.tsx	Silent failure / state divergence	Privacy and consent writes have no error handling on any screen in the group, and 631 mutates memory before persisting Both failure codes are already mapped to localised strings (mod/674.js:276-277 `E_COULD_NOT_SAVE_CHANGES: "seCouldNotSaveChanges", E_STORAGE_FULL: "seStorageFull"`), so the messages exist and are simply never surfaced. Because memory is mutated first, a failed write leaves the in-memory store and the persisted store disagreeing for the rest of the session. Evidence: mod/2478.js:19-21 `U = async () => { t && (W((t) => (t ? Object.assign({}, t, l) : t)), W(await (0, B.setPrivacySettings)(t.id, l))); };` — no catch. mod/2478.js:16 `t && ((0, B.fetchPrivacySettings)(t.id).then(W), (0, B.fetchBlockedList)(t.id).then((t) => J(t.length)));` — no catch either. mod/2477.js:20-29 has only `try { R(await (0, C.updatePrivacy)(t.id, a)); } finally { W(!1); }`. mod/2473.js:24-26 `q = async (t, a) => { o && H && (await (0, k.updatePassportPrivacy)(o.id, { [t]: a })), Y()); };` — no catch. In the backend the in-memory row is updated before the persist that can throw: mod/631.js:14807-14809 `n >= 0 ? (Qt.privacySettings[n] = i) : Qt.privacySettings.push(i), await Za(qe, Qt.privacySettings),` and mod/631.js:1002-1005 does the same for `mockUpdatePrivacy`. `Za` throws on quota: mod/631.js:1771-1774 `throw new Error([quota]QuotaExceeded[NS_ERROR_DOM_QUOTA_EXCEEDED].test(a) ? "E_STORAGE_FULL" : "E_COULD_NOT_SAVE_CHANGES");`.	When storage is full or the write fails, the toggle animates to its new position, the in-memory backend agrees, and nothing tells the user. On the next reload the setting silently reverts to the old value. For `profile_visibility` and `allow_messages` that means a user believes they are locked down and is not, with no error to lead them back. On /privacy a failed `fetchPrivacySettings` additionally makes the entire Safety card — visibility, messages, online status, blocked accounts — vanish from the page (the `L &&` guard at mod/2478.js:52) with no explanation.	Add catch blocks around each write (mod/2478.js:20, mod/2478.js:24, mod/2477.js:24, mod/2473.js:25) that revert the optimistic state and a `storeErrorText(e.message)`, the pattern already used at mod/2473.js:122. In 631, move the in-memory assignment after a successful `mockSetPrivacySettings` (mod/631.js:14807) and `mockUpdatePrivacy` (mod/631.js:1002).

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-44 Medium	/privacy.tsx	Concurrency / lost update	mockSetPrivacySettings is an unguarded read-modify-write, so two quick toggles on /privacy lose one of the changes Last write wins over the whole row, and the snapshot is stale across the `await ei()` boundary. The final `W(i)` on the screen paints whichever response resolves last, so the UI can end up agreeing with the losing write. Evidence: mod/631.js:14802-14813 `r.mockSetPrivacySettings = async (e, t) => { await ei(); const a = al(e), i = Object.assign({}, a, t, { user_id: e }); n = Qt.privacySettings.findIndex((t) => t.user_id === e); return (n >= 0 ? (Qt.privacySettings[n] = i) : Qt.privacySettings.push(i), await Za(qe, Qt.privacySettings), ...) — the snapshot `a = al(e)` is taken after an `await`, the whole row is then replaced, and there is no version or compare-and-set. The screen fires a write on every tap with no in-flight guard: mod/2478.js:19-21 `U = async () => { t && (W(...), W(await (0, B.setPrivacySettings)(t.id, l))); },` used by three independent controls (mod/2478.js:74 `onChange: (t) => U({ profile_visibility: t })`, mod/2478.js:89 `U({ allow_messages: t })`, mod/2478.js:111 `U({ show_online: t })`). A second writer exists on another screen: mod/1805.js:566-568 `await (0, l.setPrivacySettings)(e.id, { profile_visibility: ye ? "followers" : "public" })`. Contrast mod/2477.js:21 `if (t && !B)` which does guard.	A user who sets visibility to "private" and immediately switches messages to "none" can end up with only one of the two applied — and the segment controls may still show both as selected, because the screen renders the optimistic merge and then the response of whichever call returned last. The failure is silent and the setting that was lost is a privacy restriction, so the user is left more exposed than the UI shows. The same race occurs between /privacy and the profile-tab editor, which both write `profile_visibility`.	Apply the patch as a field-level merge onto the current row read synchronously at write time (or add an `updated_at`/version check to reject a stale write) in mod/631.js:14802. On the screen, add the guard already used at mod/2477.js:21 and re-read after each write.
P-CARCH-45 Medium	/refunds.tsx	Payment lifecycle / dead end	Court-share payment requests expire 30 minutes after they are created, with no way for the payer to recover A share of a court fee for a match days away is given the lifetime of a checkout hold. When it lapses there is no self-service re-request — only the organizer can regenerate a plan, and mod/1838.js:258 hides that control once any payment row exists (`0 === L.length ? ...`). Evidence: mod/631.js:7599 `reserved_until: t <= 0 ? null : new Date(Date.now() + 18e5).toISOString(),` — 1,800,000 ms = 30 minutes, regardless of when the game starts (contrast the seat path, mod/631.js:7665-7666, which scales the hold to the time until kickoff). The sweeper then kills it: `Zr` (mod/631.js:8262) flips "pending" rows whose `reserved_until` has passed to "expired", and it runs on every /refunds load via mod/631.js:7639 `await Zr()`. The screen drops expired rows from "Still to pay": mod/2479.js:37 `E.filter((t) => "pending" === t.status && (!t.reserved_until    Date.parse(t.reserved_until) > e))`. Opening the request directly then offers nothing: mod/2474.js:118 `:"pending" !== R.status ? (0, _jsx)(x.Card, { ... children: (0, _jsx)(f.Badge, { label: H(`pstatus_\${R.status}`), tone: "neutral" }) })` — a badge, no action, no link to the game.	A player who gets the payment-request notification and comes back an hour later finds the item gone from /refunds; if they tap the notification's deep link to /pay/<id> they see a grey "expired" badge and a back button. Their share is uncollectable, the organizer cannot re-issue it, and the venue is short.	Scale `reserved_until` in `mockCreatePaymentPlan` (mod/631.js:7665) to match the booking's `starts_at`, as `Hr` does at mod/631.js:7665; and give a recovery action on expired/cancelled states — at minimum a link to the game or booking and a "request again" path.
P-CQUAL-46 Medium	/rewards/index.tsx	Catalog correctness	Every reward whose category is not partner or reward is permanently invisible, including three seeded ones The category allow-list and the writer disagree. Either the two categories are unimplemented (in which case seeding and admin creation should refuse them) or the filter is wrong (in which case the seeds should show). Evidence: mod/631.js:11429 `const xs = new Set(["partner", "reward"]);` filters the catalog at mod/631.js:11580 `.filter((e) => e.active && xs.has(e.category))` and blocks redemption at mod/631.js:11592 `if (!a    !a.active    !xs.has(a.category)) throw new Error("E_REWARD_NOT_AVAILABLE");`. Three of the six seeded rewards are outside that set: mod/631.js:11457 `10% off a court booking`, ..., "discount", 200, mod/631.js:11465 `e("Free court hour", ..., "discount", 600, ...)` , mod/631.js:11466 `e("1 month Premium", ..., "premium", 800, ...)` . Admins can keep creating more of them — `mockUpsertReward` stores `category: t.category` unvalidated at mod/631.js:11679.	Half the shipped reward catalogue never appears on /rewards, silently. An admin who adds a `discount` reward sees it saved with no error and it is never visible to a single user — a failure mode with no signal anywhere in the product.	Validate `t.category` against `xs` in `mockUpsertReward` (mod/631.js:11465) and either implement fulfillment for `discount`/`premium` or remove those three entries from the seed block at mod/631.js:11429.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-47 Medium	/rewards/index.tsx	Rewards integrity	Loyalty points survive cancellation and refund of the transaction that earned them Earning is derived from a live scan but recorded as an immutable append-only row keyed on `(user, action, ref_id)`. When the underlying booking is later cancelled — `Mr` sets `status = "cancelled"`, which removes it from the filter at 11531 — the already-written row is neither removed nor offset, and the dedupe guarantees it will never be re-evaluated. Evidence: mod/631.js:11530-11533 — points are granted from current booking state: `for (const t of Qt.courtBookings.filter((t) => t.organizer_id === e && ("confirmed" === t.status    "released" === t.status))) await Ws(e, "booking", t.id, t.created_at);` and mod/631.js:11525-11527 for match play. `Ws` writes once and never withdraws (mod/631.js:11504-11517): `if (Qt.loyaltyLedger.some(i) => i.user_id === e && i.action === t && i.ref_id === a)) return 0; ... Qt.loyaltyLedger.push({ ... type: "earn", points: r, ... ref_id: a, ... })`. The only other writer is redemption (mod/631.js:11601-11610), which pushes a negative row for a reward. No cancellation path touches `Qt.loyaltyLedger`: `Mr` (mod/631.js:7455-7493), `cancelGameLocked` (mod/631.js:3742-3796) and `Br` (mod/631.js:7691-7735) never reference it. Points convert to goods at mod/631.js:11589 — `r.mockRedeemReward`, gated only on `qs(e) < a.cost_points`.	A user can earn loyalty points on a booking or match, cancel inside the refund window, get their money back in full, and keep the points. Repeating book-and-cancel cycles mints points at no cost, and those points are exchanged for real rewards with stock and value through `mockRedeemReward`.	Give `Ws` a reversal counterpart that appends a compensating n for `(user, action, ref_id)`, and call it from `Mr` (mod/631.js:7466), `cancelGameLocked` (mod/631.js:3775) and `Br` whenever a ref issued. Keep the ledger append-only but make the balance net o
P-CARCH-46 Medium	/rewards/ledger.tsx	Data completeness / pagination	The points ledger is hard-capped at 50 rows, so it never reconciles with the balance it sits under A truncated list is presented as a complete one. It needs either pagination (cursor + load-more, as /wallet does at mod/2506.js:520-527) or an explicit "showing the latest 50" marker. Evidence: mod/631.js:11562 `r.mockGetLoyaltyLedger = async (e) => (await ei(), js(e).sort((e, t) => new Date(t.created_at).getTime() - new Date(e.created_at).getTime()).slice(0, 50));` — a fixed slice with no cursor and no total. The screen renders that array as the whole history with no footer, no count and no load-more: mod/2483.js:79-87 `data: _, keyExtractor: (e) => e.id, ... ListEmptyComponent: ...`. The balance shown one screen up is computed over the *whole* ledger: mod/631.js:11552 `balance: qs(e)` where `qs = (e) => js(e).reduce((e, t) => e + t.points, 0)`. The empty-state copy promises the opposite: `rewardsLedgerEmpty: "Nothing yet. Every point you earn or spend is listed here."`	For any player past 50 events the rows on /rewards/ledger do not add up to the balance on /rewards, with nothing on screen to explain the difference — the exact situation in which a user checks a ledger. This is aggravated by earn rows being backdated at write time (mod/631.js:11517 `created_at: i ?? new Date().toISOString()`, called with `t.ends_at` at mod/631.js:11526), so a newly accrued point can be inserted below the 50-row window and be invisible from the moment it is created.	Give `mockGetLoyaltyLedger` (mod/631.js:11562) a cursor/limit s returning `{ rows, next_cursor, total }`, and add a load-more contr mod/2483.js matching the `showMore` pattern already used in mod/2506.js:521.
P-CTEST-16 Medium	/settings/access.tsx	Dead end / incomplete feature	/settings/access is a settings screen with no settings — six paragraphs of prose and a disabled element The one accessibility control that does exist in the app — high contrast — lives somewhere else entirely, inside the profile-tab edit sheet (mod/1805.js:600-607 `value: K, onChange: Q`), where a user looking for accessibility settings will not find it. Evidence: mod/2484.js:38-65 — the entire body is `w.map((o) => ... Card { Icons, Text(K(o.tKey)), Text(K(o.bKey)) })` over the static list at mod/2484.js:85-92, followed by mod/2484.js:64 `!1,` — a literal false left in the children array where a control used to be. The styles for that control survive unused: mod/2484.js:108-116 `osBtn: { flexDirection: "row", alignItems: "center", justifyContent: "center", borderWidth: 1.5, borderRadius: 999, paddingVertical: 12, marginTop: x.spacing.sm }`; Two imports are evaluated and immediately discarded at mod/2484.js:70 `(t(r[d[1]]), t(r[d[2]]));` (deps 454 and 137 — the Linking/Platform pair the removed button needed). The screen also takes no `user`/`profile`, so nothing on it is per-user.	A user navigates to "Accessibility" expecting to change something and gets a read-only description of features the app claims to have, with no control, no link to OS settings, and nothing to do but go back. The dangling `!1` and orphaned `osBtn` style show a button was removed rather than never added, so this shipped as a regression.	Either move the high-contrast switch from mod/1805.js:600 onto i and add the real controls the copy describes, or restore the OS-s the `osBtn` style and the discarded imports at mod/2484.js:70 we Remove the `!1` placeholder at mod/2484.js:64 and the unused s way.
P-CSEC-93 Medium	/settings/region.tsx	Price misrepresentation	Choosing a non-Kuwait currency relabels KWD amounts with a foreign symbol and drops the fils digit There is a currency selector but no FX layer. The stored value is unchanged; only the symbol and the decimal count change. For the four 2-decimal currencies, `toFixed(2)` also rounds away the fils digit that the rest of the system charges in. Evidence: mod/2485.js:122-123 — the picker sets the currency directly with no conversion: `active: E.currency === l, onPress: () => l({ currency: l })`, iterating `Object.keys(f.CURRENCIES)`. Module 912 then formats every amount against that global: `e.formatMoney = (n, r) => { const s = t[o.currency] ?? t.KWD, l = "ar" === r ? s.symbolAr : s.symbolEn; return `\$(f(n.toFixed(s.decimals), r)) \$\${l}`; }` with `CURRENCIES` declaring `KWD: { ..., decimals: 3 }, SAR: { ..., decimals: 2 }, GBP: { symbolEn: "£", decimals: 2 }`. Every price flows through it — mod/1311.js: `e.formatPrice = (o) => (o ? (0, a.formatMoney)(o, ...) : (0, t.t)("free"))` — including the Pay button label at mod/2474.js: `title: L ? H("paying") : `\${H("payNow")} · \$(f(0, w.formatPrice)(R.amount_kwd))`. The amounts themselves are unconditionally KWD: `A.WALLET_CURRENCY = "KWD"` (module 655) and `YI` stamps `currency: e.currency ?? A.WALLET_CURRENCY` at mod/631.js:15931.	A user who picks SAR or GBP sees every price, wallet balance and transaction in a currency they are not being charged in — a 3.255 KWD seat renders as "3.26 £" on the Pay Now button while 3.255 KWD is captured. The displayed amount and the charged amount differ, and the displayed currency is simply wrong, on the screen where consent to the charge is given.	Either drop the currency control from /settings/region.tsx (mod/2485.js:122-123) and pin display to the transaction's own `currency` field, or add a conversion layer so `formatMoney` receives a converted value. U exists, `formatMoney` should use the amount's currency rather than the region's. `getRegionSettings().currency`.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-48 Medium	./squad/[gameId]	Input validation	Removing a squad player requires a reason the UI treats as optional, and the dialog discards what was typed The client does not mirror the backend's mandatory-reason rule, and it tears down the input before the request resolves, so the typed text is no longer on screen when the error appears. Evidence: mod/2486.js:441-448 `Button, { title: k("removePlayer"), variant: "danger", loading: N, onPress: () => { const e = V; (Y(null), \$(() => kickPlayer(o.id, D.game_id, e.id, G    void 0))); } }` — the button is never disabled and the note field (mod/2486.js:420-431) has no required marker or minimum length. The backend demands three characters: mod/631.js:3310 `if (sanitizeText(i    "", 200).length < 3) throw new Error("E_A_REASON_IS_REQUIRED");`. `Y(null)` closes the dialog before the call, so the failure lands in the page-level error line at mod/2486.js:291-295 while the form is gone.	A host removing a no-show leaves the note blank (or types 'ok'), the card vanishes, and a red line appears at the top of the page saying a reason is required — with the note box gone. They must reopen the dialog and guess the minimum length; some will conclude removal is broken.	Disable the remove button until `G.trim().length >= 3` in mod/2486.js:441-448, and add a helper line stating the requirement, and keep the dialog mounted so the note survives a rejection.
P-CQUAL-49 Medium	./stories/[id].tsx	Render-phase side effect / silent dead end	/stories/[id] calls the router during render when an author has no live stories, and closes silently on every error A router transition is a state update on another component; performing it in the render phase rather than in an effect produces React's 'Cannot update a component while rendering a different component' warning and an unpredictable double-render. The condition is reachable in ordinary use, not just in theory: a story can expire between the tray fetch at mod/2487.js:26 and the author fetch at mod/2487.js:38, and a shared /stories/<author> link opened after the stories expire hits it directly. Separately, every error is mapped to 'close the screen' with no message. Evidence: mod/2487.js:90 navigates from inside the render body: if (0 === F.length) return (U(), null); where U is the navigation callback at mod/2487.js:19-21: U = (0, t.useCallback)(() => { L.canGoBack() ? L.back() : L.replace("/(tabs)/feed"); }, [L]); Every failure path in the screen is the same silent close - mod/2487.js:26-30: (0, b.fetchStoryTray)(_id) .then((t) => { (V(t.map((e) => e.author_id)), A(e)); }) .catch(() => U()); and mod/2487.js:38-42: (0, b.fetchAuthorStories)(_id, z) .then((t) => { e && (P(t.items), D(t.author_name), H(t.is_self), B(0)); }) .catch(() => U()); Those calls can throw 'blocked' (mod/631.js:14906) and E_THAT_RECORD_IS_IN_THE_OTHER_WORLD (via oa -> ra, mod/631.js:766-768).	Opening a shared story link a few minutes too late, or tapping a tray entry whose last story just expired, bounces the user straight back to the feed with no explanation and a console error - which is exactly the console-error condition the repo's smoke harness is required to keep clean. The same silent bounce happens when the author has blocked the viewer or is in the other audience partition, so a blocked user cannot tell the difference between 'blocked' and 'the app is broken'.	Move the empty-list exit in mod/2487.js:90 into a useEffect that fires when the list is empty, becoming a neutral placeholder in the render phase. Replace the bare `.catch(() => U())` at mod/2487.js:30 and mod/2487.js:42 with an error state that shows the localized message (the 672 prc localizes it) and an explicit Close button.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CTEST-17 Medium	/stories/compose.tsx	Error handling / silent failure	/stories/compose swallows every error: the Share button has a try/finally with no catch There is no catch block and no error state in the whole screen - no FormError, no Toast, no aria-live region. The finally clause resets the busy flag, so a failure is indistinguishable from a mis-tap. Errors reaching the screen are already localized by the 672 store proxy (mod/672.js:43-45 -> localizeStoreError), so a one-line catch could render a correct human message. Evidence: mod/2488.js:52-68: <pre>onPress: async () => { if (t && q && !U) { E(10); try { const l = V ? "photo" : F ? "highlight" : "text"; (await (0, C.createStory)(t.id, { kind: l, text: T, clipId: F?.cover_clip_id ?? null, bgColor: ..., mediaUri: V?.uri ?? null })), P.replace("/feed"); } finally { E(11); } } },</pre> Every throwable on that path is unhandled: mod/631.js:14822 `throw new Error("empty_story")`, mod/631.js:1733 `throw new Error("E_IMAGE_TOO_LARGE_PICK_A_SMALLER")`, mod/631.js:1772-1774 E_STORAGE_FULL / E_COULD_NOT_SAVE_CHANGES. The same omission appears on the collections fetch - mod/2488.js:19: <pre>t && (0, C.fetchHighlightCollections)(t.id).then(W);</pre>	A user composes a story, taps Share, and the button flashes and re-enables with no message and no navigation. On a full localStorage quota (E_STORAGE_FULL) or an oversized image this happens on every retry, and the user has no way to learn why - their story is simply never posted. The unhandled rejection surfaces only in the browser console, which the smoke harness is supposed to keep clean. The missing .catch at mod/2488.js:19 has the same shape: if the collections fetch rejects, the highlight chip row silently never renders and the user concludes they have no clips.	Add `catch (err) { setError(err.message) }` around the createStory mod/2488.js:55 and render the message with the FormError component sibling upload screen already uses (mod/2454.js:88), and add a `the fetchHighlightCollections call at mod/2488.js:19.
P-CARCH-47 Medium	/teams/[id].tsx	Performance / redundant load	The clan page refetches its entire dataset, including a global ranking recompute, on every tab tap `ye` is only needed by one line of the loader (mod/2489.js:82, the chat preload), yet it is a dependency of the whole callback. Six tabs are rendered (mod/2489.js:547), so simply reading the page costs six full reloads. Evidence: mod/2489.js:51-84 — the loader closes over the active tab `ye`, so the callback identity changes on every tab press: <pre>... 51 Ue = (0, t.useCallback)(async () => { ... 62 const [a, l, s, n, o, i, c, m, u] = await Promise.all([63 (0, L.fetchTeamMembers)(e, r.id).catch(() => []), ... 69 (0, L.fetchTeamRankings)(void 0, r?.id).catch(() => []), 70 (0, L.fetchClanBattles)(r.id).catch(() => []), 71 (0, L.fetchH2HMap)(r.id, e).catch(() => ({})), 72]); 84 }, [e, r, ye]); ... mod/2489.js:85-89 re-runs it whenever that identity changes: ... 85 (0, x.useFocusEffect)(86 (0, t.useCallback) (() => { 87 Ue(); 88 }, [Ue]), 89); ... One of those ten calls is the most expensive query in the backend — mod/631.js:9177-9183 walks every team and, via `ToQ`, recomputes every team's ladder score: ... 9177 r.mockGetTeamRankings = async (e = {}, t) => { 9178 (await ei(), await mo(), await Bo(), await ToQ()); ... </pre>	Tapping through Overview → Roster → Battles → Calendar → Badges → Chat fires 60 backend calls and six global ranking recomputes, each of which is O(teams × events) and rewrites the whole teams array to localStorage via `Za(Ot, ...)`. On a phone on a slow connection against a real `rpc` backend this is seconds of spinner per tab and heavy server load; in the local store it means every tab tap blocks the UI thread on a full-table scan and a serialization pass.	Split the loader in mod/2489.js:51 into a tab-independent `useCal` deps `[e, r]` for the nine shared fetches, and a small separate effect `ye` that loads only the chat preview — mirroring the pattern the sibling already uses for battles at mod/2489.js:32-48. Also hoist `fetchTeamRankings` out of the per-view path or scope it to the component's sport.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-94 Medium	/teams/[id].tsx	Access control / private-group membership	Team invite code is returned for any team id with no caller identity Every other team read takes the caller and checks privacy and membership — <code>`mockGetTeam` (mod/631.js:8636)</code> and <code>`mockGetTeamMembers` (mod/631.js:8656-8658)</code> both call <code>`no(e, t)`</code> for private teams. The invite lookup, which returns the most sensitive artefact of the three, takes no caller at all. Evidence: <code>mod/631.js:9199-9203</code> — the whole function: <pre>... r.mockGetTeamInvite = async (e) => { await ei(); const t = to(e); return t ? { code: t.invite_code, link: `https://playora.app/t/\${t.invite_code}` } : null; }; ...</pre> No caller parameter, so no membership or role check is expressible. The code is a join credential that grants immediate active membership to an invite-only team: <code>mod/631.js:8734-8746</code> <pre>... "invite_only" === i.privacy ? eo(i.id, async () => { ... (a.status = "active"), (a.joined_at = ...), ... }) ...</pre> Facade <code>mod/671.js:586</code> <code>`e.fetchTeamInvite = (o) => t.store.mockGetTeamInvite(o)`</code>; screen <code>mod/2489.js:68</code>.	The credential that bypasses a private team's approval flow is readable by team id. Anyone who obtains a code enters an invite-only squad as an active member without any captain approval, and the team's chat, calendar and member list open to them. The screen currently bails out before the call for teams it cannot load, so today this is a latent hole — but it survives intact when the code moves behind a real API.	Change the signature to <code>`mockGetTeamInvite(callerId, teamId)`</code> a <code>`so(teamId, callerId)`</code> or at minimum <code>`no(teamId, callerId)`</code> before the code; update <code>mod/671.js:586</code> and <code>mod/2489.js:68</code> to pass the
P-CQUAL-50 Medium	/teams/battle/[battleId].tsx	Stale state / missed deadline	An accepted battle freezes its clock, so the score form and deadline warnings never appear while the screen is open The ticker is enabled in exactly the state that does not need it (pending, where the shot clock is a countdown) and disabled in the state that does (accepted, where the kick-off time and the seven-day result deadline both have to cross a threshold to reveal UI). Nothing else triggers a re-render of the accepted branch. Evidence: <code>mod/2495.js:34-38</code> — the one-second ticker that drives <code>`J`</code> runs only while the challenge is pending: <pre>... 34 (0, t.useEffect)() => { 35 if ("pending" !== U?.view_status) return; 36 const e = setInterval(() => K(Date.now()), 1e3); 37 return () => clearInterval(e); 38 }, [U?.view_status]); ...</pre> <code>mod/2495.js:49-51</code> — but the accepted-state values all read from that frozen <code>`J`</code>: <pre>... 49 ae = U ? new Date(U.result_deadline).getTime() - J : 0, 50 le = !!U && new Date(U.starts_at).getTime() <= J, 51 se = (0, S.resultDeadlineUrgency)(ae), ...</pre> <code>`le`</code> gates the deadline banner at <code>mod/2495.js:515</code>, and <code>mod/2495.js:634</code> gates the whole score form on a value evaluated only when the component re-renders: <code>`A = Date.now() > new Date(e.starts_at).getTime() + 54e5`</code>.	A captain who opens the battle page before kick-off and keeps it open sees <code>`resultOpensAfter`</code> ("the result opens after the match") indefinitely — the score entry form never appears, the "result deadline" warning never escalates from calm to final, and <code>`resultDeadlinePassed`</code> never shows. They have to guess that navigating away and back is required. Missing the seven-day window sends the result to escalation (<code>mod/631.js:10137-10170</code>), so a UI that silently stops updating costs the clan a recorded result.	In <code>mod/2495.js:34</code> run the interval whenever <code>`view_status`</code> is <code>`per`</code> <code>`accepted`</code> (a coarser 30s tick is enough for the accepted case), <code>`A`/`le`/`se`</code> from <code>`J`</code> consistently and key the ticker off "any live d the future" rather than off the pending status.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-51 Medium	./teams/battle/[battleId].tsx	Incorrect data rendering	Accepting a gauntlet replaces the battle with a rank-less copy, so ranks and the underdog bonus disappear from the screen With `a` undefined, `from_rank`, `from_pts`, `to_rank`, `to_pts` all resolve to null (mod/631.js:9968-9971) and `underdog` stays null (9943-9950, guarded by `if (k && v)`). The screen renders those conditionally (`null != de.rank &&` at mod/2495.js:201, `(be    je) &&` at 369), so they silently vanish. Evidence: mod/631.js:9910 — `Wo` takes a third argument, the rank map: `Wo = (e, t, a) => {` and reads it at 9941-9942: <pre> ... 9941 k = a?.get(e.from_team_id) ?? null, 9942 v = a?.get(e.to_team_id) ?? null; ... </pre> Every read path supplies it — mod/631.js:10052 `return (i && ra(t, i.audience), Wo(a, t, Oo(aa(t))))` and mod/631.js:9984 `.map((t) => Wo(t, e, i))`. But `mockRespondClanChallenge` omits it, twice — mod/631.js:10718 `Wo(i, e)` and mod/631.js:10822 `Wo(i, e)`. mod/2495.js:57 writes that degraded object straight into screen state: `H(await (0, k.respondClanChallenge)(c.id, U.id, e));`	The moment a captain taps "Accept the gauntlet" — the single most important action on the screen — both clans' ladder ranks and points disappear from under the crests and the "+50% underdog" stake card disappears entirely, exactly when the user wants to confirm what they just committed to. It reappears only if they navigate away and return. It reads as data loss and undermines trust in the stake figures.	Pass the rank map at both call sites in mod/631.js — `Wo(i, e, Oo(aa(e)))` at 10822 — matching `mockGet` at 10052. Alternatively make `Wo`'s third parameter required so this is a type error.
P-CQUAL-52 Medium	./teams/battle/ceremony/[battleId].tsx	Navigation dead end	Every screen in the clans and clubs group uses bare router.back(), which is a no-op on a cold deep link These routes are all deep-linkable — serve.json rewrites every path to the SPA and the app itself shares battle and ceremony URLs to WhatsApp (mod/2496.js:364-367, 386) and clan invite links (mod/2489.js:423). On a cold load there is no history entry, so `router.back()` does nothing at all. Evidence: mod/9001.js:77 is the codebase's own correct pattern: `A = I ?? () => (T.canGoBack?.() ? T.back() : T.replace("/(tabs)"));` mod/2487.js:20 uses it too: `L.canGoBack() ? L.back() : L.replace("/(tabs)/feed");` None of the ten in-scope screens do. The ceremony's only escape from its not-ready state is unguarded — mod/2496.js:164-169: <pre> ... 164 (0, N.jsx)(f.Button, { 165 title: V("backToBattle"), 166 onPress: () => L.back(), ... </pre> Same at mod/2495.js:118 `onPress: () => B.back()`, mod/1847.js:70 `onPress: () => w.back()`, mod/2489.js:111/151, mod/2497.js:39, mod/2498.js:30, mod/2500.js:56, mod/2501.js:30, mod/2371.js:30, mod/2370.js:21.	A user who taps a shared ceremony or battle link opens the app directly on that route. If the ceremony is not ready (mod/2496.js:156 — result not yet confirmed, a very common case for a link shared right after a match) the entire screen is one message and one button, and that button does nothing when pressed. The user is trapped on a page with no other control; the ceremony screen has no header back affordance either.	Replace the bare `router.back()` calls in all ten modules with the guard already used in mod/9001.js:77 — `router.canGoBack?.() ? router.replace("/(tabs)")` — ideally by exporting that helper from r and importing it, so the pattern cannot drift again.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-53 Medium	/teams/chat/[teamId].tsx	Error handling / dead end	The clan chat screen has no error handling, so an unloadable clan leaves a permanent spinner 2489 and 2495 both classify `isAudienceError` and render `NotAvailable`; 2497 does neither and does not even use a `finally` to clear the loading flag. A `null` team (deleted, or private and the viewer is not a member) is also not handled — `P(null)` leaves `L?.viewer_member` falsy, which at least reaches the members-only panel, but a throw does not. Evidence: mod/2497.js:18-22 — the loader is unguarded, so a throw skips `E(!1)` and the spinner never clears: <pre>... 18 J = (0, t.useCallback)(async () => { 19 if (!e !z) return; 20 const t = await (0, k.fetchTeam)(e, z.id); 21 (P(!t, t?.viewer_member && A(await (0, k.fetchTeamChat)(e, z.id).catch(() => [])), E(!1)); 22 }, [e, z]); ...</pre> `fetchTeam` does throw — mod/631.js:8636 calls `ra(t, a.audience)` which raises `AUDIENCE_MISMATCH`; `return a ? (ra(t, a.audience), ...)`. The sibling screens handle it; mod/2489.js:54-59 does: <pre>... 56 } catch (e) { 57 if ((0, L.isAudienceError)(e)) return (we(!0), void be(!1)); 58 throw e; 59 } ...</pre> Module 2497's dependency list (mod/2497.js:347-350) contains no entry for module 2385 (`NotAvailable`), confirming it has no equivalent branch.	A user who opens a clan-chat deep link for a clan in the other audience partition — or whose session audience changed — gets a spinner that never resolves, plus an unhandled promise rejection in the console (which the project's own smoke harness treats as a failure condition). The header back button is present, so it is not fully trapping, but the screen never tells them anything.	Wrap mod/2497.js:20 in try/catch/finally: on `isAudienceError` rer `NotAvailable` (module 2385), on any other error show an inline € `RetryButton` from module 9001, and always call `E(!1)` in `finally`
P-CTEST-18 Medium	/teams/league.tsx	Demo data in production path	The league screen renders hardcoded demo fixtures, and with demo seeding off it is an empty table claiming seven clans `mockGetLeague` is a static fixture with no relationship to `Qt.teams`. Real clans created by users can never enter the division, `mine` is false for everyone outside the seed roster, and the clan count is reported from the hardcoded array length rather than from the rows actually rendered. Evidence: mod/631.js:10868-10895 — the division is a fixed list of seed team ids, never the user's clans: <pre>... 10868 Rs = (e) => 10869 "female" === aa(e) 10870 ? { div: As, results: Ds, fixtures: Os, totalMd: 3, ... } 10883 : { div: hs, results: ys, fixtures: ks, totalMd: 6, ... divisionName: "Division 1", ... }, ...</pre> with `hs = [_s, us, ms, ws, ps, fs, gs]` (mod/631.js:10832) naming `team-0000-strikers` ... `team-0006-fintas`. mod/631.js:9699-9700 — the seeding those ids depend on is a no-op outside demo mode: `Bo = async () => { if (!Fa()) return;` with `Fa = () => !0 === Ne0().demo` (mod/631.js:1722). mod/631.js:10911-10929 then drops every row whose team is missing (`.filter((e) => !!e)`), while mod/631.js:10971 still reports `clans: t.div.length` — a constant 7. mod/2500.js:99 prints that constant and 133 maps the empty array: `children: N("leagueMeta", { n: (0, C.formatNumber)(R.clans, slot: R.slot }},` ... `R.table.map((t, r) => {`	On a non-demo deployment the league page renders a header, "Division 1", the text "7 clans", a table with column headers and zero rows, and then a tie-break explainer and a "finale week" card dated from a computed future Friday — a visibly broken screen presented as live standings. Even in demo mode, a real user's own clan can never appear, so the promotion/relegation/playoff lines describe a competition no customer can enter.	Either build the division from `Qt.teams` ranked by `Ao()` (as `mockGetTeamRankings` does at mod/631.js:9177) so real clans or gate the whole route behind `Fa()` and render an explicit "leag next season" state when seeding is off. At minimum derive `clans` `table.length` at mod/631.js:10971 so the header cannot contradi
P-CARCH-48 Medium	/teams/rankings.tsx	Read amplification	Team rankings return every team with three full-table scans each, while the callers use only the top 8 No `limit` parameter and no per-team index. `Ao` at mod/631.js:9171 correctly short-circuits on the cached `ladder_score`, but the three other per-team scans in the map at mod/631.js:9189-9193 are not cached and run regardless. Evidence: mod/631.js:9177-9197 — `mockGetTeamRankings` maps over every active public team and per team runs `Qt.teamEvents.filter(...)` (9189), `Qt.teamMembers.filter(...)` (9190), `So(e.id)` which filters and sorts `Qt.teamEvents` again (mod/631.js:9120-9123), and `cs(e.id)` which scans `Qt.clanBattles` (mod/631.js:10620). It returns the full sorted array with no limit. Callers: mod/1677.js:51-53 (home, on every focus) and mod/1629.js:39 — and mod/1629.js:75 immediately does `J.filter(...).slice(0, 8)`.	Both the home tab and the compete tab pull the entire league table on every focus in order to display eight rows. At 500 teams and 20,000 team events that is ~30 M comparisons plus 500 sorts per tab switch, twice over, for 8 visible rows.	Add `{limit}` to `mockGetTeamRankings` (mod/631.js:9177), sort already-cached `ladder_score` first, slice, and only then compute and `cs` for the surviving rows. Build `Map<team_id, events[]>` a `Map<team_id, activeMemberCount>` once per call.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-95 Medium	/venue/[id]	Data exposure	Venue listing rules are enforced on the directory query but not on single-record reads Both single-record readers bypass the visibility rule their list counterpart enforces. <code>`mockGetVenue`</code> should apply the same <code>`!1 !== e.listed`</code> predicate, and <code>`mockGetVenueBookingDetail`</code> should apply <code>`yr(e)`</code>, or return an explicit “not available” status the screens can render. Evidence: <code>mod/631.js:2488-2490</code> — the comment states the intent (“The public directory only lists venues that have not been withheld pending review.”) and <code>`r.mockGetVenues = async () => (await pi()).filter((e) => !1 !== e.listed);`</code>, but <code>mod/631.js:2491-2493</code> — <code>`r.mockGetVenue = async (e) => { const t = (await pi()).find((t) => t.id === e); if (!t) return null;`</code> has no <code>`listed`</code> check. The same asymmetry exists for approval: <code>mod/631.js:6946</code> — <code>`if (!yr(r.id)) continue;`</code> guards the bookable search (with <code>`yr = (e) => "approved" === hr(e)?.status`</code> at <code>mod/631.js:6818</code>), while <code>mod/631.js:6971-6976</code> — <code>`r.mockGetVenueBookingDetail = async (e) => { ... const t = fi().find((t) => t.id === e); return t ? { venue: t, profile: hr(e) ?? null, courts: Qt.courts.filter((t) => t.venue_id === e && t.active) } : null;`</code> returns everything with no status check.	A venue withheld pending review is invisible in the directory and on the map but fully readable at <code>`/venue/<id>`</code> — name, address, description, rating and all reviews — to anyone holding or guessing the id. A venue that is not approved for bookings still serves its full court list and per-hour prices at <code>`/booking/venue/<id>`</code> , letting a user pick a slot that <code>`mockReserveCourt`</code> will reject at <code>mod/631.js:7342</code> (<code>`await kr(r.venue_id)`</code>). Delisting is therefore not an effective control.	Add the <code>`listed`</code> filter to <code>`mockGetVenue`</code> (<code>mod/631.js:2491</code>) and <code>`</code> — or a soft <code>`{ bookable: false }`</code> flag — to <code>`mockGetVenueBookingDetail`</code> ( <code>mod/631.js:6971</code> ), and have <code>mod/2502.js</code> and <code>mod/1842.js</code> render existing <code>`venueNotFound`</code> / an “not accepting bookings” state for results.
P-CARCH-49 Medium	/venue/portal.tsx	Access provisioning / dead end	Staff accounts are created with a synthetic user id that no real account can ever match, so staff can never use the portal or scanner Adding staff writes a display-only record. There is no invitation, no account lookup by email or phone, and no way to bind the row to a person, yet <code>`vr`</code> is written as though staff rows grant access. Evidence: <code>mod/631.js:7078-7083</code> — <code>`n.staff.push({ user_id: `staff-\${ea().slice(0, 8)}`, name: r, role: i, added_at: new Date().toISOString() })`</code> — the id is minted locally from a name string, not resolved from any account. The only place staff ids are ever read back, <code>mod/631.js:6827</code> — <code>`!a.staff.some((t) => t.user_id === e)`</code> inside <code>`vr`</code>, compares against the caller’s real account id, and <code>mod/631.js:7052</code> — <code>`Qt.venueProfiles.filter((t) => t.owner_id === e    t.staff.some((t) => t.user_id === e))`</code>. A <code>`staff-xxxxxxx`</code> string can never equal an account id produced by <code>`ea()`</code> (<code>mod/631.js:744</code>, a UUID) or a <code>`guest`</code> id. The screen only ever sends a name and a fixed role: <code>mod/2504.js:478-481</code> — <code>`onPress: () => oe(async () => { (await (0, S.addVenueStaff)(e.id, re.id, te.trim(), "scanner"), se("")); })`</code>, while rendering <code>`W(\\role_\${t.role}\\)`</code> at <code>mod/2504.js:447</code> as if more than one role were reachable.	The entire Staff accounts section (<code>mod/2504.js:420-486</code>) is a dead end: an owner adds their front-desk employee, sees them listed, and the employee still cannot sign in and reach <code>/venue/portal</code> or <code>/venue/scan</code> . The owner must hand over their own credentials to get anyone else scanning — the exact outcome the staff-role feature exists to prevent. The staff branch of <code>`vr`</code> and of <code>mockGetMyVenues</code> is unreachable code.	Change <code>mockAddVenueStaff</code> (<code>mod/631.js:7071</code>) to take an account (email or phone), resolve it against <code>`Qt.profiles`</code> , and reject with a message when no account matches; or issue a pending invitation row that <code>`user_id`</code> when the invitee accepts. Update the portal input at <code>mod/2504.js:467-482</code> to collect that identifier and to offer the role: implies.
P-CQUAL-54 Medium	/venue/portal.tsx	Configuration gap / user-visible failure	New courts are locked to 08:00–24:00 and their opening hours can never be edited, and the last generated slot is always rejected Opening hours are a first-class court attribute that the portal renders but provides no control for. Every court created here is fixed at 08:00–24:00, and the 24:00 boundary is incompatible with the same-calendar-day rule in <code>withinOperatingHours</code>. Evidence: <code>mod/2504.js:214-217</code> — <code>`onPress: () => { (ee("12.000"), Y({ sport: "padel", open_minutes: 480, close_minutes: 1440, price_per_hour_kwd: 12 })); }`</code> — the only place hours are set for a new court. The editor form has fields for name, sport and price only (<code>mod/2504.js:279-327</code>) and re-sends whatever was there: <code>mod/2504.js:350-351</code> — <code>`open_minutes: X.open_minutes ?? 480, close_minutes: X.close_minutes ?? 1440,`</code>. The values are nonetheless displayed: <code>mod/2504.js:246-248</code> — <code>`(0, T.fmtHHMM)(t.open_minutes), "-" , (0, T.fmtHHMM)(t.close_minutes)`</code>. With <code>close_minutes</code> 1440 the slot generator offers a 23:00 slot — module 669: <code>`generateSlots=(t,n,a,o,s,l)=>{ ... for(let c=n;c+o<=a;c+=s){ ... }`</code> (<code>1380+60<=1440</code>) — but the reservation check rejects it: <code>`withinOperatingHours=(t,e,a,o)=>{ ... return new Date(t).toString()===new Date(e).toString()&&s>=a&&w<=o&&s<w`</code>, and a 23:00–00:00 booking ends on the next calendar day. <code>mockReserveCourt</code> enforces it at <code>mod/631.js:7348-7349</code> — <code>`if (!t(0, H.withinOperatingHours)(a, i, r.open_minutes, r.close_minutes)) throw new Error("E_THAT_TIME_IS_OUTSIDE_THE_COURT");`</code>	A venue that opens at 06:00 or closes at 22:00 cannot be configured at all from the consumer app, and the displayed hours are a lie it cannot correct. Worse, the forced 24:00 close makes the booking UI advertise a final 23:00 slot as available that always fails at reservation time with “that time is outside the court’s hours” — a player-facing error on a slot the app itself offered.	Add open/close time inputs to the court editor at <code>mod/2504.js:279</code> to send the user’s values at <code>mod/2504.js:350-351</code> . Cap the default <code>close_minutes</code> at 1440 (23:00) rather than 1440, or make <code>`withinOperatingHours`</code> (module 669) a midnight end timestamp on the following calendar day as minutes from the start day.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-55 Medium	/venue/portal.tsx	Error handling / data loss	Saving a court discards the form and everything typed in it whenever the save fails The success and failure paths are not distinguished: because `oe` never rejects, the comma-sequenced `Y(null)` runs after a validation error exactly as it does after a successful save. Evidence: mod/2504.js:35-44 — `const oe = async (e) => { U(!0); try { (await e(), await le()); } catch (e) { l.default.alert(W("error")), (0, H.storeErrorText)(e?.message ?? ""); }    W("error")); } finally { U(!1); } }`; — the catch swallows, so `oe` always resolves. mod/2504.js:340-354 — `onPress: async () => { e && L && X && (await oe()) => (0, S.upsertCourt)(e.id, L.venue.id, { id: X.id, name: X.name ?? "", ... }), Y(null)); }` — `Y(null)` closes the editor unconditionally after the awaited `oe`. The save button has no disabled guard (mod/2504.js:337-357), and the new-court seed carries no name (mod/2504.js:216), while the backend requires one: mod/631.js:7226-7228 — `const i = (0, v.sanitizeText)(a.name, 60); if (!i) throw new Error("E_ADD_A_COURT_NAME"); if (a.close_minutes <= a.open_minutes) throw new Error("E_CLOSING_TIME_MUST_BE_AFTER_OPENING");`	A venue owner taps Add court, picks a sport, types a price, forgets the name, taps Save — gets an alert and finds the form gone along with the sport and price they entered. The same happens for any transient network failure on an edit of an existing court. Because the Add-court seed has no name and Save is never disabled, this is the default first-run experience of the court editor, not an edge case.	Have `oe` (mod/2504.js:35) rethrow after alerting, or return a suc boolean, and only call `Y(null)` at mod/2504.js:354 when the save succeeded. Add `disabled: !(X?.name ?? "").trim()` to the Save cc at mod/2504.js:337.
P-CTEST-19 Medium	/venue/portal.tsx	Error handling / silent failure	Errors from the portal's courts, bookings and revenue fetches are swallowed and rendered as empty state A catch-all that maps every failure onto the same value as "there is genuinely no data" erases the distinction between an empty venue, an authorization failure and a server outage. Evidence: mod/2504.js:24-26 — `\$(await (0, S.fetchCourts)(s, e.id).catch(() => [])), K(await (0, S.fetchVenueBookings)(e.id, s).catch(() => [])), G(await (0, S.fetchVenueRevenue)(e.id, s).catch(() => null))`;" Those calls can fail for reasons the operator needs to know about — mod/631.js:7548 (`await Sr(e, t)` in mockGetVenueBookings) and mod/631.js:8364 (`await Sr(e, t)` in mockGetVenueRevenue) throw `E_YOU_ARE_NOT_AUTHORIZED_TO_MANAGE_4` after writing a `venue.access_denied` audit entry (mod/631.js:6834). The empty results then render as ordinary no-data UI: mod/2504.js:491-495 — `0 === ie.length ? Text({ ... children: W("noBookings") })` ("No bookings yet.") — and the revenue block is simply skipped: mod/2504.js:108 — `F && Fragment({ ... })`.	A venue owner whose access has been revoked, or whose backend is down, sees "No bookings yet.", no revenue section and an empty court list — and concludes their bookings have vanished or that nobody has booked. They act on a screen that is silently wrong: they will not chase a support ticket, and the audit trail records repeated access-denied events that nobody on the venue side ever sees.	Replace the three `.catch` swallows at mod/2504.js:24-26 with a error-capture that records the failure in state, then render the error. Remove the `GateScreen`/`RetryButton` from mod/9001.js — using `classifyError` flag (mod/9001.js:59-63) to show a denied screen for `E_YOU_ARE_NOT_AUTHORIZED_TO_MANAGE_4` rather than the list.
P-CARCH-50 Medium	/venue/portal.tsx	Multi-tenancy	Only the first of a user's venue profiles is reachable; the rest are invisible with no venue switcher The screens model a one-venue-per-user world that the backend does not enforce and the organizer flow actively violates. `[0]` is whichever profile happens to sit first in the persisted array. Evidence: mod/2504.js:21 — `const t = (await (0, S.fetchMyVenues)(e.id))[0] ?? null;` and mod/2505.js:17-19 — `const t = await (0, _fetchMyVenues)(e.id); if (!t[0]) { const s = await (0, _fetchVenueBookings)(e.id, t[0].venue.id)... }` — both take index 0 of a list and there is no selector anywhere in either screen. The API is plural by design: mod/631.js:7048-7055 — `r.mockGetMyVenues = async (e) => (... Qt.venueProfiles.filter((t) => t.owner_id === e    t.staff.some((t) => t.user_id === e)).map((e) => ({ venue: fi(), find((t) => t.id === e.venue_id), profile: e })).filter((e) => !e.venue));` Multiple profiles per user arise two ways: mockApplyVenue has no already-applied guard (mod/631.js:6978-7010 checks only admin role, `venue_id` reuse and a non-empty name), and every organizer custom venue mints another owned profile — mod/631.js:3455-3457 — `const vp9 = { venue_id: r.id, owner_id: e, status: "pending", ...`. Ordering is insertion order in `Qt.venueProfiles`, not a deliberate default.	An organizer who typed a custom venue name for one match silently becomes a venue owner, and from then on the profile tab sends them to /venue/portal for that phantom venue instead of /venue/apply (mod/1805.js:683). A genuine operator with two locations, or one who reapplied after a rejection, can only ever manage one of them — the other's bookings, courts and revenue are unreachable from the app, and the scanner points at the wrong site. There is no UI hint that more than one exists.	Render a venue picker when `fetchMyVenues` returns more than one (mod/2504.js:21, mod/2505.js:17), persisting the selection, and pick a chosen venue id to the scanner. Separately, reject a second application for a user who already holds a pending or approved profile in mockA (mod/631.js:6978).
P-CSEC-96 Medium	/venue/portal.tsx	Financial reporting	Venue revenue omits refunds and pending payout, so the venue's gross does not agree with the platform's The venue-facing report exposes a gross that is not net of refunds and hides both the refund total and the unpaid portion of the payout, even though the API hands the screen all three numbers. Evidence: mod/631.js:8368-8382 returns eight figures — `grossKwd`, `commissionKwd`, `netKwd`, `settledKwd`, `pendingKwd: roundKwd(a.filter((e) => "pending" === e.status).reduce((e, t) => e + t.net_to_venue_kwd, 0)), paidPlayers, refundsKwd: roundKwd(n.filter((e) => "refunded" === e.status).reduce((e, t) => e + t.amount_kwd, 0))`. The portal renders six and drops `pendingKwd` and `refundsKwd`: mod/2504.js:118-143 — tiles for `grossRevenue`, `platformCommission`, `netPayout`, `settledLabel`, `confirmedBookings`, `paidPlayersLabel`. `grossKwd` is the raw settlement sum, mod/631.js:8371 — `grossKwd: roundKwd(a.reduce((e, t) => e + t.gross_kwd, 0))` — with no refund deduction, whereas the platform's own view of the same money does deduct: mod/631.js:8399-8400 — `grossKwd: o, netGrossKwd: (0, z.roundKwd)(o - s),` where `s` is the refunded total.	A venue reconciling its Playora income reads a Gross revenue that overstates what was actually collected by exactly the refunded amount, and a Net payout with no way to see how much of it is still pending versus settled. Their books will not tie to the platform's `netGrossKwd`, and the first refund dispute turns into a reconciliation argument with no shared figure to point at.	Add `pendingKwd` and `refundsKwd` tiles to the grid at mod/2504.js:118-143 and either rename the gross tile or expose a refund-netted gross venue and admin views (mod/631.js:8371 vs 8399-8400) report the difference.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-51 Medium	./venue/portal.tsx	Scalability / dead end	Venue bookings are fetched in full with no paging, and the portal renders only the first twenty with no way to see the rest An unbounded, unindexed fetch on every screen focus, truncated client-side with no continuation. The same call is repeated by the scanner (mod/2505.js:19) for a different slice of the same data. Evidence: mod/631.js:7546-7554 — `r.mockGetVenueBookings = async (e, t) => (await ei(), await Sr(e, t), await Cr(), Qt.courtBookings.filter((e) => e.venue_id === t).sort((e, t) => new Date(t.starts_at).getTime() - new Date(e.starts_at).getTime()).map(Pr));` — no limit, offset or date window. Each row costs a linear court scan and a venue-name lookup: mod/631.js:7528-7535 — `Pr = (e) => { const t = fr(e.court_id); return Object.assign({}, e, { court_name: t?.name ?? "Court", venue_name: Ai(e.venue_id), sport: t?.sport ?? "padel", settlement: Rr(e) }); };` with `fr = (e) => Qt.courts.find((t) => t.id === e)` (mod/631.js:6815). The sweep it runs first walks every booking in the system: mod/631.js:7516 — `for (const a of Qt.courtBookings)`. The screen holds the whole array (mod/2504.js:25) and truncates only at render: mod/2504.js:496-497 — `ie.slice(0, 20).map(...)` with no pager or Show more.	For a venue in its second season the request grows without limit, re-sorted and re-mapped on every focus of two different screens, on the phone the front desk uses. And the operator has no route to booking twenty-one: the history section simply stops, with no date filter, no pagination and no indication that anything was omitted — so older bookings become unverifiable from the app.	Give mockGetVenueBookings (mod/631.js:7546) a status filter, a and limit/offset parameters, and index `Qt.courts` by id so `Pr` stc scanning. In the portal, request pending and recent bookings sep add a pager or date filter to the history list at mod/2504.js:487-52
P-CQUAL-56 Medium	./venue/scan.tsx	Dead end	Selecting a court booking with no match in the scanner shows a status word and nothing else The booking list is not filtered to bookings that can actually be checked in, and the zero-length case reuses a per-player status label as the whole screen body, with no empty-state component, no explanation and no action. Evidence: mod/631.js:8312-8328 — `if (a.game_id) { const e = Qt.bookings.filter((e) => e.game_id === a.game_id && "confirmed" === e.status); for (const t of e) Qt.checkins.some(...)    Qt.checkins.push({ ... state: "pending" ... }); await Za(pe, Qt.checkins); } return Qt.checkins.filter((e) => e.booking_id === t);` — with `game_id` null nothing is seeded and the filter returns `[]`. Plain court reservations always have a null game: mod/631.js:7377 — `game_id: null,` in mockReserveCourt. The scanner nevertheless lists them: mod/2505.js:25 — `.filter((e) => "confirmed" === e.status    "released" === e.status)` with no game filter. The resulting screen body, mod/2505.js:102-106 — `0 === H.length ? Text({ style: [...], children: R("pendingCheckin") }) : H.map(...)` — and `pendingCheckin` is the string "Not arrived".	A venue that takes direct court bookings — the default product, since a court reservation only gains a game_id if the organizer later publishes a match onto it — taps a booking in the scanner and lands on a card showing the court name, the time and the single word "Not arrived", with no players, no buttons and no way to record anything. The back chevron returns to the list (mod/2505.js:63), so the feature simply does not work for them, with no error to report.	Either filter the scanner list at mod/2505.js:25 to bookings with a or make mockGetBookingCheckins (mod/631.js:8312) seed a ch for the booking's organizer when there is no match, so a direct re can be checked in. Replace the bare `R("pendingCheckin")` at mod/2505.js:103-106 with an `EmptyState` that says why there is check in.
P-CSEC-97 Medium	./venue/scan.tsx	Input validation	scanCheckin does not validate the token, so an absent qr_token matches the first booking that has none The lookup treats a missing token as a legitimate search key. Any booking row lacking `qr_token` — a row restored from an older persisted store, or one produced by a future code path that forgets the field — becomes the match for a scan carrying no token. Evidence: mod/631.js:8330-8333 — `r.mockScanCheckin = async (e, t, a, l) => { await ei(); const n = Qt.courtBookings.find((e) => e.qr_token === t); if (!n) throw new Error("E_INVALID_QR_CODE");` — `t` is never checked for being a non-empty string, and `undefined === undefined` satisfies the predicate. The screen passes it straight through without a check: mod/2505.js:47 — `(await (0, _scanCheckin)(e.id, q.qr_token, t, s), L(await (0, _fetchBookingCheckins)(e.id, q.id)));` The field is only ever written at creation time, mod/631.js:7388 — `qr_token: \`PLQR-\${(0, C.randomCode) (20)}\`,` — and the store is persisted across bundle versions under `playora.mock.*.v1`, so rows written before the field existed carry no token. The `PLQR-` prefix is never validated anywhere.	The venue marks a player present on their own screen and the check-in, along with the `booking.attendance` write at mod/631.js:8355-8359, lands on a completely different booking. Nothing in the UI reveals the mismatch; the operator sees the state badge update and moves on. Once the real server at `backendUrl` implements this contract, an empty token parameter becomes a way to address an arbitrary booking.	In mockScanCheckin (mod/631.js:8332) reject before the lookup: `string" == typeof t && /^PLQR-[A-Z0-9]{20}\$/i.test(t)` , throwing `E_INVALID_QR_CODE` otherwise. Guard the caller too — mod. should refuse to call when `q.qr_token` is falsy and surface the b unsceannable.
P-CTEST-20 Medium	./wallet/index.tsx	Error handling / resilience	The whole wallet is blocked when the unrelated friends list fails to load A required resource (the wallet) and an optional one (friends) are fetched with the same all-or-nothing combinator. The friends call should degrade like the court-bookings call on My Bookings does (`catch(() => [])`, mod/2457.js:19). Evidence: mod/2506.js:26 `const [t, l] = await Promise.all([(0, C.fetchWallet)(e.id), (0, C.listFriends)(e.id)]; (K(t), V(l.friends));` — one rejection fails both, and the catch at mod/2506.js:29 sets the single error state. The render then swaps the entire screen out: mod/2506.js:78 `null === F && N ? (0, W.jsxs)(i.default, { ... children: [(0, W.jsx)(x.EmptyState, { icon: "cloud-offline-outline", title: M("error"), body: N }), (0, W.jsx)(y.Button, { title: M("retry"), ... }) ] })`. `listFriends` is needed only by the Send/Request recipient chips (mod/2506.js:251).	If the friends list errors, the user cannot see their balance, their pending money requests, or any transaction history — the entire wallet is replaced by a generic offline card, even though the wallet data loaded fine.	In mod/2506.js:26, wrap the `listFriends` call in `catch(() => ({ fri so only the Send/Request recipient picker degrades, and keep th error state for a `fetchWallet` failure only.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-52 Medium	./wallet/index.tsx	Scalability / payload size	Every wallet load scans the entire ledger five times and returns the user's full transaction history uncapped Balances are derived by replaying the full ledger on every read instead of being maintained as account rows, and the transaction list has no server-side limit or cursor. Both are O(total ledger) per wallet open, and the write path is O(total ledger) per transaction. Evidence: `WI` is a full-table scan per account: mod/631.js:15936 `function WI(e) { let t = 0; for (const a of Qt.walletLedger) for (const i of a.postings) i.account === e && (t += i.delta_files); return t; }`. `ZI` calls it three times plus a fourth full pass of its own: mod/631.js:15982 `for (const e of Qt.walletLedger) { let t = 0; for (const a of e.postings) a.account.startsWith(i) && (t += a.delta_files); ...}` and mod/631.js:16008-16011 `available_files: WI(A.acct.available(a)), pending_files: WI(A.acct.pending(a)) + o, credits_files: WI(A.acct.credits(a)), points: js(a).reduce(...)` . The result carries every transaction ever — mod/631.js:16017 `transactions: n` — and the screen only paginates after the fact: mod/2506.js:517 `.slice(0, G)` with `G` starting at 20 (mod/2506.js:15). Each posting also rewrites the entire ledger: mod/631.js:15934 `await Za(at, Qt.walletLedger)` .	The cost of opening ./wallet grows without bound with platform history, and the payload grows without bound with a user's own history — this runs on focus every time the screen is entered (mod/2506.js:32-36). On the current storage backend it also walks toward the quota failure described above; ported to a server it is a full-table scan per wallet view.	Maintain materialised balances per account (updated inside `YI`, mod/631.js:15924) instead of recomputing in `WI`, and give `moc (mod/631.js:15970) a limit/cursor for `transactions` that ./wallet/in drives from its existing `showMore` control.
P-CSEC-98 Medium	./wallet/index.tsx	KYC / AML control	mockSubmitWalletKyc marks the submitter verified in the same call, with no review Identity verification is self-asserted. The function should persist `status: "pending"` and require an admin review action, as the venue and club verification flows do. Evidence: mod/631.js:16087 — `r.mockSubmitWalletKyc = async (e, t, a) => { / `const i = t.trim(); if (!i) throw new Error("E_ENTER_YOUR_LEGAL_NAME"); / `if (!((0, A.looksLikeCivild)(a)) throw new Error("E_ENTER_A_VALID_12_DIGIT_CIVIL"); / mod/631.js:16093 `const n = { user_id: e, status: "verified", full_name: i, id_masked: (0, A.maskidNumber)(a), submitted_at: new Date(), toISOString(), reviewed_at: new Date().toISOString() };` — `status` is set to `verified` and `reviewed_at` stamped by the same statement that records the submission; no reviewer, no queue, no `pending` state. `looksLikeCivild` is a format test only. That status is the sole gate on cashing out: mod/631.js:16068 `if ("verified" !== (V(e)??.status ?? "unverified")) throw ... new Error("E_IDENTITY_VERIFICATION_KYC_IS_REQUIRED_BEFORE");` in `mockWalletWithdraw` . Every other reviewed artefact in the file has a real two-step flow — `mockAdminReviewVenue`, `mockReviewClubVerification` (mod/631.js:13182, gated by `await sn(e)`), `mockReviewSanction` .	Any user who can reach the withdraw-eligible state supplies a plausible-looking 12-digit string and immediately clears the KYC gate, so funds can leave the platform against an unverified identity. For a Kuwait payments product this is a regulatory control that is present in the UI and absent in the logic, and the audit trail is misleading: `wallet.kyc_verified` is logged with no reviewer.	In `r.mockSubmitWalletKyc` (mod/631.js:16087) write `status: "pe`reviewed_at: null`, and add an admin-only `mockReviewWalletK`userId, approved)` guarded by `await sn(adminId)` and audited th `logAdminAudit`, mirroring `mockReviewClubVerification` .
P-CARCH-53 Medium	./wallet/index.tsx	Concurrency / double spend	Wallet money requests can be paid twice — the pending guard sits outside the serialisation lock The author knew this function needed serialisation — the balance read is deliberately inside `ql(e, ...)` — but left the idempotency guard outside it. Two calls issued in the same tick both clear the status check (the mutation to `paid` happens later, inside the queued callback), then both callbacks run and each writes a `request_payment` transfer. Evidence: mod/631.js:16173-16195 — `r.mockRespondWalletRequest = async (e, t, a) => { await ei(); const i = Qt.walletRequests.find((e) => e.id === t); if (!i i.from_user_id !== e) throw new Error("E_REQUEST_NOT_FOUND"); if ("pending" !== i.status) throw new Error("E_THIS_REQUEST_WAS_ALREADY_RESOLVED"); return a ? (await ql(e, async () => { if (WI(A.acct.available(e)) < i.amount_files) throw ...; (await YI({ kind: "request_payment", postings: (0, A.transferPostings)(A.acct.available(e), A.acct.available(i.requester_id), i.amount_files), ... })), (i.status = "paid"), ...` — the balance check is inside `ql` but the `pending` guard at 16177 is not, and it is not re-checked inside the callback. The screen fires it from a plain button handler with no in-flight guard on the request row: mod/2506.js:53 — `pe = (t, l) => ge() => (0, C.respondWalletRequest)(e.id, t, l), l ? "walletSent" : "walletRequestDeclined")` .	A double tap on Accept, or a React re-invocation of the handler, transfers the requested amount twice from the payer to the requester for a single request. Both postings balance so no invariant check catches it, and the request ends as `paid` once, so the wallet history shows two outgoing transfers against one visible request and the payer has no way to reverse either.	Move the status read and the `pending` guard inside the `ql(e, .` in `mockRespondWalletRequest` (mod/631.js:16181) so the chec mutation are in the same critical section, and disable the Accept c ./wallet/index.tsx while a response is in flight.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-57 Medium	./wallet/index.tsx	Input handling / dead end	Wallet amounts are parsed with parseFloat, so Arabic-Indic digits silently become zero The screen shows the user eastern digits everywhere but parses their input as if it were western. <code>parseFloat("١.٠")</code> is NaN, <code> 0</code> turns it into 0 fils, and the resulting error message talks about allowed limits rather than the digits. The normaliser needed to fix it is already in the bundle and used only on the output side. Evidence: <code>mod/2506.js:39</code> — <code>`ue = Math.round(1e3 * (parseFloat(U)    0))</code>, where <code>`U</code> is the raw text state set by the amount field. <code>`ue</code> is the only amount passed to every wallet mutation: <code>mod/2506.js:224</code> — <code>`onPress: () => ge() => (0, C.walletAddFunds)(e.id, ue, te)</code>, <code>"walletTopupSuccess"</code>), <code>mod/2506.js:298</code> — <code>`() => ge() => (0, C.walletTransfer)(e.id, ae.id, ue, Z)</code>, <code>"walletSent"</code>), <code>mod/2506.js:380</code> — <code>`onPress: () => ge() => (0, C.walletWithdraw)(e.id, ue)</code>, <code>"walletWithdrawn"</code>). The backend then rejects with a range error, <code>mod/631.js:16022-16025</code>: <code>`if ((await ei(), !Number.isInteger(t))    t < A.WALLET_LIMITS.topup_min_fils    t > A.WALLET_LIMITS.topup_max_fils) throw new Error("E_AMOUNT_IS_OUTSIDE_THE_ALLOWED_TOP");`</code>. The app renders numbers in Arabic-Indic form by default — module 912: <code>`c = (n) => { const t = o.numerals; return "eastern" === t    ("auto" === t && "ar" === n); }`</code> with <code>`numerals: "auto"</code> — and module 912 already exports the western-normalising inverse <code>`y = (n) => n.replace(/[\u0600-\u06FF]/g, (n) => String(s.indexOf(n)))`</code>, which the wallet screen never applies to input.	An Arabic-locale user typing an amount on an Arabic keyboard cannot top up, send, request or withdraw. Every attempt fails with "amount is outside the allowed top-up range" against a field that visibly reads as a valid number, and there is no other entry point for these actions — the wallet is a dead end for that user.	Normalise before parsing in <code>./wallet/index.tsx</code> (<code>mod/2506.js:39</code>): <code>n</code> through the <code>western-digit</code> mapper exported by module 912 (and <code>s</code> Arabic decimal separator) before <code>parseFloat`</code> , and set <code>`inputMode="decimal"</code> on the field. Add a client-side range hint <code>s</code> range value is reported against the field rather than as a backend
P-CSEC-99 Medium	./wallet/index.tsx	Unsolicited contact / cross-partition exposure	Wallet transfers and money requests reach any user id, bypassing the audience partition and the messaging privacy setting Money requests are an unsolicited inbound message with attached free text, but they are governed by none of the controls that govern messages. The UI limits the picker to friends (<code>mod/2506.js:26</code> loads <code>`listFriends`</code>), so the constraint exists only in the client. Evidence: <code>mod/631.js:16140-16143</code> and <code>mod/631.js:16109-16110</code> — the only relationship check is a block: <pre>... r.mockCreateWalletRequest = async (e, t, a, i) => { if ((await ei(), t === e)) throw new Error("E_YOU_CANNOT_REQUEST_MONEY_FROM_YOURSELF"); if (!Qt.profiles.some((e) => e.id === t)) throw new Error("E_RECIPIENT_NOT_FOUND"); if (Xd(e, t)) throw new Error("E_YOU_CANNOT_TRANSACT_WITH_THIS_ACCOUNT"); ... }</pre> No <code>`oa(e, t)`</code> audience check and no <code>`al(t).allow_messages`</code> check. The call then pushes a notification into the target's inbox carrying the requester's name and a free-text note: <code>mod/631.js:16157</code> <code>`note: i?.trim().slice(0, 140)    null`</code> and <code>mod/631.js:16170</code> <code>`await Jl(t, "request_payment", "out", a, Td(e))`</code>, where <code>`Jl`</code> (<code>mod/631.js:15953-15965</code>) writes <code>`counterparty_name: n`</code>. The DM path by contrast enforces both: <code>mod/631.js:14649-14654</code> <code>`gl`</code> calls <code>`oa(e, t)`</code> then rejects <code>`allow_messages === "none"</code> and <code>""followers"</code>.	A caller can push a named, note-bearing notification to any user id in the system — across the gender partition, and to users who set "Allow messages: none". That is a harassment and identity-disclosure channel dressed as a payment request, rate-limited only by <code>`requests_per_hour`</code> (<code>mod/631.js:16146-16151</code>).	Call <code>`oa(e, t)`</code> and the <code>`gl`-style <code>`allow_messages`</code> check (<code>mod/631.js:16140</code>) at the top of <code>`mockCreateWalletRequest`</code> (<code>mod/631.js:16140</code>) and <code>`mockWalletTransfer`</code> (<code>mod/631.js:16108</code>), or restrict both to access the way the wallet screen already assumes.</code>

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-100 Medium	631 (NPN candidate matcher Sn)	Behavioural data / location inference	Every profile is silently enrolled as a Need-a-Player candidate, exposing attendance reliability and home-area distance There is no user-facing opt-in or opt-out for being an NPN candidate — the loop takes all profiles unconditionally — and the derived reliability score is a behavioural record about the user that they cannot see, correct, or suppress. The privacy filters that every other people-listing applies are all absent here. Evidence: mod/631.js:5254-5266 — the candidate pool is the entire profile table: <pre>... Sn = (e, t) => { const a = e.npn_radius_km, i = []; for (const t of Qt.profiles) t.id !== e.organizer_id && "admin" !== t.role && ia(aa(t.id), e.audience) && i.push({ id: t.id, name: t.full_name ?? "Player", sports: ..., skill: ... }); ... }</pre> No block check (`Xd`), no `profile_visibility` === "private" check, no `Ho` shadow-ban check, no opt-out flag, and `analyst` is not excluded though `mockSearchPlayers` (mod/631.js:14198) and `mockSearchUsers` (mod/631.js:13543) exclude both roles. Each candidate's home-area coordinates are then resolved and an attendance-derived reliability computed: mod/631.js:5277-5284 <pre>... const i = Sa(r.id), o = (0, f.distanceKm)(i.lat, i.lng, t.lat, t.lng); if (null !== a && o > a) continue; ... l = Qt.bookings.filter((e) => e.user_id === r.id && null !== e.attendance), c = l.length ? l.filter((e) => "attended" === e.attendance).length / l.length : 0.8, ... </pre> and surfaced with the name at mod/631.js:5296-5302 (`display_name`, `distance_km`, `reliability`, `played_before`).	An organizer a user has never met, and a user they have blocked, receives their name, their no-show rate expressed as a reliability fraction, and their distance from the venue derived from their home area. Users who set their profile private are included. `En` (mod/631.js:5303-5326) additionally pushes a notification to every one of them, with no way to stop receiving it.	In `Sn` (mod/631.js:5257) skip profiles where `Xd(e.organizer_id, ``private" === al(t.id).profile_visibility`, `Ho(t.id)`), or `analyst" === an explicit `npn_opt_in` profile flag surfaced on ./preferences.tsx it, and coarsen `reliability` into the same category key already used to display (`reliabilityCategoryKey`, mod/631.js:5455) rather than show raw fraction.
P-CSEC-101 Medium	631 (avatar_mode read sites)	Privacy controls / data exposure	The "initials" avatar setting is honoured in four DTOs and ignored in five others The setting is applied per-DTO by hand rather than at a single projection point, so five of nine call sites forgot it. The screens that matter most for the setting — the match roster, the squad sheet and the lineup, i.e. the places strangers actually look at who is playing — are among the ones that leak. Evidence: Honoured: mod/631.js:8673 `avatar_url: "initials" === a?.avatar_mode && e.user_id !== t ? null : (a?.avatar_url ?? null)` (team members); mod/631.js:14132 (follow list); mod/631.js:14246 `avatar_url: "photo" === u.avatar_mode ? u.avatar_url : null` (player search); mod/631.js:14288 (player profile). Ignored on paths that ship the same photo: - mod/631.js:2907-2908 (match roster) `const o = new Map(Qt.profiles.map((e) => [e.id, e.avatar_url ?? null])); return i.map((e) => Object.assign({}, e, { avatar_url: e.user_id ? (o.get(e.user_id) ?? null) : null }));` - mod/631.js:4052 (squad state) `return { name: t?.full_name ?? "Player", avatar_url: t?.avatar_url ?? null };` - mod/631.js:16435 (lineup slots) `avatar_url: t?.avatar_url ?? null,` and mod/631.js:16459 (bench) - mod/631.js:14113 (follow requests) `avatar_url: t?.avatar_url ?? null,` - mod/631.js:2789 (venue reviews) `author: { full_name: t.full_name, avatar_url: t.avatar_url }`	A user who selects the initials avatar (the default for female-audience accounts, mod/631.js:2297 `avatar_mode: d ? "initials" : "photo"`) still has her photo rendered to every other player on the roster, in the squad list, on the lineup pitch, in the follow-request inbox, and on any venue she reviewed. The setting's own copy, avatarModeSub = "Connections always see your chosen photo.", implies non-connections do not.	Add a single helper in 631 — `avatarFor(subjectId, viewerId)` returns when `avatar_mode` === "initials" and viewer ≠ subject — and ro sites through it, starting with mod/631.js:2907, 4052, 14113, 16435 and 2789.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-102 Medium	671 (fetchMatchParticipants) / 631	Data exposure / private-group membership	Match participant list is exported with no caller parameter, returning raw booking rows for any game The function has no viewer argument, so it cannot be authorized at all — not by the organizer check `zi` (mod/631.js:2967), not by the audience check `ra`, not by game visibility. It is reachable through the same facade every consumer screen imports. Evidence: mod/631.js:4204-4214: ... <pre>const Qi = async (e) => { (await ei(), await Oi(e)); const t = Qt.bookings.filter((t) => t.game_id === e); return { confirmed: t.filter((e) => "confirmed" === e.status).sort(xi), reserved: ..., pending: ..., waitlist: ..., }; }; r.mockGetMatchParticipants = Qi; ...</pre> The booking rows are returned verbatim — they carry `user_id`, `display_name`, `status`, `attendance`, `reserved_until`, `group_id` and `created_at` (mod/631.js:13704-13716). Exposed on the shared facade with the same single argument: mod/671.js:448 `e.fetchMatchParticipants = (o) => t.store.mockGetMatchParticipants(o);`.	Any caller with a game id obtains the complete participant roster of that match, private matches included, with each person's attendance history flag and their group linkage. Because the raw row is returned rather than a projection, any field added to bookings later leaks automatically.	Change to `Qi(gameld, callerId)`, call `zi(hi(gameld), callerId)` (or confirmed participants), and return a projection listing only `user_` `display_name` and `status` rather than spreading the booking ro mod/671.js:448.
P-CSEC-103 Medium	671 API facade (fetchTeams) / 631 mock backend	Audience partition leakage	Clan search returns clans from the other audience partition, unlike every sibling query `mockGetTeams` is the only clan-listing function in the backend that omits the audience partition. Individual clan reads do enforce it (`ra(t, a.audience)` at mod/631.js:8636 throws `AUDIENCE_MISMATCH`), so the partition is clearly intended to cover clans — the list endpoint simply forgot it. Evidence: mod/631.js:8611-8626 filters only on status and privacy — no audience predicate: ... <pre>8611 r.mockGetTeams = async (e = {}, t) => { 8612 (await ei(), await mo()); 8613 let a = Qt.teams.filter(8614 (e) => 8615 !!(("suspended" !== e.status (t && ro(t))) && !!(("private" !== e.privacy no(e.id, t) (t && ro(t))), 8616); ... Every comparable query applies `na`/`ia` — mod/631.js:9180-9183: ... 9180 let i = na(9181 Qt.teams.filter((e) => "active" === e.status && "private" !== e.privacy), 9182 t, 9183); ... and mod/631.js:10057 `.filter((e) => "active" === e.status && "private" !== e.privacy && ia(e.audience, a))`, and mod/631.js:9992 in `mockGetRecentBattleResults`. The helper is right there at mod/631.js:759-762: `na = (e, t) => { const a = aa(t); return e.filter((e) => ia(e.audience, a)); };` It is exposed publicly at mod/671.js:547: `e.fetchTeams = (o, c) => t.store.mockGetTeams(o, c);`</pre>	`fetchTeams` returns every women's clan (`wteam-*`, audience `female`) — name, home area, crest colour, member count and ladder score — to a male-audience caller, and vice versa. No consumer screen calls it today, so nothing leaks in the shipped UI, but it is a published function on the API facade: the first search or directory screen wired to it, or any client of the `/rpc` endpoint once 631 moves server-side, silently exposes a roster the women's partition exists to protect. In a Kuwait-market women's-sport context that is a safety issue, not a cosmetic one.	Wrap the filter in mod/631.js:8613 with `na(...)`, matching `mockGetTeamRankings` at mod/631.js:9180, so the listing enfor same partition the single-clan read already enforces. Add a regre to tools/rules.mjs asserting that no clan-listing function returns a t audience mismatches the caller's.
P-CSEC-104 Medium	index.html line 6 (CSP) + serve.json	Clickjacking	No frame-ancestors in the CSP and no X-Frame-Options from the host, so the wallet and payment screens can be framed Every other directive is locked down to `none`, which makes the omission look deliberate rather than accidental — but framing is the one thing left open, and the app has one-tap destructive actions behind it. Evidence: index.html line 6 — `content="default-src 'none'; script-src 'unsafe-inline'; style-src 'unsafe-inline'; img-src data: blob: https://*.basemaps.cartocdn.com; font-src data: blob: connect-src 'self' https://api.rush-x.xyz; base-uri 'none'; form-action 'none'` — there is no `frame-ancestors` directive, and `default-src` does not cover it. serve.json lines 4-6 set only `Cache-Control: no-cache` on index.html; a HEAD against the deployed command returns no X-Frame-Options header.	A third-party page can iframe rush-x.xyz, overlay it, and get a signed-in user to tap through .pay/[id].tsx (Pay now) or .wallet/index.tsx (withdraw/transfer) without seeing what they are confirming. Sessions last 30 days, so a returning user is almost always signed in.	Add `frame-ancestors 'none'` to the CSP in index.html line 6 and Options: DENY` to the serve.json headers block for index.html.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-58 Medium	mod/631.js	Silent failure	mockUpdateProfile silently succeeds when the profile row does not exist Every other write in 631 throws a mapped code for a missing subject ('mockGetPrivacy' throws 'E_PROFILE_NOT_FOUND' at mod/631.js:990; 'mockGetPlayerProfile' throws 'not_found' at mod/631.js:14274). This one resolves, so there is no way for a caller to detect the no-op. Evidence: mod/631.js:2366-2368 `r.mockUpdateProfile = async (e, t) => { await ei(); const a = Qt.profiles.findIndex((t) => t.id === e); if (a >= 0) { — the entire body is inside the guard, with no `else` and no throw; the function resolves normally when the id is unknown. Callers treat resolution as success: mod/2476.js:118-120 `await T(), M.back());` and mod/1805.js:569-573 `await \$((), ... Te(1));`, both of which only surface problems via `catch` (mod/2476.js:121, mod/1805.js:574).	A guest (id `guest:<audience>`, no profile row — mod/630.js:74) fills in preferences or the profile editor, taps Save, the button spinner stops, the sheet closes and the router navigates back — and nothing was written. Same for any signed-in user whose profile row failed to provision. The user believes their name, bio, avatar and sports were saved.	Throw `E_PROFILE_NOT_FOUND` from the missing-row branch mod/631.js:2368 so the existing localised message (mod/674.js:1 the alert handlers already present at mod/2476.js:121-122 and mod/1805.js:574-575.
P-CARCH-54 Medium	mod/631.js	Side effects on read	Reading someone's passport writes to their account: unlocks achievements and pushes notifications Achievement evaluation is a write triggered by a GET-shaped read on behalf of a third party. There is no idempotency key beyond "already unlocked", no rate limit, and no restriction to self-reads. Evidence: mod/631.js:6611 `(await ei(), await tn(), await ir(e));` — `e` is the subject, not the viewer. `ir` mutates the subject's records: mod/631.js:6500-6516 `Qt.achievements.push({ user_id: e, achievement_id: r.id, unlocked_at: new Date().toISOString() }), ... Qt.notifications = [{ id: ea(), user_id: e, type: "achievement", emoji: r.emoji, achievement_key: r.key, read: !1, created_at: ... }, ...Qt.notifications],` and persists at mod/631.js:6543 `n && (await Promise.all([Za(Ut, Qt.achievements), Za(\$, Qt.notifications)]));`. `dr` is reached from any viewer: mod/2473.js:17 `fetchPassport(t, o?.id)` with an arbitrary `t`, and from mod/631.js:14318 `const u = await dr(t, e);` inside mockGetPlayerProfile.	A stranger opening your passport can cause achievement unlocks and notification rows to be created on your account at a time you did not act — notifications land in your tray attributed to nothing you did. Moved to a server, this is an unauthenticated write amplification: repeatedly fetching many users' passports drives writes to every one of their notification lists. It also makes passport reads non-cacheable and makes the two persisted arrays a contention point under concurrency.	Split the evaluation out of the read: call `ir(e)` only when the view subject (`t === e` at mod/631.js:6611), and move third-party achievement recomputation to the events that actually change the inputs (after recorded, match completed, booking confirmed).
P-CSEC-105 Medium	mod/631.js	Consent withdrawal / data retention	There is no way to delete an upload, a clip, or a story, despite an explicit consent checkbox and player tagging The upload flow collects an affirmative consent ('consentLabel', required to enable the Generate button at mod/2454.js:116) and supports tagging up to eight players into generated reels, but the data model offers no revocation: an owner cannot delete their upload or any derived clip, a tagged player cannot request removal, and a story author cannot take down a story before its 24-hour expiry. Consent that cannot be withdrawn is not consent. Evidence: No delete path exists for any media or story collection anywhere in 631 - the only statements touching these arrays are the initialisers and hydration assignments at mod/631.js:2057-2060 and mod/631.js:2080-2081: (Qt.mediaUploads = Xt), (Qt.mediaClips = ea), (Qt.mediaShares = ta), (Qt.mediaReports = aa), (Qt.stories = Sa), (Qt.storyViews = Ea), plus the pushes. There is no mockDelete* for media or stories, and none of ./media/[id], ./media/index, ./media/upload, ./stories/[id] renders a delete affordance. Meanwhile consent and third-party tagging are explicit - mod/2454.js:104 and mod/2454.js:107: consent: !0, player_tags: X ? [e.id] : [], stored at mod/631.js:13387 and mod/631.js:13390: consent: !0, player_tags: (t.player_tags ?? []).slice(0, 8), Stories only age out logically - mod/631.js:14817 filters at read time and never removes: const hl = () => Qt.stories.filter((e) => new Date(e.expires_at).getTime() > Date.now()),	A user who posts a photo story they immediately regret has no recourse for 24 hours. A player tagged into someone else's player_reel has no mechanism to be untagged or removed. An owner who set a match to 'private' after realising it should not have been shared cannot delete the clips that were already generated - and, per the story finding above, those clips may already have been republished. Operationally, nothing is ever pruned, so every upload, clip, share row, report row, story and story-view row accumulates in localStorage for the life of the browser profile.	Add mockDeleteMediaUpload / mockDeleteClip / mockDeleteStory cascading to mediaClips, mediaShares, mediaReports and storyViews on owner-or-admin, and surface delete actions on ./media/[id] and ./stories/[id]. Add an untag path for entries in player_tags. Prune entries in hl() (mod/631.js:14817) rather than only filtering them.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-55 Medium	mod/631.js	Storage growth / lost updates	Every media and story write serialises the entire table, nothing is pruned, and two open tabs clobber each other Each of these tables is read-modify-write-whole-array with no row-level store and no optimistic concurrency. Two tabs each hydrate their own Qt and each write the complete array back, so the later write silently discards everything the other tab did. Nothing prunes mediaShares, mediaReports, stories or storyViews, and the read helpers scan the full arrays on every call. Evidence: mod/631.js:1766-1768, Za falls back to a whole-array JSON rewrite for any key outside the small Ga set: const a = l.default.setRows; if (a && Ga[e] && Array.isArray(t)) return void (await a(e, t)); await l.default.setItem(e, JSON.stringify(t)); mod/631.js:1682 shows Ga holds only six keys, none of them media or stories: Ga = { [W]: "asc", [we]: "asc", [at]: "desc", [\$]: "desc", [me]: "asc", [ue]: "asc" }, while the media/story keys are m/ut/wt/pt/We/Ke (mod/631.js:543-544, mod/631.js:563-566). So a single clip view rewrites the whole clips table - mod/631.js:13424: t && ((t.views += 1), await Za(mt, Qt.mediaClips)); a share rewrites two tables - mod/631.js:13441-13442; and each 1.5s poll tick can rewrite the whole uploads table - mod/631.js:13366: (!(e.status = "processing"), (e.progress = a), await Nd()), The per-upload in-flight guard is per-tab only - mod/631.js:13289 `Md = new Map()` and mod/631.js:13294 `const t = Md.get(e.id); if (!t) return t`.	With two tabs open on /media/[id] - which is easy, since the screen is reached by a shareable URL - both poll every 1.5 seconds, both can run clip generation for the same upload (Md is not shared across tabs), and each Za(mt, ...) overwrites the other's clip set. The result is duplicated or orphaned clips, and orphaned clips are exactly the input that makes mockGetClipSignedUrl fail open. Independently, storyViews grows by one row per (story, viewer) pair forever, mediaShares and mediaReports never shrink, and clip-view traffic costs an O(total clips) serialisation per view - the store hits E_STORAGE_FULL well before the dataset is interesting, which in turn triggers the stuck-processing state.	Register the media and story keys in Ga (mod/631.js:1682) so the row-level setRows path instead of whole-array setItem, or add a check in Za (mod/631.js:1763) so a stale whole-array write is rejected than applied. Prune expired stories and their storyViews inside hl (mod/631.js:14817), and cap or age out mediaShares and media
P-CQUAL-59 Medium	mod/631.js (push copy, surfaced on ./notifications.tsx)	Localisation / duplicated logic	Push copy is hardcoded English and generic for most notification types Notification copy exists in two divergent implementations, one localised and one not, with no shared source. Types added to the in-app renderer are not added to the push renderer, so they fall through to the default. Evidence: mod/631.js:2523-2567 `const Si = (e) => { const t = e.venue_name ?? "your match"; switch (e.type) { case "award_voting_closing": return { title: "Last call to vote", body: "MVP voting for \${t} closes in a few hours." }; ... default: return { title: "Playora", body: "Something new is waiting for you." }; } }` — 15 cases, all literal English, against the 41 notification types actually produced. The in-app renderer is a completely separate, fully localised implementation at mod/2458.js:232-441 (`x = (t, n) => { if ("match_reminder" === t.type) { ... n("notifReminderTitle") ... } }`). A third English-only body is baked into the payload itself at mod/631.js:10818 `summary: "\${s.name} declined your challenge."`, which mod/2458.js:391 renders verbatim as the notification body.	An Arabic user's lock screen is entirely English. More than half the types — including `sanction_issued`, `payment_request`, `booking_decision`, `refund_issued`, `payment_forfeited` and `award_voting_open` — arrive as "Playora / Something new is waiting for you", which tells the user nothing and trains them to ignore pushes. `team_announcement` bodies are English strings baked into the stored record, so they cannot be localised later even in-app.	Derive push copy from the same key-based table as mod/2458.js pass the recipient's locale, so `Si` (mod/631.js:2523) returns translated strings; store i18n keys plus parameters in `team_announcement` (mod/631.js:10818) instead of an English sentence.
P-CARCH-56 Medium	mod/631.js (shared backend: mockEditFutureOccurrences)	Payment integrity / price consistency	Changing a series price rewrites future occurrences that already hold paid players, with no top-up or refund Single-match editing correctly refuses to change price once players have joined; the series editor does it for every future occurrence at once, without reconciling the payments already taken. Existing paid rows keep the old amount, existing pending rows keep the old amount, and already-confirmed free players are never billed at all. Evidence: mod/631.js:6104-6109 — the price is overwritten on every future scheduled occurrence: `for (const t of Kn(e)) { const e = new Date(t.starts_at); ((t.starts_at = Hn(e, r.start_minutes)), (t.ends_at = Hn(e, r.end_minutes)), (t.duration_minutes = r.end_minutes - r.start_minutes), (t.skill_level = r.skill_level), (t.notes = r.notes), (t.price_kwd = r.price_kwd));` and `Kn` selects occurrences with confirmed rosters (mod/631.js:6087-6091): `Qt.games.filter((t) => t.series_id === e && "scheduled" === t.status && new Date(t.starts_at).getTime() > Date.now())`. Nothing in the function touches `Qt.payments`. Seat charges are only ever minted at join time — `Hr` is called from `ji` (mod/631.js:3086), `Di` (mod/631.js:2636) and `mockAcceptReplacement` (mod/631.js:12231) — and entitlement is judged against the current price at mod/631.js:7656: `xr = (e, t) => !(Number(e.price_kwd) > 0)    "paid" === Gr(e.id, t)?.status;`. Contrast `mockUpdateMatch` (mod/631.js:3675), which deliberately offers no price field.	Players on the same occurrence pay different amounts for the same seat depending on when they joined, and no one is topped up or refunded the difference when the organizer moves the price. Raising the price on a previously free series gives every existing participant a permanent free ride — `xr` returns true for them only because their seat predates the price — while new joiners pay, which is both a revenue hole and visibly unfair to the paying players.	Reject a `price_kwd` change in `mockEditFutureOccurrences` (mod/631.js:6098) for any occurrence that already has a confirmed participant, matching the stance `mockUpdateMatch` takes; or, if must be allowed, issue a delta payment request or partial refund participant and notify them of the amount, not just "series scheduled"

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-57 Medium	mod/631.js (shared backend: mockGatewayWebhook)	Booking/payment consistency	The gateway webhook resurrects cancelled bookings and can push a match past capacity The webhook is the asynchronous twin of `Sr` but skips both of its guards. It also has no equivalent of the `Wr` fallback: if the payer has no booking row at all, the payment is set to "paid" at 7961 and the function returns, leaving money taken with no seat and no refund. Evidence: mod/631.js:7969-7972 — `const e = Qt.bookings.find((e) => e.game_id === n.game_id && e.user_id === n.payer_id); e && "confirmed" !== e.status && ((e.status = "confirmed"), (e.updated_at = new Date().toISOString()), await Za(W, Qt.bookings));` — no exclusion of "cancelled"/"rejected" and no capacity test. The interactive path is written defensively by comparison, mod/631.js:7867-7869: `const t = Qt.bookings.find((t) => t.game_id === i.game_id && t.user_id === e && "cancelled" !== t.status && "rejected" !== t.status);`, and when it finds nothing it hands off to `Wr`, which checks the roster before seating (mod/631.js:7749): `Qt.bookings.filter((e) => e.game_id === a.id && "confirmed" === e.status).length < a.max_players`.	A player who starts a 3-D Secure flow and then leaves the match (or is removed) before the callback lands has their cancelled booking flipped back to confirmed by the webhook — potentially as the (max_players + 1)th player, over-filling the match and the court. The inverse case leaves a confirmed payment with no seat and no automatic refund; only a later `mockReconcileSeatPayments` sweep, which nothing schedules, would notice.	In `mockGatewayWebhook` (mod/631.js:7969), reuse the same Id as `Sr` — exclude cancelled and rejected rows — and route the no-t case through `Wr` so the roster check and the refund fallback both Run the whole block inside `Xt(n.game_id, ...)` as `mockPayReq` 7822.
P-CARCH-58 Medium	mod/631.js (shared store — affects every write path)	Data integrity	Mutations mutate the in-memory store before persisting and never roll back, so a quota failure permanently desynchronises memory from disk There is no transaction boundary: the `Qt` mutation and the persistence call are two independent steps and the error path unwinds only the second. Evidence: mod/631.js:14392-14397 — `return (Qt.posts.unshift(i), await Za(Ce, Qt.posts), ...)`. mod/631.js:2589 — `((Qt.notifications = [e, ...Qt.notifications]), await Za(\$, Qt.notifications), Ei(e))`. mod/631.js:15934 — `return (Qt.walletLedger.unshift(t), await Za(at, Qt.walletLedger), t)`. In every case `Za` (mod/631.js:1763-1778) throws `E_STORAGE_FULL` or `E_COULD_NOT_SAVE_CHANGES` *after* the array has already been mutated, and no caller restores the previous array.	Once the origin quota is reached (which takes only a few image posts — see the inline-base64 finding), the user sees an error toast but the app keeps showing the post/booking/ledger entry as if it succeeded, for the rest of the session. A wallet ledger entry that exists in memory but not on disk means the displayed balance is wrong until reload, and the next *successful* write to that same key silently persists the phantom entry alongside the real one.	In the hot write paths, snapshot the array reference (or the insert before mutating and restore it in a catch around `Za` before rethrowing e.g. `const prev = Qt.posts; Qt.posts = [...prev]; try { await Za(C) } catch (e) { Qt.posts = prev; throw e }`. Better: give `Za` a `mutate` signature so the swap only happens after a successful write.
P-CQUAL-60 Medium	mod/643.js (audit log)	Lost updates / write amplification	The audit log is a read-modify-write of a 500-entry JSON array with no serialisation, called from 280 sites including concurrent Promise.all paths Classic lost-update race: two in-flight `h` calls both read the same array, both unshift their own entry, and the second `setItem` overwrites the first. There is also no batching — each of the 280 call sites parses and re-serialises up to 500 entries. Evidence: mod/643.js:32-46 — `const h = async (e, a, i) => { const l = await t.default.getItem(e), u = l ? JSON.parse(l) : []; u.unshift(...); await t.default.setItem(e, JSON.stringify(u.slice(0, a))); }` — an await sits between the read and the write, and nothing queues concurrent calls. mod/643.js:47-51 — `logAudit` awaits `h` directly. mod/631.js contains 280 `logAudit`() call sites. A concurrent path is reachable: mod/631.js:2688 `await Promise.all(n.map((e) => Oi(e.id)))` fans out `Di`, which calls `logAudit` for each expired reservation (mod/631.js:2622) and each waitlist promotion (mod/631.js:2635).	Audit entries are dropped silently under exactly the conditions that matter most (a burst of waitlist promotions or reservation expiries when a game fills). Since `logAudit` sits in the critical path of ordinary user actions — liking a post (mod/631.js:14504), sending a DM (mod/631.js:14739) — every such action also pays a full parse-and-reserialise of the audit array.	Serialise `h` behind a per-key promise chain the way `ql` does for writes (mod/631.js:15915-15923), and buffer entries in memory with a debounced flush so a burst of 20 audit writes costs one `setItem` 20.
P-CTEST-21 Medium	mod/671.js (API facade, shared)	Operational tooling / dead code	The seat-payment reconciler and consistency audit are exported but called from nowhere The repair job that exists specifically to resolve half-finished payments is not wired to anything — no admin screen, no startup hook, no interval. The anomalies it detects therefore accumulate untouched. Evidence: mod/671.js declares and defines them — `e.reconcileSeatPayments = () => t.store.mockReconcileSeatPayments();` and `e.auditSeatConsistency = () => t.store.mockAuditSeatConsistency();` — and the implementation is substantial: mod/631.js:8038-8089 (`Jr`, a 100-row cursored sweep that re-seats or refunds orphaned paid seats and voids abandoned holds, persisting its cursor at mod/631.js:8088). Searching the shipped bundle for `reconcileSeatPayments` returns exactly three hits: the export declaration list, the facade definition, and the backend assignment `r.mockReconcileSeatPayments = Jr;` (mod/631.js:8090). No screen and no scheduler invokes it.	The exact failure classes this audit found have a written remedy that never runs: orphaned `paid` seat payments with no booking (mod/631.js:8055) are never re-seated or refunded, and abandoned holds (mod/631.js:8065-8083) are never voided. Shipping a data-repair path with no caller also means it has never executed against production data, so its correctness is unverified.	Either wire `reconcileSeatPayments` and `auditSeatConsistency` to admin-triggered control and/or a scheduled run and cover them in regression suites under tools/, or delete them from mod/671.js and mod/631.js so the surface does not imply a safety net that is not there.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-59 Medium	mod/673.js	Availability / data durability	One network blip permanently downgrades the app to the local store, so privacy, consent and deletion writes stop reaching the server There is no retry, no backoff, no recovery to remote, and no signal to the UI that writes are now local-only. A 900 ms probe on a slow mobile connection is enough to pin the whole session to the mock store from first load. Evidence: mod/673.js:65-67 <code>`}` catch { return ((o = "mock"), console.warn("[[backend]] unreachable, falling back to local store"), l()); }`</code> — the module-level mode variable is set to <code>"mock"</code> on the first fetch rejection. The gate never re-evaluates once the mode is non-null: mod/673.js:26-34 <code>`k = () => null != o ? Promise.resolve() : (l (l = {async () => { c = n(); o = c && (await u(c, 900)) ? "remote" : "mock"; }(), l);`</code> and mod/673.js:53-54 <code>`e.rpc = async (t, n, l) => { if ((await k(), "remote" != o)) return l();`</code>. The initial health probe is also a hard 900 ms timeout (mod/673.js:20 <code>`l = await fetch(`\${t}/health`, { signal: o.signal })`</code> with <code>`setTimeout(() => o.abort(), n)`</code> at mod/673.js:19).	After one transient failure — or one slow first request — every subsequent write in this group goes only to the browser's localStorage: privacy visibility, message permissions, consent grants and revocations, blocked accounts, and account deletion. The user sees each one succeed. Nothing is ever sent to the server, and clearing site data destroys it all. Consent and deletion are precisely the writes that must be durable.	Make the fallback per-request rather than sticky: leave <code>`o`</code> uncaught at mod/673.js:66 and retry the probe with backoff, or expose degraded state so screens can show an offline banner and queue a minimum, raise the 900 ms probe timeout at mod/673.js:32 and retry the next write after a fallback.
P-CQUAL-61 Medium	mod/673.js (every screen's writes go through this)	Error handling	The RPC transport parses the response body without checking the status, so a 502 surfaces as a raw JSON parse error Only a network-layer rejection reaches the fallback. An HTTP error that returns an HTML body — a Railway cold-start 502, a proxy 504, a rate-limit page — resolves the fetch, then throws a SyntaxError from <code>`json()`</code>, which mod/674.js:316-322 passes through untouched because it is not a mapped code. Evidence: mod/673.js:59-70 — <code>`try { y = await fetch(`\${c}/rpc`, {...}) } catch { return ((o="mock"), console.warn(...), l()) } const f = await y.json(); if (!f.ok) throw new Error(f.error ?? "Server error.");`</code> — there is no <code>`y.ok`</code> check between the fetch and the <code>`json()`</code>. mod/913.js:39-49 shows the correct shape (<code>`if (429 === e.status) throw ...; try { o = await e.json() } catch { throw new Error("SERVER_TROUBLE")`</code>) but the main RPC path does not use it.	During any API hiccup every screen in the app shows <code>`Unexpected token '<', '<!DOCTYPE'...</code> is not valid JSON` in its error banner instead of a localized message, and the request is not retried or degraded — it just fails. <code>`f.error`</code> from a real server also arrives as a free-text string with no <code>`code`</code>, so it cannot be localized into Arabic either.	In mod/673.js, check <code>`y.ok`</code> (and status 429) before parsing, wrap <code>try/catch</code> mapping to a known code, and have the server return <code>`{ error:"E_SOME_CODE"}`</code> so mod/674.js can localize it.
P-CTEST-22 Medium	mod/9001.js	Error-state coverage	The whole profile/privacy group ships without the shared error-gate helper, so no screen in it has a retry path A shared, localised, accessible error surface was built for exactly these cases and no screen in this group uses it. Every screen instead invents its own — or no — failure state, and the profile tab swallows nine independent load failures with empty catches so partially-loaded screens render as if the missing data were zero. Evidence: mod/9001.js:72-123 defines <code>`GateScreen`</code> with <code>`kind: "denied"   "error"`, a back control that falls back to <code>`T.replace("/(tabs)")`</code> (mod/9001.js:77), and a retry button (mod/9001.js:117 <code>`lz`</code> && <code>o`</code> && (0, k.jsx)(b.Button, { title: E("retry"), fullWidth: 10, onPress: o })), plus <code>`classifyError`</code> at mod/9001.js:59-63 and <code>`RetryButton`</code> at mod/9001.js:65-71. Module 9001 appears in the dependency array of none of the eight modules in this group: mod/2476.js:198, mod/2477.js:271, mod/2478.js:330, mod/2484.js:120, mod/2485.js:279, and the multi-line dependency arrays at the end of mod/1805.js, mod/2473.js and mod/2475.js. The consequences are the individually filed dead ends at mod/2477.js:30, mod/2473.js:30 and mod/2475.js:53, and the swallowed failures at mod/1805.js:44-79 (nine <code>`catch(() => {})`</code> handlers) and mod/2478.js:16.</code>	Every failure mode in the profile, passport and privacy surface is either invisible (profile tab shows 0 followers, 0 friends, 0 wallet when those loads fail) or terminal (permanent spinners, a wrong "private account" message). No screen offers retry, so a user whose first load failed has no way to recover except backing out and re-entering — and on /privacy-controls not even that. The uniform absence across eight routes indicates error paths were never exercised in review or test.	Adopt <code>`GateScreen`/`classifyError`</code> from mod/9001.js across the group, add a <code>`retry`</code> button to error+retry for mod/2477.js:30, mod/2473.js:30 and mod/2475.js:53, and inline <code>`RetryButton`</code> for the individually-failing panels on mod/1805.js:44-79 instead of empty catches. Add a smoke-harness case per route to ensure that each fetch to reject and asserts a retry control is present.
P-CQUAL-62 Medium	mod/9001.js (shared role-gate/error helpers)	Inconsistent error handling	A shared retry/denied screen exists but exactly one of the ~70 consumer routes uses it, so error handling is re-invented (or omitted) per screen The correct pattern was built and then not adopted. Each screen invents its own loading/error shape, which is why <code>`/pay/[id]`</code> (mod/2474.js:32-50) and <code>`/game/[id]`</code> (mod/2379.js:104-122) get retry-and-back right while <code>`/group/[gameld]`</code>, <code>`/teams/battle/[battleld]`</code>, <code>`/privacy-controls`</code>, <code>`/passport/[id]`</code> and <code>`/teams/league`</code> render unrecoverable spinners. Evidence: mod/9001.js:59-63 defines <code>`classifyError`</code> and mod/9001.js:72-123 defines <code>`GateScreen`</code>, which renders a back button, an <code>`EmptyState`</code>, a <code>`retry`</code> button and a <code>`goBack`</code> button. Grepping the dump for <code>`GateScreen useRoleGate classifyError`</code> matches only mod/9001.js itself and mod/2503.js (<code>`./venue/apply.tsx`</code>); checking which modules declare 9001 in their dependency array confirms the same single consumer. Meanwhile 43 route modules call <code>`useFocusEffect`</code>, and nine of them contain no <code>`catch`</code> at all (mod/1835.js, mod/1837.js, mod/1840.js, mod/2375.js, mod/2456.js, mod/2473.js, mod/2488.js, mod/2500.js, mod/2501.js).	Error recovery quality varies per screen with no way to reason about it, and every new screen starts from zero. The nine catch-free focus effects each produce an unhandled promise rejection plus a stuck screen on the first store error — and a store error is not exotic here, since one bad localStorage key fails every read (see the hydration finding).	Make <code>`GateScreen`/`classifyError`</code> (mod/9001.js) the required load for all screens, add a small <code>`useResource(fetcher)`</code> hook returning <code>`{data, error, loading}`</code> that screens render through <code>`GateScreen`</code> when <code>`error`</code> is set. Standardize the nine catch-free modules listed above.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-107 Low	./(auth)/sign-in.tsx and ./(auth)/sign-up.tsx (defect in mod/631.js)	Credential disclosure	In production a randomly generated OTP equal to "123456" skips the SMS send and is returned to the client and displayed on screen The branch uses <code>'a !== sa'</code> (is the code the literal string "123456") as a proxy for "are we in demo mode". <code>`randomSixDigit`</code> is uniform over 000000-999999, so roughly one request in a million produces "123456" in a fully live deployment and takes the demo branch. Evidence: mod/631.js:771 defines the demo code and mod/631.js:772 the generator: <code>`sa = "123456";`</code>, <code>`la = () => (Fa() ? sa : (0, c.randomSixDigit()))`</code>. <code>mockRequestOtp</code> then branches on value equality rather than on the demo flag — mod/631.js:834-844: <pre>... 836 await (0, w.logAudit)("auth.otp_requested", null), 837 a !== sa) 838 } { 839 try { 840 await ca(t, a); ... 845 return { sent: l0, demo_code: "" }; 846 } 847 return { sent: l0, demo_code: a }; ...</pre> The screens render whatever <code>`demo_code`</code> comes back, unconditionally: mod/613.js:136-141 (<code>`q && U && ... children: E("otpDemoHint", { code: U })`</code>) and mod/917.js:513-520.	For that request no SMS is sent, so the legitimate user never receives a code; simultaneously the code is returned in the API response and rendered on screen. The failure is silent — the UI reports the code as sent — and the user is left waiting for an SMS that will never arrive until the code expires.	Branch on the demo flag directly at mod/631.js:837 (<code>`Fa()`</code> is already a predicate used at line 772 and 773) rather than comparing the <code>ge</code> value, and have the screens gate the demo hint on the same flag on a non-empty <code>`demo_code`</code> .
P-CQUAL-64 Low	./awards/[gameld].tsx	Misleading copy	The awards screen tells a non-participant "No eligible players" when the ballot is full of them One string is doing three jobs: "you are not eligible to vote", "there are no nominees", and "nobody voted". Every award row is simultaneously rendered <code>`disabled`</code> (mod/1835.js:153) with no explanation for why it cannot be tapped. Evidence: mod/1835.js:78-82 <code>`"voting" === N.state && (0, z.jsx)(u.default, { ... children: N.can_vote ? H("matchAwardsSub") : H("noNominees") })`</code>, where <code>`noNominees`</code> is "No eligible players" (bundle-src/909.js). <code>`can_vote`</code> is false for anyone outside the participant set: mod/631.js:15092 <code>`can_vote: "voting" === c && r && !i && Ol(i)`</code> with <code>`r = n.includes(e)`</code> from <code>`Sl(t)`</code> (mod/631.js:15055-15056). The same string is reused for a published ballot with no winners at mod/1835.js:85-89 <code>`0 === N.winners.length ? (0, z.jsx)(v.EmptyState, { icon: "trophy-outline", title: H("awardResults"), body: H("noNominees") })`</code>, a state that occurs when nobody voted (<code>`tallyVotes`</code> skips awards with a top count of zero).	A spectator or a player whose booking was cancelled opens a ballot with ten nominees and is told there are no eligible players, then finds every row inert. A published ballot that nobody voted in reports the same thing, so the organizer cannot tell whether voting failed to open or simply got no turnout.	Split the copy at mod/1835.js:81 into a not-eligible message and nominees message keyed off <code>`N.nominees.length`</code> , and add a <code>dis</code> votes were cast" string for the empty-winners case at mod/1835.js:909.
P-CQUAL-65 Low	./contact.tsx	Dead-end navigation	The contact screen's social links point at the platforms' generic homepages These are placeholder URLs that were never replaced with the product's own accounts, and the failure is swallowed by the empty <code>`catch`</code>. Evidence: mod/2373.js:245-250 <code>`const z = [ { icon: "logo-instagram", url: "https://instagram.com" }, { icon: "logo-tiktok", url: "https://tiktok.com" }, { icon: "logo-youtube", url: "https://youtube.com" }, { icon: "logo-facebook", url: "https://facebook.com" } ]`</code>, opened at mod/2373.js:204 <code>`onPress: () => o.default.openURL(t.url).catch(() => {})`</code> under the heading <code>`B("followUs")`</code>.	A user who taps "Follow us" is dropped on instagram.com's logged-out landing page rather than Playora's profile — visible on a production marketing surface, and the only outbound brand links in the consumer app.	Replace the four URLs at mod/2373.js:245-250 with the real account handles, or hide the block until they exist; move them into the run block in index.html alongside the other deployment-specific value are not buried in the bundle.
P-CQUAL-66 Low	./discover.tsx	Privacy controls	Player search hides home area based on the viewer's audience instead of the listed player's The condition reads as "hide areas for female users" but is evaluated against the wrong party. It should be <code>"female" === aa(u.id)`</code> — or better, driven by the subject's own privacy setting — not by who is looking. Evidence: mod/631.js:14186-14187 binds the flag to the caller: <pre>... d = aa(e), l = "female" === d, ...</pre> and mod/631.js:14249 applies it to every listed player: <pre>... area: l ? null : (u.home_area ?? null), ...</pre> <code>`e`</code> is the viewer (<code>`mockSearchPlayers = async (e, t = {}) =>`</code>, mod/631.js:14174); <code>`u`</code> is the profile being listed. The same DTO gets the subject-keyed form right one line earlier — mod/631.js:14246 <code>`avatar_url: "photo" === u.avatar_mode ? u.avatar_url : null`</code>.	Every female user of /discover sees <code>`area: null`</code> for all results, so the area column is permanently blank for them even though the screen offers an area filter (mod/2374.js:125) — a broken screen for half the user base. The protection the line was written to provide only ever holds as a side effect of the partition; the moment a cross-partition grant or an <code>"open"</code> audience is in play, a female player's home area is disclosed to a male viewer.	At mod/631.js:14249 key the redaction to the subject — compute as <code>"female" === aa(u.id)`</code> , or better, read the subject's own privacy rule is explicit rather than inferred from gender.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-67 Low	/group/[gameId]	Internationalisation / error messaging	The audience-mismatch error has no translation entry, so users are shown the raw constant Only `mockGetGameScreen` handles this code specially (`isAudienceError`, mod/2379.js:36-37, mod/631.js:749). Every write path leaks the constant to the UI, in Arabic as well as English. mod/2444.js also re-implements a subset of the map that 674 already provides, so the two drift. Evidence: The store throws a bare constant: mod/631.js:748 `ta = "AUDIENCE_MISMATCH";` raised by `ra`/`oa` (mod/631.js:763-768) from `mockCreateGroupBooking` (mod/631.js:13670 `oa(e, t)`), `mockConfirmSquadSpot` (mod/631.js:4010) and `mockOptOutSquad` (mod/631.js:4031). The error map has no entry for it — mod/674.js:53 carries `AUDIENCE_IMMUTABLE`: `seAudienceImmutable`, but no `AUDIENCE_MISMATCH` (verified by search over mod/674.js) — so `storeErrorText` returns the input unchanged (mod/674.js tail: `const s = e[A]; return s ? t(s) : A;`). Both screens then print it: mod/2444.js:60-72 `fe` maps nine group codes and returns `e` otherwise, rendered at mod/2444.js:222; mod/2486.js:25 `R(storeErrorText(e.message));` rendered at mod/2486.js:291-295.	A woman adding a male friend to a group, or confirming a squad spot on a mismatched match, sees the literal text 'AUDIENCE_MISMATCH' — in a product whose gender partition is a core promise, the one message explaining it is untranslated developer output.	Add `AUDIENCE_MISMATCH`: "seAudienceMismatch"" to mod/6`en`/`ar` copy in module 909, and delete the local `fe` map in mod/2444.js in favour of `storeErrorText` so there is one mapping.
P-CQUAL-68 Low	/media/upload.tsx	Input validation / unenforced contract	Upload visibility is never validated against its enum, and /media[id] ignores the backend's can_manage flag Neither the enum nor the ownership contract is enforced. An unrecognised visibility string is simultaneously not-private (so mockGetMediaUpload serves it to everyone) and not-public (so mockShareClip refuses every non-owner), producing a record in a state the UI has no label for and no path to repair. And the backend explicitly tells the screen whether the viewer may manage the upload, but the screen ignores it and renders the same controls to owners and strangers. Evidence: mod/631.js:13386 accepts any string: visibility: t.visibility ?? "unlisted", while the only legal values are defined client-side at mod/2454.js:149: B = ["public", "unlisted", "private"]. Downstream the value is only ever compared for equality - mod/631.js:13418 `if (li && "private" === a.visibility)`, mod/631.js:13431 `""public" !== n?.visibility`, mod/631.js:13452 `""private" === i?.visibility`. Separately, mod/631.js:13419 computes and returns an ownership flag the screen never consults: return { upload: a, clips: Qt.mediaClips.filter((e) => e.upload_id === t), can_manage: i }; mod/2451.js:65 destructures only two of the three fields: const { upload: re, clips: ae } = W;	A mistyped or injected visibility value creates a permanently half-broken upload: readable by every signed-in user, shareable by none, with no UI to correct it because /media/upload only ever creates. Meanwhile a non-owner who opens an unlisted upload's page sees the identical owner UI - share sheet, link button, report button - with nothing indicating the media is not theirs, which makes it easy to mint links for, and re-share, footage belonging to someone else.	Validate against the allowlist in mockUploadMatchVideo (mod/631.js:13386) e.g. `visibility: ["public","unlisted","private"].includes(t.visibility) ? t.visibility : "unlisted"`. In mod/2451.js:65 destructure can_manage and hide share controls, or mark the page read-only, when it is false.
P-CQUAL-69 Low	/media/upload.tsx	Error handling / dead code	The upload button reports 'consent required' for an expired session, and its consent error branch is unreachable The guard conflates two unrelated conditions: a missing session and missing consent both render 'consent required'. And the catch branch tests `e.message` against the raw backend code, but by the time the screen sees the error the message is already the localized string - the comparison never matches, so that branch is dead. The correct discriminator is the `.code` property the localizer attaches. Evidence: mod/2454.js:96 and mod/2454.js:113: if (e && J) { ... } else le(M("consentRequired")); where e is the session user (mod/2454.js:6) and J is the consent toggle. Because the button is already disabled without consent (mod/2454.js:116 `disabled: !J,`), the only way to reach the else branch is `e` being null. mod/2454.js:111 compares against a raw code that can never arrive: (le("consent_required" === e.message ? M("consentRequired") : e.message), ee(!1)); The 672 store proxy rewrites the message first - mod/672.js:41-45: return await (0, t.rpc)(String(n), e, () => c(...e)); } catch (e) { throw (0, r.localizeStoreError)(e); } and mod/674.js:316-321 replaces Error.message with the translated text, keeping the code only on a separate `.code` property: const E = new Error(e); return ((E.code = __.message), E); with consent_required mapped at mod/674.js:22.	A user whose 30-day session expired while the form was open fills everything in, taps Generate, and is told to tick a consent box that is already ticked - with no sign-in prompt and no way forward. The dead comparison is a latent maintenance trap: it reads as working consent handling, so the next person to add error cases to this screen will copy the same broken pattern.	In mod/2454.js:96 split the guard: when `!e`, route to sign-in or su session-expired message; keep the consent message only for `!J` mod/2454.js:111, test `e.code === "consent_required"` instead of `e.message`, or simply render the already-localized `e.message` since the 674 map covers this code.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CARCH-60 Low	/my-bookings.tsx	N+1 / missing pagination	/my-bookings is N+1 over games and payments with no pagination, so the screen slows down with account age Two per-row lookups into unindexed global tables, and no server-side cap on a list whose length only ever increases. Evidence: mod/631.js:2717-2748 — `mockGetMyBookings` filters the user's bookings then, per booking, calls `hi(t.game_id)` (mod/631.js:2503 — `gi().find(...)`), which in demo mode reallocates the games array first, mod/631.js:2502) and `Qt.payments.find((a) => "seat" === a.kind && a.game_id === t.game_id && ...)` — a full scan of the global payments table per row. There is no limit, offset or date filter; every booking the user has ever made is returned.	A user who has played weekly for two years has ~100 bookings; the screen performs 100 games scans and 100 payments scans against tables that hold every game and every payment in the system. The cost grows with both the user's history and total platform volume, and the whole history is rendered at once.	Build `Map<game_id, game>` and `Map<game_id+payer, payme` before the map at mod/631.js:2721, and add `{limit, before}` so it can page. Splitting upcoming from past bookings server-side would be the common case return a handful of rows.
P-CQUAL-70 Low	/passport/[id].tsx	Localisation / identity display	The passport header invents a handle from the display name, producing a bare "@@" for Arabic-named users A cosmetic handle is derived client-side from a display name instead of reading the real, unique `username`. The character class `[^\a-z0-9]` strips every non-ASCII letter. Evidence: mod/2473.js:41 `X = `@\${H.display_name.toLowerCase().replace(/^[^\a-z0-9]/g, "")}`;` rendered at mod/2473.js:80-83. The real handle exists on the profile and is used elsewhere: mod/631.js:14284 `username: a.username ?? null,` and mod/2475.js:157-161 `!!F.username && ... children: ["@", F.username],`, with uniqueness enforced per audience at mod/631.js:2377-2379. `dr` never returns `username` at all (mod/631.js:6649-6677).	For any user whose name is in Arabic — the primary market — the regex removes every character and the passport renders a bare "@@" under their name. For Latin names the derived handle is not unique and does not match the handle shown on /player/[id] and on the profile card (mod/1805.js:237-238), so the same person appears under two different handles on two screens.	Add `username` to the passport payload in `dr` (mod/631.js:6650 it at mod/2473.js:41, falling back to rendering nothing when it is not a derived string.
P-CQUAL-71 Low	/refunds.tsx	Error reporting	Three screens compute a specific error message and then throw it away in favour of generic copy The 674 error map exists precisely to turn `E_*` codes into actionable sentences; these three screens discard the result. An authorization failure, a storage-full failure and a transient network failure all render identically. Evidence: mod/2479.js:24 `O((0, C.storeErrorText)(e?.message ?? ""))    b("error"));` stores the localised reason in `D`, and mod/2479.js:83 then renders `body: b("refundsErrorBody")` — `D` is used only as a truthiness test at mod/2479.js:76. The same pair appears at mod/2481.js:29 `Y((0, C.storeErrorText)(e?.message ?? ""))    A("error"));` / mod/2481.js:80 `body: A("rewardsErrorBody")`, and at mod/2483.js:19 `D((0, S.storeErrorText)(e?.message ?? ""))    B("error"));` / mod/2483.js:67 `body: B("rewardsErrorBody")`. Their siblings do it correctly: mod/2506.js:82 `body: N` and mod/2474.js:42 `body: \$ ?? H("notFound")`.	A user on /refunds, /rewards or /rewards/ledger sees the same generic sentence whatever went wrong, so they cannot tell a retryable problem from a permanent one, and support has nothing to work from. The mapping work is done and simply not displayed.	Render the stored message with a fallback in each screen — `body: b("refundsErrorBody")` at mod/2479.js:83, `body: U ?? A("rewardsErrorBody")` at mod/2481.js:80, `body: O ?? B("rewardsErrorBody")` at mod/2483.js:67 — matching the pattern used at mod/2474.js:42.
P-CARCH-61 Low	/rewards/ledger.tsx	N+1	The loyalty earn breakdown rescans the entire ledger once per action type The per-user ledger slice is recomputed inside the loop instead of being hoisted above it. Evidence: mod/631.js:11568-11575 — `return L.LOYALTY_ACTIONS.map(({ action: a }) => { const i = js(e).filter((e) => e.action === a); return { action: a, rate: t[a] ?? 0, earned: i.reduce(...), count: i.length }; });` where `js = (e) => Qt.loyaltyLedger.filter((t) => t.user_id === e)` (mod/631.js:11497). `js(e)` is called inside the map, so the whole loyalty ledger is filtered once per entry in `LOYALTY_ACTIONS`.	With N action types the ledger is scanned N times instead of once, for a result that could be produced in a single pass. `Qt.loyaltyLedger` is append-only (mod/631.js:11497 has no pruning counterpart), so this grows with platform lifetime.	Hoist `const mine = js(e);` above the `LOYALTY_ACTIONS.map` mod/631.js:11570, or build `Map<action, {points, count}>` in one place and read from it.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CSEC-108 Low	<code>/teams/[id].tsx</code>	Abuse / notification spam	Throwing gauntlets has no rate limit, so one captain can spam every clan in the ladder No per-actor or per-clan throttle on challenge creation, and no bound on the proposed fixture time. The three fixed slots offered in the challenge sheet (module 1630, <code>`M()`</code>) are a client-side convenience, not an enforced constraint. Evidence: <code>mod/631.js:10665-10681</code> — the only duplicate guard is per opposing clan pair: <pre>... 10676 const s = (e) => (e.from_team_id === t && e.to_team_id === a) (e.from_team_id === a && e.to_team_id === t), 10678 if (Qt.clanBattles.some((e) => "pending" === e.status && s(e))) 10679 throw new Error("E_THERE_IS_ALREADY_A_PENDING_CHALLENGE"); 10680 if (Qt.clanBattles.some((e) => "accepted" === e.status && s(e) && new Date(e.starts_at).getTime() > d)) 10681 throw new Error("E_YOU_ALREADY_HAVE_A_BATTLE_BOOKED_WITH_THIS_CLAN"); ...</pre> Each accepted call writes two system chat messages and a push-style notification — <code>mod/631.js:10705-10717</code>: <pre>... 10705 await qo(t, `🏹 We threw a gauntlet at \${o.name} — ...`), 10706 await qo(a, `🏹 \${r.name} challenged us — answer on the battle board`), 10707 await bi({ ... type: "challenge_received", ... user_id: o.owner_id, ... }), ...</pre> Contrast the limits the codebase applies elsewhere: <code>mod/631.js:8540</code> caps clan creation at five per day, and <code>mod/631.js:13202-13206</code> caps club invites with <code>`DEFAULT_QUOTA.free.invitesPerDay`</code>. <code>`starts_at`</code> is also taken verbatim from the client at <code>mod/631.js:10689</code> (<code>`starts_at: i,`</code>) with no validation that it is a real, future date.	One captain can fire a pending gauntlet at every other clan in the ladder in a loop. Each target clan's owner gets a notification and both clans' chat histories get a system message, so the abuse is loud, persistent and visible to every member. Unanswered challenges expire into recorded "dodges" (<code>mod/631.js:10593-10598</code>, surfaced as the 🏹 badge at <code>mod/2489.js:224-231</code>), so a spammer can also manufacture public dodge shame for clans that simply ignore the flood. A crafted past <code>`starts_at`</code> additionally produces a battle whose seven-day result deadline has already lapsed.	Add a rolling-window cap in <code>`mockSendClanChallenge`</code> (<code>mod/631</code> — for example at most three pending outgoing challenges per clan per day, mirroring the shape of the check at <code>mod/631.js:8540</code> — <code>`starts_at`</code> (starts_at): reject non-dates, anything in the past, and anything a few weeks out.
P-CQUAL-72 Low	<code>/venue/portal.tsx</code>	Dead end / incorrect copy	A rejected venue is shown suspension copy and left with no route forward Rejected and suspended share one hint string, so a rejected applicant is told they were suspended. Neither status offers a re-apply action, an appeal, a contact route, or the reviewer's note — which the admin was forced to write (<code>mod/631.js:7139</code> <code>`if ("approve" !== a    "pending" !== n.status) && !r) throw new Error("E_A_REASON_IS_REQUIRED")`</code>). Evidence: <code>mod/2504.js:100-106</code> — <code>`approved`</code> <code>!== ae.status && Card({ ... children: "pending" === ae.status ? W("venuePendingHint") : W("venueSuspendedHint") })`</code> — a two-way branch covering four statuses. The transition table has four: <code>mod/631.js:7119-7124</code> — <code>`{ pending: ["approve", "reject"], approved: ["suspend"], rejected: ["approve"], suspended: ["approve"] }`</code>. The badge above it names the real one: <code>mod/2504.js:89</code> — <code>`Badge({ label: W(`vstatus_\${ae.status}`), tone: ce })`</code>, and <code>`vstatus_rejected`</code> resolves to "Rejected" while <code>`venueSuspendedHint`</code> reads "This venue has been suspended and cannot accept bookings." The admin's rejection reason is stored but never surfaced: <code>mod/631.js:7146</code> — <code>`(n.review_note = r null)`</code>, and <code>`review_note`</code> appears nowhere in <code>mod/2504.js</code>.	A venue whose registration was rejected opens the portal, sees a "Rejected" badge contradicted by text about suspension, a full management UI that quietly does nothing useful, and no explanation or next step. The reason an admin was required to provide never reaches them, so the feedback loop the review workflow is built around is broken at the last hop.	Branch on all four statuses at <code>mod/2504.js:105</code> with a distinct string <code>`rejected`</code>, render <code>`ae.review_note`</code> when present, and give the reviewer an action — re-apply or contact support — instead of an inert management screen.
P-CSEC-109 Low	<code>/venue/portal.tsx</code>	Auditability	Staff removal is an unconfirmed one-tap action and, unlike staff addition, writes no audit entry One half of a paired privileged operation is audited and the other is not, and the destructive half has no confirmation step in the UI. Evidence: <code>mod/631.js:7089-7093</code> — <code>`r.mockRemoveVenueStaff = async (e, t, a) => { (await ei()), await Sr(e, t); const i = hr(t); return ((i.staff = i.staff.filter((e) => e.user_id !== a)), await Za(ce, Qt.venueProfiles, i)); }`</code> — no <code>`logAudit`</code> call. The matching add does log: <code>mod/631.js:7085</code> — <code>`await (0, w.logAudit)("venue.updated", (0, w.actorRef)(e), { venue: t.slice(-6), staff: "added" })`</code>. Every other venue mutation logs too — <code>mod/631.js:7067</code> (<code>`venue.updated`</code>), <code>7241</code> (<code>`court.updated`</code>), <code>7261</code> (<code>`court.created`</code>), <code>7272</code>, <code>7434</code> (<code>`booking.confirmed`</code>), <code>7437</code> (<code>`booking.rejected`</code>). The trigger is a bare label with no confirmation: <code>mod/2504.js:451-458</code> — <code>`onPress: () => oe() => (0, S.removeVenueStaff)(e.id, re.id, t.user_id), hitSlop: 8, children: Text({ ... children: W("removeStaff") })`</code>.	Revocation of venue access leaves no trace: an owner, a co-owner, or an admin (who <code>`vr`</code> also admits, <code>mod/631.js:6828</code>) can strip staff and the event is invisible to any later investigation, while the corresponding grant is on record. Combined with the one-tap control, an accidental removal is both easy to cause and impossible to reconstruct.	Add a <code>`logAudit("venue.updated", actorRef(e), { venue: t.slice(-6) "removed" })`</code> call to <code>mockRemoveVenueStaff</code> (<code>mod/631.js:7092</code>) line 7085, and put a confirmation dialog in front of the remove call at <code>mod/2504.js:451</code>.

ID / Claimed severity	Interface	Risk Area	Candidate gap (with evidence)	Claimed impact	Suggested remediation
P-CQUAL-73 Low	/venue/portal.tsx	Concurrency / UI state	A single shared busy flag spins and disables every action button on the screen at once, while the switches it does not cover stay live Per-action pending state is modelled as one screen-wide boolean, and only the Button-based controls consult it — the Switch-based ones do not, so they remain tappable during an in-flight mutation. Evidence: mod/2504.js:15 — <code>`[Q, U] = (0, t.useState)(1),`</code> is the only busy flag, set in <code>`oe`</code> (mod/2504.js:36 <code>`U(0)`</code>), and passed to every action: <code>`loading: Q`</code> at mod/2504.js:178 (accept), 186 (reject), 339 (save court) and 476 (add staff). The Button component treats <code>`loading`</code> as disabled and replaces the label with a spinner — module 626: <code>`onPress:()=>{j h (...p?.())},disabled:j h, ... children:h?ActivityIndicator:...`</code>. The two switches are not covered — mod/2504.js:261-264 — <code>`Switch({ value: t.active, onValueChange: (s) => oe() => (0, S.setCourtActive)(e.id, t.id, s) })`</code> — and mod/2504.js:385-388 for auto-accept. The scanner repeats the pattern: mod/2505.js:14 <code>`[F, A] = useState(1)`</code> is passed as <code>`busy`</code> to all three buttons of every player row (mod/2505.js:153, 161, 169).	Accepting one booking blanks the labels of every Accept and Reject button in a list of pending bookings, which reads as the screen breaking rather than one row working; in the scanner, marking one player present freezes the controls for every other player, so a queue at the door is checked in one blocking round-trip at a time. Meanwhile a court-active or auto-accept switch can still be flipped mid-flight, and since <code>`oe`</code> reloads everything on completion (mod/2504.js:38 <code>`await le()`</code>), the later reload can overwrite the result of the earlier toggle.	Key the pending state by the entity being mutated — a <code>`Set`</code> of in booking/court/player ids — and have each control read only its own mod/2504.js:178/186/261/339/385/476 and mod/2505.js:153/161 the same flag to the two <code>`Switch`</code> components so they are disabled when mutation is running.
P-CSEC-110 Low	671 (fetchUserPosts)	Data exposure	A user's posts are readable by people they have blocked and by strangers when the profile is private Two readers of <code>`Qt.posts`</code> apply different rules. The per-author reader omits both the block check and the profile-visibility check that its neighbours enforce. Evidence: mod/631.js:14488-14495 — the only check is the audience partition: <pre>... r.mockGetUserPosts = async (e, t) => (await ei(), e !== t && oa(e, t), Qt.posts .filter((e) => e.author_id === t) map((t) => sl(t, e))); ... The feed builder over the same table does check blocks and mutes — mod/631.js:14481-14483: ... i = (t) => (t.author_id === e ia(a, aa(t.author_id))) && (t.author_id === e (!t(e, t.author_id) && !Xd(e, t.author_id))), ... and `mockGetPlayerProfile` gates the whole profile on `al(t).profile_visibility` (mod/631.js:14280-14281). Exposed at mod/671.js:664 <code>`e.fetchUserPosts = (o, c) => t.store.mockGetUserPosts(o, c);`</code>.</pre>	A blocked user can still read everything the person who blocked them has posted, and a private profile's posts are readable by any same-partition stranger. No consumer screen calls this today, so the exposure is latent — but the endpoint is on the shared facade and will be wired up by the next screen that needs a profile post grid.	In <code>`mockGetUserPosts`</code> (mod/631.js:14488) reuse the feed predicate on <code>`Xd(e, t)`</code> and on <code>`"private" === al(t).profile_visibility`</code> (or <code>`"follow" === al(t).profile_visibility`</code> without <code>`Vd(e, t)`</code>), matching <code>`mockGetPlayerProfile`</code> at mod/631.js:14281.
P-CARCH-62 Low	mod/1676.js (useLiveRefresh — used by home, games, /game/[id], /squad, /teams/chat)	Reconnect storm	The live-refresh EventSource reconnects every 3 seconds forever with no backoff or attempt cap Fixed 3-second retry with no exponential backoff, no jitter and no maximum attempt count. The loop is only entered when <code>`remoteEnabled()`</code> is true — i.e. when <code>`window.__PLAYORA_CONFIG__.backendUrl`</code> is set and <code>`/health`</code> answered — so it activates precisely when a real backend is deployed and its <code>`/events`</code> endpoint is unhealthy. Evidence: index.html module 1676 — <code>`n.onerror=()=>{n?.close(),n=null,l  (t=setTimeout(c,3e3))}`</code> inside <code>`useLiveRefresh`</code>. The guard <code>`t`</code> is only set by the effect cleanup, so as long as the screen stays mounted the retry loop is unbounded. Consumers: mod/1677.js:71, mod/1629.js:60, mod/2379.js:99, mod/2486.js:33, mod/2497.js:28.	Once <code>`backendUrl`</code> is configured, an <code>`/events`</code> endpoint that is reachable but failing puts every open client into a 20-requests-per-minute reconnect loop per mounted screen, with five screens capable of mounting it. A partial outage becomes a self-inflicted load amplifier against the recovering server, and each successful reconnect fires the refresh callback — which on home is six backend calls (mod/1677.js:42-62).	Replace the fixed <code>`setTimeout(c, 3e3)`</code> with exponential backoff capped at ~60 s, and stop retrying after a bounded number of consecutive failures until the next focus or visibility change.

6. What this audit did not cover

Discovery was thorough; verification was not. All sixteen auditors completed and produced 305 candidate findings across the full consumer surface, but only about forty of those reached an adversarial verifier before the run hit a session limit. The 35 confirmed findings below are trustworthy. The 269 listed as unverified are raw auditor output, and on the portion that was checked roughly one in eight was rejected.

Not examined	Why it matters here	What a follow-up pass should do
Arabic and RTL rendering	Half the intended market reads Arabic, and the app ships both locales	Drive every consumer screen with the locale forced to ar and check layout mirroring, numeral localisation and string truncation
The gender-audience partition under adversarial use	It is the rule with the most social consequence if it leaks	Attempt every cross-partition read and write directly against the store, as tools/rules.mjs does for admin roles
Under-18 players	Minimum age is 13 and the app arranges real-world meetings between strangers	Trace what an account aged 13 to 17 can see, book, message and be messaged by
Prayer times, Ramadan hours and Kuwait timezone handling	Scheduling is the core of the product and the home screen already groups by a UTC date key	Test scheduling and day-grouping across the local midnight boundary and during shifted hours
Physical safety at a venue	The product puts strangers on a pitch together; no auditor was assigned to it	Review check-in, no-show, conduct reporting and blocking as one safety path rather than as separate screens